

Aula 03

*DataPrev (Perfil 3: Desenvolvimento de
Software) Banco de Dados - 2026
(Pós-Edital)*

Autor:

Lucas Fargoni Di Ianni

09 de Julho de 2026

Índice

1) 5.0 SQL - Básico - Teoria	3
2) 5.0 SQL - Básico - Questões Comentadas - FGV	126
3) 5.0 SQL - Básico - Lista de Questões - FGV	192

SQL - BÁSICO

Introdução

Em 1970 Edgar F. Codd publicou um artigo propondo um modelo teórico de como organizar dados em tabelas como relações chamado "modelo relacional".

Baseados nas ideias introduzidas por Codd, Donald D. Chamberlin and Raymond F. Boyce, pesquisadores da IBM, começaram a construir o "System R", um dos primeiros protótipos de banco de dados relacional funcional.

Ocorre que, era **necessário criar uma linguagem para interagir com os dados** (como os consultar, adicionar, remover, atualizar etc.) no "System R", então esses pesquisadores criaram a SEQUEL (Structured English Query Language), baseada em inglês simples para facilitar sua utilização.

Porém, como o nome "SEQUEL" já era marca registrada de uma empresa de engenharia (a Hawker Siddeley), decidiram remover as suas vogais, ficando apenas com "SQL". Depois, retroativamente, **SQL** passou a significar **Structured Query Language**.



É por isso que até hoje existem duas pronúncias: a oficial "éss-kiú-él" (letra por letra) e a histórica "síquel" (resquício do nome original).

Enquanto a IBM não lançava seu produto comercial, mantendo o System R principalmente como um protótipo de pesquisa, uma pequena empresa (que depois se tornaria a Oracle) percebeu cedo o potencial do SQL e saiu na frente ao lançar, em 1979, o Oracle V2, apontado como a primeira implementação comercialmente disponível de SQL.

Na sequência, após testes com sucesso em clientes reais, a IBM começou a lançar seus produtos comerciais baseados no System R: o "System/38" (1979), "SQL/DS" (1981) e "IBM Db2" (1983).

O sucesso comercial desse tipo de produto unido ao peso da IBM no mercado fez com que a indústria inteira adotasse o SQL como padrão, iniciando a era dos bancos de dados relacionais.



(CEBRASPE (CESPE)/Ana (FINEP)/2024) Considerando-se que determinada empresa possui vários tipos de banco de dados para armazenamentos de dados estruturais, é correto afirmar que a linguagem SQL, nesse caso, tem a finalidade de

- a) desenvolver aplicações baseadas na linguagem Java.
- b) realizar cálculos matemáticos simples e complexos.
- c) gerenciar sistemas operacionais.
- d) desenvolver aplicativos para dispositivos móveis.
- e) manipular dados em banco de dados.

Comentários:

a) Errada. SQL não é uma linguagem para desenvolver aplicações Java. Java é uma linguagem de programação de propósito geral, enquanto SQL é específica para interagir com bancos de dados.

b) Errada. Embora SQL possa realizar alguns cálculos simples em consultas (como somas e médias), essa não é sua finalidade principal. SQL foi criada para consultar, adicionar, remover e atualizar dados em bancos de dados.

c) Errada. SQL não gerencia sistemas operacionais. Ela foi criada para interagir com os dados dentro de bancos de dados relacionais.

d) Errada. Desenvolver aplicativos para dispositivos móveis exige linguagens de programação específicas, e não SQL. SQL pode até ser usada dentro desses aplicativos para acessar dados, mas não é sua finalidade.

e) Correta. A SQL foi criada justamente para interagir com os dados de bancos de dados relacionais, permitindo consultá-los, adicioná-los, removê-los e atualizá-los. Ou seja, sua finalidade principal é manipular dados em banco de dados.

Gabarito: E

(CEBRASPE (CESPE)/Ana (CAU BR)/2024) A principal finalidade da linguagem SQL (structured query language) em um banco de dados relacional é permitir operações como consulta, adição e exclusão de dados.

Comentários:

A afirmação está correta porque descreve exatamente o que o SQL faz em um banco de dados relacional. O SQL é a linguagem usada para interagir com os dados, permitindo justamente operações como consultar (com o SELECT), adicionar (com o INSERT) e excluir (com o DELETE),

além de outras como atualizar dados. Repare que a banca usou a expressão "operações como", o que indica que está dando exemplos, e não uma lista fechada. Os três exemplos citados (consulta, adição e exclusão) estão todos dentro das finalidades reais do SQL, por isso a questão está certa.

Gabarito: Certo

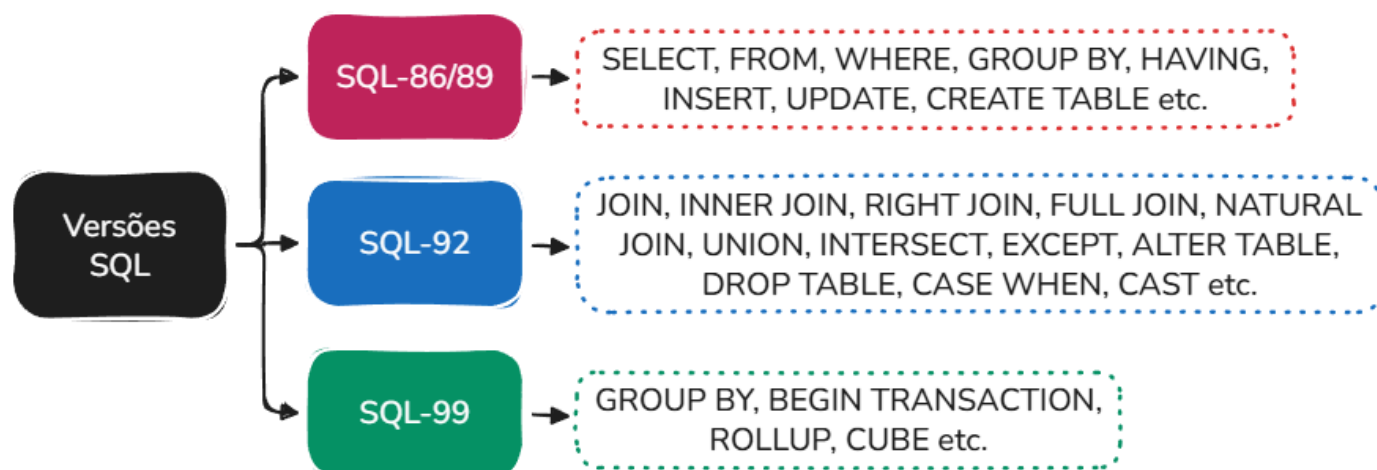
Padronização

À época, cada fabricante de banco de dados relacional (Oracle, IBM etc.) criou sua própria versão de SQL, diferente entre si. Para o mercado como um todo isso foi péssimo, porque criava dependências aos produtos de cada fabricante.

Por exemplo, um analista de banco de dados que trabalhasse com Oracle v2, poderia não conseguir emprego com o IBM Db2, por incompatibilidades entre os SQLs de cada produto.

Para corrigir isso, alguns órgãos padronizam a linguagem, ou seja, definiram um "SQL oficial" que todos deveriam seguir, garantindo um mínimo de compatibilidade entre os produtos. Criou-se, assim, o padrão americano, através da ANSI (em 1986) e, depois, unificou-se tudo no padrão internacional, através da ISO (em 1987).

A partir daí, ao longo dos anos, diversas revisões no padrão SQL ocorreram, sendo que a cada revisão adicionaram-se recursos importantes. Não precisamos saber qual mudança ocorreu exatamente em qual versão do SQL, porém apresento a seguir um diagrama informativo para fins de curiosidade.



De todo modo, consideramos o SQL-92 como o marco da indústria, pois muitos recursos que usamos hoje vieram dessa versão.

Por exemplo, foi nessa versão que as operações de álgebra relacional sobre conjuntos foram adicionadas, como UNION (união de conjuntos), UNION ALL (união de multiconjuntos) e CROSS JOIN (notação moderna para o produto cartesiano entre conjuntos, que antes ocorria apenas com a cláusula "FROM").

Além disso, também foi no SQL-92 que ocorreu a formalização dos tipos de junção com a sintaxe JOIN explícita: INNER JOIN, LEFT JOIN, RIGHT JOIN e FULL OUTER JOIN.

Embora aprofundaremos nesse tema ao longo da aula, saibam desde já que antes, nos padrões SQL anteriores, só era possível fazer o INNER JOIN, e, ainda assim, utilizando condições de junção após o FROM.



Na prática, cada produto (Oracle, MySQL, PostgreSQL, SQL Server etc.) implementa o SQL do seu jeito, seguindo o padrão apenas em alguma medida. Ou seja, um código SQL escrito para MySQL pode quebrar se tentarmos rodar no PostgreSQL, por exemplo.

Por isso que, além de estudar a aula de SQL, precisamos estudar os seus dialetos específicos (como PL/SQL da Oracle, ou T-SQL do SQL Server), quando o edital exige.



(CEBRASPE (CESPE)/TInf (Pref Cach Itapemirim)/2024) SQL é uma linguagem utilizada para declarar e acessar tabelas em um banco de dados relacional, caracterizando-se como um padrão para especificar o esquema das tabelas e o acesso aos dados.

Comentários:

A afirmação está correta porque descreve exatamente o que o SQL faz no dia a dia de um banco de dados relacional. Quando falamos em "declarar tabelas", estamos falando da parte que define a estrutura, ou seja, criar as tabelas com suas colunas e tipos de dados. Já "acessar" se refere a consultar, inserir, atualizar e remover os dados que ficam guardados nessas tabelas. Além disso, o enunciado menciona que o SQL é um padrão, e isso também procede, pois existem padronizações oficiais (como a da ANSI em 1986 e a da ISO em 1987) que garantem que diferentes bancos sigam uma base comum da linguagem.

Gabarito: Certo

(CEBRASPE (CESPE)/Ana Proc (DATAPREV)/2023) Somente são possíveis os seguintes quatro tipos de JOIN em SQL, segundo o padrão ANSI: INNER JOIN, LEFT JOIN, RIGHT JOIN e CROSS JOIN.

Comentários:

Embora muitos autores classifiquem o CROSS JOIN como um tipo de JOIN, segundo o padrão SQL-92, os tipos de JOIN são INNER JOIN, LEFT JOIN, RIGHT JOIN e FULL OUTER JOIN.

Gabarito: Errado

Linguagem Declarativa

Antes de nos aprofundarmos nas instruções em **SQL**, precisamos saber que **se trata de uma linguagem declarativa**.

Isso significa que, ao escrever um código em SQL, descrevemos o que queremos que o computador faça, em vez de descrevermos como o computador deve fazê-lo como nas linguagens imperativas ou procedurais.

Ou seja, **em linguagens declarativas escrevemos diretamente o resultado desejado, sem precisar explicar em como obter esse resultado desejado**.

Por exemplo, o Python é uma linguagem procedural, então, se quisermos retomar alguma informação, temos que escrever exatamente como obter nossas informações.

A seguir, escrevemos um pseudocódigo em Python que resolve o seguinte problema: *"quero o nome e o salário dos funcionários do departamento de TI, ordenados do maior salário para o menor"* (não precisa tentar entender o código, basta ler os comentários em verde).

```
# Versão imperativa em Python (pseudocódigo)

# Passo 1: ler todos os funcionários da tabela
funcionarios = ler_tabela("funcionarios")
# Passo 2: criar uma lista vazia para guardar quem vai entrar no resultado
resultado = []

# Passo 3: percorrer cada funcionário, um por um
for func in funcionarios:
    # Passo 4: verificar se o funcionário atende ao filtro (departamento de TI)
    if func.departamento == "TI":
        # Passo 5: se atender, adicionar à lista de resultado
        # apenas com as colunas que queremos (nome e salário)
        resultado.append({"nome": func.nome, "salario": func.salario})

# Passo 6: ordenar a lista pelo salário, do maior para o menor
# (reverse=True faz a ordem ser decrescente)
resultado.sort(key=lambda x: x.salario, reverse=True)

# Passo 7: devolver a lista final, já filtrada e ordenada
return resultado
```

Vejam o inferno que foi conseguir resolver nosso problema, já que fomos responsáveis por todos os detalhes de implementação: como percorrer os dados, como filtrar, como acumular os resultados e como ordenar.

Agora vejam como resolver esse mesmo problema usando uma linguagem declarativa como SQL.

```
SELECT nome, salario -- Busque para mim o nome e salário
FROM funcionarios -- dos registros que estão na tabela 'funcionarios'
WHERE departamento = 'TI' -- cujo departamento vale 'TI'
ORDER BY salario DESC; -- e me retorne os dados em ordem decrescente considerando
os valores de salário de cada um
```

Agora reparem como o paradigma mudou completamente: cada cláusula do SQL descreve uma característica do resultado desejado, e não um passo de execução.

Ou seja, não escrevemos "comece percorrendo as linhas, depois compare cada departamento, depois ordene...", mas apenas informamos o que queremos: quais colunas, de qual tabela, com qual filtro e em que ordem.

Certo professor, então quem de fato então “pensa” a forma de buscar os dados passo a passo e executa seu planejamento? O **Sistema Gerenciador de Banco de Dados (SGBD)** gente! Ou seja, nós só informamos o que queremos e o próprio SGBD se vira para conseguir retornar esse resultado da forma ordenada.



(FCC - Ana Leg (ALAP)/2020) Um auditor está checando práticas de aplicação de desconto comercial para clientes, executando consultas em SQL (Structured Query Language) nos bancos de dados digitais de uma empresa, para validar se os cálculos e regras de autorização estabelecidas pelo negócio estão sendo cumpridos. É correto afirmar que o uso de SQL em auditoria corresponde à aplicação de

- a) uma linguagem de programação orientada a objetos que funciona integrada ao sistema gerenciador de bancos de dados e permite simular operações.
- b) um software de auditoria especializado que permite, entre outras coisas, a simulação das funções do software auditado e seu banco de dados.
- c) uma linguagem de programação declarativa que funciona integrada ao sistema gerenciador de bancos de dados e permite consultar dados, entre outras operações.
- d) um sistema de segurança computacional que permite coletar dados, analisá-los e realizar medidas protetivas contra práticas incorretas de operação.

e) um software de aplicação de usuário final, operado com a finalidade de realizar registros de atividades transacionais do dia a dia da empresa.

Comentários:

a) **Errada.** SQL não é uma linguagem orientada a objetos. É uma linguagem declarativa. Além disso, o enunciado descreve corretamente o uso de SQL para consultas, não para "simular operações" como afirma a alternativa.

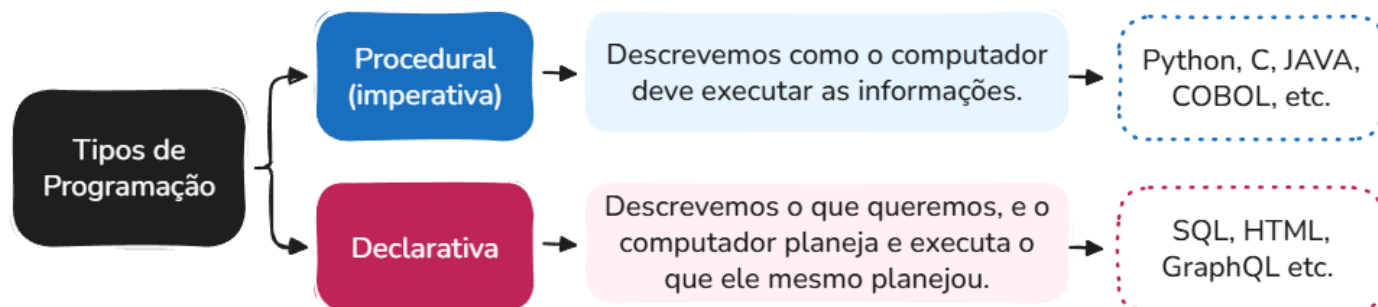
b) **Errada.** SQL não é um software de auditoria especializado, mas sim uma linguagem padronizada integrada aos SGBDs.

c) **Correta.** SQL é de fato uma linguagem declarativa: o usuário descreve o que quer obter, e não como obtê-lo. Além disso, funciona integrada ao SGBD. Ela permite realizar consultas (SELECT), manipulação de dados (INSERT, UPDATE, DELETE) e outras operações, sendo amplamente usada em auditoria para verificar registros e validar regras de negócio.

d) **Errada.** SQL não é um sistema de segurança computacional. Embora possa ser usada para identificar inconsistências e irregularidades, ela não realiza por si só "medidas protetivas".

e) **Errada.** SQL não é um software de aplicação de usuário final voltado a registros transacionais do dia a dia. É uma linguagem de consulta e manipulação de dados, podendo ser usada por desenvolvedores, analistas, auditores e administradores de banco de dados para diversas finalidades.

Gabarito: C



Subconjuntos SQL

A **abrangência dos comandos SQL** (há instruções para criar tabelas, deletar dados, conceder acesso aos dados etc.) **permite que os separemos em diversos subconjuntos** de acordo com suas características.

Infelizmente, não há consenso geral na literatura dos exatos subconjuntos que existem no SQL. Por exemplo, alguns autores estabelecem a existência de quatro subconjuntos básicos: **DQL** (Data Query Language), **DDL** (Data Definition Language), **DCL** (Data Control Language) e **DML** (Data Manipulation Language).

Entretanto, outros autores não separam o DQL como categoria própria, o tratando como parte do DML.

Além disso, há quem diga que há o TCL (Transaction Control Language) como um subconjunto à parte. Porém, isso também é debatido, já que alguns materiais tratam o TCL como parte do DCL ou do DML.



Em concursos, essas discussões permanecem, então precisamos estar atentos a qual visão a banca está cobrando para "marcar o 'x' no lugar certo".

Pois bem, na linguagem de consulta de dados (**Data Query Language - DQL**) temos apenas o **SELECT**, instrução usada para realizar consultas nos dados.

Já na linguagem de manipulação de dados (**Data Manipulation Language - DML**) temos mais comandos, como adicionar (**INSERT**), deletar (**DELETE**), modificar (**UPDATE**) e mesclar (**MERGE**) dados em um banco de dados.

Mas **cuidado!** Alguns autores defendem que o comando de selecionar dados (**SELECT**) exige certa manipulação deles, então o **SELECT deveria ser um DML**.

Continuando, enquanto o DML lida com os dados (o conteúdo dentro das tabelas), a linguagem de definição de dados (**Data Definition Language - DDL**) lida com a própria estrutura do banco de dados.

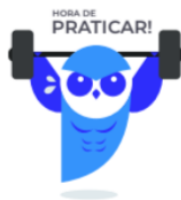
Porém, saibam que os objetos que compõem a estrutura do banco vão muito além das tabelas, pois incluem outras coisas como índices, usuários, views, schemas e outros elementos. Dessa forma, cabe ao DDL manipular esses objetos.

Os comandos fundamentais do DDL são **CREATE** (que cria um novo objeto no banco), **ALTER** (que modifica um objeto existente), **DROP** (que remove um objeto completamente) e **TRUNCATE** (que remove todos dados de uma tabela de forma mais eficiente que o **DELETE**).

Já a linguagem de controle de dados (**Data Control Language - DCL**) agrupa as instruções responsáveis por controlar o acesso aos dados e objetos do banco, ou seja, os comandos que estabelecem quem pode fazer o quê no banco de dados, estabelecendo permissões às informações.

Os dois comandos fundamentais de DCL são o **GRANT**, que concede permissões (como "o usuário João pode fazer **SELECT** na tabela Clientes"), o **REVOKE**, que remove permissões já concedidas (como "agora o usuário João não pode mais fazer **DELETE** na tabela Clientes") e **DENY**, que bloqueia terminantemente o acesso e a possibilidade de conceder acesso, seja por

qual meio for (como "não é possível mais conceder o usuário João permissão de DELETE na tabela Clientes").



(CEBRASPE (CESPE)/APC (FUNPESP,EXE)/Estatística Ciências de Dados e Inovação/2025) DCL (data control language) é responsável pelas tarefas administrativas de controle do próprio banco de dados; em SQL, comandos de GRANT, REVOKE e DENY são exemplos de comandos utilizados para esse fim.

Comentários:

A DCL (Data Control Language) é a linguagem responsável por controlar o acesso aos dados e objetos do banco, estabelecendo permissões (tarefas de natureza administrativa). Os comandos GRANT (que concede permissões) e REVOKE (que as remove) são os dois fundamentais do DCL, e o DENY, implementado em alguns bancos, também integra essa linguagem ao bloquear terminantemente o acesso independentemente de qualquer GRANT.

Gabarito: Certo

Finalmente, na linguagem de controle de transação (Transaction Control Language - **TCL**) ou linguagem de transação de dados (Data Transaction Language - **DTL**) temos as instruções SQL responsáveis por gerenciar as transações que ocorrem nos dados do banco.

Lembrem-se que uma transação se trata de um grupo de operações sobre os dados que precisa ser tratado como uma unidade atômica, ou seja, ou tudo acontece, ou nada acontece. Assim, com os comandos da **TCL** tornamos permanentes (ou descartamos) no banco de dados as operações executadas pelos comandos DML.

Dessa forma, o TCL possui comandos para confirmar todas as alterações da transação, tornando-as permanentes no banco (**COMMIT**), desfazer tudo que foi feito desde o início da transação, retornando o banco ao estado anterior (**ROLLBACK**), criar um ponto intermediário dentro da transação, para não ser necessário desfazer tudo com ROLLBACK (**SAVEPOINT**) e também configurar comportamentos avançados a transação (**SET TRANSACTION**).

Vale lembrar que alguns autores e fabricantes (como a Oracle) tratam o TCL como categoria separada. Porém, com muita frequência, os comandos de transação são classificados como parte do DML ou do DCL.



(FGV/Tecno Jr (INPE)/2024) SQL (Structured Query Language) é uma linguagem declarativa padrão usada para gerenciar e manipular bancos de dados relacionais. Ela fornece um conjunto de comandos que permitem aos usuários realizarem diversas operações, como consultar, inserir, atualizar e excluir dados de bancos de dados relacionais. Relacione as linguagens declarativas às suas respectivas propriedades.

1. DDL – Data Definition Language.
 2. DML – Data Manipulation Language.
 3. DTL – Data Transaction Language.
 4. DCL – Data Control Language.
- () Contém o comando SELECT.
- () Contém os comandos COMMIT e ROLLBACK.
- () Utilizada para dar acesso aos usuários.
- () Contém os comandos CREATE, ALTER e DROP.
- () Contém os comandos INSERT, UPDATE e DELETE.

Assinale a opção que indica a relação correta, segundo a ordem apresentada.

- a) 1 – 4 – 4 – 1 – 2.
- b) 2 – 3 – 4 – 1 – 2.
- c) 2 – 3 – 3 – 1 – 2.
- d) 3 – 4 – 3 – 2 – 1.
- e) 2 – 3 – 4 – 3 – 1.

Comentários:

SELECT → DML (2);

COMMIT e ROLLBACK → DTL/TCL (3);

controle de acesso → DCL (4);

CREATE, ALTER e DROP → DDL (1);

INSERT, UPDATE e DELETE → DML (2),

resultando na sequência 2 – 3 – 4 – 1 – 2,

Gabarito: B

(FGV/Analista Legislativo - Analista de Informática (CM Fortaleza)/2024) No contexto das linguagens de manipulação de dados, relacione as linguagens com seus respectivos comandos:

1. DDL

2. DML
3. DQL
4. DTL
5. DCL

() GRANT () TRANSACTION () SELECT () INSERT () DROP

Assinale a opção que indica a relação correta na ordem apresentada.

- a) 1 – 2 – 3 – 4 – 5.
- b) 2 – 1 – 5 – 3 – 4.
- c) 3 – 4 – 1 – 5 – 2.
- d) 4 – 3 – 2 – 1 – 5.
- e) 5 – 4 – 3 – 2 – 1.

Comentários:

GRANT → DCL (5);

TRANSACTION → DTL (4);

SELECT → DQL (3);

INSERT → DML (2);

DROP → DDL (1),

formando a sequência 5 – 4 – 3 – 2 – 1

Gabarito: E

(COGEPE/FIOCRUZ/Ciência de Dados em Saúde/2024) Sobre a linguagem SQL, é INCORRETO afirmar que:

- a) os comandos CREATE, ALTER, DROP e TRUNCATE são usados para a definição da estrutura do banco de dados.
- b) os comandos INSERT, UPDATE, DELETE e SELECT são usados para modificar os dados.
- c) os comandos GRANT e REVOKE são usados para controlar as opções de acesso aos dados.
- d) os comandos COMMIT, ROLLBACK, SAVEPOINT e SET TRANSACTION são usados para gerenciar transações.
- e) o comando SELECT é usado para selecionar dados.

Comentários:

O SELECT pertence ao DQL e serve para consultar dados, não para modificá-los. Os comandos de modificação (DML) são INSERT, UPDATE, DELETE e MERGE. As demais alternativas estão corretas: CREATE, ALTER, DROP e TRUNCATE são DDL; GRANT e REVOKE são DCL; COMMIT, ROLLBACK, SAVEPOINT e SET TRANSACTION são TCL.

Gabarito: B

(CEBRASPE (CESPE)/SUSEP/Tecnologia da Informação e Ciência de Dados/2025) O comando SELECT nome, idade FROM CLIENTE; pertence à DML, pois consulta os dados armazenados no banco para eventual tomada de decisão.

Comentários:

Embora alguns autores incluam o SELECT no DML, para a CESPE devemos considerar que o SELECT pertence ao DQL (Data Query Language).

Gabarito: Errado

Outros subconjuntos SQL

Na prática, os SGBDs podem aceitar diversos subconjuntos SQL além das mais comuns que estudamos até aqui.

Por exemplo, há alguns que aceitam a **SDL (Storage Definition Language**, ou Linguagem de Definição de Armazenamento), que trata como e onde os dados são fisicamente armazenados.

Percebam que, enquanto a DDL (CREATE TABLE, ALTER TABLE) define a estrutura lógica dos dados (quais colunas existem, quais tipos têm, quais são as restrições etc.), **a SDL define a estrutura física**, ou seja, como os bytes são de fato organizados em disco.

Em outras palavras, a SDL estabelece o esquema interno do banco de dados, a camada mais baixa que descreve como as tabelas são realmente guardadas nos arquivos físicos. Assim, aqui definimos coisas **como ordem dos campos, quantos bytes por campo serão usados no storage, como os registros serão acessados, quais índices existem** etc.

Também temos a **VDL (View Definition Language**, ou Linguagem de Definição de Visão), linguagem dedicada a especificar interfaces de usuário para os dados do banco, ou seja, a responsável por definir como os dados serão apresentados por quem os consome.

Dessa forma, o **foco da VDL é definir visões** (views) sobre os dados, organizando o esquema externo, isto é, as "janelas" personalizadas que diferentes usuários usam para ter acesso específicos aos dados.



(FEPESE/UDESC/Analista de Suporte/2022) São linguagens de sistemas gerenciadores de banco de dados válidas:

1. SDL (Storage Definition Language)
2. VDL (View Definition Language)
3. DML (Data Manipulation Language de alto nível)

4. DML (Data Manipulation Language de baixo nível)

Assinale a alternativa que indica todas as afirmativas corretas.

- a) É correta apenas a afirmativa 3.
- b) São corretas apenas as afirmativas 2 e 3.
- c) São corretas apenas as afirmativas 3 e 4.
- d) São corretas apenas as afirmativas 2, 3 e 4.
- e) São corretas as afirmativas 1, 2, 3 e 4.

Comentários:

A SDL (Storage Definition Language) e a VDL (View Definition Language) são linguagens reconhecidas por alguns SGBDs além das mais comuns. Já a DML pode se apresentar em dois níveis: de alto nível, com comandos mais declarativos e próximos da linguagem humana (como os do SQL padrão), e de baixo nível, com operações mais procedurais e próximas da estrutura interna do banco, usadas geralmente em conjunto com linguagens de programação.

Gabarito: E

(CEBRASPE (CESPE)/ANTT/Tecnologia da Informação/Desenvolvimento de Sistemas da Informação/2013) Existem várias categorias de linguagens de SGBD: a de definição de dados (DDL), usada para especificar esquemas (conceitual e externo); a de manipulação de dados (DML), que provê um conjunto de operações básicas para manipulação de dados; a de definição de visões (VDL), utilizada para especificar a visão do usuário (esquema externo) e seu mapeamento para o esquema conceitual e a de definição de armazenamento (SDL), usada para especificar o esquema interno de armazenamento dos dados no banco.

Comentários:

A DDL especifica os esquemas conceitual e externo (estrutura lógica); a DML provê operações básicas de manipulação de dados; a VDL é dedicada a especificar visões do usuário, ou seja, também o esquema externo, definindo como os dados são apresentados a quem os consome; e a SDL especifica o esquema interno, isto é, como os dados são fisicamente armazenados em disco: incluindo organização dos bytes, acesso aos registros e definição de índices.

Gabarito: Certo

Definição de Dados e Integridade (DDL)

Na Data Definition Language, temos comandos, como CREATE, ALTER, DROP e TRUNCATE, para manipular objetos da estrutura do banco de dados. Então, vamos estudar esses comandos a partir do estudo dos objetos que podem ser criados, alterados e apagados.

Esquema de Banco de Dados

Vamos começar estudando o objeto esquema (**schema**) do banco de dados, que nada mais é que um **organizador dos outros objetos do banco**. Se ficou confuso, pense no esquema como a uma "pasta" do seu computador, e os outros objetos (como tabelas, views, restrições etc.) como os arquivos dentro dessa pasta.

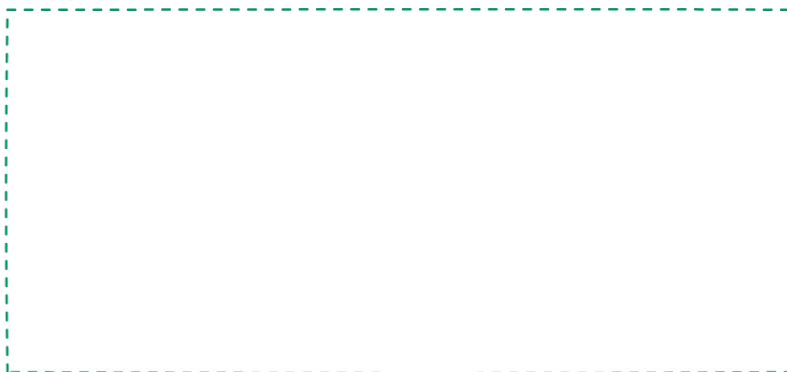
Assim, ao criarmos um objeto como uma tabela, ele não fica "solto" dentro do banco, porque está associado a um schema.

Na prática, podemos criar um esquema através do comando "CREATE SCHEMA" seguido do seu nome. A seguir, vamos criar um esquema "Faculdade" e vou representá-lo visualmente como um "local" em que posteriormente colocaremos objetos dentro.

```
CREATE SCHEMA FACULDADE;
```



FACULDADE



Existe um objeto na própria infraestrutura do sistema de banco de dados chamado catálogo, que está um nível acima do schema na hierarquia de organização de seus elementos. Assim, o catálogo é uma coleção de schemas.

Pensem assim, se o schema é uma pasta, então o catálogo é o armário que contém várias pastas. Em resumo, a hierarquia básica fica assim:

Sistema de Banco de dados → Catálogo → Schemas → Objetos (tabelas etc.)

Se criarmos um esquema e depois tabelas dentro dele, podemos acessar esses elementos os referenciando por um "." no SQL. Por exemplo, se houver uma tabela "Professor" no esquema "FACULDADE", dentro de um SQL podemos a invocar com "FACULDADE.Professor".

De modo oposto, podemos remover um esquema do banco de dados com o comando "DROP SCHEMA".

```
DROP SCHEMA FACULDADE;
```

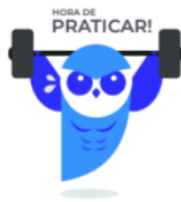
Ocorre que esse comando é muito perigoso, porque, após esse comando, o esquema FACULDADE simplesmente não existe mais. Assim, qualquer referência a esse esquema (como em consultas, views etc.), em qualquer local do banco de dados, quebrará.

Por isso, podemos inserir a cláusula RESTRICT ao comando para que só seja possível remover o objeto se nada depender da existência dele. Ou seja, se houver qualquer referência ao schema, o banco rejeita o comando.

```
DROP SCHEMA FACULDADE RESTRICT;
```

Porém, se quisermos criar um efeito cascata dessa remoção do esquema para seus objetos subordinados (como as tabelas associadas a ele), podemos adicionar a cláusula CASCADE.

```
DROP SCHEMA FACULDADE CASCADE;
```



(FACTO/IF ES/Laboratório/Informática/2025) Um esquema de um banco de dados relacional, denominado IFES, precisa ser removido completamente, bem como todos os elementos a ele associados, incluindo tabelas, domínios, restrições e o esquema propriamente dito. Para realizar essa ação, qual comando SQL deve ser utilizado?

- a) DELETE SCHEMA IFES;
- b) ERASE SCHEMA IFES CASCADE;
- c) FORMAT SCHEMA IFES;
- d) DROP SCHEMA IFES CASCADE;
- e) CANCEL IFES;

Comentários:

- a) **Errada.** DELETE não é usado para remover esquemas, e sim para apagar registros de dentro de uma tabela.
- b) **Errada.** ERASE não é um comando válido em SQL para remoção de objetos do banco.
- c) **Errada.** FORMAT não é um comando SQL para remoção de esquemas.
- d) **Correta.** O comando DROP SCHEMA remove o esquema do banco, e a cláusula CASCADE faz com que essa remoção seja em efeito cascata, eliminando junto todos os objetos subordinados a ele, como tabelas, domínios e restrições.
- e) **Errada.** CANCEL não é um comando SQL para remover esquemas.

Gabarito: D

Tabelas (tables)

As **tabelas** nos permitem armazenarmos os dados no banco, já que, para SGBDs relacionais, todo dado deve estar associado a uma tabela.

Podemos criar tabelas utilizando o comando "CREATE TABLE" seguido do nome da tabela, passando, na sequência, os nomes das suas colunas e os tipos de dados que aceitam.

A seguir, vamos criar uma tabela "Professores", com um atributo "CPF" sendo um número inteiro (INTEGER) e um atributo "nome" sendo uma palavra de até 50 caracteres (VARCHAR(50)), no schema "FACULDADE".

```
CREATE TABLE
FACULDADE.Professor (
    CPF INTEGER,
    nome VARCHAR(50)
);
```



FACULDADE

Professor	
CPF	nome

Em concursos públicos raramente o examinador referencia o schema nos SQLs. Isso é possível porque, ao criarmos um objeto sem referenciar esquema nenhum, o SGBD o atribui a algum esquema padrão previamente existente no banco, ou ao esquema que estivermos trabalhando no momento.

```
CREATE TABLE Professor (
-- Tabela criada em um
schema padrão.
    CPF INTEGER,
    nome VARCHAR(50)
);
```



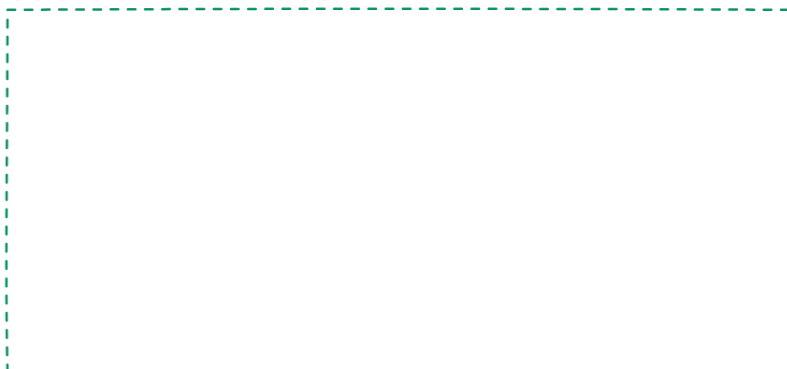
SCHEMA PADRÃO DO SGBD

Professor	
CPF	nome

Do mesmo modo que aprendemos na seção anterior, podemos descartar nossa tabela usando o comando DROP, perdendo, consequentemente, todos os dados dentro dela.

SCHEMA PADRÃO DO SGBD

```
DROP TABLE Professor;
```



(INAZ do Pará/Técnico de Tecnologia da Informação (CORE SP)/2019) SQL é uma linguagem de banco de dados abrangente, que possui instruções para definições de dados, consultas e atualizações. Apresenta alguns comandos principais e, entre eles, o "CREATE TABLE", que serve para:

- a) Definir uma nova tabela de banco de dados relacional, criando as relações necessárias aos registros dessa tabela.
- b) Especificar uma nova relação, dando-lhe um nome e especificando seus atributos e restrições iniciais.
- c) Criar uma nova tabela a partir dos registros encontrados em tabelas diversificadas de banco de dados distintos.
- d) Juntamente com o comando "CREATE DOMAIN", estabelecer todos os endereços dos registros de uma tabela.
- e) Definir uma tabela centralizada a partir dos registros encontrados em banco de dados distribuídos.

Comentários:

- a) **Errada.** O CREATE TABLE não cria relações entre tabelas, ele apenas cria uma nova tabela com seus atributos e tipos de dados.
- b) **Correta.** O CREATE TABLE serve exatamente para criar uma nova tabela, dando um nome a ela e definindo seus atributos (colunas com seus tipos) e restrições iniciais, como PRIMARY KEY, UNIQUE, NOT NULL, entre outras.
- c) **Errada.** O CREATE TABLE cria uma tabela nova e vazia, não a partir de registros de outras tabelas. Para preencher a tabela com dados, usamos o INSERT depois.

d) **Errada.** O CREATE DOMAIN serve para definir tipos de dados personalizados (como TIPO_CPF AS CHAR(11)) para reaproveitar em vários atributos, e não para estabelecer endereços de registros.

e) **Errada.** O CREATE TABLE simplesmente cria a estrutura de uma tabela no banco, não tem relação com centralizar registros vindos de bancos distribuídos.

Gabarito: B

(CEBRASPE (CESPE)/Ana Proc (DATAPREV)/DATAPREV/2023) Na DDL, que é uma linguagem declarativa, é descrito o que se deseja fazer, em vez de como fazê-lo, como comandos para definir tabelas e procedures, tal qual exemplificado a seguir.

```
CREATE TABLE <tablename> (col1 int, col2 int, col3 int)
```

Comentários:

A afirmação está correta porque toca em duas características importantes do SQL. Primeiro, o SQL é uma linguagem declarativa, ou seja, a gente diz o que quer que seja feito, e não o passo a passo de como o banco deve fazer. Segundo, a DDL (Data Definition Language) é justamente o subconjunto do SQL responsável por definir a estrutura do banco, usando comandos como CREATE, ALTER, DROP e TRUNCATE. O exemplo dado, um CREATE TABLE criando uma tabela com três colunas inteiras, é um caso clássico de DDL em ação: estamos apenas declarando "quero uma tabela com essas colunas", sem nos preocupar como o banco vai armazenar isso fisicamente.

Gabarito: Certo

Tipos de dados de atributo

Vimos no exemplo da seção anterior que CPF é um número inteiro (tipo INTEGER) e nome é uma palavra de até 50 caracteres (tipo VARCHAR(50)). Ocorre que esses tipos de dados são apenas alguns de todos possíveis que podemos definir para nossos atributos.

Assim, antes de avançarmos no estudo dos comandos DDL, precisamos passar pelos principais tipos de dados que podem existir nas nossas tabelas.

Os tipos de dados se organizam em numéricos, cadeia de caracteres, cadeia de bits, booleano, data e hora, porém, como existem diversas espécies nesses gêneros, fica melhor aprendermos tudo por uma tabela resumo.

Categoria	Tipo de Dado	Explicação	Observações
Numérico	INTEGER / INT	Inteiro padrão	Números inteiros sem casa decimal
	SMALLINT	Inteiro pequeno	Inteiro com menor faixa de valores possíveis
	FLOAT / REAL	Ponto flutuante	Números reais com precisão simples

Categoria	Tipo de Dado	Explicação	Observações
	DOUBLE PRECISION	Ponto flutuante duplo	Números reais com mais casas após a vírgula
	DECIMAL(i, j) / DEC / NUMERIC	Decimal exato	i = total de dígitos (precisão); j = dígitos após o ponto (escala).
Cadeia de caracteres	CHAR(n) / CHARACTER(n)	Tamanho fixo	Palavras de exatos "n" caracteres. Cadeias menores são preenchidas com espaços à direita
	VARCHAR(n) / CHARACTER VARYING(n)	Tamanho variável	Palavras de até "n" caracteres. Não há preenchimento de espaços.
	CLOB(tamanho)	Texto longo	Para grandes textos (documentos). Tamanho em K, M ou G. Ex: CLOB(20M)
Cadeia de bits	BLOB(tamanho)	Binário longo	Para grandes valores binários (como imagens). Tamanho em K, M ou G. Ex: BLOB(30G)
Booleano	BOOLEAN	Lógico	Valores: TRUE, FALSE ou UNKNOWN (lógica de 3 valores)
Data e hora	DATE	Data	Formato YYYY-MM-DD. Componentes: YEAR, MONTH, DAY
	TIME	Hora	Formato HH:MM:SS. TIME(i) para frações de segundo. WITH TIME ZONE para fuso horário (+13:00 a -12:59)
	TIMESTAMP	Data + hora	Campos DATE + TIME + frações de segundo (mín. 6 posições). Aceita WITH TIME ZONE
	INTERVAL	Intervalo	Valor relativo para incrementar/decrementar DATE, TIME ou TIMESTAMP. Qualificado como YEAR/MONTH ou DAY/TIME

Na prática do dia a dia, é muito comum o tipo de dado de determinada coluna ser inconveniente para nossas necessidades, precisando ser convertido em outro. Devido a isso, o padrão SQL estabeleceu uma função muito útil para conseguirmos isso, a CAST.

Assim, com a CAST, convertemos um valor (de qualquer tipo) em um outro tipo de dado específico, bastando passarmos o nosso valor antigo e o tipo de dado a ser convertido após um AS. A seguir, vamos converter um número com vírgula (FLOAT) para um inteiro (INT).

```
CAST(25.65 AS int); -- O resultado será 25 ou 26, a depender do banco de dados.
```

Domínio (domain)

Muitas vezes temos um tipo de dado específico ao nosso negócio e que é comum a vários atributos de várias tabelas ao longo do nosso banco de dados.

Então, ao invés de ficar repetindo esse mesmo tipo em vários SQLs, podemos criar um **domínio** (CREATE DOMAIN) que **representa** esse **tipo de dado para utilizá-lo nos atributos das tabelas**.

Por exemplo, se definirmos que um atributo CPF será necessariamente um CHAR(11), podemos criar o domínio "TIPO_CPF" para armazenar essa informação.

```
CREATE DOMAIN TIPO_CPF AS CHAR(11);
```

A partir de agora, em vez de escrever CHAR(11) toda vez que precisarmos de um campo de CPF, podemos usar o TIPO_CPF.

```
CREATE TABLE Professor (  
    CPF TIPO_CPF,  
    nome VARCHAR(50)  
);
```

Vejam que, assim, ganhamos em legibilidade, já que, ao ler a definição da tabela "TIPO_CPF" contextualiza muito mais do que "CHAR(11)", e em manutenção, se um dia o formato do CPF mudar, alteramos apenas a definição do domínio e todos os atributos que o utilizam são afetados automaticamente (sem domínios, teríamos que encontrar e alterar cada CHAR(11) em cada CPF manualmente em todas as tabelas).

Restrições (constraints)

As restrições (**constraints**) são basicamente **regras de comportamento para os valores dos atributos de nossas tabelas**.

Acredito que a constraint mais clássica seja **PRIMARY KEY** (chave primária), que força um **atributo (ou conjunto de atributos) a sempre possuir um valor** (ou seja, não pode ser nulo,

ausente) **e também que seja único**, (ou seja, não permite que existam dois valores iguais em linhas distintas para o(s) atributo(s)).

É possível criar essa restrição diretamente no atributo quando da criação da tabela. Para isso, basta informar o comando PRIMARY KEY após a definição do seu tipo.

Por exemplo, na criação da tabela (CREATE TABLE) Professores a seguir, definimos CPF como chave primária.

```
CREATE TABLE Professor (  
    CPF CHAR(11) PRIMARY KEY,  
    nome VARCHAR(50)  
);
```



Professor	
<u>CPF</u>	nome

Se nossa chave primária for um conjunto de atributos (chave composta), podemos declarar a PRIMARY KEY ao final do comando CREATE TABLE passando a lista de atributos. A seguir, adicionamos um atributo RG à tabela Professor e criamos uma chave primária composta para essa tabela com CPF e RG.

```
CREATE TABLE Professor (  
    CPF INTEGER,  
    RG INTEGER,  
    nome VARCHAR(50),  
    PRIMARY KEY (CPF, RG)  
);
```



Professor		
<u>CPF</u>	<u>RG</u>	nome

Na prática, muitos SGBDs oferecem a possibilidade de criarmos chaves primárias auto incrementais, isto é, colunas em que o valor único é atribuído pelo próprio banco de dados de forma sequencial aos valores já existentes na tabela.

Porém, como não se trata de um padrão SQL, cada fornecedor tem sua própria implementação para essa funcionalidade, como AUTO_INCREMENT (no MySQL), IDENTITY (no SQL Server), SERIAL (no PostgreSQL) etc.

Vejamos a seguir como declarar um campo "id" auto incremental na tabela Professor em diversos bancos de dados.

MySQL	SQL Server	PostgreSQL
<pre>CREATE TABLE Professor (id INT AUTO_INCREMENT PRIMARY KEY, CPF INTEGER, RG INTEGER, nome VARCHAR(50));</pre>	<pre>CREATE TABLE Professor (id INT IDENTITY(1,1) PRIMARY KEY, -- (1,1): comece em 1 e incremente de 1 em 1 CPF INTEGER, RG INTEGER, nome VARCHAR(50));</pre>	<pre>CREATE TABLE Professor (id SERIAL PRIMARY KEY, CPF INTEGER, RG INTEGER, nome VARCHAR(50));</pre>

Continuando, com a restrição **UNIQUE** definimos que **determinado atributo deve ter valores únicos**, porém aceitando valores ausentes.



No contexto do modelo relacional de bancos de dados, **o valor especial usado para representar a ausência ou desconhecimento de informação é denominado NULL** (ou valor nulo, em português).

De todo modo, essa é justamente a situação que se enquadram as chaves alternativas (secundárias).

```
CREATE TABLE Professor (  
    CPF INTEGER PRIMARY KEY,  
    RG INTEGER UNIQUE,  
    nome VARCHAR(50)  
);
```



(CEBRASPE (CESPE)/ATM (Pref Aracaju)/2021) Em um banco de dados relacional, a condição que garante que valores não possam se repetir dentro da mesma coluna denomina-se

- a) foreign key.
- b) cláusula unique.

- c) reference key.
- d) índice cluster.
- e) domain restriction.

Comentários:

a) Errada. A foreign key (chave estrangeira) serve para ligar tabelas, fazendo com que um atributo de uma tabela referencie a chave de outra tabela, garantindo integridade referencial. Não tem a função de impedir repetição de valores em uma coluna.

b) Correta. A cláusula UNIQUE define que determinado atributo deve ter valores únicos dentro da coluna, ou seja, não permite repetição de valores. A diferença para a PRIMARY KEY é que o UNIQUE aceita valores ausentes (NULL).

c) Errada. "Reference key" não é uma restrição padrão do SQL para impedir repetição de valores em uma coluna.

d) Errada. Índice cluster está relacionado à forma de organização física dos dados no armazenamento, e não a uma regra que impede repetição de valores em uma coluna.

e) Errada. "Domain restriction" não é a restrição usada para garantir unicidade de valores. Lembre-se que o domínio (DOMAIN) serve para criar tipos de dados personalizados reutilizáveis, e não para impedir repetições.

Gabarito: B

(FGV/Ana (DPE RS)/Desenvolvimento de Sistemas/2023) Para criar o cadastro dos clientes da sua empresa de TI, Paulo implementou a tabela CLIENTE e definiu o atributo CLIENTE_ID para identificar os clientes com um número sequencial único.

O recurso de banco de dados que Paulo deve usar para gerar automaticamente um número único quando novos registros são inseridos na tabela CLIENTE é o:

- a) index;
- b) unique key;
- c) constraints;
- d) primary key;
- e) auto increment.

Comentários:

a) Errada. Index é um recurso para acelerar buscas, não tem relação com gerar números sequenciais automaticamente.

b) Errada. Unique key apenas garante que os valores não se repitam na coluna, mas não gera valores automaticamente.

c) Errada. Constraints são regras de comportamento gerais (como PRIMARY KEY, UNIQUE, CHECK), não um recurso específico para gerar números sequenciais.

d) **Errada.** Primary key garante que o valor seja único e não nulo, mas sozinha não gera o número automaticamente.

e) **Correta.** O auto increment é o recurso usado para gerar automaticamente um número único e sequencial sempre que um novo registro é inserido na tabela. Cada SGBD tem sua sintaxe (AUTO_INCREMENT no MySQL, IDENTITY no SQL Server, SERIAL no PostgreSQL), mas a ideia é a mesma: o próprio banco atribui o valor da chave de forma sequencial.

Gabarito: E

(VUNESP/Pref Santo André/Administração de Bancos de Dados/2024) Considerando o modelo relacional de bancos de dados relacionais, é possível que não se conheça ou que não exista o valor de um determinado atributo de um registro de uma relação. Nesse caso, recomenda-se que seja atribuído um valor especial a esse atributo, denominado valor

a) nulo.

b) temporário.

c) de junção.

d) parcial.

e) de seleção.

Comentários:

a) **Correta.** Quando não se conhece ou não existe o valor de um atributo, atribui-se o valor especial NULL, que em português chamamos de valor nulo, justamente para representar essa ausência ou desconhecimento de informação.

b) **Errada.** Não existe esse conceito de valor temporário para representar ausência de informação no modelo relacional.

c) **Errada.** Junção se refere à combinação de linhas de tabelas (como no INNER JOIN), e não a um valor atribuído a um atributo desconhecido.

d) **Errada.** Não há essa categoria de valor parcial para indicar ausência de dado em um atributo.

e) **Errada.** Seleção está ligada à filtragem de registros (como no WHERE), e não a um valor especial atribuído a um atributo sem informação.

Gabarito: A

Caso alguma chave secundária **não possa ficar ausente sob nenhuma circunstância**, podemos criar uma restrição específica para isso: **NOT NULL** ao final da definição do atributo.

```
CREATE TABLE Professor (  
    CPF INTEGER PRIMARY KEY,  
    RG INTEGER UNIQUE NOT NULL,  
    -- NOT NULL impede que RG seja  
    ausente, nulo  
    nome VARCHAR(50)  
);
```



Professor		
<u>CPF</u>	RG	nome
999999999999	9999999	Márcio
888888888888	8888888	Márcia
777777777777	7777777	-

Pode haver valor
desconhecido em nome

Também podemos atribuir um valor padrão para um atributo em caso de ausência de valor. Para isso, basta passar qual o valor padrão após o comando **DEFAULT**.

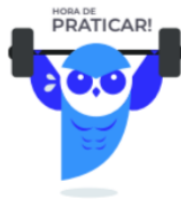
Por exemplo, vamos **definir** que o **nome padrão** de um professor seja "Nome nulo".

```
CREATE TABLE Professor (  
    CPF INTEGER PRIMARY KEY,  
    RG INTEGER UNIQUE NOT NULL,  
    nome VARCHAR(50) DEFAULT  
    'Nome nulo'  
);
```



Professor		
<u>CPF</u>	RG	nome
999999999999	9999999	Márcio
888888888888	8888888	Márcia
777777777777	7777777	Nome nulo

O valor ausente, por padrão,
virou "Nome nulo"



(FGV/Ana (DPE RS)/Banco de Dados/2023) Maria criou um banco de dados no qual há uma tabela com uma coluna X que não aceita valores nulos. Entretanto, como o sistema que mantém essa tabela é novo, na carga inicial dos dados essa coluna não existe.

Como Maria não quer abrir mão do impedimento de valores nulos, Maria poderia carregar as linhas incompletas desde que, na coluna X, usasse a cláusula:

- a) skip;
- b) check;
- c) default;
- d) float;

e) case when ... then ... end.

Comentários:

- a) **Errada.** Não existe uma cláusula skip em SQL para preencher valores ausentes em colunas.
- b) **Errada.** A cláusula CHECK serve para criar uma condição personalizada de validação dos valores, como exigir que uma idade esteja entre 14 e 24 anos, e não para fornecer um valor quando a coluna está ausente na carga.
- c) **Correta.** A cláusula DEFAULT atribui um valor padrão para um atributo em caso de ausência de valor. Assim, mesmo que a carga inicial não traga a coluna X, o banco preenche automaticamente com esse valor padrão, respeitando a regra NOT NULL.
- d) **Errada.** FLOAT é um tipo de dado para números reais com casas decimais, não tem relação com tratamento de valores ausentes.
- e) **Errada.** Não é uma cláusula válida para esse contexto de preenchimento automático de valores em colunas NOT NULL.

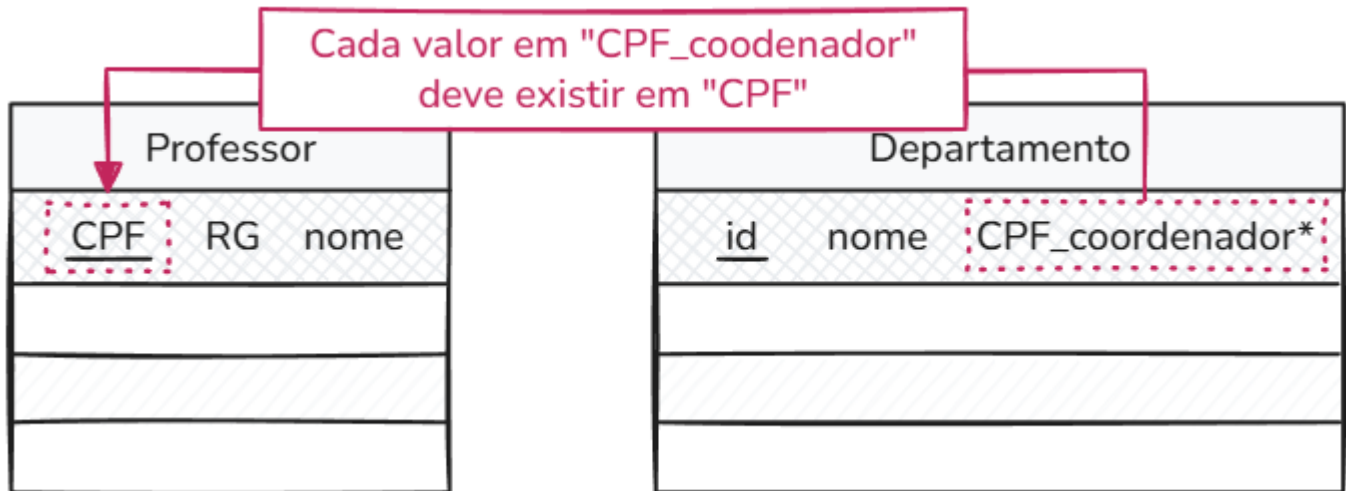
Gabarito: C

Continuando, com a *constraint* de chave estrangeira (FOREIGN KEY) conseguimos ligar tabelas através de seus atributos, estabelecendo uma integridade referencial.

Em outras palavras, com o **FOREIGN KEY** criamos regras como "**este atributo dessa tabela referencia aquela chave daquela outra tabela**" no nosso banco de dados.

Por exemplo, se quisermos uma tabela "Departamento", em que todo departamento tenha um professor coordenador, podemos criar uma FOREIGN KEY em um atributo seu referenciando (comando REFERENCES) o atributo CPF da tabela Professor.

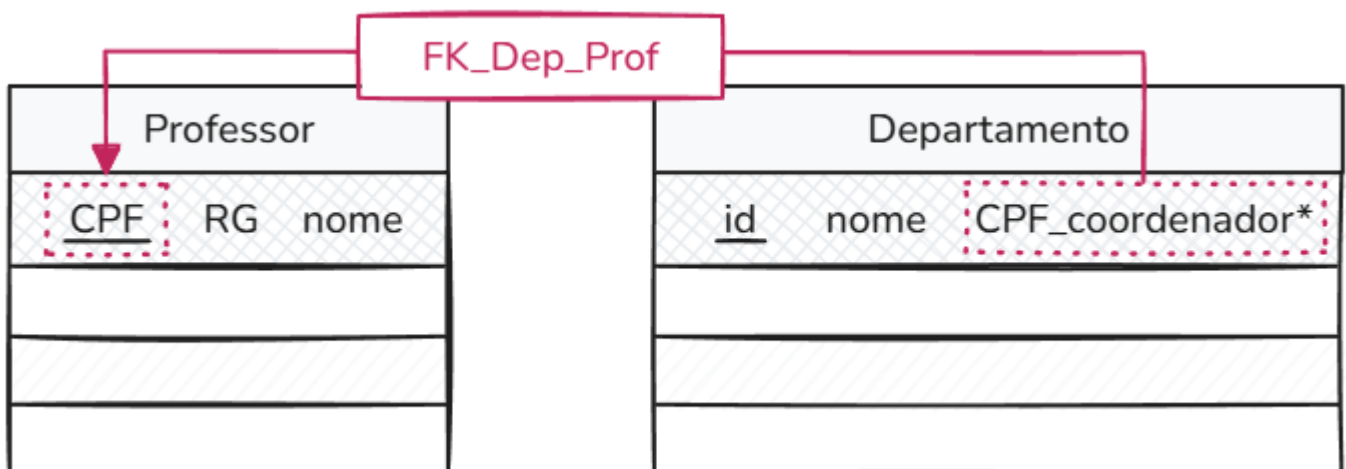
```
CREATE TABLE Professor (  
    CPF INTEGER PRIMARY KEY,  
    RG INTEGER UNIQUE,  
    nome VARCHAR(50) NOT NULL  
);  
  
CREATE TABLE Departamento (  
    id INTEGER PRIMARY KEY,  
    nome VARCHAR(100),  
    CPF_coordenador INTEGER,  
    FOREIGN KEY (CPF_coordenador) REFERENCES Professor(CPF) -- CONSTRAINT de  
    chave estrangeira atribuído à coluna "CPF_coordenador" na tabela Departamento  
    referenciando um CPF da tabela Professor  
);
```



Também podemos criar nossas restrições escrevendo expressamente o comando **CONSTRAINT**, para ser possível dar um nome à restrição.

Vejam só, a seguir, como conseguimos atribuir o nome "FK_Dep_Prof" à nossa restrição FOREIGN KEY entre as tabelas Departamento e Professor.

```
CREATE TABLE Professor (  
    CPF INTEGER PRIMARY KEY,  
    RG INTEGER UNIQUE,  
    nome VARCHAR(50) NOT NULL  
);  
  
CREATE TABLE Departamento (  
    id INTEGER PRIMARY KEY,  
    nome VARCHAR(100),  
    CPF_coordenador INTEGER,  
    CONSTRAINT FK_Dep_Prof FOREIGN KEY (CPF_coordenador) REFERENCES Professor(CPF)  
);
```



Ações de disparo referencial

No dia a dia do banco de dados, podemos realizar operações que violam a restrição de integridade referencial estabelecida pela FOREIGN KEY, como deletar registros que estão referenciados ou os atualizar de forma inconsistente.

Vamos supor um banco de dados com uma tabela Professor e Departamento já populadas para continuarmos a explicação posteriormente.

Professor		
<u>CPF</u>	RG	nome
999999999999	9999999	Márcio
888888888888	8888888	Márcia
000000000000	0000000	-

Departamento		
<u>id</u>	nome	CPF_coordenador*
1	Matemática	999999999999
2	Português	888888888888

Agora pensem assim: e se tentarmos deletar um coordenador na tabela "Professor"? Automaticamente o registro do departamento ficaria inconsistente, pois referencia um professor que não existe mais.

Professor		
<u>CPF</u>	RG	nome
999999999999	9999999	Márcio
888888888888	8888888	Márcia
000000000000	0000000	-

Departamento		
<u>id</u>	nome	CPF_coordenador*
1	Matemática	999999999999
2	Português	888888888888

Não podemos deletar o registro de "Márcia", porque seu CPF está referenciado em "CPF_coordenador*"

Nesses casos, por padrão, o **sistema gerenciador do banco de dados simplesmente rejeita a operação**. Essa funcionalidade implícita de rejeitar esse tipo de operação se chama **"RESTRICT"**.

Para melhorar a expressividade do nosso código em SQL, podemos orientar expressamente o SGBD a ter esse mesmo comportamento.

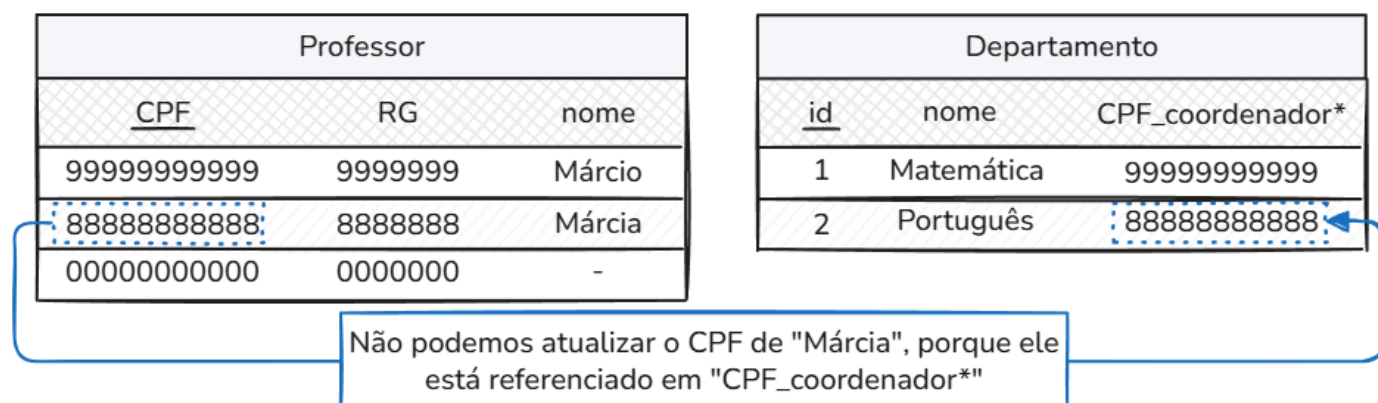
Para isso, precisamos, ao criar uma restrição (CONSTRAINT), passar a cláusula que, ao (tentarem) deletar (ON DELETE) um valor do atributo, rejeite a operação (RESTRICT).

```
CREATE TABLE Departamento (  
    id INTEGER PRIMARY KEY,  
    nome VARCHAR(100),  
    CPF_coordenador INTEGER,  
    FOREIGN KEY (CPF_coordenador) REFERENCES Professor(CPF) ON DELETE RESTRICT  
);
```

Em outras palavras, se existir algum departamento com CPF_coordenador apontando para o CPF de determinado professor, não conseguimos excluir esse professor da tabela Professor, porque o banco rejeita a operação para proteger a referência.

A mesma lógica vale para uma tentativa de update inválida, porém, ao invés de estabelecermos "ON DELETE RESTRICT", adicionamos o comando ON UPDATE RESTRICT.

```
CREATE TABLE Departamento (  
    id INTEGER PRIMARY KEY,  
    nome VARCHAR(100),  
    CPF_coordenador INTEGER,  
    FOREIGN KEY (CPF_coordenador) REFERENCES Professor(CPF) ON UPDATE RESTRICT  
);
```



Além do RESTRICT, existe também uma opção chamada **NO ACTION**. À primeira vista, parecem a mesma coisa, já que **ambos bloqueiam operações que violariam a nossa integridade**. Mas há uma diferença sutil e importante entre eles, relacionada ao momento em que o banco verifica a integridade.

O RESTRICT verifica imediatamente se a restrição foi violada, no exato momento em que a operação é executada, não permitindo nem mesmo uma violação momentânea dos dados, antes do seu efetivo registro no banco de dados.

Já o NO ACTION verifica no final da instrução, um pouquinho depois. Assim, o banco permite que a operação seja "tentada" e só no final verifica se algum CONSTRAINT foi violado. Se houver violação, a operação inteira é desfeita.



Em consultas simples, essa diferença é imperceptível por nós, e o resultado é o mesmo: rejeição da operação violadora.

Mas há situações em que a diferença aparece, principalmente quando temos operações complexas que apenas temporariamente violam a integridade, pois conseguem se resolver no final.

Mas, além de simplesmente rejeitar a operação, também podemos definir outros comportamentos.

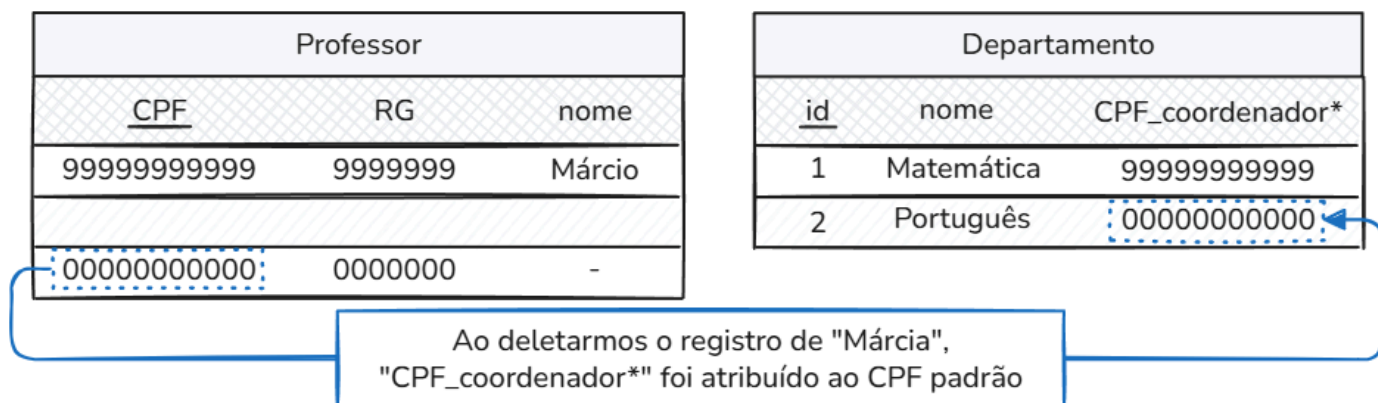
Por exemplo, podemos definir que, ao deletar um registro na tabela Professor, o campo "CPF_coordenador" na tabela Departamento fique ausente (nulo). Para isso, basta adicionarmos a orientação SET NULL.

```
CREATE TABLE Departamento (  
  id INTEGER PRIMARY KEY,  
  nome VARCHAR(100),  
  CPF_coordenador INTEGER,  
  FOREIGN KEY (CPF_coordenador) REFERENCES Professor(CPF) ON DELETE SET NULL  
);
```

Na mesma lógica, se o campo possuir um valor padrão (DEFAULT), podemos orientar que, quando da exclusão (ON DELETE) do CPF do professor referenciado como coordenador, deve-se atribuir ao campo CPF_coordenador o valor padrão (SET DEFAULT).

Por exemplo, supondo que o CPF_coordenador tenha como valor *default* o CPF do dono da instituição de ensino "00000000000" (porque, além de dono, é professor). Então podemos exigir que, ao se deletar o professor coordenador, o CPF_coordenador fique automaticamente atribuído ao dono (isto é, o valor de CPF default).

```
CREATE TABLE Departamento (  
  id INTEGER PRIMARY KEY,  
  nome VARCHAR(100),  
  CPF_coordenador INTEGER DEFAULT 000000000000,  
  FOREIGN KEY (CPF_coordenador) REFERENCES Professor(CPF) ON DELETE SET DEFAULT  
);
```



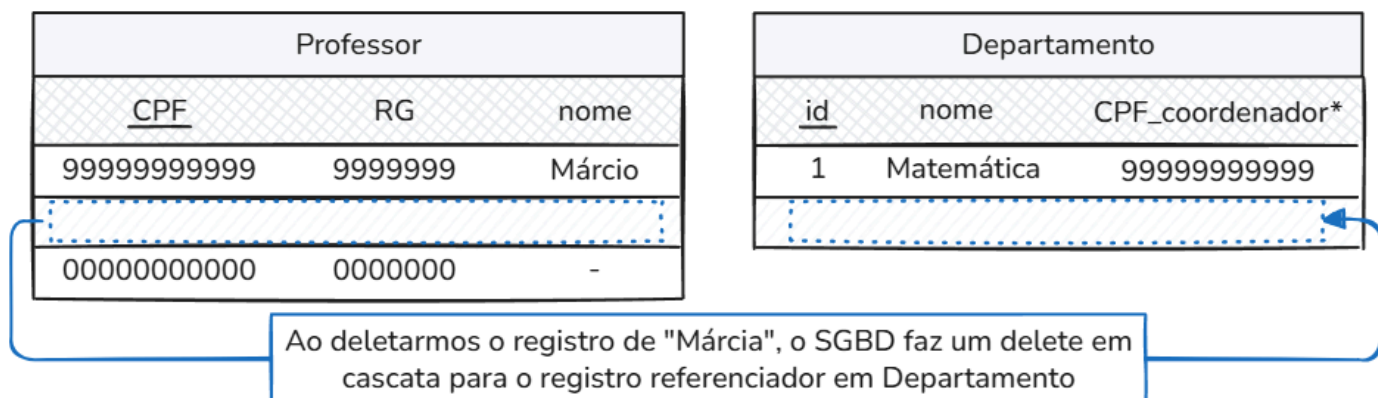
Também podemos desejar propagar a ação realizada na outra tabela, resolvendo a mesma operação em ambas.

Se ficou confuso, pense em um efeito dominó: ao deletar o professor referenciado, o SGBD deve deletar, automaticamente, o departamento que utilizava esse professor como coordenador.

Ou seja, a manipulação de um registro referenciado leva a manipulação do registro referenciador.

Para isso, basta atribuirmos ao evento de deleção (ON DELETE) um efeito em cascata (CASCADE).

```
CREATE TABLE Departamento (  
  id INTEGER PRIMARY KEY,  
  nome VARCHAR(100),  
  CPF_coordenador INTEGER DEFAULT 000000000000,  
  FOREIGN KEY (CPF_coordenador) REFERENCES Professor(CPF) ON DELETE CASCADE  
);
```



Do mesmo modo, podemos querer que automaticamente o banco fique consistente quando da atualização (UPDATE) do valor do atributo referenciado.

No nosso caso, poderíamos querer que, ao corrigir o CPF do Professor, o valor desse CPF no atributo CPF_coordenador de Departamento também se atualize automaticamente.

Para forçar esse "cascateamento" de operações de atualização, precisamos definir o comando "ON UPDATE CASCADE".

```
CREATE TABLE Departamento (  
    id INTEGER PRIMARY KEY,  
    nome VARCHAR(100),  
    CPF_coordenador INTEGER DEFAULT 000000000000,  
    FOREIGN KEY (CPF_coordenador) REFERENCES Professor(CPF) ON UPDATE CASCADE  
);
```

Professor		
<u>CPF</u>	RG	nome
999999999999	9999999	Márcio
111111111111	8888888	Márcia
000000000000	0000000	-

Departamento		
<u>id</u>	nome	CPF_coordenador*
1	Matemática	999999999999
2	Português	111111111111

Ao atualizarmos o CPF de "Márcia", o SGBD atualiza em cascata no registro referenciador em Departamento



(FGV/Auditor Estadual de Controle (CGE SP)/2025) A integridade referencial entre a tabela Pedidos (Chave Estrangeira FK_ClienteID) e a tabela Clientes (Chave Primária PK_ClienteID) foi estabelecida. O administrador do banco de dados precisa garantir que, se um registro for excluído da tabela Clientes, todos os registros de pedidos associados a esse cliente na tabela Pedidos sejam automaticamente excluídos em cascata.

Assinale a opção que indica a cláusula específica da restrição de Chave Estrangeira que deve ser usada para implementar esse comportamento de propagação da exclusão.

- a) ON UPDATE CASCADE
- b) ON DELETE NO ACTION
- c) ON DELETE SET NULL
- d) ON DELETE CASCADE
- e) ON DELETE RESTRICT

Comentários:

a) Errada. ON UPDATE CASCADE propaga atualizações, não exclusões. Ou seja, se o valor da chave referenciada fosse alterado, a mudança se refletiria na tabela que referencia, mas não é isso que o enunciado pede.

b) Errada. ON DELETE NO ACTION não propaga a exclusão; ele apenas verifica no final da instrução se a integridade foi violada e, se foi, bloqueia a operação.

c) Errada. ON DELETE SET NULL não exclui os pedidos associados, apenas faz com que o campo da chave estrangeira (FK_ClienteID) fique com valor nulo nos pedidos quando o cliente for excluído.

d) Correta. ON DELETE CASCADE faz exatamente o que o enunciado pede: ao excluir um registro na tabela referenciada (Clientes), o banco de dados automaticamente exclui, em efeito dominó, todos os registros relacionados na tabela que referencia (Pedidos).

e) Errada. ON DELETE RESTRICT bloqueia imediatamente a exclusão do cliente caso existam pedidos associados a ele, ou seja, faz justamente o oposto da propagação desejada.

Gabarito: D

(FGV/Ana Gest (COMPESA)/COMPESA/Administrador de Banco de Dados/2016) Nos sistemas gerenciadores de bancos de dados, a cláusula "ON DELETE CASCADE" aplica-se, usualmente, à especificação de

a) chaves estrangeiras.

b) índices.

c) procedures armazenadas.

d) tabelas dependentes.

e) views.

Comentários:

a) Correta. A cláusula ON DELETE CASCADE é usada na criação de chaves estrangeiras (FOREIGN KEY) para indicar que, ao deletar um registro da tabela referenciada, os registros que dependem dele na tabela referenciadora também serão deletados em efeito cascata.

b) Errada. Índices servem para acelerar buscas e não utilizam a cláusula ON DELETE CASCADE.

c) Errada. Procedures armazenadas são blocos de código executáveis, não possuem relação com a cláusula ON DELETE CASCADE.

d) Errada. Embora a ideia de "tabela dependente" pareça relacionada, a cláusula é aplicada especificamente na definição da chave estrangeira, e não na tabela em si.

e) Errada. Views são tabelas derivadas (visões) e não utilizam ON DELETE CASCADE, que é um comportamento próprio das chaves estrangeiras.

Gabarito: A

Especificando outras restrições

Na prática, podemos ter restrições bem diferentes desses comandos que o SQL fornece por padrão, como PRIMARY KEY e FOREIGN KEY.

Por exemplo, podemos querer que o campo "idade" em uma tabela "Jovem Aprendiz" seja um valor entre 14 anos e 24 anos.

Para definirmos esse tipo de **condição personalizada**, usamos a cláusula **CHECK**.

Assim, a partir do momento em que temos uma condição estabelecida com CHECK, o banco verifica sempre que um registro é inserido ou modificado, para decidir se aceita ou rejeita a operação.

Para nosso exemplo de jovem aprendiz, poderíamos inserir a CONSTRAINT com CHECK ao final da criação da tabela, vejamos.

```
CREATE TABLE Jovem_Aprendiz(  
    CPF INTEGER PRIMARY KEY,  
    nome VARCHAR(50),  
    idade INTEGER,  
    CHECK (idade >= 14 AND idade <= 24) -- CONSTRAINT que verifica se a idade do  
registro é um valor maior ou igual a 14 anos e menor ou igual a 24 anos  
);
```



(FGV/Aud CE (TCE,PA)/TCE PA/Administrativa/Ciência de Dados/2024) Ao projetar uma tabela para acompanhamento de processos, deseja-se evitar inconsistência temporal entre as etapas, permitindo que o sistema verifique se o valor a ser inserido em um campo preserve a cronologia das etapas.

Nesse contexto, a cláusula SQL que implementa essa restrição é a

- a) CHECK
- b) FOREIGN KEY
- c) NOT NULL
- d) PRIMARY KEY
- e) UNIQUE

Comentários:

a) **Correta.** A cláusula CHECK permite definir uma condição personalizada que os valores devem cumprir, como verificar se uma data respeita a cronologia das etapas anteriores.

b) **Errada.** FOREIGN KEY serve para criar ligações entre tabelas, referenciando uma chave de outra tabela, e não para validar regras cronológicas.

- c) **Errada.** NOT NULL apenas impede que o campo fique vazio (sem valor), mas não verifica nenhuma regra sobre o conteúdo do valor.
- d) **Errada.** PRIMARY KEY garante que o campo tenha valor único e não nulo para identificar registros, mas não valida cronologia.
- e) **Errada.** UNIQUE garante que não existam valores repetidos na coluna, mas não verifica se o valor respeita uma ordem temporal.

Gabarito: A

(CESGRANRIO/Ana (BNDES)/2024) No momento da criação de uma tabela em um Sistema Gerenciador de Banco de Dados (SGBD), por meio do comando CREATE TABLE, definem-se regras para o controle de integridade e para a garantia de qualidade dos dados dessa tabela. Considere que W e Y são duas tabelas. A é um atributo de W, e B é um atributo de Y. Foi criada corretamente uma cláusula CHECK no atributo A. Essa cláusula permite verificar se o atributo A

- a) tem valor único em todas as linhas da tabela W.
- b) tem valor maior ou igual a zero.
- c) tem valor existente em uma ou mais linhas da tabela Y na coluna B.
- d) inviabiliza o acesso a alguma linha da tabela Y.
- e) está preenchido com algum valor ou não.

Comentários:

- a) **Errada.** Garantir valor único em uma coluna é função da cláusula UNIQUE, e não do CHECK.
- b) **Correta.** O CHECK permite criar condições personalizadas, como exigir que o valor seja maior ou igual a zero, e o banco verifica essa regra a cada inserção ou alteração.
- c) **Errada.** Verificar se um valor existe em outra tabela é função da cláusula FOREIGN KEY, que estabelece integridade referencial entre tabelas.
- d) **Errada.** O CHECK não controla acesso a linhas; ele apenas valida se os valores inseridos atendem a uma condição lógica.
- e) **Errada.** Garantir que um campo esteja preenchido é função da restrição NOT NULL, e não do CHECK.

Gabarito: B

(Com. Org. (IFSP)/Tec (IF SP)/2019) Os tipos de dados são uma forma de limitar os dados que podem ser armazenados na tabela. Entretanto, para muitos aplicativos, a restrição obtida não possui o refinamento necessário. Por exemplo, uma coluna contendo preços de produtos pode provavelmente aceitar somente valores positivos, mas não existe nenhum tipo de dado em um banco de dados que aceite apenas números positivos. Dessa forma, a utilização de constraints (restrições), ou regras, específicas aplicadas em colunas de uma tabela, ou na própria tabela, pode permitir maior controle sobre os dados que são armazenados em um banco de dados. Assim, pode-se afirmar que as constraints são usadas para limitar os tipos de dados que são

inseridos. As constraints podem ser especificadas no momento da criação de uma tabela (CREATE TABLE) ou após a tabela ter sido criada (ALTER TABLE). Analise a descrição abaixo:

Esta constraint permite especificar se o valor em uma dada coluna deve satisfazer a uma expressão lógica (booleana), ou seja, que pode retornar um valor verdadeiro ou um valor falso. Também, é possível criar uma restrição desta constraint que faça referência a mais de uma coluna, ou ainda usar mais de uma desta constraint na tabela.

O texto refere-se a qual constraint no postgresql?

- a) CHECK.
- b) FOREIGN KEY.
- c) UNIQUE.
- d) NOT NULL.

Comentários:

a) **Correta.** A constraint CHECK é exatamente a que permite definir uma expressão lógica (booleana) que retorna verdadeiro ou falso para validar os valores de uma coluna, como por exemplo exigir que um preço seja maior que zero. Ela pode envolver mais de uma coluna e podemos ter várias dessas restrições em uma mesma tabela.

b) **Errada.** A FOREIGN KEY serve para criar integridade referencial entre tabelas, ou seja, garantir que um valor em uma coluna exista como referência em outra tabela, e não para validar expressões booleanas.

c) **Errada.** A UNIQUE garante que os valores de uma coluna sejam únicos (sem duplicidade), aceitando valores nulos, mas não avalia expressões lógicas personalizadas.

d) **Errada.** A NOT NULL garante que os valores de uma coluna nunca sejam ausentes.

Gabarito: A

O comando ALTER

Ao longo dessa seção estudamos os objetos que podem ser manipulados com os comandos CREATE e DROP, focando nas diversas sintaxes que o SQL aceita. Porém, pela flexibilidade de utilização comando ALTER, precisamos estudá-lo especificamente.

Pois bem, vamos supor que temos uma tabela Professor criada com atributos "CPF", "RG" e "nome", e queremos alterá-la para adicionar um novo atributo, a "classe". Para isso, podemos utilizar o comando ALTER TABLE, adicionando (ADD) esse atributo "classe".

```
ALTER TABLE  
  Professor  
ADD  
  classe INTEGER;
```



Professor			
<u>CPF</u>	RG	nome	classe

Essa não é a única alteração possível em nossa tabela, já que também podemos remover atributos. Para isso, invocamos o comando "DROP COLUMN". A seguir, vamos remover a coluna "classe" que acabamos de adicionar.

```
ALTER TABLE  
  Professor  
DROP COLUMN  
  classe;
```



Professor		
<u>CPF</u>	RG	nome



É interessante notar a assimetria entre adicionar uma coluna, já que basta o "ADD", para remover a coluna, que precisamos do COLUMN: "DROP COLUMN".



(CESGRANRIO/CNU/Bloco Temático 2/Tecnologia, Dados e Informação/2024) Para criar uma tabela em um banco de dados relacional, foi utilizado o seguinte comando:

```
CREATE TABLE PESSOA (  
ID INTEGER PRIMARY KEY,  
NOME VARCHAR(255)  
);
```

Qual comando SQL permite modificar a tabela para incluir o campo CPF do tipo VARCHAR(11)?

- a) ALTER TABLE PESSOA ADD COLUMN CPF VARCHAR(11);
- b) ALTER TABLE PESSOA INCLUDE COLUMN CPF VARCHAR(11);
- c) ALTER TABLE PESSOA INSERT COLUMN CPF VARCHAR(11);
- d) MODIFY TABLE PESSOA ADD COLUMN CPF VARCHAR(11);
- e) MODIFY TABLE PESSOA INCLUDE COLUMN CPF VARCHAR(11);

Comentários:

a) Correta. Para adicionar uma nova coluna em uma tabela já existente, usamos o comando ALTER TABLE seguido do nome da tabela e da cláusula ADD com o nome e tipo da nova coluna, exatamente como visto no comando que adiciona o atributo RG à tabela Professor.

b) Errada. Não existe a cláusula INCLUDE COLUMN no SQL para adicionar colunas; a palavra correta é ADD.

c) Errada. INSERT é usado para adicionar dados (linhas) dentro de uma tabela, e não para adicionar colunas na estrutura dela.

d) Errada. Não existe o comando MODIFY TABLE no SQL padrão; para alterar a estrutura de uma tabela, usamos ALTER TABLE.

e) Errada. Além de MODIFY TABLE não existir, INCLUDE COLUMN também não é uma cláusula válida do SQL.

Gabarito: A

A verdade é que existe uma infinidade de alterações que podemos fazer nos mais diversos tipos de objetos (além das tabelas) do banco de dados. Então, para condensar nosso estudo, veremos o que há de mais importante para concursos na tabela a seguir.

Ação	Exemplo	SQL
Adicionar uma restrição com check	Adicionar uma restrição chamada "idade_positiva" à coluna "idade" de uma tabela "Pessoas".	<pre>ALTER TABLE Pessoas ADD CONSTRAINT idade_positiva CHECK (idade > 0);</pre>

Ação	Exemplo	SQL
Remover uma restrição	Adicionar a restrição chamada "idade_positiva".	<pre>ALTER TABLE Pessoas DROP CONSTRAINT idade_positiva;</pre>
Adicionar uma restrição de chave primária	Adicionar uma restrição de chave primária à coluna "CPF" de uma tabela "Pessoas".	<pre>ALTER TABLE Pessoas ADD PRIMARY KEY (CPF);</pre>
Adicionar uma restrição de chave secundária	Adicionar uma restrição de chave secundária à coluna "RG" de uma tabela "Pessoas".	<pre>ALTER TABLE Pessoas ADD UNIQUE (RG);</pre>
Alterar o tipo de uma coluna	Alterar o tipo da coluna "CPF" da tabela "Pessoas" para VARCHAR(11).	<pre>ALTER TABLE Pessoas ALTER COLUMN CPF TYPE VARCHAR(11);</pre>
Renomear uma tabela	Altera o nome da tabela "Pessoas" para "Professores".	<pre>ALTER TABLE Pessoas RENAME TO Professores;</pre>
Renomear uma coluna	Altera o nome da coluna "CPF" da tabela Pessoas para "id".	<pre>ALTER TABLE Pessoas RENAME COLUMN CPF TO id;</pre>
Alterar o usuário responsável por uma tabela	Altera o responsável pela tabela Pessoas para o usuário "Joao".	<pre>ALTER TABLE Pessoas OWNER TO Joao;</pre>



(FCC/AP (MANAUSPREV)/MANAUSPREV/Tecnologia da Informação/2021) Um analista criou a tabela aposentado por meio do comando abaixo em um banco de dados relacional.

```
CREATE TABLE aposentado (
cpf VARCHAR(18) NOT NULL,
anoNascimento INT NULL,
nome VARCHAR(70) NULL,
anosContribuicao INT NULL,
valorReceberAno DOUBLE NULL,
```

totalReceber DOUBLE NULL);

Após executar o comando, percebeu que não se definiu a chave primária da tabela. Para definir o campo cpf como chave primária deve utilizar o comando

- a) ADD PRIMARY KEY (cpf) TO aposentado;
- b) ALTER TABLE aposentado ADD PRIMARY KEY (cpf);
- c) ALTER COLUMN (cpf) TO PRIMARY KEY TO aposentado;
- d) ADD CONSTRAINT PRIMARY KEY (cpf) TO aposentado;
- e) ALTER TABLE aposentado ADD COLUMN (cpf) PRIMARY KEY;

Comentários:

a) Errada. A sintaxe está incorreta, pois para alterar uma tabela existente devemos começar pelo comando ALTER TABLE seguido do nome da tabela, e não por ADD.

b) Correta. Para adicionar uma chave primária a uma tabela já existente, usamos ALTER TABLE seguido do nome da tabela e, em seguida, ADD PRIMARY KEY com a coluna entre parênteses.

c) Errada. Não existe essa sintaxe ALTER COLUMN ... TO PRIMARY KEY no padrão SQL. Para adicionar chave primária a uma tabela existente, o correto é usar ALTER TABLE com ADD PRIMARY KEY.

d) Errada. Mesmo que CONSTRAINT seja usado para nomear restrições, a estrutura do comando está errada, pois falta o ALTER TABLE no início e a cláusula TO não é válida nesse contexto.

e) Errada. O comando ADD COLUMN serve para adicionar uma nova coluna à tabela, e não para transformar uma coluna existente em chave primária. Como o campo cpf já existe, não devemos adicioná-lo novamente.

Gabarito: B

(FUNDATEC/Ana Sist (BRDE)/2023) Considere a tabela ALUNOS criada usando o comando SQL (padrão SQL99 ou superior) abaixo.

Create table Alunos

(pk int not null primary key,

nomea varchar(100),

idade int,

nascimento date not null);

Assinale qual, dentre as instruções ALTER TABLE listadas abaixo, representa um comando SQL INVÁLIDO, que causa um erro de execução.

- a) alter table Alunos add constraint adulto check(idade >= 18);
- b) alter table Alunos drop column nascimento;
- c) alter table Alunos add unique(nomea);
- d) alter table Alunos add index idx_idade on idade;

e) alter table Alunos rename to discipulos;

Comentários:

a) **Errada.** Adicionar uma restrição CHECK com nome é uma alteração válida, pois ALTER TABLE permite incluir constraints como CHECK para validar valores, no caso, garantir que idade seja maior ou igual a 18.

b) **Errada.** Remover uma coluna com DROP COLUMN é uma operação válida do ALTER TABLE, eliminando o atributo nascimento da tabela.

c) **Errada.** Adicionar uma restrição UNIQUE a uma coluna é válido, pois define que os valores em nomea não poderão se repetir entre as linhas.

d) **Correta.** Essa sintaxe é inválida no padrão SQL. O ALTER TABLE possui comandos como ADD, DROP COLUMN, ADD CONSTRAINT, RENAME, entre outros, mas não existe a forma "add index ... on ..." dentro do padrão SQL para alteração de tabela, o que provoca erro de execução.

e) **Errada.** Renomear uma tabela com RENAME TO é uma alteração válida, mudando o nome de Alunos para discipulos.

Gabarito: D

(ADAPTADA - PROGEF FURG/Tec (FURG)/2022) Quando você cria uma tabela e percebe que cometeu um erro, ou os requisitos do aplicativo mudam, você pode precisar fazer modificações nela. Qual das alternativas abaixo aponta a sintaxe correta para corrigir nome de campo cd_area para cod_area, sem perder os dados já inseridos.

a) UPDATE TABLE cursos COLUMN cd_area RENAME cod_area;

b) ALTER TABLE cursos DROP COLUMN cd_area; ALTER TABLE cursos ADD COLUMN cod_area text;

c) ALTER COLUMN cd_area RENAME cod_area INTO cursos;

d) ALTER TABLE cursos.cd_area RENAME TO cursos.cod_area;

e) ALTER TABLE cursos RENAME COLUMN cd_area TO cod_area;

Comentários:

a) **Errada.** UPDATE é comando para modificar dados dentro das linhas da tabela, não para alterar a estrutura. Para mudar o nome de uma coluna usamos ALTER TABLE.

b) **Errada.** Esse comando até "renomeia" no sentido de remover a coluna antiga e criar uma nova com o novo nome, mas o DROP COLUMN apaga a coluna junto com todos os dados que estavam nela, o que viola a exigência de não perder os dados já inseridos.

c) **Errada.** A sintaxe está completamente fora do padrão. O comando ALTER deve começar com ALTER TABLE seguido do nome da tabela, e não com ALTER COLUMN.

d) **Errada.** Não se usa a notação tabela.coluna no RENAME. A sintaxe correta indica primeiro a tabela com ALTER TABLE e depois usa RENAME COLUMN para apontar a coluna a ser renomeada.

e) **Correta.** Essa é a sintaxe padrão para renomear uma coluna preservando os dados: ALTER TABLE seguido do nome da tabela (cursos), depois RENAME COLUMN com o nome antigo (cd_area), TO e o nome novo (cod_area).

Gabarito: E

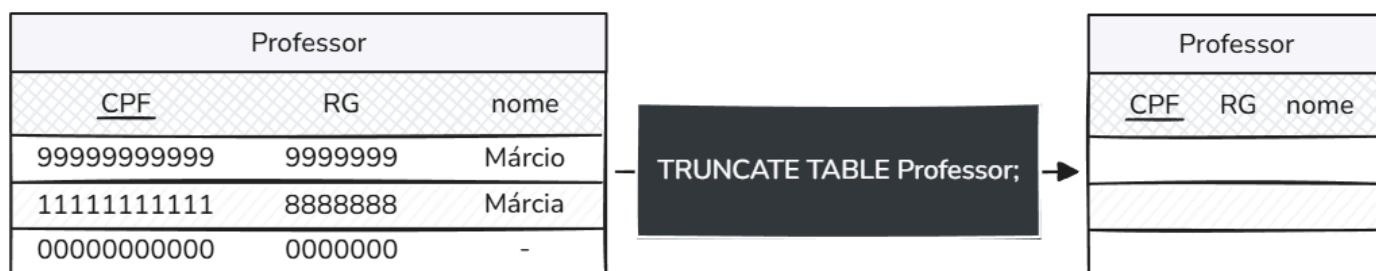
O comando TRUNCATE

O **TRUNCATE TABLE** **deleta todas as linhas de uma tabela**, a "resetando" para seu estado inicial vazio.

Ou seja, o resultado é bem simples: a tabela continua existindo com toda sua estrutura (colunas, tipos, constraints, etc.), mas fica completamente vazia, como se tivesse acabado de ser criada.

Vamos zerar nossa tabela professores, então.

```
TRUNCATE TABLE Professor;
```



Saibam que esse comando é bem "bruto", não se preocupando em registrar no log do banco cada linha que foi removida.

Assim, ganha-se em velocidade, porque o TRUNCATE atua diretamente na estrutura física de armazenamento da tabela, marcando as páginas de seus registros como livres para escrita, reiniciando, na prática, a tabela.

É por isso que se trata de uma operação no nível estrutural (DDL), não no nível de dados (DML).

Porém, como contrapartida, dependendo do banco, não temos volta ao aplicar o TRUNCATE. Tudo se apaga, e perde-se todos os dados. O estrago é permanente assim que o comando termina.



(Com. Org. (IFTO)/Tec (IFTO)/2018) O comando TRUNCATE, pertence a qual tipo de linguagem de dados SQL:

- a) DCL (Data Control Language)
- b) DTL (Data Transaction Language)
- c) SDL (Storage Definition Language)
- d) DDL (Data Definition Language)
- e) DML (Data Manipulation Language)

Comentários:

a) Errada. A DCL é a linguagem de controle de acesso aos dados, com comandos como GRANT e REVOKE, e não inclui o TRUNCATE.

b) Errada. A DTL (ou TCL) cuida do controle de transações, com comandos como COMMIT, ROLLBACK e SAVEPOINT, não sendo o caso do TRUNCATE.

c) Errada. A SDL trata da definição de armazenamento físico dos dados, ou seja, como os bytes são organizados em disco, o que não é a função do TRUNCATE.

d) Correta. O TRUNCATE pertence à DDL porque atua diretamente na estrutura física da tabela, esvaziando-a de forma rápida e em bloco, sendo classificado como uma operação no nível estrutural junto com CREATE, ALTER e DROP.

e) Errada. Apesar de remover dados como o DELETE (que é DML), o TRUNCATE não trabalha linha por linha nem registra cada exclusão no log, por isso não é considerado DML.

Gabarito: D

(CEBRASPE (CESPE)/Ana Min (MPE TO)/2024) O comando TRUNCATE é usado para remover todas as linhas de uma tabela.

Comentários:

O TRUNCATE TABLE realmente apaga todas as linhas de uma tabela de uma só vez, deixando ela vazia, mas mantendo sua estrutura (colunas, tipos, restrições) intacta. É como se a tabela voltasse ao estado em que estava logo após ser criada. Vale lembrar que ele é diferente do DROP, que remove a tabela inteira (estrutura e dados), e também do DELETE, que apaga linha por linha e permite usar WHERE para escolher quais registros remover. O TRUNCATE é mais rápido justamente porque não fica registrando cada linha removida no log, ele simplesmente esvazia a tabela de forma mais bruta.

Gabarito: Certo

(CEBRASPE (CESPE)/Ana CT I (CNPq)/2024) As expressões SQL a seguir geram resultados que são equivalentes.

delete from dados;

drop table dados;

Comentários:

Os dois comandos parecem fazer a mesma coisa à primeira vista, mas têm efeitos bem diferentes. O DELETE FROM dados apaga todas as linhas da tabela, mas a tabela continua existindo, vazia, com toda sua estrutura preservada (colunas, tipos, restrições). É como esvaziar uma gaveta: a gaveta continua lá. Já o DROP TABLE dados elimina a tabela inteira do banco, ou seja, apaga tanto os dados quanto a própria estrutura. É como jogar a gaveta fora junto com o que estava dentro. Depois do DELETE, ainda dá para inserir novos registros na tabela. Depois do DROP, a tabela simplesmente não existe mais, e qualquer comando que tente usá-la dará erro. Por isso os resultados não são equivalentes.

Gabarito: Errado

Manipulação de Dados (DML)

Nesta seção vamos tratar dos comandos que nos permitem manipular os dados em um banco de dados, INSERT, DELETE e UPDATE.



(CEBRASPE (CESPE)/Ana Proc (DATAPREV)/DATAPREV/2023) A DML é uma linguagem que interage com os objetos do banco de dados, em vez de interagir com os dados.

Comentários:

A banca inverteu os papéis. Quem interage com os objetos do banco (tabelas, índices, views, schemas) é a DDL, com comandos como CREATE, ALTER e DROP. A DML faz justamente o contrário: trabalha com os dados que estão dentro das tabelas, usando comandos como INSERT, UPDATE, DELETE e MERGE. Por isso a afirmação está errada.

Gabarito: Errado

O comando INSERT

Vamos supor que já criamos nossa tabela Professor com "CPF" chave primária e "nome" utilizando o comando CREATE TABLE. Ou seja, nossa estrutura está pronta, mas vazia.

Professor	
<u>CPF</u>	nome

Agora, para preencher essa tabela com dados, utilizamos o INSERT.

Pois bem, a forma mais rápida de realizarmos essa operação seria através do comando "INSERT INTO" passando o nome da tabela e os valores a serem inseridos após "VALUES". Vejam.

```
INSERT INTO
  Professor
VALUES
  (999999999999, 'Alex');
```



Professor	
<u>CPF</u>	nome
999999999999	Alex

Percebam que nessa sintaxe foi necessário fornecer todos os valores, na mesma ordem em que as colunas foram definidas no CREATE TABLE.

Porém, aqui temos um problema: se alguém alterar a tabela no futuro (adicionar ou reordenar colunas), esse comando pode quebrar ou colocar valores nas colunas erradas.

Para resolver isso, podemos especificar quais colunas preencher após o nome da tabela. Fazendo dessa forma, nomeamos exatamente quais colunas estamos preenchendo, na ordem que desejamos. As que não foram mencionadas recebem automaticamente NULL ou o valor DEFAULT definido na criação da tabela.

```
INSERT INTO  
    Professor(CPF, nome)  
VALUES  
    (888888888888, 'Alexandre');
```



Professor	
<u>CPF</u>	nome
999999999999	Alex
888888888888	Alexandre



Vale lembrar que não podemos omitir colunas que foram definidas como NOT NULL e não possuem valor DEFAULT. Se a coluna exigir um valor e não fornecermos, o banco rejeita a operação.

A verdade é que é trabalhoso inserir vários registros um por um, então podemos passar múltiplos registros separados por vírgula para fazer tudo de uma vez.

```
INSERT INTO  
    FUNCIONARIO(CPF, nome)  
VALUES  
    (777777777777, 'Jonas'),  
    (666666666666, 'Joana'),  
    (555555555555, 'João');
```



Professor	
<u>CPF</u>	nome
999999999999	Alex
888888888888	Alexandre
777777777777	Jonas
666666666666	Joana
555555555555	João



Saibam que, se a tabela tiver uma chave primária auto incremental, só precisamos passar os valores dos outros atributos no INSERT, porque o banco preenche o valor da chave sozinho.



(CEBRASPE (CESPE)/Ana CT I (CNPq)/2024) As expressões SQL a seguir, após serem executadas, gerarão o mesmo resultado.

```
insert into dados values (2,'C',40);
```

```
insert into dados (categoria, modelo, valor) values (2,'C',40);
```

Comentários:

As duas formas de escrever o INSERT produzem exatamente o mesmo efeito quando os valores são passados na mesma ordem em que as colunas foram definidas na tabela. Na primeira expressão, omitimos os nomes das colunas e passamos apenas os valores. Nesse caso, o banco assume que os valores estão na ordem em que as colunas foram criadas no CREATE TABLE. Já na segunda expressão, especificamos as colunas (categoria, modelo, valor) e passamos os valores na mesma ordem. Como a ordem coincide e os valores são idênticos (2, 'C', 40), o resultado final no banco será o mesmo registro inserido. A diferença prática entre as duas formas é de segurança e clareza: nomear as colunas evita que o comando quebre se um dia a tabela for alterada. Mas, executando as duas hoje, com a tabela como está, o resultado é igual.

Gabarito: Certo

(FCC/AFTE (SEFAZ PE)/SEFAZ PE/2022)

```
CREATE TABLE nfe (  
Numero_NFe VARCHAR(9) NOT NULL,  
Modelo_NFe VARCHAR(2) NULL,  
Serie_NFe VARCHAR(3) NULL,  
codigo_UF VARCHAR(2) NULL,  
ano_Emissao VARCHAR(2) NULL,  
mes_Emissao VARCHAR(2) NULL,
```

```
CNPJ_Emitente VARCHAR(14) NULL,  
Codigo_Chave VARCHAR(8) NULL,  
Digito_Chave VARCHAR(1) NULL,  
PRIMARY KEY (Numero_NFe));
```

Para inserir um registro com valores de teste na tabela nfe, utiliza-se a instrução SQL:

- a) INSERT INTO nfe VALUES('123456789','55','1','RJ','22','10','15145076000106','87654321','6');
- b) APPEND TO nfe VALUES('123456789','55','1','RJ','22','10','15145076000106','87654321','6');
- c) INSERT INTO nfe('123456789','55','1','RJ','22','10','15145076000106','87654321','6');
- d) INSERT TO nfe VALUES('123456789','55','1','RJ','22','10','15145076000106','87654321','6');
- e) ADD INTO nfe('123456789','55','1','RJ','22','10','15145076000106','87654321','6');

Comentários:

- a) **Correta.** A sintaxe para inserir um registro é INSERT INTO seguido do nome da tabela e da cláusula VALUES com os valores na mesma ordem das colunas definidas no CREATE TABLE.
- b) **Errada.** Não existe o comando APPEND TO em SQL para inserir registros.
- c) **Errada.** Faltou a cláusula VALUES antes da lista de valores; sem ela, o banco não reconhece o que deve ser inserido.
- d) **Errada.** A sintaxe correta usa INSERT INTO, e não INSERT TO.
- e) **Errada.** Não existe o comando ADD INTO em SQL para inserir registros em uma tabela.

Gabarito: A

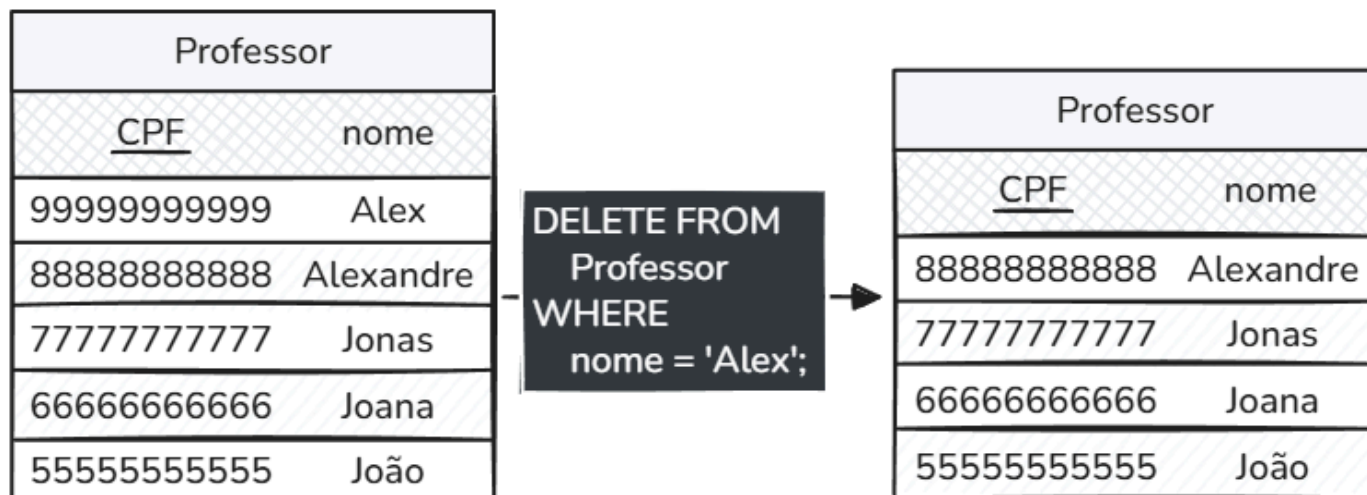
O comando DELETE

Usamos o **DELETE** para **remover dados de uma tabela**.

Para isso, usamos o comando "DELETE FROM" especificando o nome da tabela que possui os registros a serem excluídos e, na sequência, o comando "WHERE" com as regras que o banco de dados deve utilizar para encontrar os exatos registros que queremos de fato remover.

A seguir, vamos deletar todos registros de Professor cujo nome é Alex.

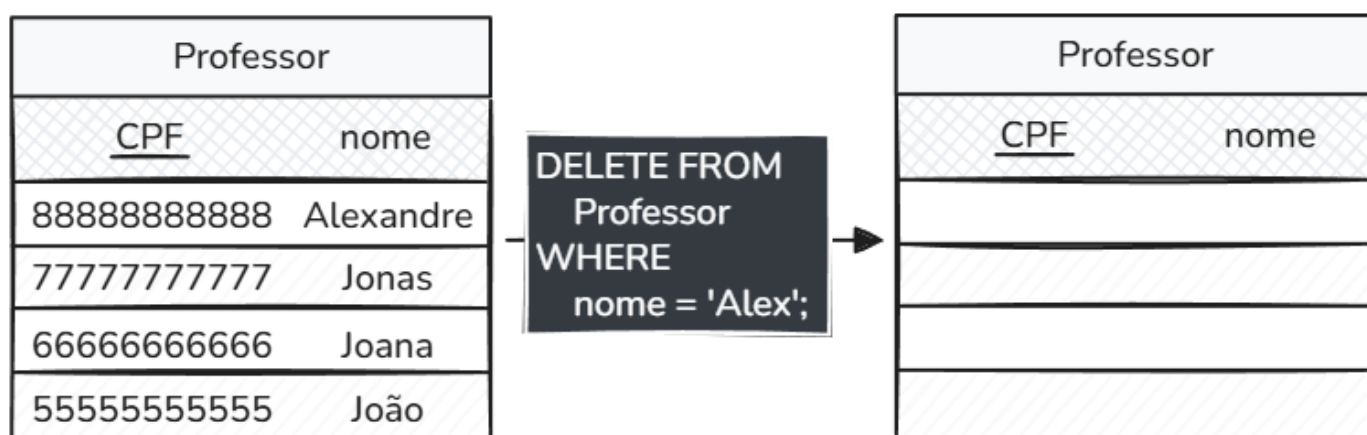
```
DELETE FROM  
    Professor  
WHERE  
    nome = 'Alex';
```



No nosso caso, como há apenas 1 Alex registrado na tabela Professores, só houve 1 exclusão. Porém, se houvesse vários Professores com nome Alex, todos seriam deletados.

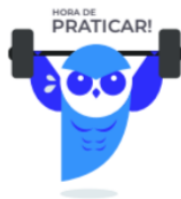
É interessante saber que, caso não seja utilizada nenhuma condição com WHERE, todos os registros da tabela são deletados.

```
DELETE FROM Professor;
```



Saibam que o comando DELETE remove linhas uma a uma, registrando no log do banco de dados cada remoção, respeitando constraints linha por linha, disparando gatilhos se houver, e participando do sistema de transações. Ou seja, é uma operação no nível dos dados, por isso é classificada em DML.

Por esses motivos também, o DELETE ocorre de forma mais lenta que o TRUNCATE, porém com a vantagem de ser reversível, já que para desfazer as operações basta verificar o que ocorreu no log do sistema e retroceder.



(FCC/TRE (SEFAZ MA)/2016) Atenção: Para responder à questão, considere a figura abaixo.



Considere que na tabela **Contribuinte** estão cadastrados os seguintes dados:

IDContribuinte	NomeContribuinte	CPF_CNPJ
1	Paulo da Silva	154.246.037-12
2	Maria Pereira	143.172.129-50

Para excluir o contribuinte cujo conteúdo do campo CPF_CNPJ é 143.172.129-50 utiliza-se a instrução SQL:

- a) DELETE FROM Contribuinte WHERE CPF_CNPJ = 143.172.129-50;
- b) DELETE FROM Contribuinte WHERE CPF_CNPJ = '143.172.129-50';
- c) DELETE FROM Contribuinte WHERE CPF_CNPJ = "143.172.129-50" && IDContribuinte = 2;
- d) DELETE * FROM Contribuinte WHERE CPF_CNPJ = 143.172.129-50;
- e) DELETE * FROM Contribuinte WHERE CPF_CNPJ LIKE 143.172.129-50;

Comentários:

- a) **Errada.** O valor do CPF_CNPJ está sem aspas.
- b) **Correta.** A sintaxe DELETE FROM seguida do nome da tabela e da cláusula WHERE com a condição é a forma correta de excluir registros específicos. Como CPF_CNPJ é um texto (contém pontos e traço), o valor deve estar entre aspas simples.
- c) **Errada.** Em SQL padrão, valores de texto são delimitados por aspas simples e não duplas. Além disso, o operador correto para combinar condições é AND, e não &&.
- d) **Errada.** O comando DELETE não aceita asterisco depois dele, pois o DELETE já remove a linha inteira por natureza. Além disso, o valor do CPF_CNPJ está sem aspas.
- e) **Errada.** Apresenta o mesmo problema do asterisco após o DELETE e o valor sem aspas. O operador LIKE também não é adequado quando se quer uma comparação exata, sem caracteres curinga.

Gabarito: B

De todo modo, devido à similaridade de comportamento entre os comandos DELETE, TRUNCATE e DROP, vejamos uma tabela comparativa entre eles.

Critério	DELETE	TRUNCATE	DROP
Classificação SQL	DML	DDL	DDL
O que remove?	Apenas os dados (tuplas)	Apenas os dados (tuplas)	Dados e estrutura (tabela inteira)
Tabela continua existindo?	Sim, vazia	Sim, vazia	Não, é eliminada do schema
Aceita cláusula WHERE?	Sim. Pode excluir registros específicos	Não. Sempre remove todos registros.	Não. Remove o objeto inteiro.
Velocidade	Mais lento (linha por linha)	Muito rápido (operação em bloco)	Rápido
Registra no log de transações?	Sim, cada linha excluída	Geralmente não (mínimo)	Geralmente não
Pode ser desfeito (ROLLBACK)?	Sim	Depende do SGBD (geralmente não)	Depende do SGBD (geralmente não)



(CEBRASPE (CESPE)/SEPLAN RR/Tecnologia da Informação/2023) Os comandos TRUNCATE e DROP TABLE removem todas as linhas de uma tabela, porém o comando DROP TABLE exclui também a estrutura da tabela do banco de dados bem como todos os dados armazenados na tabela.

Comentários:

A questão acerta ao mostrar a diferença essencial entre os dois comandos. O TRUNCATE apaga todos os registros da tabela, mas a tabela continua existindo, vazia, pronta para receber novos dados. Já o DROP TABLE é mais radical: além de remover os dados, elimina a própria estrutura da tabela do banco, ou seja, a tabela deixa de existir, com colunas, restrições e tudo mais. Pense assim: o TRUNCATE esvazia uma gaveta, enquanto o DROP TABLE joga a gaveta inteira fora.

Gabarito: Certo

(CEBRASPE (CESPE)/Tecno Jr (CTI)/CTI/Inovação e Gestão de Infraestrutura de P&D/Desenvolvimento Tecnológico voltado à Infraestrutura de Tecnologia da Informação e Comunicação/2024) No DML (data manipulation language), a instrução TRUNCATE elimina todas as linhas de uma tabela simultaneamente, enquanto a instrução DELETE oferece a possibilidade de excluir dados específicos ou todos os dados.

Comentários:

O erro da questão está logo no começo: o TRUNCATE não pertence ao DML, e sim ao DDL (Data Definition Language). Isso acontece porque ele atua diretamente na estrutura física de armazenamento da tabela, funcionando no nível estrutural e não no nível dos dados. A parte final da afirmação até está correta, já que o TRUNCATE realmente apaga todas as linhas de uma vez e o DELETE permite excluir registros específicos (com WHERE) ou todos eles (sem WHERE). Mas como a classificação do TRUNCATE foi feita errada, a questão inteira fica errada.

Gabarito: Errado

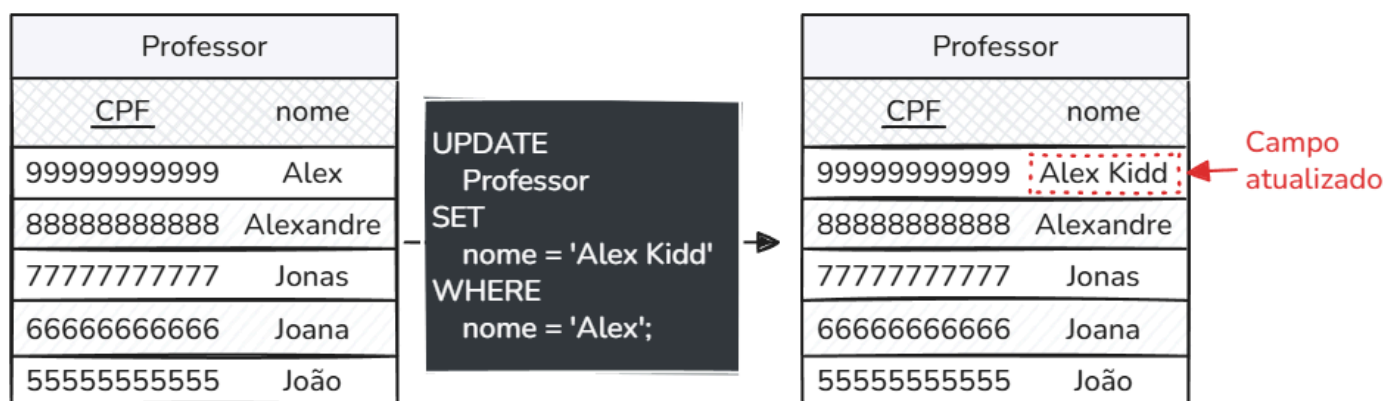
O comando UPDATE

O comando **UPDATE** nos possibilita **modificar dados que já existem em uma tabela**.

Nessa sintaxe, aplicamos o "UPDATE" passando o nome da tabela que contém as informações a serem modificadas, depois usamos "SET" para definir quais valores devem ser alterados e fechamos com um "WHERE" indicando a condição para encontrar quais tuplas serão afetadas.

Por exemplo, no SQL seguir vamos modificar, na tabela Professor, o nome para "Alex Kidd", para todos os professores cujo nome é "Alex" (no nosso caso específico, teremos apenas 1 registro atualizado).

```
UPDATE
  Professor
SET
  nome = 'Alex Kidd'
WHERE
  nome = 'Alex';
```



Existe também uma sintaxe muito interessante que podemos usar com UPDATE (mas também com outros comandos, como SELECT) para definir condições complexas de atualização dos dados.

Nessa sintaxe, usamos a cláusula CASE, juntamente com condições após a sintaxe WHEN para decidir como manipular os dados.

Por exemplo, supondo agora que nossa tabela Professores tenha também os atributos "idade" e "classe", podemos querer que a classe de professores com menos de 30 anos seja atualizada para "Júnior", com menos de 50 para "Pleno" e, se nenhuma das cláusulas for cumprida, seja "Senior".

```
UPDATE Professor
SET classe = CASE
    WHEN idade < 30 THEN 'Júnior' -- Menos de 30 anos, classe = Júnior
    WHEN idade < 50 THEN 'Pleno'  -- Entre 30 e 49 anos, classe = Pleno
    ELSE 'Sênior'                 -- Se não se enquadrar em nenhum anterior,
classe = Sênior
END; -- Obrigatório para fechar o CASE
```



(Instituto AOCP/Esp (MGI)/2024) Considerando a Linguagem de Manipulação de Dados (DML – Data Manipulation Language) em SQL, assinale a alternativa correta.

- a) O comando DELETE remove todas as linhas de uma tabela e a estrutura da tabela do banco de dados.
- b) O comando INSERT INTO é utilizado para atualizar os dados existentes em uma tabela.
- c) O comando UPDATE é usado para modificar os dados existentes em uma tabela com base em uma condição específica.
- d) O comando SELECT cria uma nova tabela com os resultados de uma consulta.
- e) O comando TRUNCATE TABLE adiciona uma nova coluna a uma tabela existente.

Comentários:

- a) **Errada.** O DELETE remove linhas (dados) da tabela, mas não exclui a estrutura da tabela. Quem remove a estrutura é o DROP TABLE.
- b) **Errada.** O INSERT INTO serve para inserir novos registros na tabela, e não para atualizar dados existentes. Quem atualiza é o UPDATE.
- c) **Correta.** O UPDATE realmente modifica dados já existentes em uma tabela, e usamos a cláusula WHERE para indicar a condição que define quais registros serão alterados.
- d) **Errada.** O SELECT apenas consulta (recupera) dados das tabelas, ele não cria uma nova tabela com os resultados.

e) **Errada**. O TRUNCATE TABLE apaga todas as linhas de uma tabela, deixando ela vazia. Ele não adiciona colunas, quem faz isso é o ALTER TABLE com ADD.

Gabarito: C

(CEBRASPE (CESPE)/APO (MPO)/2024) No exemplo de comando UPDATE a seguir, é utilizada a construção case, em que as atualizações nas tuplas são realizadas com um único comando, evitando-se problemas com a ordem das atualizações.

UPDATE tabela

SET coluna = case

When predicado1 then resultado1

When predicado2 then resultado2

...

When predicado3 then resultadoN

else resultadoN

End;

Comentários:

A questão está correta porque descreve exatamente o funcionamento do CASE dentro de um UPDATE. Pense assim: sem o CASE, se quisermos atualizar a coluna com valores diferentes dependendo de condições diferentes, seríamos tentados a rodar vários UPDATEs separados, um para cada situação. O problema é que a ordem em que esses UPDATEs são executados pode bagunçar o resultado, porque uma atualização pode acabar sobrescrevendo outra. Com o CASE, resolvemos tudo em um único comando. O banco vai testando cada WHEN em sequência e, no primeiro predicado que for verdadeiro, aplica o resultado correspondente. Se nenhum for atendido, usa o valor do ELSE. Como tudo acontece em uma só execução, não existe risco de uma atualização atropelar a outra. É o mesmo raciocínio do exemplo da aula, em que o professor era classificado como Júnior, Pleno ou Sênior conforme a idade, tudo dentro de um único UPDATE.

Gabarito: Certo

As funções COALESCE e NULLIF

Sabemos que o valor NULL em SQL representa a ausência de informação e que, por ser um valor desconhecido, operações envolvendo NULL apresentam comportamento frequentemente problemático (por exemplo, NULL + 5 dá NULL).

Então, ao longo de toda especificação SQL, foram sendo criados recursos para lidar com os problemas que esse tipo de valor pode causar. E é justamente pra resolver alguns desses problemas que servem as funções COALESCE e o NULLIF fazem.

Vamos começar pelo **COALESCE**, que, do inglês, significa "coalescer", ou seja, "unir-se" ou "combinar".

É interessante saber isso porque seu funcionamento é totalmente aderente ao seu nome: essa função **combina vários valores retornando o primeiro que não for NULL**.

Ou seja, na prática o COALESCE funciona como se fosse um CASE com várias condições:

1. Se o valor do primeiro elemento é diferente de NULL (WHEN), então retorna ele (THEN);
2. Se o valor do segundo elemento é diferente de NULL (WHEN), então retorna ele (THEN);
3. Se o valor do terceiro elemento é diferente de NULL (WHEN), então retorna ele (THEN);
4. ...
5. Se todos elementos forem NULL, então retorna NULL.

```
COALESCE(NULL, NULL, 3, 4, NULL) -- retorna '3'  
  
COALESCE(1, NULL, 3, 4, NULL) -- retorna '1'  
  
COALESCE(NULL, NULL, NULL, NULL, NULL) -- retorna 'NULL'
```



(FAURGS/Ana (HCPA)/2024) No PostgreSQL, a função _____ retorna o primeiro de seus argumentos que não é nulo. NULL será retornado somente se todos os argumentos forem nulos. Essa função é usada para substituir um valor padrão por valores nulos quando os dados são recuperados para exibição.

- a) CASE
- b) COALESCE
- c) NULLIF
- d) GREATEST
- e) LEAST

Comentários:

a) Errada. CASE é uma estrutura condicional usada para definir comportamentos diferentes conforme regras (WHEN/THEN), não tem como função específica retornar o primeiro argumento não nulo.

b) Correta. COALESCE recebe vários valores e retorna o primeiro que não for NULL. Se todos forem NULL, ela retorna NULL. É usada justamente para substituir valores ausentes por um valor padrão.

c) Errada. NULLIF tem comportamento oposto: ela transforma um valor em NULL quando os dois argumentos passados são iguais.

d) **Errada.** GREATEST não está descrita como função para tratar nulos da forma indicada na questão.

e) **Errada.** LEAST também não tem a finalidade de retornar o primeiro argumento não nulo.

Gabarito: B

(FEPESE/Ana Sis (CELESC)/2022) Analise as afirmativas abaixo sobre a função COALESCE do Oracle.

1 Trabalha com um ou mais argumentos.

2 Retorna nulo somente se todos os argumentos forem nulos.

3 Retorna todos os argumentos não nulos.

Assinale a alternativa que indica todas as afirmativas corretas.

a) É correta apenas a afirmativa 2.

b) São corretas apenas as afirmativas 1 e 2.

c) São corretas apenas as afirmativas 1 e 3.

d) São corretas apenas as afirmativas 2 e 3.

e) São corretas as afirmativas 1, 2 e 3.

Comentários:

Apenas a afirmativa 2 está certa, pois a função COALESCE retorna NULL somente quando todos os argumentos passados forem nulos.

A afirmativa 1 está incorreta, pois COALESCE não trabalha com apenas um argumento, ela combina vários valores para retornar o primeiro que não for NULL, exigindo mais de um argumento para fazer sentido.

A afirmativa 3 está incorreta, pois COALESCE não retorna todos os argumentos não nulos, ela retorna apenas o primeiro valor não nulo encontrado na lista.

Gabarito: A

(CONSULPLAN/Ana TI (CM BH)/2018) "Essa função do SQL funciona como uma expressão CASE, onde são testados os valores diferentes de NULL. Desse modo, o primeiro valor que for diferente de NULL será retornado por tal função." Assinale-a.

a) NULLIF.

b) COALESCE.

c) INTERSECT.

d) EXCEPT DISTINCT.

Comentários:

a) **Errada.** O NULLIF faz o oposto: ele compara dois valores e retorna NULL quando eles são iguais, em vez de retornar o primeiro valor não nulo.

b) **Correta.** O COALESCE recebe vários valores como argumentos e retorna o primeiro que não for NULL, funcionando como se fosse um CASE que testa cada valor até achar um que tenha conteúdo. Se todos forem NULL, ele retorna NULL.

c) **Errada.** INTERSECT é um operador de conjunto, usado para combinar resultados de duas consultas, retornando apenas o que existe em ambas. Não tem relação com tratamento de valores NULL.

d) **Errada.** EXCEPT DISTINCT também é um operador de conjunto, usado para retornar registros de uma consulta que não aparecem em outra. Não serve para retornar o primeiro valor diferente de NULL.

Gabarito: B

Já o **NULLIF** possui um comportamento bem mais estranho (e até pouco intuitivo), porque **transforma um valor específico em NULL**

Para isso funcionar, passamos dois valores ao NULLIF, e, caso ambos forem iguais, retorna NULL.

```
NULLIF(3, 0) -- retorna '3'
NULLIF(0, 3) -- retorna '0'
NULLIF(0, 0) -- retorna 'NULL'
NULLIF(3, 3) -- retorna 'NULL'
```



(ADAPTADA - FGV/AF (SEFAZ PR)/SEFAZ PR/2025) A expressão NULLIF(comissao, 0) retorna NULL apenas quando o valor de comissao é exatamente igual a zero.

Comentários:

A função NULLIF compara dois valores e retorna NULL quando ambos são iguais. Se forem diferentes, ela devolve o primeiro valor informado. No caso da expressão NULLIF(comissao, 0), o segundo argumento é o número zero. Então, só haverá retorno de NULL quando comissao também for zero, pois é nessa situação que os dois valores se igualam. Para qualquer outro valor de comissao (como 100, 50, -3), a função simplesmente devolve o próprio valor da comissao. A afirmação está correta porque descreve exatamente esse comportamento: NULL aparece somente quando comissao é igual a zero.

Gabarito: B

Consulta a Dados (DQL)

O comando SELECT

Na Data Query Language temos basicamente o comando **SELECT**, usado para recuperar (consultar) dados das tabelas de um banco de dados, sejam tabelas bases (as criadas com CREATE TABLE) ou tabelas derivadas (VIEWS).

Esse é provavelmente o comando mais usado em SQL no dia a dia dos profissionais de TI, e, como não poderia deixar de ser, não faltam recursos dentro do SELECT para estudarmos (é muito extenso mesmo). Consequentemente, as instruções SQL produzidas com SELECT tem capacidade de se tornarem extremamente complexas.

Pois bem, sua sintaxe básica é composta por três cláusulas: SELECT, FROM e WHERE.

Iniciamos invocando o SELECT e passando a lista de colunas que desejamos retornar do banco de dados. Na sequência, chamamos o FROM com a lista de tabelas que desejamos para consultar as colunas. Finalmente, usamos o WHERE com um predicado, ou seja, com as condições que devem ser cumpridas para encontrarmos os registros corretos nas tabelas.

Na literatura, esse conjunto básico de comandos é chamado de **mapeamento** ou **bloco select-from-where**.



É justamente por o SELECT ser seguido de comandos que manipulam os dados, como FROM e WHERE, que alguns autores o classificam como DML (Data Manipulation Language).

Antes de aplicarmos nosso primeiro SELECT, precisamos apresentar nossa tabela Professor, com atributos novos, já criada e com dados inseridos. Vejam.

Professor					
<u>CPF</u>	RG	nome	idade	departamento	classe
999999999999	999	Alex	31	Matemática	Junior
888888888888	888	Alex	32	Português	Pleno
777777777777	777	Alexandre	25	Matemática	Senior
666666666666	666	Marcos	31	Português	Junior
555555555555	555	Barcos	32	Matemática	Pleno
444444444444	444	Carlos	25	Biologia	Senior

A seguir, vamos selecionar as colunas "CPF" e "nome" (SELECT) da tabela Professor (FROM) cujo "departamento" vale "Matemática" (WHERE).

```
SELECT
    CPF, nome
FROM
    Professor
WHERE
    departamento = 'Matemática';
```



<u>CPF</u>	nome
999999999999	Alex
777777777777	Alexandre
555555555555	Barcos

Vejam que, depois do WHERE, fizemos "departamento = 'Matemática'". Esse tipo de comparação, com um valor previamente estabelecido, como 'Matemática', chama-se **condição de seleção**, e se trata do **filtro utilizado para capturar apenas as linhas desejadas**.

Porém, não estamos limitados a comparar se os valores de um atributo é exatamente igual a outro, porque temos diversos operadores usados para realizar outros tipos de comparações entre valores.

Como acredito que a maioria desses operadores são bem conhecidos, então podemos estudá-los diretamente por uma tabela resumo de todo o tema.

Operador	Descrição	SQL de Exemplo	Explicação
<	Menor	<pre>SELECT CPF, nome FROM Professor WHERE idade < 18;</pre>	Seleciona todos os registros na "tabela" que possuem o campo "idade" com valores menores que 18.
>	Maior	<pre>SELECT CPF, nome FROM Professor WHERE idade > 18;</pre>	Seleciona todos os registros na "tabela" que possuem o campo "idade" com valores maiores que 18.
<=	Menor ou igual	<pre>SELECT CPF, nome FROM Professor WHERE idade <= 18;</pre>	Seleciona todos os registros na "tabela" que possuem o campo "idade" com valores menores ou iguais à 18.
>=	Maior ou igual	<pre>SELECT CPF, nome FROM Professor WHERE idade >= 18;</pre>	Seleciona todos os registros na "tabela" que possuem o campo "idade" com valores maiores ou iguais à 18.
=	Igual	<pre>SELECT CPF, nome FROM Professor WHERE idade = 18;</pre>	Seleciona todos os registros na "tabela" que possuem o campo "idade" com valores exatamente iguais à 18.
<>	Diferente	<pre>SELECT CPF, nome FROM Professor WHERE idade <> 18;</pre>	Seleciona todos os registros na "tabela" que possuem o campo "idade" com valores que são diferentes de 18.

Continuando, se quisermos selecionar todas colunas da nossa tabela, não precisamos listar uma a uma após o SELECT, basta utilizarmos um asterisco (*).

```
SELECT
    *
FROM
    Professor
WHERE
    departamento =
    'Matemática';
```



<u>CPF</u>	RG	nome	idade	departamento	classe
999999999999	999	Alex	31	Matemática	Junior
777777777777	777	Alexandre	25	Matemática	Senior
555555555555	555	Barcos	32	Matemática	Pleno

Precisamos saber também que, se não for passada nenhuma condição de seleção, o SELECT retorna todos registros da tabela.

```
SELECT
    CPF, nome
FROM
    Professor;
```



<u>CPF</u>	nome
999999999999	Alex
888888888888	Alex
777777777777	Alexandre
666666666666	Marcos
555555555555	Barcos
444444444444	Carlos



(CEBRASPE (CESPE)/APO (MPO)/2024) Em uma consulta SQL, a cláusula FROM corresponde à seleção do predicado que envolve atributos da relação determinada pela cláusula SELECT.

Comentários:

A questão troca o papel das cláusulas, e essa confusão é justamente a pegadinha. No SELECT temos três cláusulas básicas, cada uma com sua função bem definida. O SELECT lista as colunas (atributos) que queremos que apareçam no resultado. O FROM indica de quais tabelas (relações) esses dados serão buscados. Já o WHERE é onde colocamos o predicado, ou seja, as condições que filtram os registros. Então, quem corresponde à seleção do predicado é o WHERE, e não o

FROM. Além disso, o FROM lida com tabelas, não com atributos. Por isso a afirmação está errada em mais de um ponto.

Gabarito: Errado

(FGV/Per Crim (PC MG)/2025) A linguagem SQL (Structured Query Language) é amplamente utilizada para gerenciar e manipular bancos de dados relacionais. Assinale a opção que representa corretamente uma consulta SQL para retornar todos os dados da tabela clientes em que a coluna cidade é igual a "São Paulo".

- a) SELECT * FROM clientes WHERE cidade LIKE 'São Paulo';
- b) SELECT * FROM clientes WHERE cidade = 'São Paulo';
- c) SELECT * FROM clientes WHERE cidade IN 'São Paulo';
- d) SELECT * FROM clientes WHERE cidade NOT IN 'São Paulo';
- e) SELECT cidade, * FROM clientes WHERE cidade = 'São Paulo';

Comentários:

a) Errada. O LIKE é usado para buscar padrões com caracteres curinga como % e _. Sem curingas, embora funcione na prática, não é a forma adequada para uma comparação de igualdade exata.

b) Correta. O asterisco retorna todas as colunas, FROM clientes indica a tabela, e WHERE cidade = 'São Paulo' filtra apenas os registros cuja cidade é exatamente igual a São Paulo, com o texto entre aspas simples.

c) Errada. O operador IN exige uma lista de valores entre parênteses, como cidade IN ('São Paulo', 'Rio'). Usar IN com apenas um valor solto é sintaxe inválida.

d) Errada. Além de o NOT IN também exigir lista entre parênteses, esse comando faria o oposto do desejado, retornando os clientes cuja cidade não é São Paulo.

e) Errada. A sintaxe está inválida porque não se pode misturar uma coluna específica (cidade) com o asterisco dessa forma após o SELECT.

Gabarito: B

(FGV/AJ (TJ SE)/TJ SE/Análise de Sistemas/Web designer/2023) Davi está elaborando uma página Web com uma lista de todos os processos judiciais armazenados da tabela TB_PROCESS do banco de dados do Tribunal. Para retornar todos os campos de todos os registros da tabela TB_PROCESS, a fim de exibir em sua página Web, Davi deve implementar o comando SQL:

- a) SELECT * FROM TB_PROCESS;
- b) SELECT TOP 3 * FROM TB_PROCESS;
- c) SELECT * INTO REL_PROCESS FROM TB_PROCESS;
- d) SELECT COUNT(DISTINCT N_PROCESS) FROM TB_PROCESS;
- e) SELECT N_PROCESS FROM TB_PROCESS WHERE 1=1;

Comentários:

- a) **Correta.** O asterisco (*) significa "todas as colunas" e, sem a cláusula WHERE, o SELECT retorna todos os registros da tabela, exatamente o que Davi precisa.
- b) **Errada.** O TOP 3 limita o resultado a apenas 3 registros, mas Davi quer todos os processos.
- c) **Errada.** O SELECT * INTO não apenas consulta, mas cria uma nova tabela (REL_PROCESS) com os dados, o que não é o objetivo.
- d) **Errada.** O COUNT(DISTINCT N_PROCESS) retorna apenas um número (a contagem de processos distintos), e não os campos e registros da tabela.
- e) **Errada.** Retorna apenas a coluna N_PROCESS, e não todos os campos como solicitado.

Gabarito: A

Conjunto vs Multiconjunto (bag)

O modelo relacional fundamenta suas bases na teoria dos conjuntos da matemática, herdando, portanto, suas regras e restrições.

Uma dessas **restrições** trata-se da unicidade de cada elemento de um conjunto, ou, em palavras mais simples, **somente é possível termos uma tupla por relação**.

Assim sendo, para o modelo relacional, se uma relação Professor estabelece que seus atributos são "Nome" e "idade" e temos uma tupla "João" e "31", então é impossível haver associada a essa relação outra tupla exatamente idêntica. Ou seja, não é possível termos multiplicidade de elementos, ou mais de um "João" e "31" em Professor.

Acontece que, **os SGBDs relacionais seguem o teórico modelo relacional, porém estabelecendo algumas concessões às suas restrições**. Ou seja, as tabelas de um SGBD relacional tentam, na medida do que lhes convém, se comportar como as relações do modelo relacional. Se a restrição matemática não fizer sentido na prática, o banco de dados simplesmente a descumpra e vida que segue.

Pois bem, uma dessas concessões ao modelo relacional é justamente relativizar essa regra de existir apenas elementos únicos. Dessa forma, é plenamente possível, em uma tabela de um banco de dados relacional, haver mais de um registro exatamente com os mesmos valores de atributos.

Pessoas	
nome	idade
Alex	31
Alex	31
Alexandre	25

É possível termos,
em uma tabela,
2 registros iguais

Por isso que dizemos que os **SGBDs fundamentam-se em multiconjuntos** (também chamados de *bags*), que são os conjuntos que aceitam múltiplos elementos.

Como consequência direta das tabelas serem *bags*, comandos como o SELECT também podem retornar múltiplos elementos. No exemplo a seguir, embora a tabela original não possua registros repetidos, o resultado do SQL possui.

Professor					
<u>CPF</u>	RG	nome	idade	departamento	classe
999999999999	999	Alex	31	Matemática	Junior
888888888888	888	Alex	32	Português	Pleno
777777777777	777	Alexandre	25	Matemática	Senior
666666666666	666	Marcos	31	Português	Junior
555555555555	555	Barcos	32	Matemática	Pleno
444444444444	444	Carlos	25	Biologia	Senior

```
SELECT  
nome  
FROM  
Professor  
WHERE  
nome = 'Alex';
```

nome
Alex
Alex

Então, para conseguirmos um comportamento de SELECT que seria adequado ao esperado pela teoria dos conjuntos do modelo relacional, precisamos utilizar o comando **DISTINCT**. Isso porque, com o DISTINCT, **todos registros múltiplos são exibidos como um só**.



Na prática, não existe melhor ou pior comando, pois cada um resolve um problema específico. Por exemplo, se quisermos contar quantos registros há na

tabela, precisamos contar até os com o mesmo valor, então devemos usar o SELECT sem DISTINCT.

Do mesmo modo, a operação **UNION**, por unir diversos registros distintos, **também aceita múltiplas versões que modificam seu comportamento**, alternando entre obediência a teoria dos conjuntos e multiconjuntos.

Operador	SQL de Exemplo	Explicação	Desempenho
UNION	Combina resultados de duas consultas eliminando duplicatas.	Semanticamente igual a UNION DISTINCT. Exige o mesmo número e tipo de colunas.	Baixo
UNION DISTINCT	Variante explícita do UNION.	Idêntico a UNION em comportamento. Preferível quando a intenção deve ser clara.	Baixo
UNION ALL	Combina resultados preservando todas as tuplas, inclusive duplicadas.	Mais rápido que UNION por não realizar deduplicação. Usar quando duplicatas são esperadas.	Alto



(CEBRASPE (CESPE)/Tecno (LNA)/2024) Assinale a opção em que é apresentada a palavra-chave, em SQL, que deve ser incluída em uma instrução de SELECT para evitar a apresentação de resultados duplicados da tabela.

- a) UNIQUE
- b) NOTDUPLICATE
- c) NOTALL
- d) DISTINCT
- e) ONLY

Comentários:

a) Errada. UNIQUE em SQL é uma restrição (constraint) usada para garantir que valores em uma coluna sejam únicos, mas não é a palavra-chave usada no SELECT para eliminar duplicatas.

- b) **Errada.** NOTDUPLICATE não existe na linguagem SQL, é apenas uma palavra inventada para confundir o candidato.
- c) **Errada.** NOTALL não é uma palavra-chave do SQL. Existe o UNION ALL, que justamente preserva duplicatas, mas isso é diferente do que a questão pede.
- d) **Correta.** DISTINCT é a palavra-chave que, quando incluída em uma instrução SELECT, faz com que registros múltiplos (iguais) sejam exibidos como um só, eliminando duplicatas do resultado.
- e) **Errada.** ONLY não é a palavra-chave usada para evitar resultados duplicados em um SELECT.

Gabarito: D

(ADAPTADA - FURB/AFTM Florianópolis/2025) Avalie as afirmativas a seguir sobre operações de conjunto (UNION, INTERSECT, EXCEPT), tratamento de valores NULL e duplicação de tuplas em SQL:

- I. A operação UNION DISTINCT, embora semanticamente equivalente a UNION, pode ser utilizada para explicitar a remoção de duplicatas entre os resultados de duas consultas. Já UNION ALL preserva todas as tuplas, inclusive duplicadas, com melhor desempenho.
- IV. Embora o modelo relacional clássico trate relações como conjuntos, o SQL permite tuplas duplicadas tanto em tabelas quanto em resultados de consultas, tratando seus dados como multiconjuntos (bags).

É correto o que se afirma em:

Comentários:

As duas afirmativas estão corretas. Na afirmativa I, a ideia central é entender que UNION e UNION DISTINCT fazem exatamente a mesma coisa: combinam os resultados de duas consultas e eliminam as linhas duplicadas. A diferença é só de escrita, o DISTINCT serve para deixar a intenção explícita no código. Já o UNION ALL é diferente, porque mantém todas as tuplas, inclusive as repetidas, e por não precisar gastar tempo eliminando duplicatas, acaba tendo desempenho melhor. Na afirmativa IV, temos uma observação importante sobre como o SQL funciona na prática. Na teoria pura do modelo relacional, baseada em conjuntos da matemática, não poderia existir duas tuplas iguais numa mesma relação. Mas o SQL relaxa essa regra e trabalha com multiconjuntos (também chamados de bags), que aceitam elementos repetidos. Por isso é possível ter linhas idênticas tanto em tabelas quanto em resultados de consultas, e é justamente por causa disso que existe o DISTINCT, para quando queremos forçar o comportamento de conjunto puro.

Gabarito: I - Correta e IV - Correta

(FUNDATEC/Ana Sist (BRDE)/2025) Uma query deve combinar a lista de todos os Clientes ativos de uma região com a lista de todos os Fornecedores ativos da mesma região. O resultado deve incluir todos os registros de ambas as tabelas, sem duplicatas. Qual operador de conjunto SQL é o mais adequado para essa tarefa?

- a) UNION ALL
- b) INTERSECT

- c) CROSS JOIN
- d) UNION
- e) FULL OUTER JOIN

Comentários:

- a) **Errada.** O UNION ALL combina os resultados das duas consultas, mas preserva todas as duplicatas, o que contraria o requisito de "sem duplicatas".
- b) **Errada.** O INTERSECT retorna apenas os registros que aparecem em ambas as consultas (a interseção), mas o enunciado pede todos os registros das duas tabelas combinados.
- c) **Errada.** O CROSS JOIN faz o produto cartesiano entre tabelas, combinando cada linha da primeira com todas as linhas da segunda, não sendo um operador de união de conjuntos.
- d) **Correta.** O UNION combina os resultados de duas consultas eliminando duplicatas automaticamente, atendendo exatamente ao requisito de unir todos os registros das duas listas sem repetições.
- e) **Errada.** Alternativa não aplicável.

Gabarito: D

Lógica de 3 valores

Sabemos que o WHERE elimina todas as linhas da tabela em que o predicado de comparação não é verdadeiro. Essa formulação parece óbvia, mas tem uma sutileza importante no contexto do SQL.

Isso porque **condições em SQL**, diferentemente de outras áreas do conhecimento, como Raciocínio Lógico Matemático, **podem assumir valores Verdadeiro (True), Falso (False) e Desconhecido (Unknown).**

Assim, "eliminar tudo que não é True", significa não escolher as linhas False e Unknown!

Agora você deve estar se perguntando "*mas quando "Unknown" ocorre, professor?*". **Quando uma condição é avaliada sob um valor ausente (Null)!**

Por exemplo, vamos rodar um SELECT que retorna todos os nomes dos professores cujo departamento é Matemática. Para conseguir retornar os registros corretos, o banco de dados percorre linha a linha da tabela Professor e avalia se cada departamento encontrado é igual a 'Matemática'.

```
SELECT nome FROM Professor WHERE departamento = 'Matemática';
```

Ocorre que, nas linhas em que há algum valor, o banco consegue avaliar se a avaliação é True (isto é, o departamento é matemática) ou Falsa (isto é, não é matemática).

Porém, nas linhas cujo valor é ausente (Null), o banco de dados não consegue avaliar a condição em Verdadeira ou Falsa, porque simplesmente não sabe qual o valor a ser avaliado. Nesses casos, a avaliação da comparação com Matemática possui valor desconhecido (Unknown).

Professor					
CPF	RG	nome	idade	departamento	classe
999999999999	999	Alex	31	Matemática	Junior
888888888888	888	Alex	32	Português	Pleno
777777777777	777	Alexandre	25	Matemática	Senior
666666666666	666	Marcos	31	Português	Junior
555555555555	555	Barcos	32	NULL	Pleno
444444444444	444	Carlos	25	Biologia	Senior

SELECT nome FROM Professor
WHERE departamento = 'Matemática';

TRUE

FALSE

TRUE

FALSE

UNKNOWN

FALSE

nome
Alex
Alexandre

Além disso, não conseguimos usar comparações com NULL, como "departamento = NULL".

SELECT nome FROM Professor WHERE departamento = NULL;

Isso porque, nessas situações, o banco de dados tenta comparar cada valor de departamento com um valor desconhecido (o NULL), e sempre entenderá que a resposta é desconhecida (UNKNOWN).

Professor					
CPF	RG	nome	idade	departamento	classe
999999999999	999	Alex	31	Matemática	Junior
888888888888	888	Alex	32	Português	Pleno
777777777777	777	Alexandre	25	Matemática	Senior
666666666666	666	Marcos	31	Português	Junior
555555555555	555	Barcos	32	NULL	Pleno
444444444444	444	Carlos	25	Biologia	Senior

SELECT nome FROM Professor
WHERE departamento = NULL;

UNKNOWN

UNKNOWN

UNKNOWN

UNKNOWN

UNKNOWN

UNKNOWN

nome



Nem se um departamento for NULL a comparação "NULL = NULL" retornará TRUE.

Isso porque, o banco avalia "um valor que eu não sei quanto vale (NULL) é igual (=) a outro valor que eu não sei quanto vale (NULL)? Não sei (UNKNOWN)!".

Então, se quisermos capturar justamente os registros que possuem valor ausente, devemos utilizar um operador específico: o **IS NULL**.

```
SELECT nome FROM Professor WHERE departamento IS NULL;
```

Professor					
CPF	RG	nome	idade	departamento	classe
999999999999	999	Alex	31	Matemática	Junior
888888888888	888	Alex	32	Português	Pleno
777777777777	777	Alexandre	25	Matemática	Senior
666666666666	666	Marcos	31	Português	Junior
555555555555	555	Barcos	32	NULL	Pleno
444444444444	444	Carlos	25	Biologia	Senior

```
SELECT nome FROM Professor  
WHERE departamento IS NULL;
```

FALSE
FALSE
FALSE
FALSE
TRUE
FALSE

nome
Barcos

Finalmente, o **NOT** se trata da **negação de um operador**, ou seja, inverte a lógica de qualquer condição.

Por exemplo, IS NOT NULL capturamos justamente os valores que não são NULL.

Do mesmo modo, podemos usar NOT IN para capturarmos os valores que não estão na lista, e assim por diante.



(ADAPTADA - FURB/AFTM Florianópolis/2025) Avalie as afirmativas a seguir sobre operações de conjunto (UNION, INTERSECT, EXCEPT), tratamento de valores NULL e duplicação de tuplas em SQL:

II.A linguagem SQL utiliza lógica de três valores (TRUE, FALSE, UNKNOWN) ao avaliar expressões lógicas, sendo UNKNOWN o resultado típico de comparações com NULL.

III.Em SQL, ao comparar um valor com NULL, deve-se usar o operador de igualdade (=) ou desigualdade (<>), já que NULL é tratado como um valor especial.

É correto o que se afirma em:

Comentários:

A afirmativa II está perfeitamente alinhada com o que vimos sobre lógica de três valores. Em SQL, quando o banco avalia uma expressão, o resultado pode ser TRUE, FALSE ou UNKNOWN. Esse terceiro valor, UNKNOWN, aparece justamente quando tentamos comparar algo com NULL, porque o banco simplesmente não sabe qual é o valor real para fazer a comparação. Já a afirmativa III cai numa pegadinha clássica. O erro está em sugerir que devemos usar = ou <> para

comparar com NULL. Pense assim: se NULL representa "valor desconhecido", quando você escreve "campo = NULL", o banco entende como "esse valor é igual a um valor que eu não sei?" e a resposta é sempre UNKNOWN, nunca TRUE. Por isso, nem mesmo NULL = NULL retorna verdadeiro. Para verificar se um campo está com valor ausente, o correto é usar o operador IS NULL (ou IS NOT NULL para o contrário). Portanto, apenas a II está correta e a III está errada.

Gabarito: II - Correta e III - Errada

Operadores em Predicados

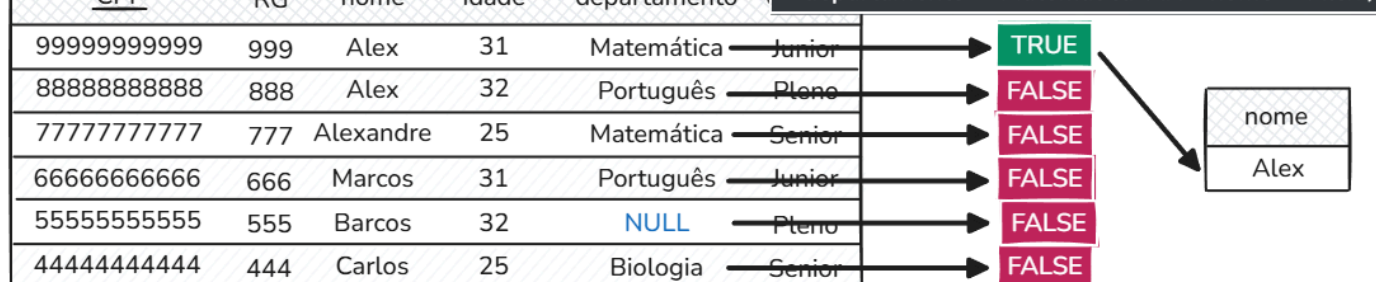
Se quisermos filtros mais complexos do que simplesmente comparar um campo com algum valor, precisamos recorrer aos diversos operadores lógicos e palavras-chave que o SQL oferece.

Com o **conectivo lógico AND**, podemos acumular **diversas restrições que precisam ser cumpridas** (isto é, todas serem verdadeiras) para que um registro possa ser selecionado.

Por exemplo, se quisermos todos professores do departamento de matemática e que tenham nome Alex, podemos escrever:

```
SELECT
    nome
FROM
    Professor
WHERE
    departamento = 'Matemática' AND nome = 'Alex';
```

Professor					SELECT nome FROM Professor WHERE departamento = 'Matemática' AND nome = 'Alex';	
CPF	RG	nome	idade	departamento		
99999999999	999	Alex	31	Matemática	Junior	TRUE
88888888888	888	Alex	32	Português	Pleno	FALSE
77777777777	777	Alexandre	25	Matemática	Senior	FALSE
66666666666	666	Marcos	31	Português	Junior	FALSE
55555555555	555	Barcos	32	NULL	Pleno	FALSE
44444444444	444	Carlos	25	Biologia	Senior	FALSE



Nesse momento eu quero que vocês percebam que a comparação na linha do NULL deu valor lógico Falso e não Unknown. Isso ocorre porque o AND exige, para ser True, que todas condições sejam True também. Assim sendo, se apenas 1 não for (no caso a condição falsa foi 'Barcos' = 'Alex'), então necessariamente ele será Falso.

Vamos ver, então, a tabela da verdade do AND considerando a lógica de 3 valores: True - **V**, False - **F** e Null/Unknown - **?**.

Condição			Resultado
V	AND	V	V
V	AND	F	F
F	AND	F	F
V	AND	?	?
F	AND	?	F
?	AND	?	?

Já com o conectivo lógico **OR** basta que apenas uma das condições seja cumprida para que nosso registro seja selecionado.

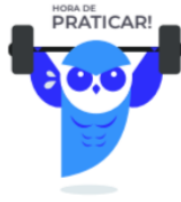
```
SELECT
    nome
FROM
    Professor
WHERE
    departamento = 'Matemática' OR nome = 'Alex';
```

Professor					SELECT nome FROM Professor WHERE departamento = 'Matemática' OR nome = 'Alex';	
CPF	RG	nome	idade	departamento		
999999999999	999	Alex	31	Matemática	Junior	TRUE
888888888888	888	Alex	32	Português	Pleno	TRUE
777777777777	777	Alexandre	25	Matemática	Senior	TRUE
666666666666	666	Marcos	31	Português	Junior	FALSE
555555555555	555	Barcos	32	NULL	Pleno	UNKNOWN
444444444444	444	Carlos	25	Biologia	Senior	FALSE

nome
Alex
Alex
Alexandre

Agora vejamos nossa tabela da verdade para o OR.

Condição			Resultado
V	OR	V	V
V	OR	F	V
F	OR	F	F
V	OR	?	V
F	OR	?	?
?	OR	?	?



(FGV/Ana (MPE SC)/MPE SC/Tecnologia da Informação/2022) Analise a tabela referente ao comportamento dos operadores lógicos em bancos de dados relacionais que trabalham com a lógica de três valores, verdadeiro (V), falso (F) e desconhecido (?).

Linha	Expressão	Valor
1	V and ?	?
2	F or ?	?
3	? or ?	?
4	not ?	?
5	? and ?	?

O valor da expressão está correto nas linhas:

- a) 1, 2 e 3, somente;
- b) 1, 2, 4 e 5, somente;
- c) 1, 3, 4 e 5, somente;
- d) 2, 3, 4 e 5, somente;
- e) 1, 2, 3, 4 e 5.

Comentários:

Linha 1: Se o valor desconhecido for V, então a saída seria V, se for F, então o resultado seria F. Como não sabemos, então a saída é desconhecida.

Linha 2: Se o valor desconhecido for V, então a saída seria V, se for F, então o resultado seria F. Como não sabemos, então a saída é desconhecida.

Linha 3: Como ambos valores são desconhecidos, não sabemos a resposta.

Linha 4: Não sabemos o valor, a negação de algo que não sabemos é algo que não sabemos também.

Linha 5: Como ambos valores são desconhecidos, não sabemos a resposta.

Gabarito: E

O operador **IN** é bem curioso: **passamos múltiplos elementos e selecionamos cada registro que valida pelo menos um deles**. Ou seja, se trata de uma forma elegante e legível de escrevermos múltiplos OR entre vários elementos.



(CEBRASPE (CESPE)/Ana (FINEP)/2024) A respeito da linguagem SQL e da manipulação de dados em um banco de dados relacional Oracle, assinale a opção correta.

- a) A cláusula MODIFY é usada para atualizar registros em uma consulta.
- b) O operador IN é usado para verificar se um valor está em um subconjunto de valores.
- c) A finalidade do comando CREATE TABLE é criar um banco de dados.
- d) A cláusula REMOVE é usada para excluir registros de uma tabela.
- e) A finalidade do comando ALTER TABLE é criar uma nova tabela.

Comentários:

- a) **Errada.** Não existe a cláusula MODIFY para atualizar registros; quem atualiza dados é o comando UPDATE, junto com SET e WHERE.
- b) **Correta.** O operador IN funciona como vários OR juntos, verificando se o valor de um atributo está dentro de uma lista de valores informada.
- c) **Errada.** O CREATE TABLE serve para criar uma tabela dentro de um banco já existente, e não para criar o banco de dados em si.
- d) **Errada.** Não existe cláusula REMOVE em SQL para excluir registros; quem remove dados de uma tabela é o comando DELETE (ou TRUNCATE, que apaga todos de uma vez).
- e) **Errada.** O ALTER TABLE serve para modificar uma tabela já existente, como adicionar ou remover colunas e restrições. Para criar uma nova tabela, usamos o CREATE TABLE.

Gabarito: B

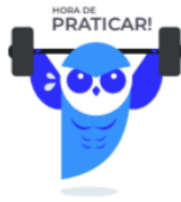
Com o **BETWEEN** filtramos os registros com base em um intervalo de valores. Saibam que os limites são inclusivos, ou seja, tanto o valor inicial quanto o final fazem parte do resultado.

```
SELECT
    nome
FROM
    Professor
WHERE
    idade BETWEEN 30 AND 50; -- restringe o resultado a quem possui idade de 30
até 50
```

Acredito que o mais surpreendente seja que podemos usar BETWEEN com strings (textos) também. Nesse caso, o banco compara as palavras usando ordem alfabética.

Por exemplo, podemos filtrar os professores cujos nomes começam entre as letras "A" e "C".

```
SELECT nome FROM Professor WHERE idade BETWEEN 'A' AND 'C';
```



(FADE UFPE/Ana (UFPE)/2023) Considerando a tabela Produto (cod, nome, valor), assinale a alternativa que constitui a única consulta SQL sintática e semanticamente correta.

- a) Select * From Produto Where valor > avg (valor)
- b) Select * From Produto Where valor in (A, B, C)
- c) Select * From Produto Where valor < any (valor)
- d) Select * From Produto Where nome like '\$A\$'
- e) Select * From Produto Where nome between 'ABC' and 'CBA'

Comentários:

- a) **Errada.** Não podemos usar uma função de agregação como AVG diretamente dentro do WHERE dessa forma, pois o WHERE filtra linha a linha e não consegue trabalhar com um valor agregado calculado sobre todas as linhas ao mesmo tempo.
- b) **Errada.** O operador IN deve receber valores válidos entre aspas quando forem textos, como ('A', 'B', 'C'). Sem as aspas, o banco interpreta A, B e C como nomes de colunas, o que gera erro.
- c) **Errada.** Comparar valor com any(valor) não faz sentido semântico, pois estaríamos comparando a coluna valor com ela mesma, sem trazer informação útil.
- d) **Errada.** Os caracteres curinga aceitos pelo LIKE são o % (zero ou mais caracteres) e o _ (exatamente um caractere). O símbolo \$ não é um curinga válido, então essa busca não funcionará como esperado.
- e) **Correta.** O operador BETWEEN também funciona com textos, comparando-os em ordem alfabética. Assim, essa consulta retorna os produtos cujo nome esteja entre 'ABC' e 'CBA' na ordem alfabética, sendo sintática e semanticamente válida.

Gabarito: E

O operador **LIKE** é bem interessante porque **permite buscarmos textos a partir de algum padrão pré estabelecido**. Para conseguirmos isso, precisamos estabelecer o padrão e utilizar algum caractere curinga, isto é, algum caractere que permite generalizarmos o padrão.

Por padrão, os SGBDs aceitam os caracteres coringas "%" e "_".

O "%" permite que **utilizarmos o padrão com zero ou mais caracteres**. Por exemplo, se escrevermos 'Alex%' encontraremos qualquer texto que contenha "Alex" no começo da palavra, como o próprio "Alex", mas também "Alexa" e "Alexandre".

Já o "_" torna **exatamente um caractere genérico**. Assim, '_arlos' encontra "Carlos" e "Marlos", mas não "Carlitos".



(ADM&TEC/TInf (Pref M de Camaragibe)/2024) Leia.

"_____ é usado em uma instrução com WHERE ou HAVING (como parte de GROUP BY) para limitar as linhas selecionadas aos itens quando uma coluna possui um determinado padrão de caracteres contido nela."

O trecho acima faz referência a qual comando da Linguagem SQL?

- a) Count.
- b) Like.
- c) As.
- d) Avg.

Comentários:

a) Errada. O Count é uma função de agregação que serve para contar linhas, não para buscar padrões de caracteres em uma coluna.

b) Correta. O Like é o operador usado justamente para buscar textos a partir de um padrão de caracteres, utilizando curingas como % (zero ou mais caracteres) e _ (exatamente um caractere). Ele aparece em cláusulas WHERE ou HAVING para filtrar linhas cujo conteúdo de uma coluna corresponda ao padrão informado.

c) Errada. O As é usado para criar apelidos (aliases) para tabelas ou colunas, não para comparar padrões de texto.

d) Errada. O Avg é uma função de agregação que calcula a média aritmética de valores numéricos, não tem relação com busca por padrões de caracteres.

Gabarito: B

(FGV/GTIFE (Sefaz AM)/SEFAZ AM/2022) Com referência à otimização de consultas SQL para bancos de dados relacionais, assinale a opção que apresenta a cláusula que potencialmente pode causar maiores problemas de desempenho, por si só, quando são manipuladas tabelas com grande número de registros.

- a) DISTINCT

- b) EXISTS
- c) LIKE
- d) NOT EXISTS
- e) ORDER BY

Comentários:

a) Errada. O DISTINCT serve para eliminar registros duplicados em uma consulta, fazendo o SELECT se comportar como na teoria dos conjuntos. Apesar de exigir esforço do banco, não é a cláusula com maior potencial de problema de desempenho entre as listadas.

b) Errada. O EXISTS é usado em subconsultas para verificar a existência de registros, e não é apontado como o maior vilão de desempenho entre as opções.

c) Correta. O LIKE permite buscar textos a partir de um padrão usando caracteres curingas como % e _. Quando temos tabelas com muitos registros, o banco precisa percorrer linha por linha comparando o padrão de texto, o que torna essa operação muito custosa em termos de desempenho.

d) Errada. O NOT EXISTS é a negação do EXISTS, também usado em subconsultas, mas não é considerado o maior problema de desempenho entre as opções listadas.

e) Errada. O ORDER BY ordena os resultados por algum critério (crescente ou decrescente). Embora ordenar muitos registros tenha custo, não é apontado como a cláusula com maior potencial de problema de desempenho aqui.

Gabarito: C

(QUADRIX/Tecno (CRC AP)/2021) Observe the instruction below:

```
SELECT * FROM Companies WHERE Name LIKE '%ar%';
```

The SQL statement finds any values that:

- a) Start with "ar" and are at least 2 characters in length.
- b) Have "ar" in any position.
- c) Have "ar" in the second position.
- d) Start with "a" and ends with "r".
- e) Have "ar" in the end.

Comentários:

a) Errada. Para começar com "ar", o padrão seria 'ar%', sem o % no início.

b) Correta. O caractere % representa zero ou mais caracteres, então '%ar%' significa que pode ter qualquer coisa antes e qualquer coisa depois de "ar", ou seja, "ar" pode aparecer em qualquer posição do texto.

c) **Errada.** Para "ar" estar exatamente na segunda posição, usariamos o curinga _ (que representa exatamente um caractere), ficando '_ar%'.

d) **Errada.** Para começar com "a" e terminar com "r", o padrão seria 'a%r'.

e) **Errada.** Para terminar com "ar", o padrão seria '%ar', sem o % no final.

Gabarito: B

FROM com Múltiplas Tabelas

Quando especificamos mais de uma tabela no FROM, o banco de dados cria uma nova tabela com todas informações das tabelas informadas. **Se não especificarmos nenhuma condição no WHERE, essa nova tabela é obtida realizando o produto cartesiano entre os registros de todas elas.**

Por exemplo, vamos utilizar duas tabelas após o FROM, Professor e Auxiliar, para buscar por todas as combinações possíveis de professores com auxiliares.

```
SELECT
*
FROM
Professor, Auxiliar;
```

Professor		
CPF	nome	departamento
99999999999	Alex	Matemática
88888888888	Alex	Português
77777777777	Alexandre	Matemática

Auxiliar		
CPF	nome	departamento
00000000000	João	Biologia
11111111111	Joana	Matemática

```
SELECT
*
FROM
Professor, Auxiliar;
```

CPF	nome	departamento	CPF	nome	departamento
99999999999	Alex	Matemática	00000000000	João	Biologia
88888888888	Alex	Português	00000000000	João	Biologia
77777777777	Alexandre	Matemática	00000000000	João	Biologia
99999999999	Alex	Matemática	11111111111	Joana	Matemática
88888888888	Alex	Português	11111111111	Joana	Matemática
77777777777	Alexandre	Matemática	11111111111	Joana	Matemática

Porém, por juntarmos duas tabelas, alguns problemas de ordem prática surgem. Por exemplo, se tivermos atributos entre as tabelas exatamente iguais e quisermos essas informações específicas no resultado, o banco não conseguirá diferenciar qual atributo se trata de qual tabela.

Por exemplo, no SELECT a seguir, queremos retornar apenas o nome de cada tabela, o que causará problemas ao SGBD.

```
SELECT
nome, nome -- isso não funciona, cada "nome" é do Professor ou do Auxiliar?
FROM
Professor, Auxiliar;
```

Para resolver esse tipo de problema, podemos qualificar cada atributo adicionando o nome da tabela em que eles estão. Vejam só.

```
SELECT
    Professor.nome, Auxiliar.nome -- prefixamos o nome das tabelas para
    identificar corretamente o atributo
FROM
    Professor, Auxiliar;
```

Também podemos facilitar nossa vida adicionando um apelido (alias) a cada nome de tabela, evitando a digitação repetida de seus nomes ao longo do SQL. Fazemos isso adicionando um AS após a tabela que receberá o apelido.

A seguir, apelidamos o a tabela Professor de "Prof" e Auxiliar de "Aux".

```
SELECT
    Prof.nome, Aux.nome
FROM
    Professor AS Prof, Auxiliar AS Aux;
```

Além das tabelas, para tornar tudo mais claro, também podemos renomear os atributos retornados no SELECT com apelidos.

Por exemplo, a seguir renomeamos o nome do Professor para "nome_do_prof" e o nome do auxiliar para "nome_do_aux".

```
SELECT
    Prof.nome AS nome_do_prof,
    Aux.nome AS nome_do_aux
FROM
    Professor AS Prof,
    Auxiliar AS Aux;
```

Professor		
<u>CPF</u>	nome	departamento
999999999999	Alex	Matemática
888888888888	Alex	Português
777777777777	Alexandre	Matemática

Auxiliar		
<u>CPF</u>	nome	departamento
000000000000	João	Biologia
111111111111	Joana	Matemática

```
SELECT
    Prof.nome AS nome_do_prof,
    Aux.nome AS nome_do_aux
FROM
    Professor AS Prof,
    Auxiliar AS Aux;
```

nome_do_prof	nome_do_aux
Alex	João
Alex	João
Alexandre	João
Alex	Joana
Alex	Joana
Alexandre	Joana

FROM e WHERE com Múltiplas Tabelas

É claro que também podemos aplicar a cláusula WHERE para restringir quais registros retornarão em um SQL em que o FROM está associado a múltiplas tabelas. Nesse caso, podemos estabelecer restrições para qualquer atributo das tabelas referenciadas.

A seguir, vamos retornar as combinações possíveis de Professor e Auxiliar que pertencem ao departamento de Matemática, filtrando tanto o departamento dos professores quanto dos auxiliares.

```
SELECT
    Prof.nome AS nome_do_prof,
    Aux.nome AS nome_do_aux
FROM
    Professor AS Prof,
    Auxiliar AS Aux
WHERE
    Prof.departamento = 'Matemática' AND Aux.departamento = 'Matemática'; -- filtro
```

Aqui, da mesma forma que aprendemos no estudo de FROM com apenas 1 tabela, podemos aplicar operadores que já conhecemos, como BETWEEN, IN, OR, AND, LIKE, NOT, comparações matemáticas, etc.

Teoricamente, a teoria dessa seção finalizaria aqui, se não fosse um outro tipo de condição que pode aparecer no WHERE com múltiplas tabelas.

Além dos filtros que vimos até aqui, também é possível utilizarmos as condições de junção, que nada mais são que critérios estabelecidos entre os atributos das tabelas para juntar suas linhas diretamente.

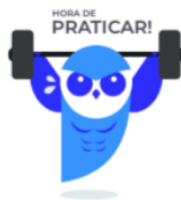
Assim, são diferentes dos predicados com filtro, porque estabelecem uma correspondência entre as linhas das tabelas envolvidas para construir o resultado.

Por exemplo, podemos estabelecer uma condição de junção para gerar as combinações de Professor e Auxiliar com o mesmo departamento.

```
SELECT
    Prof.nome AS nome_do_prof,
    Aux.nome AS nome_do_aux
FROM
    Professor AS Prof,
    Auxiliar AS Aux
WHERE
    Prof.departamento = Aux.departamento; -- condição de junção
```

Essa sintaxe, que utiliza apenas operadores de comparação (como "=") no entre campos de tabelas no WHERE para criar as condições de junção, é chamada de "junção implícita" e, mais

tarde nessa aula, veremos que se trata de um tipo específico de junção, a junção interna (INNER JOIN).



(COSEAC UFF/Tec (UFF)/2023) No SQL existe uma cláusula para junção que retorna o produto cartesiano entre a tabela primária e a tabela relacionada. Esta cláusula é conhecida como

- a) NATURAL JOIN.
- b) INNER JOIN.
- c) OUTER JOIN.
- d) NON EQUIJOIN.
- e) CROSS JOIN.

Comentários:

a) **Errada.** O NATURAL JOIN é um caso específico de INNER JOIN que junta as tabelas automaticamente pelas colunas com mesmo nome, não realiza produto cartesiano.

b) **Errada.** O INNER JOIN combina linhas das tabelas a partir de uma condição de junção definida pelo ON, retornando apenas as correspondências, e não todas as combinações possíveis.

c) **Errada.** O OUTER JOIN (LEFT, RIGHT ou FULL) preserva linhas mesmo sem correspondência na outra tabela, mas também depende de uma condição de junção, não gerando produto cartesiano.

d) **Errada.** NON EQUIJOIN não é um tipo de junção tratado como cláusula que retorna produto cartesiano entre tabelas.

e) **Correta.** O CROSS JOIN é a sintaxe moderna, introduzida no SQL-92, que indica explicitamente o produto cartesiano entre duas tabelas, combinando cada linha da primeira com todas as linhas da segunda.

Gabarito: E

(CEBRASPE (CESPE)/Ana Sist (CAGEPA)/CAGEPA/Sistemas de TI/2024) Considere o comando SQL/ANSI a seguir.

```
Select a.nome, b.nome, a.renda
from Cliente a, Cliente b
where a.cpf = b.CPF_cliente_antigo
order by 1;
```

Em relação à álgebra relacional, para que o comando apresentado no texto CC1A1 retorne os valores desejados, o sistema gerenciador deve realizar

- a) operação de junção.
- b) operação de atribuição.
- c) operações de união e de seleção.
- d) operações de produto cartesiano e de união.
- e) operações de produto cartesiano e de diferença.

Comentários:

a) Correta. Quando temos duas tabelas no FROM (no caso, a tabela Cliente referenciada duas vezes com apelidos a e b) combinadas com uma condição no WHERE que liga atributos entre elas (a.cpf = b.CPF_cliente_antigo), estamos diante de uma junção implícita, ou seja, uma operação de junção. O banco encontra as linhas em que o CPF de um cliente corresponde ao CPF do cliente antigo de outro registro.

b) Errada. Não existe operação de atribuição nesse comando, o SELECT apenas consulta dados, não atribui valores.

c) Errada. Não há união (UNION) nesse SQL, e a seleção sozinha não explica a combinação das duas tabelas no FROM.

d) Errada. Embora a sintaxe com vírgula no FROM sugira um produto cartesiano, a presença do WHERE com igualdade entre atributos das tabelas transforma isso em junção, e não há união no comando.

e) Errada. Não existe operação de diferença aqui, e novamente a condição no WHERE caracteriza junção, não apenas produto cartesiano.

Gabarito: A

Nada impede que, na sequência, também apliquemos filtros para restringir apenas as linhas cujos departamentos de ambos profissionais seja Matemática.

```
SELECT
    Prof.nome AS nome_do_prof,
    Aux.nome AS nome_do_aux
FROM
    Professor AS Prof,
    Auxiliar AS Aux
WHERE
    Prof.departamento = Aux.departamento AND
    Prof.departamento = 'Matemática' AND Aux.departamento = 'Matemática';
```

Não sei o que vocês pensam sobre o tema, mas a comunidade entendeu que misturar condições de junção com filtros após o WHERE se trata de uma solução bem pouco elegante.

Por isso, na revisão SQL-92, foi introduzida a sintaxe CROSS JOIN para indicar que o FROM se trata, na realidade, de um produto cartesiano.

Dessa forma, modernamente, podemos escrever de forma explícita que estamos aplicando um produto cartesiano entre tabelas.

```
SELECT
    Prof.nome AS nome_do_prof,
    Aux.nome AS nome_do_aux
FROM
    Professor AS Prof
CROSS JOIN
    Auxiliar AS Aux;
```

Do mesmo modo que na sintaxe anterior, aqui também podemos restringir os resultados utilizando o WHERE.

```
SELECT
    Prof.nome AS nome_do_prof,
    Aux.nome AS nome_do_aux
FROM
    Professor AS Prof
CROSS JOIN
    Auxiliar AS Aux
WHERE
    Prof.departamento = Aux.departamento,
    Prof.departamento = 'Matemática' AND Aux.departamento = 'Matemática';
```

Além disso, foi introduzido um novo tipo de comando, o JOIN, que nos permite trabalhar explicitamente com as condições de junção em diversos tipos distintos além da junção interna.

ORDER BY

Regra geral, o SGBD relacional retorna as linhas escolhidas pelo SELECT em uma ordem qualquer.

Porém, pode ser desejado ordenar esses registros usando algum critério, considerando alguns dos seus atributos. Para conseguirmos isso, utilizamos a cláusula ORDER BY.

Saibam que, por padrão, o **ORDER BY organiza os dados** em ordem crescente: se o atributo for numérico, será do menor para o maior, se for um texto, de A para Z, se for uma data, do mais antigo para o mais recente.

Vamos, então, retornar os nomes dos professores em ordem alfabética.

```
SELECT
    nome
FROM
    Professor
ORDER BY
    nome;
```

Professor					
CPF	RG	nome	idade	departamento	classe
999999999999	999	Alex	31	Matemática	Junior
888888888888	888	Alex	32	Português	Pleno
777777777777	777	Alexandre	25	Matemática	Senior
666666666666	666	Marcos	31	Português	Junior
555555555555	555	Barcos	32	Matemática	Pleno
444444444444	444	Carlos	25	Biologia	Senior

```
SELECT
    nome
FROM
    Professor
ORDER BY
    nome;
```

nome
Alex
Alex
Alexandre
Barcos
Carlos
Marcos

Embora a ordem crescente seja o padrão de comportamento, podemos deixar esse critério explícito, usando a palavra ASC (abreviação de ascending, que significa "ascendente" em inglês).

```
SELECT
    *
FROM
    Professor
ORDER BY
    nome
ASC;
```

Porém, se quisermos inverter esse comportamento, ou seja, ordenar do maior para o menor, de Z para A, podemos usar o ORDER BY com DESC (de descending, "descendente").

```
SELECT
    *
FROM
    Professor
ORDER BY
    nome
DESC;
```

Também podemos querer ordenar por mais de uma coluna ao mesmo tempo. Nesse caso, o ORDER BY organiza pelo primeiro atributo informado e, em caso de valores repetidos, organiza os valores do próximo atributo.

```
SELECT
    nome, idade
FROM
    Professor
ORDER BY
    nome, idade
DESC;
```

Professor					
CPF	RG	nome	idade	departamento	classe
999999999999	999	Alex	31	Matemática	Junior
888888888888	888	Alex	32	Português	Pleno
777777777777	777	Alexandre	25	Matemática	Senior
666666666666	666	Marcos	31	Português	Junior
555555555555	555	Barcos	32	Matemática	Pleno
444444444444	444	Carlos	25	Biologia	Senior

```
SELECT
    nome
FROM
    Professor
ORDER BY
    nome;
```

nome	idade
Alex	31
Alex	32
Alexandre	25
Barcos	32
Carlos	25
Marcos	31



Ao invés de listarmos pelo nome da coluna, poderíamos ter simplesmente usado números para representar quais colunas expressas no SELECT queremos ordenar.

Por exemplo, com `SELECT nome, departamento FROM Professor ORDER BY 2, 1;` queremos ordenar primeiro por "departamento" e depois "professor".

LIMIT, TOP e FETCH

Na prática, algumas tabelas de banco de dados possuem uma enorme quantidade de linhas, tornando inviável utilizarmos um SELECT que retorne todas elas (mesmo que com menos colunas).

Por isso, podemos utilizar cláusulas para limitar a quantidade de linhas retornadas por uma consulta. Assim, em vez de trazermos milhões de registros, limitamos os dados a até 10 linhas, 50, ou qualquer outra quantidade que faça sentido para o nosso caso prático.

A seguir queremos o top 3 de professores mais velhos, então ordenamos de forma descendente pela idade e **limitamos, com a cláusula LIMIT, a quantidade** de professores retornados em 3, vejamos.

```
SELECT
    nome,
    idade,
    departamento
FROM
    Professor
ORDER BY
    idade
DESC
LIMIT
    3;
```

Professor					
CPF	RG	nome	idade	departamento	classe
999999999999	999	Alex	31	Matemática	Junior
888888888888	888	Alex	32	Português	Pleno
777777777777	777	Alexandre	25	Matemática	Senior
666666666666	666	Marcos	31	Português	Junior
555555555555	555	Barcos	32	Matemática	Pleno
444444444444	444	Carlos	25	Biologia	Senior

```
SELECT
    nome,
    idade,
    departamento
FROM
    Professor
ORDER BY
    idade
DESC
LIMIT
    3;
```

nome	idade	departamento
Alex	32	Português
Barcos	32	Matemática
Alex	31	Matemática

Além disso, também podemos passar dois argumentos ao LIMIT.

Nessa configuração, o primeiro número representará um *offset* em relação ao início da contagem, ou seja, indica quantas linhas pular para obtermos o início do resultado (a posição do elemento de partida da contagem).

O segundo elemento funciona da forma com que já estudamos, já que indica a quantidade esperada de linhas que irão retornar a partir do início. Ou seja, é a quantidade máxima de elementos que queremos de fato

```
SELECT
    nome,
    departamento
FROM
    Professor
ORDER BY
    nome,
    idade DESC
LIMIT
    2, 3;
```

Do mesmo modo, podemos explicitamente estabelecer em uma sintaxe com OFFSET expresso.

```
SELECT
    nome,
    departamento
FROM
    Professor
ORDER BY
    nome,
    idade DESC
LIMIT 3 OFFSET 2;
```



(CEBRASPE (CESPE)/APC (FUNPRES-EXE)/2025) A execução do comando `SELECT * FROM PESSOA LIMIT 5, 10` apresentará 10 registros da tabela PESSOA, a partir do 6.º registro dessa tabela.

Comentários:

Quando o `LIMIT` recebe dois argumentos, o primeiro número é o `offset` (quantas linhas pular) e o segundo é a quantidade máxima de linhas a retornar. Então `LIMIT 5, 10` significa: pule os 5 primeiros registros e traga os 10 seguintes, ou seja, começa a contar a partir do 6.º registro e devolve até 10 linhas. É exatamente o que a afirmação descreve.

Gabarito: Certo

(CEBRASPE (CESPE)/APE (TCE-RS)/2025) Ao ser executado, o comando `SELECT * FROM PROCESSO LIMIT 10 OFFSET 5` apresenta cinco registros como resultado máximo, obtidos a partir do sexto registro da tabela.

Comentários:

A questão inverteu os números. No comando `LIMIT 10 OFFSET 5`, o número 10 é o limite máximo de registros que serão retornados, e o 5 é o `OFFSET`, ou seja, quantas linhas devem ser puladas antes de começar a contagem. Então o resultado correto seria até dez registros (e não cinco), começando a partir do sexto registro da tabela (porque pulamos os cinco primeiros). A parte sobre começar pelo sexto registro está certa, mas a quantidade máxima de registros retornados é dez, não cinco. Por isso o gabarito é Errado.

Gabarito: Errado

Infelizmente a sintaxe "`LIMIT`" para limitar resultados não é definida pelo padrão SQL, há fabricantes que utilizam outras sintaxes para esse mesmo comportamento.

Ou seja, enquanto alguns bancos utilizam o "`LIMIT`", outros preferem utilizar a cláusula "`TOP`" ou "`FETCH FIRST n ROWS ONLY`" (e também às vezes o mesmo banco permite utilizar mais de uma dessas cláusulas).

Vejamoss nossa consulta escrita de todas essas formas, lado a lado.

```
SELECT
    nome, departamento
FROM
    Professor
ORDER BY
    nome, idade
DESC
LIMIT
    3;
```

```
SELECT TOP 3
    nome, departamento
FROM
    Professor
ORDER BY
    nome, idade
DESC;
```

```
SELECT
    nome,
    departamento
FROM
    Professor
ORDER BY
    nome,
    idade DESC
FETCH FIRST 3 ROWS ONLY;
```

Tipos de Join

A partir do SQL-92, foi criado o comando JOIN que permite, de diversas formas, juntar várias tabelas em uma só.

INNER JOIN

Com o **INNER JOIN** (também chamado apenas de **JOIN**) conseguimos **juntar as linhas de duas tabelas a partir de um atributo correspondente entre elas**, de acordo com uma condição de junção definida.

Na prática, quando usamos o INNER JOIN entre duas tabelas, o banco procura, para cada linha da primeira tabela, todas as linhas na outra tabela que obedeçam à condição de junção estabelecida.

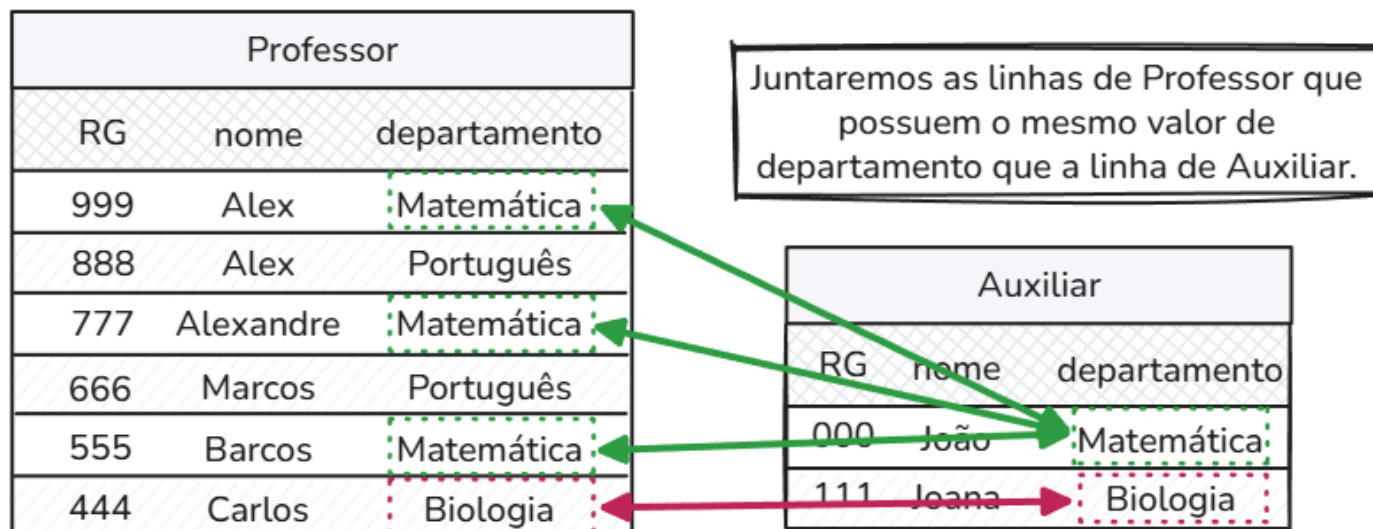
Por exemplo, podemos juntar duas tabelas Professor e Auxiliar estabelecendo como condição de junção que o nome do departamento de ambos seja igual, ou seja, que tanto o professor quanto o auxiliar estejam alocados no mesmo departamento.

Professor		
RG	nome	departamento
999	Alex	Matemática
888	Alex	Português
777	Alexandre	Matemática
666	Marcos	Português
555	Barcos	Matemática
444	Carlos	Biologia

No INNER JOIN juntaremos as linhas das duas tabelas a partir de um atributo em comum

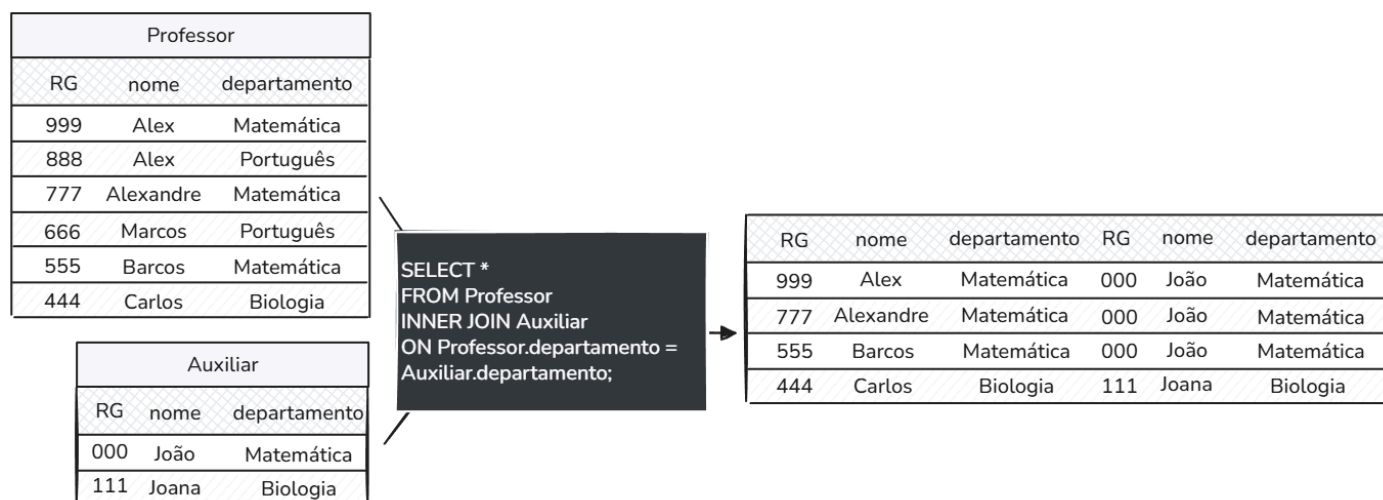
Auxiliar		
RG	nome	departamento
000	João	Matemática
111	Joana	Biologia

Nessa situação, o banco de dados olhará para cada registro da tabela Professor se há um ou mais registros na tabela Auxiliar com o mesmo nome de departamento. Se houver, juntará ambos como sendo um registro único. A seguir, vemos essa lógica para apenas o primeiro registro da tabela Professor.



Em SQL, usamos o FROM com a tabela de referência seguida da palavra-chave JOIN (opcionalmente precedida por INNER) com a outra tabela a ser juntada. Na sequência, usamos o ON para indicar a condição de junção.

```
SELECT *
FROM Professor
INNER JOIN Auxiliar
ON Professor.departamento = Auxiliar.departamento;
```





Valores NULL nunca combinam com nada, nem mesmo com outros NULLs.

Então, se houver uma linha com departamento NULL, ela não aparecerá no resultado do INNER JOIN se usado operadores de comparação (como "=" ou "<>").

Isso porque o banco simplesmente não consegue verificar se há igualdade ou desigualdade quando há valor ausente (o banco retorna "UNKWNOWN", como se fosse um "não sei").



(IBFC/Ana Tec (SAEB BA)/2023) Assinale a alternativa que apresenta a estrutura da linguagem SQL (Structure query Language), o argumento que retorna as informações de duas ou mais tabelas que estão relacionadas, e, que tem correspondência de chave entre elas.

- a) Full join
- b) Left join
- c) Outer join
- d) Inner join

Comentários:

a) Errada. O Full join preserva todas as linhas das duas tabelas, mesmo quando não existe correspondência, preenchendo com NULL os lados sem par. Ou seja, ele vai além de só retornar os registros relacionados.

b) Errada. O Left join preserva todas as linhas da tabela da esquerda, inclusive aquelas que não possuem correspondência na tabela da direita, completando com NULL quando não há par.

c) Errada. Outer join é o nome genérico que abrange Left, Right e Full join, justamente os tipos de junção que mantêm linhas sem correspondência. Não é o tipo que retorna apenas registros relacionados por chave.

d) **Correta.** O Inner join junta as linhas de duas tabelas a partir de uma condição de junção (geralmente uma correspondência de chave entre elas), retornando apenas os registros que possuem par nas duas tabelas.

Gabarito: D

NATURAL JOIN

O **NATURAL JOIN** se trata de um caso específico de INNER JOIN que não precisamos explicitar a condição de junção, ou seja, quais atributos e as regras que serão utilizados para juntar as linhas da tabela.

Isso porque nessa operação o **banco de dados descobre sozinho quais colunas devem ser usadas para a junção a partir da comparação entre seus nomes.**

Em outras palavras, aqui a regra é automática: combina-se as linhas das duas tabelas sempre que os valores das colunas com o mesmo nome forem iguais.

No nosso caso de Professor e Auxiliar, como temos dois atributos iguais, "nome" e "departamento", o natural join buscará as linhas das tabelas com o mesmo "nome" e mesmo "departamento".

```
SELECT *  
FROM Professor  
NATURAL JOIN Auxiliar;
```



(COSEAC UFF/ATI (UFF)/2019) Na linguagem SQL*Plus, o comando específico para consulta em que é possível visualizar dados provenientes apenas de duas tabelas, as quais possuem um campo em comum com o mesmo nome, tipo e tamanho, é:

- a) Create.
- b) Natural Join.
- c) Self Join.
- d) Merge.
- e) Insert.

Comentários:

a) **Errada.** O Create serve para criar objetos no banco, como tabelas, e não para consultar dados.

- b) **Correta.** O Natural Join junta automaticamente duas tabelas usando as colunas que possuem o mesmo nome, sem precisar especificar a condição de junção.
- c) **Errada.** Esse tipo de junção não foi tratado como o que combina automaticamente colunas de mesmo nome entre duas tabelas distintas.
- d) **Errada.** O Merge não é o comando usado para consultar dados a partir de colunas comuns entre duas tabelas.
- e) **Errada.** O Insert serve para inserir novos registros em uma tabela, e não para consultar dados.

Gabarito: B

(FUNDATEC/ANC (PROCERGS)/2023) Na especificação de uma consulta SQL, é necessário especificar as tabelas que serão utilizadas para realizar a consulta e como estas devem ser ligadas. Isso é feito na cláusula FROM do SELECT através das alternativas de junções (JOIN). Assinale a alternativa correta sobre o uso dos JOINS.

- a) O OUTER JOIN não contém as tuplas que o INNER JOIN retorna.
- b) O NATURAL JOIN é um tipo de INNER JOIN.
- c) A intersecção dos resultados do LEFT JOIN e do RIGHT JOIN é vazia.
- d) O uso do INNER JOIN é muito restrito, pois não relaciona chave estrangeira com chave primária.
- e) O CROSS JOIN é o tipo default de JOIN, quando se especifica apenas JOIN em uma consulta, o banco de dados executará um CROSS JOIN.

Comentários:

- a) **Errada.** O OUTER JOIN inclui sim as tuplas do INNER JOIN (linhas com correspondência) e ainda adiciona as linhas sem correspondência preenchidas com NULL.
- b) **Correta.** O NATURAL JOIN é um caso específico de INNER JOIN, em que a condição de junção é descoberta automaticamente pelo banco a partir das colunas com o mesmo nome nas duas tabelas.
- c) **Errada.** As linhas com correspondência entre as duas tabelas aparecem tanto no LEFT JOIN quanto no RIGHT JOIN, então a interseção entre os resultados não é vazia.
- d) **Errada.** O INNER JOIN é justamente muito usado para relacionar chave estrangeira com chave primária, bastando definir a condição de junção entre esses atributos.
- e) **Errada.** Não existe um tipo "default" quando se escreve apenas JOIN; nesse caso, o banco entende como INNER JOIN, e não como CROSS JOIN. O CROSS JOIN realiza o produto cartesiano entre as tabelas.

Gabarito: B

(INSTITUTO ÁGUA/ASis I (CEAGESP)/2018) No MySQL, se eu quiser uma lista contendo o nome de alunos com alguma nota maior que 7, sem repetições, devo usar:

- a) `SELECT DISTINCT nomeAluno FROM Aluno NATURAL JOIN Nota WHERE Nota.nota>7.`
- b) `SELECT nomeAluno FROM Aluno NATURAL JOIN Nota WHERE Nota.nota>7.`
- c) `SELECT nomeAluno FROM Aluno NATURAL JOIN Nota WHERE Nota.nota>7 VING Aluno.nomeAluno.`
- d) `SELECT UNIQUE nomeAluno FROM Aluno NATURAL JOIN Nota WHERE Nota.nota>7.`

Comentários:

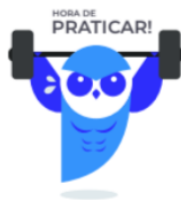
- a) **Correta.** Para eliminar repetições no resultado de um `SELECT` usamos a palavra-chave `DISTINCT`, e o `NATURAL JOIN` faz a junção automática entre `Aluno` e `Nota` pelas colunas de mesmo nome, com o filtro `nota>7` no `WHERE`.
- b) **Errada.** A consulta funciona, mas não elimina nomes repetidos, pois falta o `DISTINCT`. Se um aluno tiver várias notas maiores que 7, o nome dele apareceria várias vezes.
- c) **Errada.** Não existe a cláusula "`VING`" no SQL. Além disso, para filtrar grupos usaríamos `HAVING`, e mesmo assim ela não eliminaria duplicatas como exigido pelo enunciado.
- d) **Errada.** A palavra-chave usada para evitar resultados duplicados em SQL é `DISTINCT`, e não `UNIQUE`. O `UNIQUE` existe em SQL, mas como restrição de coluna na criação de tabelas, não dentro de um `SELECT`.

Gabarito: A

OUTER JOIN

Com o **OUTER JOIN** (junção externa) temos um comportamento diferente, porque aqui **preservamos as linhas mesmo quando não exista correspondência na outra tabela**. Assim, nenhum dado será descartado só porque faltou um par correspondente.

Assim sendo, o `OUTER JOIN` se divide em três variações, dependendo de qual tabela tem suas linhas preservadas: `LEFT OUTER JOIN` (ou apenas `LEFT JOIN`), `RIGHT OUTER JOIN` (ou apenas `RIGHT JOIN`) e `FULL OUTER JOIN` (ou apenas `FULL JOIN`).



(ADAPTADA - CETREDE/Ana (Pref Caucaia)/2024) Em um Outer Join existe a possibilidade de que algumas das linhas de uma ou ambas as tabelas de uma junção não façam parte da tabela resultante.

Comentários:

A ideia central da junção externa é preservar as linhas mesmo quando não existe correspondência na outra tabela, ou seja, nenhum dado é descartado só porque faltou um par

correspondente. Quando uma linha não tem par, ela continua aparecendo no resultado, com as colunas da outra tabela preenchidas com NULL. No LEFT JOIN preservamos todas as linhas da esquerda, no RIGHT JOIN todas as da direita, e no FULL JOIN todas das duas tabelas. Portanto, não existe possibilidade de uma linha "ficar de fora" no OUTER JOIN, exatamente o oposto do que a questão afirma.

Gabarito: Errada

LEFT JOIN

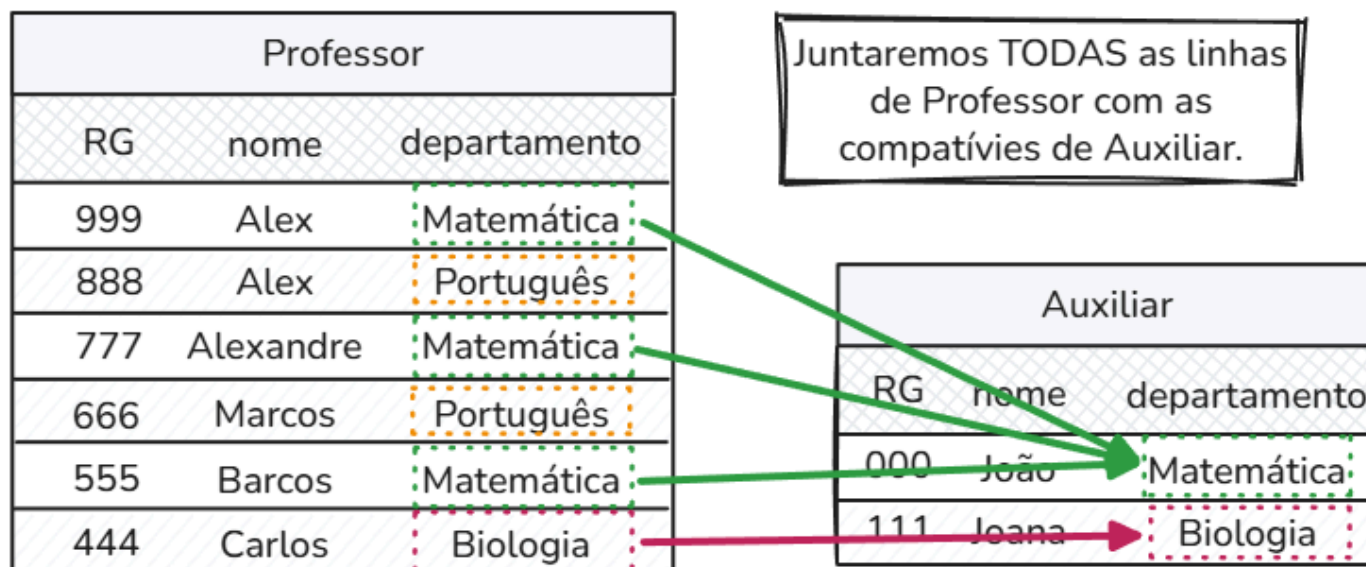
No **LEFT JOIN** preservamos todas as linhas da tabela da esquerda (a que aparece antes do JOIN), mesmo que **não** possua um par na direita.



Aqui usamos os termos "esquerda" e "direita" como recurso didático para nos referir aos lados da palavra JOIN do SQL. Por exemplo, em FROM Professor LEFT JOIN Auxiliar, a tabela Professor está à esquerda do JOIN e Auxiliar à direita.

Assim, se existir correspondência, o comportamento será idêntico ao INNER JOIN: as colunas das duas tabelas são combinadas em uma única linha do resultado.

Porém, se não houver correspondência, em vez de descartar a linha da esquerda (como faria o INNER JOIN), o LEFT JOIN mantém essa linha no resultado e preenche as colunas da tabela da direita com NULL.



Em outras palavras, com o LEFT JOIN temos uma nova tabela com todos os resultados que o INNER JOIN devolveria (as linhas com correspondência) e também todas as linhas da tabela da esquerda que não tinham par, com NULL nas colunas da direita.

```
SELECT *  
FROM Professor  
LEFT JOIN Auxiliar  
ON Professor.departamento = Auxiliar.departamento;
```

Professor		
RG	nome	departamento
999	Alex	Matemática
888	Alex	Português
777	Alexandre	Matemática
666	Marcos	Português
555	Barcos	Matemática
444	Carlos	Biologia

Auxiliar		
RG	nome	departamento
000	João	Matemática
111	Joana	Biologia

```
SELECT *  
FROM Professor  
LEFT JOIN Auxiliar  
ON Professor.departamento =  
Auxiliar.departamento;
```

RG	nome	departamento	RG	nome	departamento
999	Alex	Matemática	000	João	Matemática
888	Alex	Português	Null	Null	Null
777	Alexandre	Matemática	000	João	Matemática
666	Marcos	Português	Null	Null	Null
555	Barcos	Matemática	000	João	Matemática
444	Carlos	Biologia	111	Joana	Biologia



(FEPESE/Ana Sis (CELESC)/2014) Suponha uma consulta em PL/SQL envolvendo duas relações Empregado e Departamento. Há uma chave estrangeira em Empregado, que indica o departamento deste empregado; contudo ela não é NOT NULL. Em alguns momentos, podem existir departamentos sem nenhum empregado lotado.

Assinale a alternativa com a junção correta que deve ser utilizada em uma query que relacione as duas relações (Empregado junção com Departamento), de modo que o resultado retorne todos os empregados e somente os departamentos que possuem ao menos um empregado lotado.

- a) JOIN
- b) INNER JOIN
- c) OUTER JOIN
- d) RIGHT JOIN
- e) LEFT JOIN

Comentários:

- a) **Errada.** JOIN sozinho equivale ao INNER JOIN, que retornaria apenas empregados que possuem departamento correspondente, descartando aqueles cuja chave estrangeira está nula. Não atende ao requisito de trazer todos os empregados.
- b) **Errada.** O INNER JOIN só traz as linhas que têm correspondência nas duas tabelas. Empregados sem departamento (chave estrangeira nula) ficariam de fora, então não cumpre o pedido de retornar todos os empregados.
- c) **Errada.** OUTER JOIN sozinho não é uma sintaxe específica; ele se divide em LEFT, RIGHT e FULL. Sem indicar qual lado preservar, não conseguimos definir corretamente o comportamento desejado.
- d) **Errada.** O RIGHT JOIN preservaria todas as linhas da tabela à direita (Departamento), trazendo inclusive departamentos sem empregados. Isso é o oposto do que a questão pede, pois só queremos departamentos que tenham pelo menos um empregado.
- e) **Correta.** Colocando Empregado à esquerda e Departamento à direita, o LEFT JOIN preserva todos os empregados (mesmo os que estão com a chave estrangeira nula, sem departamento) e traz apenas os departamentos que têm correspondência com algum empregado, ou seja, que possuem pelo menos um empregado lotado.

Gabarito: E

(IBGP/Tec (CM Porto Velho)/2026) Uma consulta precisa listar processos e, quando existir, o servidor atual responsável. Há processos ainda não atribuídos. A equipe quer garantir que todos os processos apareçam no resultado, inclusive os sem responsável.

Qual junção é CORRETA para atender ao requisito?

- a) LEFT JOIN de Processo para ServidorResponsavel.
- b) INNER JOIN, pois inclui registros sem correspondência.
- c) CROSS JOIN, pois preserva linhas sem responsável.
- d) RIGHT JOIN do Servidor para Processo, pois elimina nulos.
- e) NATURAL JOIN, pois força correspondência.

Comentários:

- a) **Correta.** No LEFT JOIN, todas as linhas da tabela da esquerda (Processo) são preservadas, mesmo quando não houver correspondência na tabela da direita (ServidorResponsavel); nesses casos, as colunas do servidor virão como NULL, garantindo que processos sem responsável apareçam no resultado.
- b) **Errada.** O INNER JOIN retorna apenas linhas com correspondência em ambas as tabelas, então processos sem responsável seriam descartados.
- c) **Errada.** O CROSS JOIN faz produto cartesiano, combinando cada processo com todos os servidores, o que não atende ao requisito de listar o servidor responsável quando existir.

d) **Errada.** O RIGHT JOIN preservaria todas as linhas de Servidor, podendo descartar processos sem responsável; além disso, o JOIN não "elimina nulos", ele os gera quando não há correspondência.

e) **Errada.** O NATURAL JOIN é um tipo de INNER JOIN que junta automaticamente por colunas de mesmo nome, mas continua descartando linhas sem correspondência, deixando de fora os processos não atribuídos.

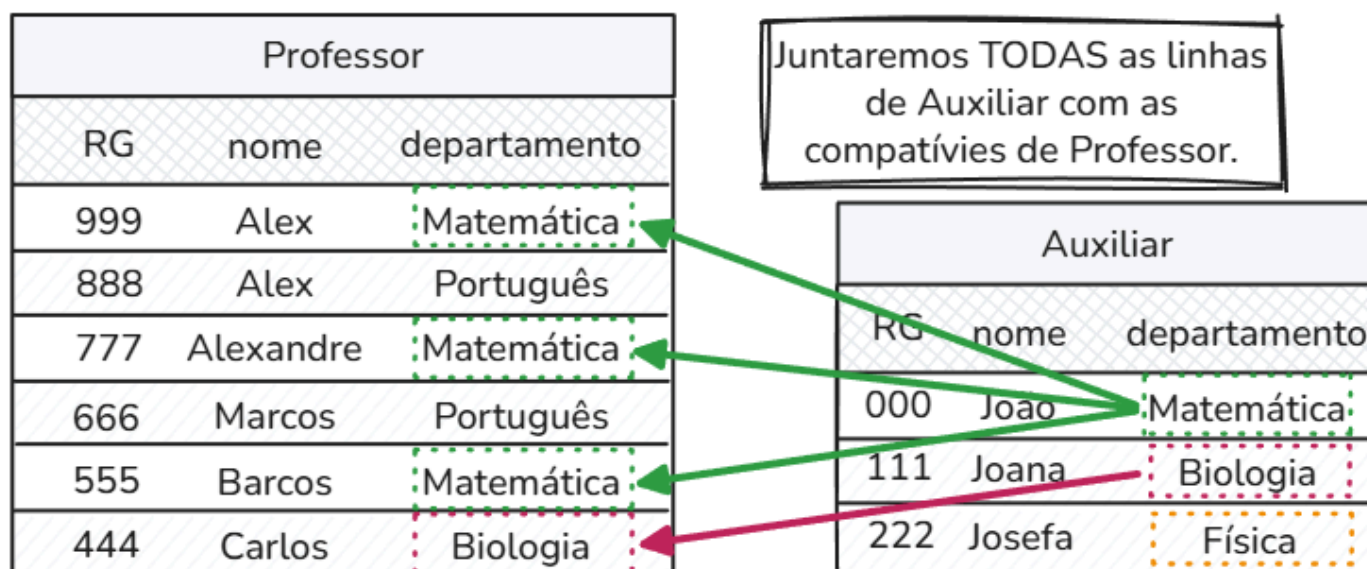
Gabarito: A

RIGHT JOIN

O RIGHT JOIN têm comportamento "invertido" do LEFT JOIN: enquanto no LEFT JOIN preservávamos todas as linhas da tabela da esquerda, no **RIGHT JOIN preservamos todas as linhas da tabela da direita.**

Dessa forma, cada linha da tabela da direita aparecerá no resultado pelo menos uma vez, mesmo que não exista correspondência na tabela da esquerda que cumpra com a condição de junção. Nesse caso de não haver correspondência, as colunas da tabela da esquerda são preenchidas com NULL.

Por exemplo, podemos juntar as tabelas Professor e Auxiliar por RIGHT JOIN estabelecendo como condição de junção que o nome do departamento de ambas seja igual, ou seja, que tanto o professor quanto o auxiliar estejam alocados no mesmo departamento.



Nesse caso, o banco de dados verificará, para cada registro de Auxiliar, se há registros em Professor com o mesmo departamento. Se houver, junta-se todos os registros de Professor ao registro de Auxiliar, criando-se múltiplas linhas na tabela conjunta. Se não houver, cria-se uma única linha com os valores da tabela da esquerda ausentes (NULL).

```
SELECT *
FROM Professor
RIGHT JOIN Auxiliar
ON Professor.departamento = Auxiliar.departamento;
```

Professor		
RG	nome	departamento
999	Alex	Matemática
888	Alex	Português
777	Alexandre	Matemática
666	Marcos	Português
555	Barcos	Matemática
444	Carlos	Biologia

Auxiliar		
RG	nome	departamento
000	João	Matemática
111	Joana	Biologia
222	Josefa	Física

```
SELECT *
FROM Professor
RIGHT JOIN Auxiliar
ON Professor.departamento =
Auxiliar.departamento;
```

RG	nome	departamento	RG	nome	departamento
999	Alex	Matemática	000	João	Matemática
888	Alex	Português	000	João	Matemática
777	Alexandre	Matemática	000	João	Matemática
444	Carlos	Biologia	111	Joana	Biologia
Null	Null	Null	222	Josefa	Física



(FUNDATEC/AnaTI (IF Sertão PE)/2025) bancos de dados relacionais, a cláusula JOIN é utilizada para combinar registros de duas ou mais tabelas com base em uma coluna comum entre elas. Em relação aos tipos INNER JOIN, LEFT JOIN, RIGHT JOIN e CROSS JOIN no MySQL, analise as assertivas abaixo, assinalando V, se verdadeiras, ou F, se falsas.

- () O comando INNER JOIN retorna apenas os registros que possuem correspondência em ambas as tabelas, conforme a condição estabelecida.
- () O LEFT JOIN garante a exibição de todos os registros da tabela à esquerda, mesmo que não haja correspondência na tabela à direita.
- () A operação CROSS JOIN pode ser usada com cláusula ON para limitar o número de combinações geradas, funcionando de forma semelhante a um INNER JOIN.
- () O RIGHT JOIN produz os mesmos resultados que um LEFT JOIN, desde que as posições das tabelas sejam invertidas corretamente.
- () Quando não há correspondência no RIGHT JOIN, os valores da tabela à esquerda serão substituídos por valores padrão (por exemplo, zero), e não por NULL.

A ordem correta de preenchimento dos parênteses, de cima para baixo, é:

- a) V – F – F – V – V.
- b) F – V – V – F – F.
- c) V – V – F – V – F.

d) F – V – V – F – V.

e) V – F – V – V – F.

Comentários:

I. **Correta.** Definição exata do INNER JOIN: só sobrevivem as linhas que satisfazem a condição em ambos os lados;

II. **Correta.** As colunas da tabela à direita vêm preenchidas com NULL quando não há match

III. **Errada.** CROSS JOIN não aceita cláusula ON. Ele produz o produto cartesiano puro (toda linha de A combinada com toda linha de B). Para filtrar, usa-se WHERE, ou então troca-se por INNER JOIN. Se você escrever CROSS JOIN ... ON, é erro de sintaxe na maioria dos SGBDs

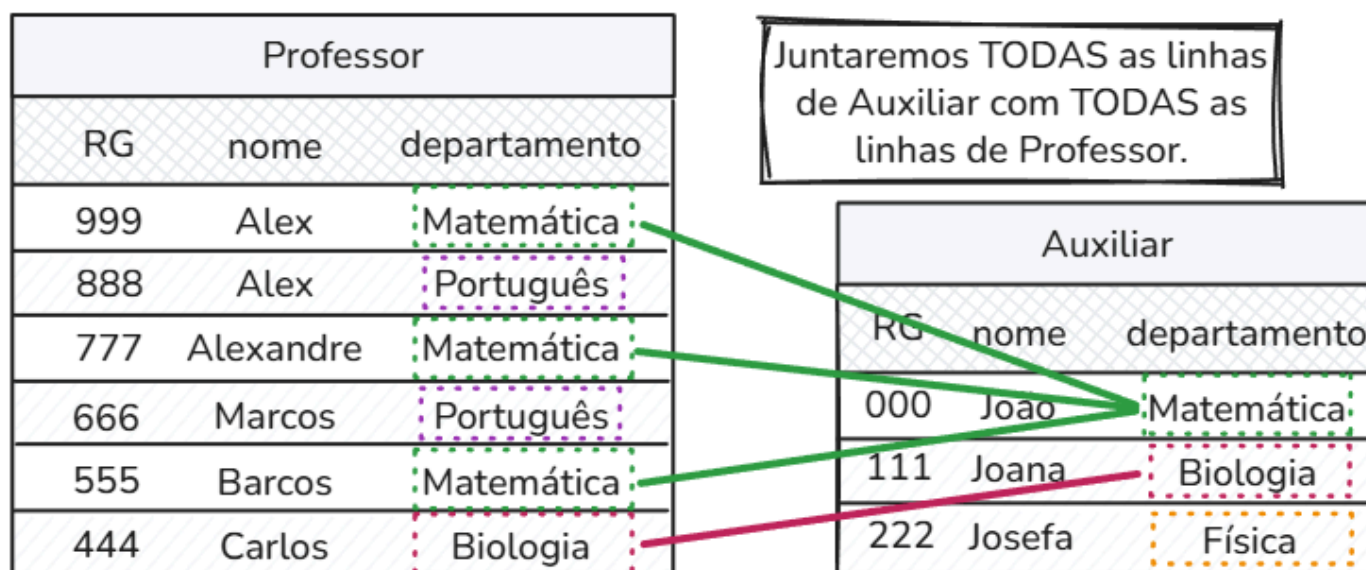
IV. **Correta.** A LEFT JOIN B e B RIGHT JOIN A são operações simétricas.

V. **Errada.** São substituídos por NULL, não por zero ou qualquer outro valor padrão. SQL nunca "inventa" valores em joins externos

Gabarito: C

FULL JOIN

O FULL JOIN combina o efeito de aplicar ao mesmo tempo um LEFT JOIN e um RIGHT JOIN. Em outras palavras, ele preserva todas as linhas das duas tabelas, independentemente de existir correspondência entre elas.



Para conseguir fazer isso, basta verificar se cada linha da tabela da esquerda possui algum correspondente na da direita e, se não houver, a complementa com NULL.

Na sequência, para as linhas da tabela da direita que não foram juntadas, o banco as complementa com um null.

Com isso, o FULL JOIN é capaz de retornar todas as linhas com correspondência mais todas as linhas órfãs da tabela da esquerda, com NULL à direita (como o LEFT JOIN adiciona) mais todas as linhas órfãs da tabela da direita, com NULL à esquerda (como o RIGHT JOIN adiciona).

```
SELECT *
FROM Professor
FULL JOIN Auxiliar
ON Professor.departamento = Auxiliar.departamento;
```

Professor		
RG	nome	departamento
999	Alex	Matemática
888	Alex	Português
777	Alexandre	Matemática
666	Marcos	Português
555	Barcos	Matemática
444	Carlos	Biologia

Auxiliar		
RG	nome	departamento
000	João	Matemática
111	Joana	Biologia
222	Josefa	Física

```
SELECT *
FROM Professor
FULL JOIN Auxiliar
ON Professor.departamento =
Auxiliar.departamento;
```

RG	nome	departamento	RG	nome	departamento
999	Alex	Matemática	000	João	Matemática
888	Alex	Português	000	João	Matemática
777	Alexandre	Matemática	000	João	Matemática
444	Carlos	Biologia	111	Joana	Biologia
888	Alex	Português	Null	Null	Null
666	Marcos	Português	Null	Null	Null
Null	Null	Null	222	Josefa	Física



(UNESC/Prof (Pref Meleiro)/2026) A linguagem SQL (Structured Query Language) é o padrão para manipulação de dados em Bancos de Dados Relacionais, permitindo a extração de informações combinadas de múltiplas tabelas através de junções. Sobre os tipos de junção (JOIN), registre V, para as afirmativas verdadeiras, e F, para as falsas:

- () O comando INNER JOIN seleciona registros que possuem valores correspondentes em ambas as tabelas envolvidas na consulta.
- () O comando LEFT JOIN (ou LEFT OUTER JOIN) retorna todos os registros da tabela da esquerda e os registros correspondentes da tabela da direita, preenchendo com NULL onde não houver correspondência.
- () O comando FULL OUTER JOIN retorna apenas os registros que não possuem correspondência em nenhuma das tabelas, excluindo as interseções de dados.
- () O comando CROSS JOIN realiza o produto cartesiano entre duas tabelas, combinando cada linha da primeira tabela com todas as linhas da segunda, exigindo obrigatoriamente a cláusula ON.

Assinale a alternativa que apresenta a sequência correta, de cima para baixo.

- a) V, V, V, F.
- b) V, F, F, V.
- c) F, V, V, F.
- d) V, V, F, F.
- e) F, F, V, V.

Comentários:

I. Certa. Essa é exatamente a definição do INNER JOIN: ele retorna somente as linhas em que a condição de junção (geralmente expressa no ON) é satisfeita nas duas tabelas. Linhas sem correspondência em qualquer um dos lados são descartadas do resultado.

II. Certa. Descrição precisa do comportamento do LEFT OUTER JOIN: a tabela à esquerda é preservada integralmente, e onde não há linha correspondente na tabela à direita, as colunas dela aparecem como NULL no resultado.

III. Errada. É justamente o contrário: o FULL OUTER JOIN retorna todos os registros de ambas as tabelas, incluindo os que têm correspondência (interseção) e também os que não têm (preenchendo com NULL nos lados sem match).

IV. Errada. A primeira parte está certa (CROSS JOIN combina cada linha da primeira tabela com todas as linhas da segunda, gerando o produto cartesiano), mas a segunda parte invalida a afirmação: o CROSS JOIN não aceita cláusula ON.

Gabarito: D

(FGV/Tec NS (TJ MS)/TJ MS/Analista de Sistemas Computacionais/Analista de Banco de Dados/2024) João está escrevendo uma consulta que envolve várias tabelas e precisa garantir que todas as suas linhas sejam incluídas no resultado, mesmo que não haja correspondências entre elas.

Para tanto, João deverá utilizar o seguinte operador de junção:

- a) LEFT JOIN;
- b) INNER JOIN;
- c) RIGHT JOIN;
- d) CROSS JOIN;
- e) FULL OUTER JOIN.

Comentários:

a) Errada. O LEFT JOIN preserva apenas as linhas da tabela da esquerda, descartando as da direita que não tenham correspondência.

b) Errada. O INNER JOIN só traz as linhas que possuem correspondência em ambas as tabelas, descartando as demais.

c) Errada. O RIGHT JOIN preserva apenas as linhas da tabela da direita, descartando as da esquerda que não tenham correspondência.

d) **Errada.** O CROSS JOIN realiza o produto cartesiano, combinando cada linha de uma tabela com todas as linhas da outra, e não trata de preservar linhas sem correspondência segundo uma condição de junção.

e) **Correta.** O FULL OUTER JOIN combina o efeito do LEFT JOIN com o RIGHT JOIN, preservando todas as linhas de ambas as tabelas, mesmo quando não há correspondência entre elas, completando com NULL os valores ausentes.

Gabarito: E

Funções de Agregação e Agrupamento

Funções de Agregação

As funções de agregação nos permitem extrair propriedades de conjuntos de linhas que o SELECT retorna.

Assim sendo, com esse recurso podemos encontrar coisas como a média dos valores de determinado atributo, a soma deles e a contagem total de linhas.

Todas as funções de agregação possuem o mesmo comportamento: agregamos os registros em apenas 1 valor de resposta. Por isso, iremos estudar apenas uma função de agregação, a MAX, e depois veremos a funcionalidade das outras através de uma tabela resumo.

Pois bem, basicamente a função MAX retorna o maior de uma coluna. Então, vamos aplicar essa função no atributo "idade" para encontrar a maior idade dentre todos professores.

```
SELECT MAX(idade) FROM Professor; -- retorna a idade do professor mais velho
```

Professor		
nome	idade	departamento
Alex	31	Matemática
Alex	33	Português
Alexandre	25	Matemática
Marcos	31	Português
Barcos	32	Matemática
Carlos	25	Biologia

```
SELECT  
MAX(idade)  
FROM  
Professor;
```

MAX(idade)
33

Função de Agregação	Explicação	Detalhes	SQL
AVG	Calcula a média aritmética dos valores de uma coluna numérica. Soma todos os valores e divide pela quantidade de linhas.	Ignora valores NULL automaticamente. Se 10 de 100 funcionários têm salário NULL, a média será calculada sobre os 90 restantes, não sobre os 100.	<pre>SELECT AVG(salario) FROM funcionarios;</pre>
COUNT(*)	Conta todas as linhas da tabela, sem exceção.	Retorna o total de registros independentemente de valores NULL em qualquer coluna. Se há 100 funcionários cadastrados, o resultado é 100.	<pre>SELECT COUNT(*) FROM funcionarios;</pre>
COUNT(coluna)	Conta apenas as linhas em que a coluna especificada não é NULL.	Útil para saber quantos registros têm um dado específico preenchido. Se dos 100 funcionários apenas 80 têm email cadastrado, o resultado é 80.	<pre>SELECT COUNT(email) FROM funcionarios;</pre>
COUNT(DISTINCT coluna)	Conta quantos valores únicos (distintos) aparecem na coluna.	Muito útil para responder perguntas como 'quantos clientes diferentes compraram?' ou 'em quantas categorias temos produtos?'. Se 100 funcionários moram em 7 cidades, retorna 7.	<pre>SELECT COUNT(DISTINCT cidade) FROM funcionarios;</pre>
SUM	Calcula a soma de todos os valores de uma coluna numérica.	Só funciona com números. Ignora valores NULL. Linhas com valor NULL simplesmente não entram na conta.	<pre>SELECT SUM(valor) FROM vendas;</pre>

Função de Agregação	Explicação	Detalhes	SQL
MAX	Retorna o maior valor de uma coluna.	Funciona com números, textos e datas. Em textos, o 'maior' é o que viria por último em ordem alfabética. Em datas, é a data mais recente.	<pre>SELECT MAX(salario) FROM funcionarios;</pre>
MIN	Retorna o menor valor de uma coluna.	Funciona com números, textos e datas. Em textos, o 'menor' é o primeiro em ordem alfabética. Em datas, é a data mais antiga.	<pre>SELECT MIN(salario) FROM funcionarios;</pre>
STDDEV	Função estatística que calcula o desvio padrão. Mede quão dispersos estão os valores em relação à média.	A disponibilidade varia entre bancos: PostgreSQL e Oracle suportam diretamente, SQL Server usa STDEV.	<pre>SELECT STDDEV(nota) FROM alunos;</pre>

Mas cuidado, porque, pelo comportamento peculiar das funções de agregação, elas não podem ser usadas indiscriminadamente.

Por exemplo, se misturarmos colunas comuns com funções de agregação num SELECT, o banco não conseguirá executar o SQL e retornará um erro. Assim, a consulta a seguir está errada:

```
SELECT nome, MAX(idade) FROM Professor; -- SGBD retorna um ERRO!
```

Isso acontece porque é impossível encaixarmos na mesma tabela todas suas linhas em uma coluna e o único valor retornado pela função de agregação na coluna seguinte.

Ou seja, trata-se de um problema lógico: MAX(idade) produz um único valor agregado, mas a coluna "nome" tem valores diferentes (um por professor). Assim, o banco não consegue combinar essas informações incompatíveis e devolve um erro.



(INSTITUTO ACESSO/Ana Leg (CM Manaus)/2024) O SQL (Structured Query Language) é a linguagem padrão utilizada para manipulação de bancos de dados relacionais. Apesar de ser

padronizado, várias implementações de bancos de dados possuem variações, e o conhecimento dessas variações pode influenciar a escolha de comandos e funções.

Considerando os padrões de SQL aceitos no mercado, qual das alternativas abaixo está correta?

- a) O FULL OUTER JOIN retorna apenas os registros que têm correspondências em ambas as tabelas envolvidas na consulta.
- b) A cláusula WHERE pode ser usada após a cláusula GROUP BY para filtrar os dados agregados.
- c) A função AVG() retorna a média aritmética, ignorando todos os valores nulos de uma coluna, sem a necessidade de filtros adicionais.
- d) A função COUNT(*) retorna o número total de linhas, incluindo as linhas que possuem valores nulos.
- e) A função COUNT() pode ser usada com a cláusula GROUP BY para retornar a soma de valores de uma coluna em grupos definidos.

Comentários:

- a) **Errada.** O FULL OUTER JOIN preserva todas as linhas das duas tabelas, mesmo quando não há correspondência, preenchendo com NULL os campos sem par. Quem retorna apenas os registros com correspondência em ambas é o INNER JOIN.
- b) **Errada.** Para filtrar dados agregados após o GROUP BY usamos a cláusula HAVING, e não o WHERE. O WHERE filtra linhas antes do agrupamento.
- c) **Errada.** De fato a AVG ignora valores nulos automaticamente, mas a alternativa erra ao dizer "todos os valores nulos de uma coluna" como se isso fosse a regra completa: a média é calculada apenas sobre as linhas com valor preenchido, e isso não é exatamente "sem necessidade de filtros adicionais", pois pode ser necessário tratar esses casos a depender da análise.
- d) **Correta.** O COUNT(*) conta todas as linhas da tabela sem exceção, incluindo aquelas que possuem valores nulos em alguma coluna, pois ele não olha para o conteúdo das colunas, apenas para a existência da linha.
- e) **Errada.** O COUNT serve para contar linhas, e não para somar valores. Quem soma valores de uma coluna é a função SUM.

Gabarito: D

(FGV/ACE (TCE RR)/TCE RR/Tecnologia da Informação/Análise de Dados/2025) Na linguagem SQL, funções de agregação são utilizadas para resumir informações de várias tuplas em uma síntese de tupla única.

As opções a seguir apresentam exemplos de funções de agregação, à exceção de uma. Assinale-a.

- a) SUM.
- b) MIN.
- c) AVG.
- d) COUNT.

e) HAVING.

Comentários:

a) **Errada.** SUM é uma função de agregação que calcula a soma de todos os valores de uma coluna numérica.

b) **Errada.** MIN é uma função de agregação que retorna o menor valor de uma coluna.

c) **Errada.** AVG é uma função de agregação que calcula a média aritmética dos valores de uma coluna numérica.

d) **Errada.** COUNT é uma função de agregação que conta as linhas, podendo contar todas (COUNT(*)) ou apenas as não nulas de uma coluna.

e) **Correta.** HAVING não é uma função de agregação, mas sim uma cláusula usada para filtrar grupos formados pelo GROUP BY, geralmente aplicando condições sobre funções de agregação.

Gabarito: E

Agrupamento (GROUP BY)

A cláusula GROUP BY permite que todas as linhas que compartilham o mesmo valor nas colunas selecionadas sejam agrupadas juntas em uma única linha só.

Por exemplo, podemos agrupar os registros de Professor de acordo com o departamento deles

```
SELECT departamento FROM Professor GROUP BY departamento;
```

A grande vantagem do GROUP BY é poder aplicar a função de agregação várias vezes, uma para cada grupo obtido.

Dessa forma, as funções de agregação são aplicadas dentro de cada grupo formado pelo GROUP BY separadamente, e o resultado final da consulta terá uma linha para cada grupo.

Por exemplo, vamos obter a idade média dos professores por departamento.

```
SELECT departamento, AVG(idade) FROM Professor GROUP BY departamento;
```

Professor		
nome	idade	departamento
Alex	31	Matemática
Alex	33	Português
Alexandre	25	Matemática
Marcos	31	Português
Barcos	32	Matemática
Carlos	25	Biologia

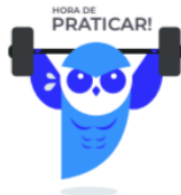
```
SELECT
departamento,
AVG(idade)
FROM
Professor
GROUP BY
departamento;
```

departamento	idade
Matemática	29,33
Português	32
Biologia	25

Vale saber que também podemos agrupar por mais de uma coluna de uma vez. Quando fazemos isso, o banco entende que cada grupo é composto pela combinação dos valores de cada coluna, ou seja, agrupa as linhas que coincidem em todas as colunas listadas.

Essa situação é típica de consultas mais complexas, como, por exemplo, quando temos uma tabela de valores de vendas por vendedor por mês em uma loja e queremos saber na média o quanto cada um vendeu em um mesmo mês.

```
SELECT mês,vendedor, AVG(valor_venda) FROM Venda GROUP BY mês, vendedor;
```



(CEBRASPE (CESPE)/SEBRAE/Cientista de Dados/2024) Assinale a opção que apresenta corretamente a consulta SQL que retornará a média de salários por departamento de uma empresa, considerada uma tabela funcionarios com colunas departamento e salario.

a) SELECT departamento, COUNT(salario)

FROM funcionarios

GROUP BY departamento;

b) SELECT departamento, AVG(salario)

FROM funcionarios

GROUP BY departamento;

c) SELECT departamento, SUM(salario)

FROM funcionarios

GROUP BY departamento;

```
d) SELECT departamento, MAX(salario)
FROM funcionarios
GROUP BY departamento;
```

Comentários:

- a) **Errada.** A função COUNT conta a quantidade de registros, não calcula a média dos salários.
- b) **Correta.** A função AVG calcula a média aritmética dos valores, e o GROUP BY departamento faz com que essa média seja calculada separadamente para cada departamento.
- c) **Errada.** A função SUM soma todos os valores dos salários, retornando o total e não a média.
- d) **Errada.** A função MAX retorna o maior valor da coluna, ou seja, o maior salário de cada departamento, e não a média.

Gabarito: B

(CEBRASPE (CESPE)/SEBRAE/Cientista de Dados/2024) Caso um cientista de dados precise extrair, transformar e analisar grandes volumes de dados em um banco de dados relacional, usando SQL, a prática mais recomendável para garantir que as operações de análise sejam eficientes e precisas consiste em

- a) aplicar funções de agregação como SUM(), AVG() e COUNT() com cláusulas GROUP BY para resumir e agregar os dados de forma eficiente.
- b) ignorar o uso de índices ao executar consultas, já que a criação e a manutenção de índices são responsabilidades do administrador do banco de dados.
- c) usar comandos SELECT * em todas as consultas para garantir que todas as colunas sejam retornadas, facilitando a análise posterior.
- d) utilizar subconsultas (subqueries) complexas em vez de operações de junção (JOIN) para simplificar a lógica das consultas.

Comentários:

- a) **Correta.** Ao trabalhar com grandes volumes de dados, usar funções de agregação como SUM(), AVG() e COUNT() junto com GROUP BY permite resumir os dados em grupos, retornando informações condensadas em vez de processar linha por linha, o que torna a análise muito mais eficiente.
- b) **Errada.** Ignorar índices é uma péssima prática, pois eles ajudam o banco a localizar dados rapidamente, sendo fundamentais para o desempenho de consultas em grandes volumes de dados.
- c) **Errada.** Usar SELECT * traz todas as colunas, inclusive as desnecessárias, gerando consumo extra de memória e processamento, prejudicando a eficiência da análise.
- d) **Errada.** Subconsultas complexas tendem a ser menos eficientes que operações de JOIN, que são justamente projetadas para combinar dados de várias tabelas de forma otimizada.

Gabarito: A

Filtragem de Grupos (HAVING)

Para filtrarmos os valores associados a grupos por alguma condição, não utilizamos o WHERE como em um SELECT "normal", mas sim a cláusula HAVING.

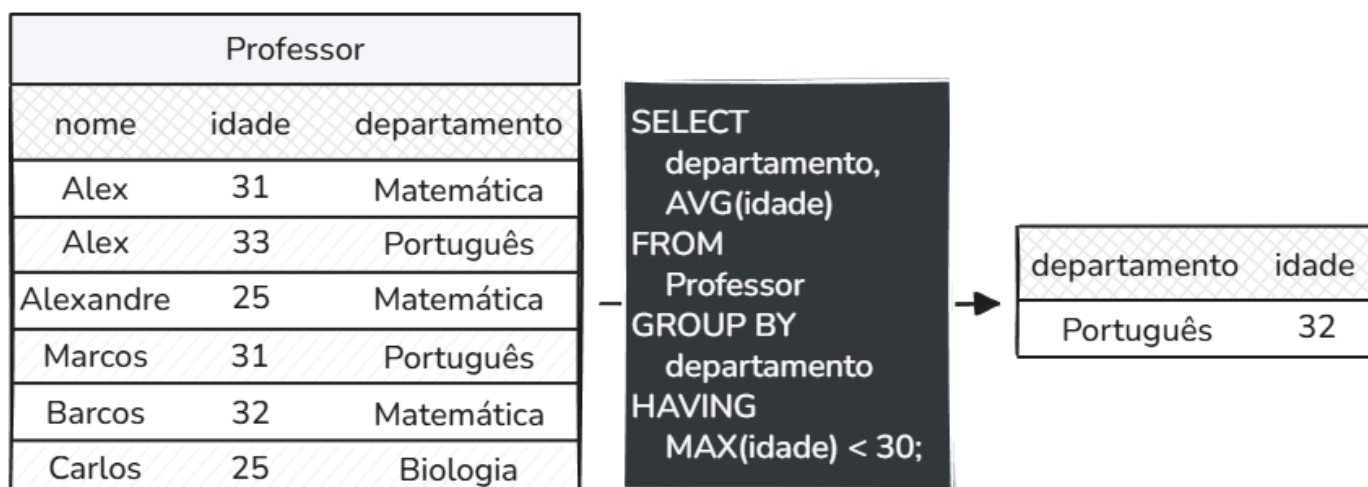
Explicando melhor, sabemos que, quando usamos o GROUP BY com alguma função de agregação (como SUM, COUNT, AVG, etc.), o banco calcula os valores agregados para cada grupo. O HAVING permite filtrarmos esses resultados calculados, mantendo apenas os grupos que passam em algum critério.

Por exemplo, vamos supor que queremos encontrar os departamentos que possuem idade máxima de professor abaixo de 30 anos. Então, podemos escrever o seguinte SELECT.

```
SELECT
    departamento,
    MAX(idade)
FROM
    Professor
GROUP BY
    departamento
HAVING
    MAX(idade) < 30;
```

É interessante notar o que acontece quando usamos o WHERE em um SQL que também possui HAVING: enquanto o primeiro comando delimita quais linhas serão analisadas para formar o GROUP BY, o segundo filtra quais grupos irão para a resposta final.

A seguir, vamos filtrar as linhas da tabela Professor para obtermos apenas aquelas que possuem idade inferior a 30 anos.





(CCV UFC/Tec (UFCA)/2014) Em relação à SQL (Structured Query Language), assinale a alternativa correta.

- a) A função agregada AVG retorna a somatória dos valores especificados na consulta.
- b) Na linguagem SQL, um RIGHT OUTER JOIN retorna todas as tuplas que não são comuns às duas tabelas.
- c) O operador SQL UNION retorna o resultado da operação de junção (join) da cláusula FROM de uma consulta.
- d) A cláusula HAVING foi adicionada à SQL devido à cláusula WHERE não poder ser usada com funções de agregação.
- e) Os comandos INSERT, CREATE, UPDATE, DELETE e SELECT fazem parte dos comandos DML (Data Manipulation Language) da SQL.

Comentários:

- a) **Errada.** A função AVG calcula a média aritmética dos valores, e não a somatória. Quem retorna a soma dos valores é a função SUM.
- b) **Errada.** O RIGHT OUTER JOIN preserva todas as linhas da tabela da direita, incluindo as que têm correspondência com a tabela da esquerda. Ele não retorna apenas tuplas não comuns.
- c) **Errada.** O UNION serve para combinar (unir) os resultados de duas consultas em um único conjunto, eliminando duplicatas. Ele não realiza junção de tabelas como o JOIN faz.
- d) **Correta.** O WHERE filtra linhas individuais antes do agrupamento e não consegue trabalhar com funções de agregação como SUM, AVG, MAX. Por isso foi criado o HAVING, que filtra os grupos formados pelo GROUP BY usando funções de agregação.
- e) **Errada.** Os comandos CREATE, ALTER, DROP e TRUNCATE pertencem ao DDL (Data Definition Language), pois lidam com a estrutura do banco. Apenas INSERT, UPDATE e DELETE (e o SELECT, dependendo do autor) são classificados como DML.

Gabarito: D

Controle de Dados (DCL)

Agora vamos estudar os comandos relacionados ao controle de acesso aos dados, ou seja, os comandos que definem quem pode fazer o quê no banco.

Lembrem-se que fazem parte desse grupo comandos como GRANT, REVOKE e DENY .

O comando GRANT

Pois bem, o GRANT é o comando que concede permissões em um determinado objeto do banco a um usuário (ou grupo de usuários). Por exemplo, se temos um usuário no banco de dados chamado "Joao" e quisermos conceder a permissão de executar SELECT na tabela Professor, podemos escrever:

```
GRANT SELECT ON Professor TO Joao;
```

A partir desse comando, o João pode consultar a tabela, mas não pode inserir nada ali, atualizar seus dados ou até deletá-lo, já que essas permissões não foram concedidas.

Para resolvermos isso, podemos simplesmente conceder as outras autorizações de INSERT, UPDATE e DELETE de uma vez.

```
GRANT INSERT, UPDATE, DELETE ON Professor TO Joao;
```

Além dos privilégios básicos bem conhecidos, como SELECT, INSERT, UPDATE e DELETE, há dois outros não tão óbvios.

Com o privilégio REFERENCES, damos a permissão ao usuário de criar chaves estrangeiras apontado para a tabela. Em outras palavras, ele concede o direito de "referenciar" a tabela a partir de outras tabelas.

A seguir, vamos conceder à João a permissão de criar FKs em outras tabelas apontando para a tabela Professor.

```
GRANT REFERENCES ON Professor TO Joao;
```

Já o USAGE é meio "genérico", pois concede o direito de usar e manipular certos objetos que não são tabelas, como schemas (conjuntos de tabelas e objetos dentro de um banco), domínios (tipos de dados personalizados), sequences (geradores de números sequenciais) e tipos definidos pelo usuário.

Pois bem, para evitar ficar escrevendo uma lista de privilégios toda vez, podemos simplesmente utilizar o atalho ALL PRIVILEGES.

```
GRANT ALL PRIVILEGES ON Professor TO Joao;
```

É válido sabermos que o GRANT se trata de um comando bem flexível, permitindo concessões bem específicas. Por exemplo, podemos conceder acessos a colunas específicas das nossas tabelas, impedindo que determinado usuário veja informações indesejadas.

A seguir vamos conceder acesso de SELECT apenas às colunas "nome" e "departamento" a João, impedindo que ele veja dados como o CPF dos professores.

```
GRANT SELECT ON Professor(nome, departamento) TO Joao;
```

Percebam que, se mais de um usuário possui esse privilégio, seria extremamente trabalhoso escrever um SQL para o conceder a cada. Então, podemos simplesmente passar todos usuários no mesmo GRANT. Vejam.

```
GRANT SELECT ON Professor(nome, departamento) TO Joao, Maria, Jose, Joana;
```

Caso seja necessário conceder o mesmo privilégio a todos usuários do banco de dados, podemos simplesmente usar a sintaxe PUBLIC.

```
GRANT SELECT ON Professor(nome, departamento) TO PUBLIC;
```

Além desses, um recurso interessante é a cláusula WITH GRANT OPTION, que permite que um usuário não só receba uma permissão, mas também conceda essa mesma permissão a outros usuários:

```
GRANT SELECT ON Funcionario TO Joao WITH GRANT OPTION;
```



(FCC/ATTIFM (Pref Manaus)/2019) No Oracle 11g um programador deseja conceder a Paulo o privilégio para selecionar dados da tabela Contribuintes e deseja permitir que Paulo conceda esse privilégio a outros usuários. Para isso, o programador deve usar o comando: GRANT SELECT

- a) ON Contribuintes TO Paulo WITH CASCADE OPTION;
- b) TO Paulo ON Contribuintes WITH CASCADE OPTION;
- c) ON Contribuintes TO Paulo WITH GRANT OPTION;
- d) TO Paulo ON Contribuintes WITH EXTENDS OPTION;
- e) FROM Contribuintes TO Paulo WITH GRANT OPTION;

Comentários:

a) Errada. A ordem ON tabela TO usuário está certa, mas "WITH CASCADE OPTION" não existe para concessão de privilégios; o correto para permitir que o usuário repasse o privilégio é WITH GRANT OPTION.

b) Errada. A sintaxe está invertida (o certo é ON tabela TO usuário, não TO usuário ON tabela) e "WITH CASCADE OPTION" não é a cláusula usada para repassar privilégios.

c) Correta. A sintaxe correta é GRANT SELECT ON Contribuintes TO Paulo WITH GRANT OPTION, que concede a Paulo o privilégio de SELECT na tabela e ainda permite que ele conceda esse mesmo privilégio a outros usuários.

d) **Errada.** A ordem TO usuário ON tabela está invertida e "WITH EXTENDS OPTION" não existe; o correto é WITH GRANT OPTION.

e) **Errada.** Para conceder privilégios usamos TO Contribuintes, não FROM (FROM é usado no comando REVOKE, que retira privilégios).

Gabarito: C

Agrupando privilégios - ROLE

Vimos que podemos atribuir as permissões a um usuário, a um grupo de usuários ou até a todos usuários do banco de dados de uma vez.

Porém, há um outro tipo de destinatário especial: a ROLE (ou papél, em português).

Com as roles podemos agrupar privilégios em perfis genéricos e atribuir esses aos usuários. Assim, conseguimos manipular acessos a diversos usuários ao mesmo tempo, porque basta alterar as permissões da ROLE para que seja automaticamente aplicada aos usuários que estão atribuídos a ela.

Em SQL, antes de concedermos os acessos a ROLE, precisamos criá-la. A seguir, vamos criar duas, o perfil de coordenador e professor.

```
CREATE ROLE role_coordenador;  
CREATE ROLE role_professor;
```

Uma vez que a ROLE foi criada, podemos atribuir a ela as permissões. Assim, vamos permitir que coordenadores consigam, além de selecionar dados no banco, os atualizar também. Já os professores terão apenas privilégio de SELECT.

```
GRANT SELECT, UPDATE ON Professor TO role_coordenador;  
GRANT SELECT ON Professor TO role_professor;
```

Pronto. Com os papéis criados e com os privilégios concedidos, podemos os atribuir aos usuários que desejamos. Então, Maria será nossa coordenadora e os outros serão os professores.

```
GRANT role_coordenador TO Maria;  
GRANT role_professor TO Joana, Joao, Jose;
```

A partir de agora não precisamos mais alterar os privilégios professor a professor. Se houver alguma mudança nas suas permissões, podemos simplesmente alterar a "role_professor" que essa capacidade se estenderá a Joana, João e José.



(VUNESP/Tec CPDJ (TJM SP)/2023) O comando do sistema gerenciador de bancos de dados H2 que faz a criação de um novo papel no banco de dados é:

- a) CREATE TRIGGER ...;
- b) CREATE DOMAIN ...;
- c) CREATE SCHEMA ...;
- d) CREATE ROLE ...;
- e) CREATE SEQUENCE ...;

Comentários:

a) **Errada.** CREATE TRIGGER serve para criar gatilhos, não papéis, e esse comando não foi tratado no conteúdo da aula.

b) **Errada.** CREATE DOMAIN cria um domínio, ou seja, um tipo de dado personalizado para reutilizar em vários atributos, e não um papel.

c) **Errada.** CREATE SCHEMA cria um esquema, que funciona como uma "pasta" para organizar tabelas e outros objetos, e não um papel.

d) **Correta.** CREATE ROLE é o comando usado para criar um papel (role), que serve para agrupar privilégios e atribuí-los a usuários de forma facilitada.

e) **Errada.** CREATE SEQUENCE cria sequências (geradores de números sequenciais), e não papéis.

Gabarito: D

(ADAPTADA - CEBRASPE (CESPE)/AFE (SEFAZ PR)/2026) Em sistemas de gerenciamento de bancos de dados que seguem o padrão SQL, os mecanismos de autorização permitem controlar o acesso de usuários e roles a objetos e dados. A respeito das formas de autorização suportadas em bancos de dados relacionais, julgue os itens a seguir.

I O comando GRANT é utilizado para conceder privilégios de acesso, e a cláusula WITH GRANT OPTION permite que o destinatário repasse o privilégio a outros usuários.

II O privilégio REFERENCES é exigido para a criação de chaves estrangeiras ou restrições CHECK que referenciem outra relação.

III O conceito de roles permite a atribuição de privilégios a grupos lógicos de usuários, podendo haver herança de privilégios entre roles.

IV (assunto fora do escopo da aula)

Em qualquer implementação SQL padrão, as autorizações podem ser concedidas sobre tuplas individuais, sem necessidade de extensões específicas do SGBD.

Estão certos apenas os itens

- a) I e II.
- b) IV e V.
- c) I, III e V.
- d) I, II, III e IV.
- e) II, III, IV e V.

Comentários:

Segundo a banca, "A ausência do item V no enunciado da questão prejudicou o seu julgamento objetivo."

https://cdn.cebraspe.org.br/concursos/sefa_pr_25/arquivos/D9B440413EEA07B9632F217AFCC A0C9508067523428E4D073480748F65B6186E.pdf

Gabarito: Anulada

O comando REVOKE

Agora vamos supor que João utilizou seu privilégio de UPDATE de forma indevida e desejamos revogar essa capacidade. Para isso, basta utilizarmos a mesma sintaxe, porém com o comando REVOKE.

```
REVOKE UPDATE ON Professor FROM Joao;
```



(FGV/FTE (SEFAZ MT)/2023) No contexto da concessão de privilégios/permisões na administração de bancos de dados relacionais, assinale a opção que indica dois dos comandos básicos disponíveis na maior parte dos SGDB.

- a) ADD e DELETE.
- b) ALLOW e CONSTRAINT.
- c) ATTACH e DETACH.
- d) GRANT e REVOKE.
- e) INSERT e REMOVE.

Comentários:

- a) **Errada.** ADD e DELETE não são comandos usados para conceder ou retirar permissões em um banco de dados.
- b) **Errada.** ALLOW e CONSTRAINT não fazem parte dos comandos de controle de acesso (DCL).
- c) **Errada.** ATTACH e DETACH não são comandos relacionados à concessão de privilégios.
- d) **Correta.** GRANT serve para conceder permissões a um usuário (como permitir que ele faça SELECT em uma tabela) e REVOKE serve para retirar uma permissão que foi concedida anteriormente. Ambos fazem parte da DCL (Data Control Language).
- e) **Errada.** INSERT é um comando de manipulação de dados (DML) e REMOVE não é um comando padrão de SQL para gerenciar permissões.

Gabarito: D

O comando DENY

O DENY é um comando usado para impedir explicitamente que um usuário receba uma permissão específica sobre um objeto do banco de dados.

Assim, com ele "negamos" permissões sobre um determinado objeto, como uma tabela ou view. Ou seja, o usuário que receber um DENY a uma permissão não pode ter essa permissão concedida de jeito nenhum, mesmo que alguém isso.

Por exemplo, a seguir negamos ao usuário Marcos a possibilidade de receber a capacidade de executar SELECT na tabela Professor.

```
DENY SELECT ON Professor TO Marcos;
```



(FCC/Tec GP (PGE AM)/PGE AM/Informática/2022) Em SQL, para definir que o usuário Paulo001 não possa executar instruções SELECT em nenhuma tabela, utiliza-se a instrução

- a) REVOKE SELECT TO Paulo001;
- b) BLOCK USER Paulo001 TO SELECT;
- c) GRANT USER Paulo001 ALLOWED=FALSE TO SELECT;
- d) DENY SELECT TO Paulo001;
- e) ROLLBACK SELECT TO Paulo001;

Comentários:

- a) **Errada.** O REVOKE serve para remover uma permissão que já foi concedida, não para bloquear terminantemente o acesso. Além disso, a sintaxe correta do REVOKE usa FROM, e não TO.
- b) **Errada.** Não existe o comando BLOCK USER em SQL.
- c) **Errada.** O GRANT serve para conceder permissões, e essa sintaxe com ALLOWED=FALSE não existe em SQL.
- d) **Correta.** O DENY é o comando que bloqueia terminantemente o acesso de um usuário a uma permissão, prevalecendo sobre qualquer GRANT, seja individual ou por grupo. Por isso, é o comando ideal para impedir que Paulo001 execute SELECT.
- e) **Errada.** Esse trecho nem sequer é um comando válido para conceder ou negar permissões, trata-se de fragmentos de comandos INSERT e COMMIT, que pertencem a outros subconjuntos do SQL (DML e TCL).

Gabarito: D

Gerenciamento de Transações (TCL)

Com o TCL (Transaction Control Language, ou Linguagem de Controle de Transações) temos o subconjunto do SQL dedicado a gerenciar transações em um banco de dados.

Percebam então que estudaremos agora o conjunto de comandos críticos para o funcionamento do banco de dados, porque garantimos a integridade dos dados em bancos de dados relacionais justamente por existir o conceito de transação.

Relembramos que uma transação é um conjunto de operações que deve ser tratado como uma única unidade lógica. Ou todas as operações são concluídas com sucesso, ou nenhuma delas é aplicada, não existe meio-termo.

É justamente para garantir o ciclo e vida das transações que existem os comandos TCL.

O comando COMMIT

Usamos o comando COMMIT para salvar no banco de dados todas as operações realizadas durante a transação atual.

Ou seja, ele marca o momento em que a transação termina e as mudanças que estavam "pendentes" se tornam oficiais e permanentes.

Ou seja, o COMMIT é como se fosse um botão "SALVAR": uma vez que uma transação é "commitada", ela se torna permanente e não pode ser desfeita, não havendo volta.

Na prática, basta invocar esse comando ao final do SQL, após uma série de operações.

Por exemplo, vamos supor que temos uma tabela Funcionário que, se receber um registro novo, então esse mesmo registro tem de ser inserido na tabela Professor. Para garantir que uma tabela

não receba um registro sem a outra ter recebido, vamos inserir um COMMIT apenas ao final das duas inserções.

```
INSERT INTO Professor(CPF, nome) VALUES (888888888888, 'Ronaldo Nazário');  
INSERT INTO Funcionario(CPF, nome) VALUES (888888888888, 'Ronaldo Nazário');  
COMMIT;
```

Curiosamente, nas primeiras versões do padrão SQL (como o SQL-92), não havia um comando específico para iniciar uma transação, então ela começava implicitamente quando se executava um DML.

A partir do SQL:1999, o padrão passou a incluir o comando START TRANSACTION. Além disso, muitos SGBDs oferecem alternativas como BEGIN TRANSACTION ou apenas BEGIN.



(CEBRASPE (CESPE)/Ana CT I (CNPq)/2024) A fim de garantir o desfazimento das operações em um SGBD, o ciclo iniciado pelo comando BEGIN TRANSACTION necessita do complemento dos comandos COMMIT, ROLLBACK e CATCH.

Comentários:

O erro da questão está na inclusão do CATCH. Uma transação iniciada com BEGIN TRANSACTION é complementada pelos comandos COMMIT (que confirma as operações, tornando-as permanentes) e ROLLBACK (que desfaz as operações). O CATCH não faz parte do ciclo de controle de transações, então a banca colocou ele ali justamente para tentar te confundir.

Gabarito: Errado

(FGV/Tec NS (TJ MS)/TJ MS/Analista de Sistemas Computacionais/Analista de Banco de Dados/2024) Observe o seguinte trecho de um script SQL:

```
START TRANSACTION;  
INSERT INTO tabA(coluna1)  
VALUES(1);
```

Para que o resultado da execução do comando INSERT, no MySQL, seja persistido no banco de dados, o script apresentado deve terminar com o comando:

- a) SET
- b) BEGIN
- c) COMMIT
- d) ROLLBACK
- e) SAVEPOINT

Comentários:

- a) **Errada.** SET sozinho não confirma transação; o SET TRANSACTION serve apenas para configurar comportamentos da transação, como nível de isolamento.
- b) **Errada.** BEGIN é usado para iniciar uma transação, não para finalizá-la salvando as alterações.
- c) **Correta.** O COMMIT é o comando que torna permanentes no banco de dados todas as operações realizadas durante a transação, funcionando como um "botão salvar".
- d) **Errada.** O ROLLBACK faz justamente o oposto do que se pede: desfaz as operações da transação atual, em vez de persisti-las.
- e) **Errada.** SAVEPOINT apenas cria um ponto intermediário dentro da transação para um possível rollback parcial, mas não confirma definitivamente as alterações no banco.

Gabarito: C

Nesse momento vocês devem estar se perguntando *“poxa professor, em todas operações em SQLs que vimos até agora não teve nada disso de BEGIN TRANSACTION e COMMIT para funcionarem como uma transação!”*

Isso é verdade, mas fazemos isso porque, por padrão, trabalhamos no modo Autocommit.

Saibam que, nesse modo, cada comando SQL executado é automaticamente confirmado (commitado) imediatamente após a execução. Assim, cada operação é tratada como uma transação independente que começa, executa e termina com COMMIT automático, sem intervenção do usuário.

Assim, se fizermos uma série de comandos na sequência, cada um será considerado uma transação.

```
INSERT INTO Funcionarios (nome) VALUES ('Ana');  
-- COMMIT implícito aqui. Ana está salva permanentemente.  
  
UPDATE Funcionarios SET idade = 51 WHERE nome = 'Alex';  
-- COMMIT implícito aqui. Atualização salva permanentemente.  
  
DELETE FROM Funcionarios WHERE nome = 'Marcos';  
-- COMMIT implícito aqui. Bruno removido permanentemente.
```

O comando ROLLBACK

Com o comando ROLLBACK queremos exatamente o oposto do COMMIT: desfazer todas as operações realizadas durante a transação atual que ainda não foram persistidas no banco.

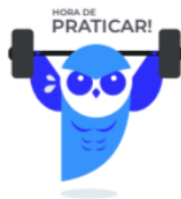
Assim, reverteremos o banco ao estado anterior caso ocorra um erro ou caso as alterações feitas não sejam desejadas. É como se tivéssemos feito alterações em um arquivo e, na hora de fechar, escolhemos não salvar, perdendo tudo.



Cuidado! O ROLLBACK só funciona para operações que ainda não foram commitadas. Uma vez que o COMMIT ocorreu, o ROLLBACK não consegue mais desfazer nada.

A seguir, colocamos um ROLLBACK após o COMMIT para garantir que, se o banco não conseguir commitar a operação, então a desfaça para o estado anterior.

```
INSERT INTO Professor(CPF, nome) VALUES (888888888888, 'Ronaldo Nazário');  
INSERT INTO Funcionario(CPF, nome) VALUES (888888888888, 'Ronaldo Nazário');  
COMMIT;  
ROLLBACK; -- Se não for possível commitar, desfaz. Se commitar, essa linha perde o efeito.
```



(CONSULPLAN/TJ (TRE SC)/2008) Ao executar o comando SQL abaixo, qual o resultado esperado?

```
1 START TRANSACTION;  
2 UPDATE Alunos SET idade=12 WHERE cod_aluno=1234;  
3 UPDATE Professores SET UF='SC' WHERE cod_professor=2345;  
4 ROLLBACK;
```

- a) Apenas o comando da linha 2 será executado.
- b) Os comandos das linhas 2 e 3 serão executados e as linhas serão alteradas na base.
- c) No fim da execução, nenhuma linha será alterada.
- d) Apenas o comando da linha 3 será executado.
- e) Não é permitido usar ROLLBACK quando é feito um Update.

Comentários:

a) Errada. O UPDATE da linha 2 até é executado durante a transação, mas o ROLLBACK na linha 4 desfaz essa alteração antes que ela se torne permanente.

b) Errada. Como não houve COMMIT antes do ROLLBACK, as alterações dos dois UPDATEs não são persistidas no banco, sendo todas desfeitas.

c) Correta. O ROLLBACK desfaz todas as operações realizadas durante a transação atual que ainda não foram commitadas, então os dois UPDATEs são revertidos e nada é alterado no banco.

d) Errada. Da mesma forma que a linha 2, o UPDATE da linha 3 ocorre dentro da transação, mas o ROLLBACK reverte tudo que ainda não foi commitado.

e) Errada. O ROLLBACK pode ser usado normalmente após comandos UPDATE, justamente para desfazer alterações que ainda não foram confirmadas com COMMIT.

Gabarito: C

(FGV/AJ (TJ SE)/TJ SE/Análise de Sistemas/Banco de Dados/2023) Os gerenciadores de bancos de dados permitem que transações mais complexas sejam controladas explicitamente, de modo a garantir a atomicidade e a integridade das operações. Nesse contexto, o ciclo iniciado por um comando begin transaction tem, como complemento, o(s) comando(s):

a) catch;

b) commit, rollback;

c) load, unload;

d) lock, unlock;

e) log.

Comentários:

a) Errada. Catch não é um comando de controle de transações em SQL, pertence a tratamento de exceções em linguagens de programação.

b) Correta. Após iniciar uma transação com begin transaction, usamos o commit para confirmar e tornar permanentes as alterações no banco, ou o rollback para desfazer tudo que foi feito desde o início da transação, retornando ao estado anterior.

c) Errada. Load e unload não fazem parte dos comandos de controle de transações (TCL).

d) Errada. Lock e unlock estão relacionados a bloqueios de recursos, não ao ciclo de uma transação iniciada por begin transaction.

e) Errada. Essa alternativa apresenta apenas trechos de comandos SQL de inserção e savepoints, não respondendo ao que se pede sobre o complemento do ciclo de uma transação.

Gabarito: B

O comando SAVEPOINT

Com o comando SAVEPOINT estabelecemos um ponto dentro de uma transação para o qual podemos voltar depois, sem precisar desfazer a transação inteira. Ou seja, esse comando permite rollbacks parciais dentro de uma transação.

Assim, em vez de termos que escolher entre "desfazer tudo" ou "confirmar tudo", podemos desfazer a transação até um ponto específico, mantendo o que veio antes.

Por exemplo, vamos supor que temos mais de uma pessoa a ser registrada em Funcionario e Professor. Não é necessário fazer o COMMIT apenas ao final de tudo, podemos criar savepoints intermediários para garantir que o SQL foi sendo salvo em cada etapa.

```
INSERT INTO Professor(CPF, nome) VALUES (7777777777, 'Edson Arantes');  
INSERT INTO Funcionario(CPF, nome) VALUES (7777777777, 'Edson Arantes');  
SAVEPOINT primeiro_registro;  
INSERT INTO Professor(CPF, nome) VALUES (7777777777, 'Marta Vieira');  
INSERT INTO Funcionario(CPF, nome) VALUES (7777777777, 'Marta Vieira');  
SAVEPOINT segundo_registro;  
COMMIT;
```



(FAURGS/Tec TI (BANRISUL)/2018) Analise as afirmações abaixo sobre transações em sistemas de banco de dados relacionais.

I - Uma transação inicia-se com o comando BEGIN TRANSACTION.

II - COMMIT e ROLLBACK são dois comandos que podem ser utilizados para terminar uma transação.

III - Autocommit é um modo de operação em que todas as transações são automaticamente confirmadas (COMMIT) após sua execução.

IV - O comando ROLLBACK TO SAVEPOINT permite desfazer uma transação somente até um dado ponto.

Quais estão corretas?

a) Apenas II.

b) Apenas II e IV.

c) Apenas I, II e IV.

d) Apenas II, III e IV.

e) I, II, III e IV.

Comentários:

I. Errada. Nas primeiras versões do padrão SQL (como o SQL-92), não havia um comando específico para iniciar uma transação, ela começava implicitamente quando se executava um DML. Apenas a partir do SQL:1999 o padrão passou a incluir o START TRANSACTION, sendo o BEGIN TRANSACTION uma alternativa oferecida por muitos SGBDs.

II. **Correta.** O COMMIT confirma e torna permanentes as alterações da transação, enquanto o ROLLBACK desfaz tudo que foi feito. Ambos finalizam a transação atual.

III. **Errada.** No modo Autocommit, cada comando SQL executado é tratado como uma transação independente que é automaticamente confirmada (COMMIT) após sua execução. O erro da assertiva está em dizer "todas as transações", quando na verdade cada comando vira uma transação própria que é commitada individualmente.

IV. **Correta.** O SAVEPOINT cria um ponto intermediário dentro da transação, e com ROLLBACK TO SAVEPOINT conseguimos desfazer apenas até esse ponto específico, sem precisar desfazer a transação inteira.

Gabarito: B

O comando SET TRANSACTION

Há ainda um último comando TCL, o SET TRANSACTION, que configura o comportamento da transação. Com definimos seus aspectos como o nível de isolamento (quão "visíveis" as mudanças de uma transação são para outras transações simultâneas) e a consistência (como conflitos são tratados).

É um comando mais avançado para essa aula, mas precisamos saber que existe.



(QUADRIX/Ana TI (DATAPREV)/2012) NÃO se trata de um comando SQL dentro da categoria Oracle de instruções de controle de transações:

- a) TRUNCATE.
- b) COMMIT.
- c) ROLLBACK.
- d) SAVEPOINT.
- e) SET TRANSACTION.

Comentários:

a) **Correta.** O TRUNCATE é um comando da DDL (Linguagem de Definição de Dados), usado para remover todas as linhas de uma tabela atuando na sua estrutura física, e não para controlar transações.

b) **Errada.** O COMMIT é um comando TCL que serve para salvar de forma permanente no banco todas as operações realizadas durante a transação.

- c) **Errada.** O ROLLBACK é um comando TCL que serve para desfazer as operações da transação atual, voltando o banco ao estado anterior.
- d) **Errada.** O SAVEPOINT é um comando TCL que cria um ponto intermediário dentro da transação, permitindo voltar até ali sem desfazer tudo.
- e) **Errada.** O SET TRANSACTION é um comando TCL que configura o comportamento da transação, como o nível de isolamento e consistência.

Gabarito: A

QUESTÕES COMENTADAS - FGV



1. (FGV/TJ TRT24/TRT 24/Apoio Especializado/Tecnologia da Informação/2025) Um banco de dados de uma loja de ferramentas será criado utilizando comandos de SQL (Structured Query Language), conforme a seguir.

<1> CREATE TABLE ferramentas (id INTEGER PRIMARY KEY, nome TEXT, estoque INTEGER, valor FLOAT);

<2> INSERT INTO ferramentas VALUES(1, 'Martelo', 40, 7.5);

<3> INSERT INTO ferramentas VALUES(2, 'Chave de fenda', 100, 5.2);

<4> INSERT INTO ferramentas VALUES(3, 'Tesoura', 230, 6.3);

<5> INSERT INTO ferramentas VALUES(4, 'Furadeira', 145, 120.8);

<6> INSERT INTO ferramentas VALUES(5, 'Lixa', 156, 3.7);

<7> INSERT INTO ferramentas VALUES(6, 'Alicate', 581, 4.5);

Após a execução dos comandos acima (linhas de <1> a <7>), nessa ordem, foram executados os comandos abaixo, nessa ordem:

<8> SELECT * FROM ferramentas ORDER BY nome;

<9> DELETE FROM ferramentas WHERE valor > 100 ;

<10> SELECT SUM(estoque) FROM ferramentas WHERE valor > 5;

O número retornado pelo comando na linha <10> é

a) 370.

b) 581.

c) 737.

d) 1107.

e) 1252.

Comentários:

- a) **Correta.** A soma dos estoques das ferramentas restantes com valor maior que 5 é $40 + 100 + 230 = 370$.
- b) **Errada.** 581 é apenas o estoque do Alicate, que nem entra no filtro porque seu valor (4.5) não é maior que 5.
- c) **Errada.** 737 seria a soma incluindo indevidamente o Alicate ($370 + 581$ não dá isso, mas esse valor não corresponde ao filtro correto).
- d) **Errada.** 1107 somaria ferramentas que não atendem ao filtro de valor maior que 5.
- e) **Errada.** 1252 seria a soma do estoque de quase todas as ferramentas, ignorando o DELETE e o filtro do WHERE.

Gabarito: A

2. (FGV/AF (SEFAZ PR)/SEFAZ PR/2025) Filtrar dados no SQL é essencial para extrair informações relevantes, reduzir o volume de resultados e melhorar o desempenho das consultas. Acerca desse cenário, avalie as afirmativas a seguir e assinale (V) para a verdadeiras e (F) para a falsa.

() A cláusula WHERE pode filtrar as linhas antes da agregação (ou seja, antes do GROUP BY).

() A cláusula HAVING filtra os grupos depois da agregação.

() Não é possível usar funções agregadas como SUM() e AVG() na cláusula WHERE, mas é permitido na cláusula HAVING.

As afirmativas são, respectivamente,

- a) F – F – F.
- b) F – V – V.
- c) V – V – F.
- d) V – F – V.
- e) V – V – V.

Comentários:

- a) **Errada.** As três afirmativas são verdadeiras, conforme explicado abaixo.
- b) **Errada.** A primeira afirmativa também é verdadeira, pois o WHERE realmente filtra linhas antes do GROUP BY.
- c) **Errada.** A terceira afirmativa também é verdadeira, já que funções de agregação não podem ser usadas no WHERE, apenas no HAVING.
- d) **Errada.** A segunda afirmativa também é verdadeira, pois o HAVING filtra os grupos depois da agregação.

e) **Correta.** O WHERE delimita quais linhas serão analisadas para formar o GROUP BY (filtra antes da agregação), enquanto o HAVING filtra quais grupos irão para a resposta final (depois da agregação). Por isso, funções como SUM() e AVG() só funcionam no HAVING, pois no momento em que o WHERE atua, os grupos e seus valores agregados ainda não existem.

Gabarito: E

3. (FGV/AF (SEFAZ PR)/SEFAZ PR/2025) Em consultas SQL que envolvem funções agregadas, é essencial compreender a sintaxe e a lógica no uso das cláusulas relevantes.

Considere uma tabela chamada funcionarios, que possui os seguintes campos: id (inteiro), nome (texto), salario (numérico), e departamento (texto).

Deseja-se escrever uma consulta que liste os departamentos cujo salário médio dos funcionários seja superior a 3000. Assinale a alternativa que apresenta a consulta SQL com sintaxe correta para esse caso.

a) SELECT departamento, AVG(salario) FROM funcionarios WHERE AVG(salario) > 3000 GROUP BY departamento;

b) SELECT departamento, AVG(salario) FROM funcionarios GROUP BY departamento WHERE AVG(salario) > 3000;

c) SELECT departamento, AVG(salario) FROM funcionarios GROUP BY AVG(salario) HAVING AVG(salario) > 3000;

d) SELECT departamento, AVG(salario) FROM funcionarios GROUP BY departamento HAVING AVG(salario) > 3000;

e) SELECT departamento, AVG(salario) FROM funcionarios HAVING AVG(salario) > 3000;

Comentários:

a) **Errada.** O WHERE não pode ser usado com funções de agregação como AVG. Para filtrar resultados de agregação, devemos usar HAVING.

b) **Errada.** A ordem das cláusulas está incorreta e, além disso, o WHERE não pode conter funções de agregação como AVG.

c) **Errada.** O GROUP BY deve agrupar pela coluna departamento (que é o que queremos listar), não pela função AVG(salario).

d) **Correta.** Agrupamos os funcionários por departamento com GROUP BY, calculamos a média salarial com AVG(salario) e filtramos os grupos cuja média é superior a 3000 usando HAVING, que é a cláusula correta para filtrar resultados de funções de agregação.

e) **Errada.** Falta o GROUP BY departamento. Sem o agrupamento, não é possível calcular a média de salário por departamento.

Gabarito: D

4. (FGV/AF (SEFAZ PR)/SEFAZ PR/2025) Considere a tabela funcionarios, que possui os campos id (inteiro), nome (texto), salario (numérico) e comissao (numérico).

Sobre o tratamento de valores faltantes em SQL, analise as afirmativas a seguir.

I. A instrução `SELECT COUNT(salario) FROM funcionarios`; conta todos os registros da tabela funcionarios.

II. A expressão `NULLIF(comissao, 0)` retorna NULL apenas quando o valor de comissao é exatamente igual a zero.

III. A cláusula `WHERE nome <> NULL` é adequada para retornar todos os nomes não nulos em uma tabela.

Está correto o que se afirma em

- a) I, apenas.
- b) II, apenas.
- c) I e II, apenas.
- d) II e III, apenas.
- e) I, II e III.

Comentários:

I. Errada. O `COUNT(coluna)` conta apenas as linhas em que o valor da coluna não é NULL, e não todas as linhas da tabela. Se quiséssemos contar todos os registros, usaríamos `COUNT(*)`. Assim, se algum funcionário tiver salário NULL, ele não seria contabilizado.

II. Correta. A função `NULLIF` compara dois valores e retorna NULL quando eles são iguais. Então `NULLIF(comissao, 0)` devolve NULL exatamente quando a comissão vale zero; caso contrário, retorna o próprio valor da comissão.

III. Errada. Em SQL, não podemos comparar valores com NULL usando os operadores `=` ou `<>`, porque o resultado dessas comparações é sempre UNKNOWN (desconhecido), devido à lógica de três valores. Para filtrar nomes não nulos, o correto seria usar `WHERE nome IS NOT NULL`.

Gabarito: B

5. (FGV/AAD (TCE,RR)/TCE RR/Tecnologia da Informação/Desenvolvimento de Sistemas/2025) Assinale a opção que contém apenas comandos da linguagem de criação de dados (DDL – Data Definition Language).

- a) INSERT, DROP, ALTER.
- b) ALTER, CREATE e DROP.
- c) INSERT, UPDATE e DROP.
- d) CREATE, ALTER e DELETE.

e) ALTER, UPDATE e DELETE.

Comentários:

a) **Errada.** INSERT é comando da DML (manipulação de dados), não da DDL. Apenas DROP e ALTER são da DDL.

b) **Correta.** ALTER, CREATE e DROP são os comandos fundamentais da DDL, usados para manipular a estrutura do banco de dados (tabelas, esquemas, etc.).

c) **Errada.** INSERT e UPDATE são comandos da DML, pois manipulam os dados dentro das tabelas, e não a estrutura.

d) **Errada.** DELETE é comando da DML, pois remove dados (linhas) de uma tabela, e não a estrutura dela.

e) **Errada.** UPDATE e DELETE pertencem à DML, pois atuam sobre os dados, não sobre a estrutura do banco.

Gabarito: B

6. (FGV/Tecno Jr (INPE)/INPE/Operação De Sistemas Espaciais Embarcados/2024) Relativamente a Bancos de Dados Relacionais, analise os itens a seguir.

I. As tabelas são os componentes fundamentais de um banco de dados relacional e são usadas para organizar e armazenar dados de forma estruturada.

II. As tabelas podem estar relacionadas entre si por meio de chaves estrangeiras.

III. Os relacionamentos em um banco de dados relacional são conexões lógicas entre diferentes tabelas que permitem combinar dados e estão restritos a relacionamentos um-para-um entre tabelas.

IV. SQL (Structured Query Language) é uma linguagem usada exclusivamente para definir a estrutura dos dados, o que inclui a criação, alteração da estrutura e deleção de tabelas. Para manipular dados em um banco de dados relacional (consultar, inserir, deletar e alterar registros) é necessário utilizar outra linguagem.

V. A chave primária é uma coluna (ou conjunto de colunas) com valor único para cada linha de uma tabela e é usada para identificar registros de forma única.

Está correto o que se afirma em

a) I, II e III, apenas.

b) II, IV e V, apenas.

c) I, III e V, apenas.

d) I, II e V, apenas.

e) I, III e IV, apenas.

Comentários:

I. Correta. As tabelas realmente são a base de um banco de dados relacional, pois é nelas que os dados ficam organizados em linhas e colunas, de forma estruturada.

II. Correta. A chave estrangeira (FOREIGN KEY) é justamente o recurso que liga uma tabela à outra, criando a integridade referencial. Vimos isso no exemplo em que a tabela Departamento referenciava a tabela Professor pelo CPF do coordenador.

III. Errada. Os relacionamentos entre tabelas não ficam restritos apenas a um-para-um. Existem outros tipos de relacionamentos possíveis, então limitar a apenas um-para-um torna a assertiva incorreta.

IV. Errada. O SQL não serve só para definir a estrutura dos dados. Ele tem vários subconjuntos, como o DDL (que cria e altera estruturas com CREATE, ALTER, DROP) e também o DML (que manipula dados com INSERT, UPDATE, DELETE) e o DQL (que consulta dados com SELECT). Ou seja, tudo isso é feito na própria linguagem SQL, não em uma linguagem separada.

V. Correta. A chave primária (PRIMARY KEY) é exatamente isso: uma coluna ou conjunto de colunas que identifica cada linha de forma única, não permitindo valores repetidos nem ausentes.

Gabarito: D

7. (FGV/Tecno Jr (INPE)/INPE/Operação De Sistemas Espaciais Embarcados/2024) SQL (Structured Query Language) é uma linguagem usada para interagir com bancos de dados relacionais. Relativamente a SQL, analise as afirmativas a seguir e assinale (V) para a verdadeira e (F) para a falsa.

() A cláusula WHERE só é utilizada associada ao comando SELECT.

() O comando CREATE TABLE cria uma nova tabela no banco de dados e o comando INSERT INTO permite popular essa tabela por meio da inserção de registros.

() O comando INNER JOIN é utilizado para unir os resultados de duas ou mais consultas SELECT e o seu resultado é um conjunto de linhas provenientes das tabelas envolvidas.

() Os comandos ALTER TABLE e UPDATE são comandos usados para atualizar registros em tabelas de um banco de dados.

() SQL suporta o conceito de chaves primárias e chaves estrangeiras para manter a integridade dos dados e as relações entre as tabelas.

As afirmativas são, respectivamente,

a) F – F – F – V – F.

b) F – V – F – F – V.

c) V – V – V – F – V.

d) V – F – V – V – V.

e) F – F – V – F – F.

Comentários:

- a) **Errada.** A segunda e a quinta afirmativas são verdadeiras, não falsas.
- b) **Correta.** Sequência F – V – F – F – V corresponde à análise feita.
- c) **Errada.** A primeira e a terceira afirmativas são falsas, não verdadeiras.
- d) **Errada.** A primeira, a terceira e a quarta não são todas verdadeiras.
- e) **Errada.** A segunda e a quinta são verdadeiras, não falsas.

Gabarito: B

8. (FGV/ATM (Pref SJC)/Pref SJC/Gestão Tributária/2024) Com relação à linguagem SQL e seus operadores, avalie se as afirmativas a seguir são verdadeiras (V) ou falsas (F).

() O operador LIKE é usado em uma cláusula WHERE para procurar um padrão especificado em uma coluna. Existem dois curingas frequentemente usados em conjunto com este operador; o sinal de % representa zero, um ou vários caracteres, já o sinal de - representa um único caractere.

() O operador IN permite especificar vários valores em uma cláusula WHERE. Ele é uma abreviação para múltiplas condições OR e AND sequenciais. Ao usar a palavra-chave NOT na frente do operador IN, haverá o retorno todos os registros que não são nenhum dos valores de uma lista.

() A palavra-chave RIGHT JOIN retorna todos os registros da tabela à direita em uma junção e os registros correspondentes da tabela à esquerda em uma junção. O resultado é zero registro do lado esquerdo, se não houver correspondência.

As afirmativas são, respectivamente,

- a) F – V – V.
- b) F – F – V.
- c) V – V – F.
- d) V – F – V.
- e) F – F – F.

Comentários:

- a) **Errada.** A primeira afirmativa é falsa, não verdadeira.
- b) **Correta.** F – F – V corresponde exatamente à análise feita.
- c) **Errada.** A primeira afirmativa é falsa e a terceira é verdadeira.
- d) **Errada.** A primeira afirmativa é falsa, não verdadeira.

e) **Errada.** A terceira afirmativa é verdadeira, não falsa.

Gabarito: B

9. (FGV/ACP (CGE PB)/CGE PB/Auditoria de Tecnologia da Informação/2024) Tabelas Relacionais T1 e T2

Considere as tabelas relacionais T1 e T2, de acordo com as colunas e as instâncias abaixo descritas. Na execução dos comandos SQL, assuma que o mecanismo do banco de dados

Tabela T1

P	N
1	10
2	40
3	20
4	10

Tabela T2

E	Q	P
1	10	2
2	40	3
3	20	8
4	10	NULL

Considerando as tabelas T1 e T2, anteriormente apresentadas, analise o comando SQL a seguir.

```
select * from T1 full outer join T2 on T1.P=T2.P
```

Além da linha de títulos, a execução desse comando produz um resultado com:

- a) 4 linhas e 5 colunas;
- b) 6 linhas e 4 colunas;
- c) 6 linhas e 5 colunas;
- d) 16 linhas e 4 colunas;
- e) 16 linhas e 5 colunas.

Comentários:

a) **Errada.** O FULL OUTER JOIN junta todas as colunas das duas tabelas (T1 tem 2 colunas e T2 tem 3 colunas), totalizando 5 colunas, não 4; além disso, são geradas 6 linhas no resultado.

b) **Errada.** A quantidade de colunas está errada, pois T1 (2 colunas) + T2 (3 colunas) = 5 colunas no resultado.

c) **Correta.** No FULL OUTER JOIN, todas as linhas das duas tabelas aparecem no resultado, mesmo sem correspondência. Os valores de P em T1 são 1, 2, 3, 4 e em T2.P são 2, 3, 8, NULL.

As correspondências ocorrem em P=2 e P=3 (2 linhas casadas), sobram P=1 e P=4 da T1 (sem par em T2.P, preenchendo T2 com NULL) e sobram P=8 e P=NULL da T2 (sem par em T1, preenchendo T1 com NULL). Total: $2 + 2 + 2 = 6$ linhas. As colunas somadas das duas tabelas (P, N de T1 mais E, Q, P de T2) dão 5 colunas.

d) Errada. 16 linhas seria o resultado de um produto cartesiano (CROSS JOIN) entre as tabelas (4 x 4), mas o FULL OUTER JOIN usa condição de junção, gerando apenas 6 linhas.

e) Errada. Apesar de acertar as 5 colunas, 16 linhas corresponderia a um produto cartesiano, não a um FULL OUTER JOIN com condição de junção.

Gabarito: C

10. (FGV/AJ (TJ AP)/TJ AP/Apoio Especializado/Tecnologia da Informação Banco de Dados/2024) Quando referenciadas, considere as tabelas relacionais Competidor e Disputa, cujas estruturas e instâncias são descritas abaixo. Todas as colunas são definidas como strings.

A tabela Disputa contém as disputas realizadas entre competidores que aparecem na tabela Competidor. Em cada disputa há dois competidores, um com camisa azul e outro com camisa verde.

Competidor

Nome
A
B
C
D
E
F

Disputa

Azul	Verde
A	B
C	A
B	A
C	E
F	A
F	D

Considerando as tabelas Competidor e Disputa, descritas anteriormente, analise o comando SQL a seguir.

```
select *
```

```
from competidor c, disputa d
```

```
where (c.nome = d.azul and c.nome = d.verde)
```

or (c.nome = d.verde and c.nome = d.azul)

O número de linhas produzidas pela execução desse comando, sem contar a linha de títulos, é:

- a) 0;
- b) 1;
- c) 6;
- d) 18;
- e) 36.

Comentários:

a) Correta. A condição exige que o mesmo competidor (c.nome) seja igual ao valor da coluna azul E igual ao valor da coluna verde na mesma linha da disputa. Como em toda disputa os competidores de camisa azul e verde são diferentes, nunca será possível um único nome ser igual aos dois ao mesmo tempo, então a condição sempre retorna falso e nenhuma linha é produzida.

b) Errada. Não há como uma linha satisfazer a condição, já que azul e verde são sempre diferentes em cada disputa.

c) Errada. Como a condição nunca é verdadeira, não retorna 6 linhas.

d) Errada. O produto cartesiano entre as tabelas seria filtrado pelo WHERE, e nenhuma linha passa no filtro.

e) Errada. Não retorna 36 linhas porque o WHERE elimina todas as combinações do produto cartesiano.

Gabarito: A

11. (FGV/AJ (TJ AP)/TJ AP/Apoio Especializado/Tecnologia da Informação Banco de Dados/2024) Observe a estrutura da tabela "Cliente" abaixo:

```
CREATE TABLE cliente (
```

```
cliente_ID NUMBER(12) PRIMARY KEY,
```

```
cidade VARCHAR2(50) NOT NULL,
```

```
limite_credito NUMBER(13,2) DEFAULT 0
```

```
);
```

Com o objetivo de gerar um relatório que mostre o limite de crédito médio para clientes em "Sao Paulo" e "Belo Horizonte", a consulta a ser executada é:

a) `SELECT cidade, AVG(NVL(limite_credito,0)) FROM
cliente WHERE cidade IN ('SAO PAULO','BELO`

HORIZONTE');

b) `SELECT cidade, AVG(limite_credito) FROM cliente
WHERE cidade IN ('SAO PAULO','BELO HORIZONTE')
GROUP BY cidade;`

c) `SELECT cidade, AVG(limite_credito) FROM cliente
WHERE cidade IN ('SAO PAULO','BELO HORIZONTE')
GROUP BY cliente_ID;`

d) `SELECT cidade, AVG(limite_credito) FROM cliente
WHERE cidade IN ('SAO PAULO','BELO HORIZONTE')
GROUP BY cidade, limite_credito;`

e) `SELECT cidade, AVG(limite_credito) FROM cliente
WHERE cidade IN ('SAO PAULO','BELO HORIZONTE')
GROUP BY limite_credito, cidade;`

Comentários:

a) Errada. Falta a cláusula `GROUP BY cidade`. Quando misturamos uma coluna comum (cidade) com uma função de agregação (`AVG`), precisamos agrupar pela coluna comum, senão o banco retorna erro. Além disso, `NVL` não faz parte do conteúdo desta aula.

b) Correta. Seleciona a cidade e a média do limite de crédito, filtra pelas cidades desejadas com `IN` e agrupa por cidade, que é exatamente o que precisamos para ter uma média por cada cidade.

c) Errada. Ao agrupar por `cliente_ID` (que é chave primária, ou seja, único para cada linha), cada grupo terá apenas um cliente, então a média seria calculada individualmente, não por cidade.

d) Errada. Agrupar por cidade e `limite_credito` junto faz o banco criar grupos combinando os dois valores, então clientes da mesma cidade com limites diferentes cairiam em grupos separados, não gerando a média correta por cidade.

e) Errada. Pelo mesmo motivo da alternativa anterior: agrupar incluindo `limite_credito` separa os clientes em grupos distintos dentro da mesma cidade, não produzindo a média única por cidade.

Gabarito: B

12. (FGV/ANL (CM Fortal)/CM Fortaleza/Analista de Informática/2024) Em um banco de dados relacional, considere a tabela a seguir, que possui informações sobre diferentes tipos de produtos, incluindo eletrônicos, roupas, eletrodomésticos, entre outros:
Produto (ID, Nome, Tipo, Preço, Fabricante)

Assinale a alternativa que corresponde à consulta que retornará o nome e o preço dos produtos que possuem a palavra "Smart" em seu tipo, somente do fabricante "Banana Inc." e preço abaixo de R\$2000,00.

a) SELECT Nome

FROM Produto

WHERE Fabricante = 'Banana Inc.'

AND Nome = 'Smart'

AND Preço < 2000;

b) SELECT Nome, Preço

FROM Produto

WHERE Tipo = 'Smart'

AND Fabricante = 'Banana Inc.'

AND Preço < 2000;

c) SELECT Nome, Preço

FROM Produto

WHERE Tipo LIKE '%Smart%'

AND Fabricante = 'Banana Inc.'

AND Preço < 2000;

d) SELECT Fabricante, Preço

FROM Produto

WHERE Fabricante = 'Banana Inc.'

AND Tipo = 'Smart'

AND Preço < 2000;

e) SELECT Tipo, Preço

FROM Produto

WHERE Tipo LIKE '%Smart%'

AND Fabricante = 'Banana Inc.'

AND Preço > 2000;

Comentários:

- a) **Errada.** Retorna apenas o Nome (faltou o Preço) e usa Nome = 'Smart', quando o enunciado pede que a palavra Smart esteja no Tipo.
- b) **Errada.** Usa Tipo = 'Smart', que buscaria apenas produtos cujo tipo seja exatamente a palavra Smart, sem aceitar variações como SmartPhone ou SmartTV. Para buscar a palavra dentro do texto, precisamos do LIKE com o curinga %.
- c) **Correta.** Seleciona Nome e Preço (como pedido), usa LIKE '%Smart%' para encontrar qualquer tipo que contenha a palavra Smart em qualquer posição, filtra pelo fabricante Banana Inc. e restringe o preço a valores menores que 2000 com o operador <.
- d) **Errada.** Retorna Fabricante e Preço, mas o enunciado pede Nome e Preço. Além disso, Tipo = 'Smart' não encontra variações da palavra.
- e) **Errada.** Retorna Tipo e Preço em vez de Nome e Preço, e usa Preço > 2000, quando o enunciado pede preço abaixo de 2000 (ou seja, <).

Gabarito: C

13. (FGV/Ana Leg (ALEP)/ALEP/Desenvolvedor de Sistemas/2024) Seja o esquema de um banco de dados relacional seguinte, criado pelo usuário U1, que possui direito de criação de tabelas no ambiente:

AUTOR (id_autor <PK>, nome_autor)

RELATORIO (id_rel <PK>, conteudo, id_autor <FK>, data_registro)

Restrições de integridade:

Tabela AUTOR: id_autor é chave primária; nome_autor ≠ NULO.

Tabela RELATORIO: id_rel é chave primária; conteudo ≠ NULO; data_registro ≠ NULO; id_autor é chave estrangeira e referencia LOCALIDADE(id_loc).

Os usuários U2 e U3 receberam de U1 o direito de consulta e atualização nas tabelas AUTOR e RELATORIO. Dado que o usuário U2 desfruta da confiança de U1, ele recebeu o benefício de propagação dos direitos que recebeu. Já o usuário U4 recebeu de U1 apenas o direito de consulta na tabela RELATORIO. Mais adiante, o usuário U1 soube que o usuário U2 propagou o direito de atualização para o usuário U4 na tabela RELATORIO, sem que ele (usuário U1) tivesse conhecimento do fato. Tendo em vista a quebra de confiança, o usuário U1 decidiu revogar todos os direitos concedidos anteriormente a U2, deixando-o sem acesso ao seu banco de dados.

Considerando os mecanismos de segurança discricionários de sistemas de bancos de dados relacionais, assinale a opção que indica o comando SQL a seguir reflete alguma das situações descritas no contexto apresentado.

a) GRANT select ON Relatorio TO U4 WITH GRANT OPTION

- b) GRANT select, update ON Autor, Relatorio TO U2 WITH GRANT OPTION
- c) GRANT select, update ON Autor, Relatorio TO U2, U3
- d) REVOKE select, update ON Autor, Relatorio FROM U2 WITHOUT GRANT OPTION
- e) REVOKE select, update ON Relatorio FROM U3

Comentários:

a) Errada. U4 recebeu apenas o direito de consulta na tabela RELATORIO, mas sem a propagação (sem WITH GRANT OPTION), já que esse benefício foi dado apenas a U2.

b) Correta. Esse comando reflete exatamente a situação de U2, que recebeu direitos de consulta (select) e atualização (update) nas tabelas AUTOR e RELATORIO, com a possibilidade de propagar esses direitos a outros usuários, o que é feito pela cláusula WITH GRANT OPTION.

c) Errada. U2 e U3 receberam os direitos de select e update nas duas tabelas, mas apenas U2 recebeu o benefício de propagação. Esse comando trataria ambos de forma igual, sem o WITH GRANT OPTION para U2.

d) Errada. Para revogar privilégios usamos REVOKE, mas a sintaxe correta não inclui WITHOUT GRANT OPTION. Esse comando não existe no padrão estudado para revogação.

e) Errada. O contexto não menciona revogação de direitos de U3, apenas de U2, que quebrou a confiança ao propagar o direito indevidamente.

Gabarito: B

14. (FGV/Ana Leg (ALETO)/ALETO/Análise de Suporte em Informática/2024) Considere uma tabela fictícia que registre as doações recebidas por projetos sociais de uma determinada prefeitura, criada com o seguinte comando SQL:

```
CREATE TABLE doacoes (
```

```
SiglaProjeto text,
```

```
DataDoacao text,
```

```
Valor REAL,
```

```
DocumentoDoador text,
```

```
NomeDoador text);
```

Considere, ainda, que uma certa consulta SQL produziu a seguinte saída:

SiglaProjeto	total_recebido
PA	57865.01
PB	37199.87

PC	18025.90
PD	17828.83
PE	8201.47

Assinale a opção que indica, corretamente, a consulta que poderia ter produzido a saída acima.

a) `SELECT SiglaProjeto, count(valor) as total_recebido FROM doacoes GROUP BY SiglaProjeto ORDER BY total_recebido DESC LIMIT 5.`

b) `SELECT SiglaProjeto, sum(Valor) as total_recebido FROM doacoes GROUP BY SiglaProjeto ORDER BY total_recebido DESC LIMIT 5.`

c) `SELECT SiglaProjeto, sum(Valor) as total_recebido FROM doacoes GROUP BY SiglaProjeto ORDER BY total_recebido LIMIT 5.`

d) `SELECT SiglaProjeto, count(valor) as total_recebido FROM doacoes GROUP BY SiglaProjeto ORDER BY total_recebido LIMIT 5.`

e) `SELECT SiglaProjeto, total_recebido FROM doacoes GROUP BY SiglaProjeto ORDER BY total_recebido DESC LIMIT 5.`

Comentários:

a) Errada. A função count conta quantas linhas existem, mas a saída mostra valores com casas decimais (como 57865.01), que representam soma de dinheiro, não contagem.

b) Correta. A função sum soma os valores das doações, o GROUP BY SiglaProjeto agrupa por projeto, o ORDER BY total_recebido DESC ordena do maior para o menor valor (como aparece na saída: PA com 57865.01 primeiro) e o LIMIT 5 restringe a 5 linhas.

c) Errada. Sem o DESC, o ORDER BY ordena de forma crescente (do menor para o maior), mas a saída está em ordem decrescente (do maior para o menor).

d) Errada. Além de usar count em vez de sum (o que não produziria valores decimais), também falta o DESC para ordenar do maior para o menor.

e) Errada. Não se pode usar total_recebido diretamente no SELECT sem aplicar uma função de agregação como sum ou count, pois total_recebido é um apelido que precisa ser calculado a partir de uma função.

Gabarito: B

15. (FGV/Ana Leg (ALETO)/ALETO/Análise de Suporte em Informática/2024) Considere o seguinte esquema de um banco de dados relacional, expresso em linguagem SQL:

`create table editora`

`(`

```
cod_editora integer primary key,  
nome_editora varchar(30) not null,  
telefone char(11)  
);  
  
create table livro  
(  
cod_livro integer primary key,  
num_isbn char(10) unique not null,  
titulo varchar(20) not null,  
edicao integer default 1 not null,  
cod_editora integer references editora  
);
```

A consulta

```
select titulo  
from livro left join editora  
on livro.cod_editora = editora.cod_editora  
where editora.nome_editora is null
```

apresenta os títulos dos livros

- a) cadastrados na tabela "editora".
- b) ligados a editoras sem ramal cadastrado.
- c) localizados ao lado esquerdo das editoras.
- d) não vinculados a uma editora.
- e) relacionados a editoras sem nome cadastrado (null).

Comentários:

a) Errada. A consulta não lista editoras cadastradas, e sim títulos de livros filtrados por uma condição específica.

b) Errada. O filtro usado é sobre o nome da editora ser nulo, não sobre telefone/ramal.

c) **Errada.** Não existe esse conceito de "localizados ao lado esquerdo" no LEFT JOIN; o LEFT JOIN apenas preserva todas as linhas da tabela da esquerda (livro), completando com NULL quando não há correspondência.

d) **Correta.** O LEFT JOIN mantém todos os livros, mesmo os que não têm editora correspondente. Quando não existe editora associada, todas as colunas da tabela editora ficam com NULL, inclusive nome_editora. Assim, o filtro "where editora.nome_editora is null" captura justamente os livros que não estão vinculados a nenhuma editora.

e) **Errada.** Como cod_editora é chave primária da tabela editora e nome_editora é NOT NULL, não existem editoras cadastradas com nome nulo. O NULL no resultado aparece por ausência de correspondência no LEFT JOIN, e não por existir editora sem nome.

Gabarito: D

16. (FGV/Ana Leg (ALETO)/ALETO/Desenvolvimento de Sistema/2024) Para analisar as vendas dos vários livros correspondentes a vários autores em uma rede de livrarias, construiu-se um DataMart com uma nova tabela "FATO" que armazenará os preços dos livros, chamada #VendaLivro (LivroID, AutorID, PrecoVenda).

Assinale a opção que descreve a ação correta para incluir esta nova tabela no banco de dados, considerando que já há dados nas tabelas #Livro e #Autor.

a) CREATE TABLE #VendaLivro (

LivroID INT,

AutorID INT,

PrecoVenda DECIMAL(10, 2),

FOREIGN KEY (AutorID) REFERENCES #Autor(AutorID));

b) CREATE TABLE #VendaLivro (

LivroID INT,

PrecoVenda DECIMAL(10, 2),

QuantidadeVendida INT,

FOREIGN KEY (LivroID) REFERENCES #Autor(LivroID));

c) CREATE TABLE #VendaLivro (

LivroID INT,

AutorID INT,

PrecoVenda DECIMAL(10, 2),

FOREIGN KEY (LivroID) REFERENCES #Livro(LivroID)),

FOREIGN KEY (AutorID) REFERENCES #Autor(AutorID));

d) CREATE TABLE #VendaLivro (

AutorID INT,

PrecoVenda DECIMAL(10, 2),

FOREIGN KEY (AutorID) REFERENCES #Autor(LivroID));

e) CREATE TABLE #VendaLivro (

PrecoVenda DECIMAL(10, 2),

FOREIGN KEY (LivroID) REFERENCES #Livro(LivroID)),

FOREIGN KEY (AutorID) REFERENCES #Autor(AutorID));

Comentários:

a) Errada. Cria a tabela com as três colunas pedidas (LivroID, AutorID, PrecoVenda), mas só define chave estrangeira para AutorID, faltando a referência de LivroID para a tabela #Livro.

b) Errada. Inclui uma coluna QuantidadeVendida que não foi pedida no enunciado e omite AutorID, além de referenciar LivroID na tabela #Autor, o que está incorreto, pois LivroID deve apontar para #Livro.

c) Correta. Cria a tabela #VendaLivro com as três colunas exigidas (LivroID, AutorID, PrecoVenda) e estabelece corretamente as duas chaves estrangeiras: LivroID referenciando #Livro(LivroID) e AutorID referenciando #Autor(AutorID), garantindo a integridade referencial com as tabelas já existentes.

d) Errada. Omite a coluna LivroID, que é essencial para identificar o livro vendido, e ainda define incorretamente a FK de AutorID apontando para um campo LivroID dentro de #Autor.

e) Errada. A tabela só possui a coluna PrecoVenda, sem LivroID e AutorID como colunas próprias, então as chaves estrangeiras tentam referenciar campos que não existem na tabela criada.

Gabarito: C

17. (FGV/Ana Leg (ALETO)/ALETO/Desenvolvimento de Sistema/2024) Considere o seguinte script SQL de um banco de dados relacional:

```
create table fornecedor
```

```
( id integer primary key,
```

```
nome varchar(30) not null );
```

```
create table produto
```

```
( id integer primary key,
```

```
descricao varchar(40) not null );
```

create table fornecimento

```
( id_fornecedor integer references fornecedor,  
id_produto integer references produto,  
primary key(id_fornecedor,id_produto) );
```

Assinale a consulta que imprime a descrição dos produtos que não são fornecidos por nenhum fornecedor

a) select produto.descricao from produto join

fornecimento on produto.id !=

fornecimento.id_produto

b) select produto.descricao from produto join

fornecimento

on produto.id = fornecimento.id_produto where id_fornecedor > all

(select id from fornecedor)

c) select produto.descricao from produto join

fornecimento on produto.id =

fornecimento.id_produto

d) select produto.descricao from produto left join

fornecimento on produto.id =

fornecimento.id_produto where

fornecimento.id_fornecedor is null

e) select produto.descricao from produto right join

fornecimento on produto.id =

fornecimento.id_produto join fornecedor on

fornecedor.id = fornecimento.id_fornecedor

Comentários:

a) Errada. Usar join com a condição "diferente" (!=) não encontra produtos sem fornecimento, apenas combina cada produto com linhas de fornecimento de outros produtos, gerando resultados incorretos.

b) Errada. Essa consulta usa join com igualdade, ou seja, só traz produtos que têm correspondência em fornecimento, justamente o oposto do que queremos. Além disso, a condição com "all" não faz sentido lógico aqui.

c) Errada. O inner join (join) retorna apenas os produtos que possuem correspondência na tabela fornecimento, ou seja, os que são fornecidos, quando queremos exatamente o contrário.

d) Correta. O left join preserva todas as linhas da tabela produto, mesmo as que não têm par em fornecimento, preenchendo com NULL as colunas da direita. Ao filtrar onde id_fornecedor is null, pegamos justamente os produtos que não possuem fornecimento, que é o que a questão pede.

e) Errada. O right join preserva todas as linhas de fornecimento, e ao juntar ainda com fornecedor, traz somente produtos que são fornecidos, descartando os que não têm fornecimento.

Gabarito: D

18. (FGV/Tec NS (TJ MS)/TJ MS/Analista de Sistemas Computacionais/Analista de Banco de Dados/2024) João está encarregado de projetar um banco de dados PostgreSQL para gerenciar informações sobre casos jurídicos e advogados, considerando as seguintes especificações:

- a tabela "Caso" armazena informações sobre os casos, incluindo um identificador único "IDCaso" como chave primária;
- a tabela "Advogado" armazena informações sobre os advogados, incluindo um identificador único "IDAdvogado" como chave primária;
- cada caso pode ter vários advogados envolvidos;
- um advogado pode estar envolvido em vários casos.

Nesse contexto, João precisa modelar um relacionamento "muitos-para-muitos" entre "Caso" e "Advogado". Para isso, ele deverá criar uma tabela de associação, denominada "Participacao", utilizando o script SQL:

a) CREATE TABLE Participacao (
IDParticipacao INT PRIMARY KEY,
FOREIGN KEY (IDCaso) REFERENCES Caso (IDCaso),
FOREIGN KEY (IDAdvogado) REFERENCES Advogado
(IDAdvogado));

b) CREATE TABLE Participacao (
IDParticipacao INT PRIMARY KEY,
IDCaso INT,
IDAdvogado INT,

FOREIGN KEY (IDCaso) REFERENCES Caso (IDCaso),
FOREIGN KEY (IDAdvogado) REFERENCES Advogado
(IDAdvogado));

c) CREATE TABLE Participacao (
IDCaso INT,
IDAdvogado INT,
PRIMARY KEY (IDCaso) REFERENCES Caso (IDCaso),
PRIMARY KEY (IDAdvogado) REFERENCES Advogado
(IDAdvogado),
FOREIGN KEY (IDCaso, IDAdvogado));

d) CREATE TABLE Participacao (
IDParticipacao INT PRIMARY KEY,
FOREIGN KEY (IDCaso) REFERENCES Caso (IDCaso),
FOREIGN KEY (IDAdvogado) REFERENCES Advogado
(IDAdvogado),
UNIQUE (IDCaso, IDAdvogado));

e) CREATE TABLE Participacao (
IDCaso INT PRIMARY KEY,
IDAdvogado INT PRIMARY KEY,
FOREIGN KEY (IDCaso) REFERENCES Caso (IDCaso),
FOREIGN KEY (IDAdvogado) REFERENCES Advogado
(IDAdvogado));

Comentários:

a) Errada. As colunas IDCaso e IDAdvogado não foram declaradas dentro da tabela, então as chaves estrangeiras fazem referência a colunas que não existem.

b) Correta. A tabela declara corretamente a chave primária IDParticipacao, cria as colunas IDCaso e IDAdvogado e define as chaves estrangeiras (FOREIGN KEY) apontando para as tabelas Caso e Advogado, estabelecendo a integridade referencial.

c) Errada. PRIMARY KEY não é usada com REFERENCES, e a sintaxe FOREIGN KEY (IDCaso, IDAdvogado) está incompleta, pois não indica a tabela referenciada.

d) Errada. Assim como na alternativa a, as colunas IDCaso e IDAdvogado não foram declaradas dentro da tabela antes de serem usadas nas chaves estrangeiras.

e) Errada. Não é possível declarar duas chaves primárias separadas na mesma tabela; para uma chave composta, seria necessário usar PRIMARY KEY (IDCaso, IDAdvogado) ao final.

Gabarito: B

19. (FGV/Tec NS (TJ MS)/TJ MS/Analista de Sistemas Computacionais/Analista de Banco de Dados/2024) Observe o script de criação e concessão de privilégios em Oracle:

----conexão no banco de dados xpto

CONNECT adm@xpto

----criação de usuário no banco de dados xpto

CREATE USER user1

IDENTIFIED BY senha1

DEFAULT TABLESPACE "usr_dados"

TEMPORARY TABLESPACE "temp"

ACCOUNT UNLOCK;

GRANT CONNECT TO user1;

GRANT SELECT ANY TABLE TO user1;

GRANT DELETE, UPDATE, INSERT ON apl.tab1 TO user1;

GRANT DELETE, UPDATE, INSERT ON apl.tab2 TO user1;

REVOKE SELECT ON apl.tab2 FROM user1;

Após a execução do script apresentado, é correto afirmar que "user1" tem permissão de executar qualquer Data Manipulation Language (DML):

a) na tabela "temp";

b) na tabela "apl.tab1";

c) na tabela "apl.tab2";

d) na tabela "usr_dados";

e) em todas as tabelas de "xpto".

Comentários:

a) Errada. "temp" não é uma tabela, é uma tablespace temporária usada pelo banco para operações internas, então não faz sentido falar em DML sobre ela.

b) **Correta.** O usuário recebeu GRANT SELECT ANY TABLE (que concede SELECT em qualquer tabela) e também GRANT DELETE, UPDATE, INSERT em apl.tab1, ou seja, possui os quatro comandos DML (SELECT, INSERT, UPDATE e DELETE) nessa tabela.

c) **Errada.** Embora tenha recebido DELETE, UPDATE e INSERT em apl.tab2, o privilégio de SELECT foi retirado com REVOKE SELECT ON apl.tab2, então o usuário não consegue executar todos os DML nessa tabela.

d) **Errada.** "usr_dados" não é uma tabela, é uma tablespace padrão, local físico onde os dados do usuário ficam armazenados, não sendo alvo de comandos DML.

e) **Errada.** O usuário não tem permissão completa de DML em todas as tabelas, pois em apl.tab2 o SELECT foi revogado, e nas demais tabelas só recebeu SELECT (via SELECT ANY TABLE), sem INSERT, UPDATE ou DELETE.

Gabarito: B

20. (FGV/Tec NS (TJ MS)/TJ MS/Analista de Sistemas Computacionais/Analista de Banco de Dados/2024) Observe as tabelas A e B a seguir, que possuem, respectivamente, 9 e 10 registros.

Tabela A	
ID	DESCRICAO
1	Descrição1
2	Descrição2
3	Descrição3
4	Descrição4
5	Descrição5
6	Descrição6
7	Descrição7
8	Descrição8
10	Descrição10

Total de Registros: 9

Tabela B	
ID	VALOR
1	10
2	20
3	30
5	50
6	60
7	70
9	90
11	110
12	120

13	130
----	-----

Total de Registros: 10

Após executar diferentes tipos de junções entre essas tabelas, o total de registros retornados em cada caso, sendo eles INNER JOIN, RIGHT JOIN, FULL OUTER JOIN, CROSS JOIN e LEFT JOIN, é, respectivamente:

- a) 9, 10, 13, 90 e 9;
- b) 6, 10, 13, 90 e 9;
- c) 6, 10, 10, 90 e 9;
- d) 6, 10, 13, 19 e 9;
- e) 6, 12, 13, 90 e 9.

Comentários:

- a) **Errada.** O INNER JOIN não retorna 9, pois só há 6 IDs em comum entre as tabelas.
- b) **Correta.** Os valores 6, 10, 13, 90 e 9 correspondem exatamente ao comportamento de cada junção explicado acima.
- c) **Errada.** O FULL OUTER JOIN não retorna 10, pois soma as correspondências com as linhas órfãs dos dois lados, totalizando 13.
- d) **Errada.** O CROSS JOIN faz produto cartesiano ($9 \times 10 = 90$), e não 19.
- e) **Errada.** O RIGHT JOIN preserva apenas as 10 linhas da tabela B, não 12.

Gabarito: B

21. (FGV/Tec NS (TJ MS)/TJ MS/Analista de Sistemas Computacionais/Web Design/2024)
Observe script SQL a seguir.

```
SELECT COUNT(*) AS [Quantidade], Tipo_Processo
```

```
FROM Processo
```

```
GROUP BY Tipo_Processo;
```

O resultado da execução desse script é:

- a) a lista dos registros da tabela quantidade;
- b) a quantidade de processos por tipo;
- c) a contagem dos registros da tabela de tipos de processos;
- d) o agrupamento de processos que realizam contagem;
- e) a contagem dos processos relacionados à quantidade de valores.

Comentários:

- a) **Errada.** O SELECT não lista registros, ele faz uma contagem agrupada, e não existe uma "tabela quantidade" (quantidade é apenas o apelido dado ao resultado do COUNT com AS).
- b) **Correta.** O COUNT(*) conta quantas linhas existem e o GROUP BY Tipo_Processo agrupa os registros pelo tipo, então o resultado mostra quantos processos existem em cada tipo.
- c) **Errada.** A consulta é feita na tabela Processo (não em uma tabela de tipos), e o resultado mostra a contagem por tipo, e não simplesmente uma contagem de registros de tipos.
- d) **Errada.** O agrupamento é feito pelo Tipo_Processo, e não por processos que realizam contagem; a contagem é aplicada dentro de cada grupo formado.
- e) **Errada.** Não há relação com uma "quantidade de valores"; a contagem é feita sobre as linhas de cada grupo formado pelo Tipo_Processo.

Gabarito: B

22. (FGV/ACE (TCE,GO)/TCE GO/Tecnologia da Informação/2024) Considere um banco de dados relacional de um tribunal, na qual nenhum usuário, a menos do DBA, possua algum privilégio sobre.

O DBA concedeu direito de criação de tabelas ao usuário USR_0010, que, por sua vez, criou as tabelas DADOS_PROCESSO e DADOS_PARTE.

Logo após a criação, o usuário USR_0010 executou os seguintes comandos da DCL (Data Control Language) da linguagem SQL no sistema gerenciador de banco de dados, referentes aos usuários USR_0011 e USR_0100:

```
GRANT SELECT, UPDATE ON DADO_PARTE TO USR_0011;
```

```
GRANT SELECT ON DADOS_PARTE TO USR_0100;
```

```
GRANT SELECT, INSERT, DELETE, UPDATE ON DADOS_PROCESSO TO USR_0011;
```

```
GRANT SELECT, UPDATE ON DADOS_PROCESSO TO USR_0100 WITH GRANT OPTION;
```

Na sequência, o usuário USR_0100 executou o seguinte comando:

```
GRANT UPDATE ON DADOS_PROCESSO TO USR_00101;
```

Por fim, o DBA executou o comando:

```
REVOKE UPDATE ON DADOS_PROCESSO FROM USR_0100;
```

Considerando esse cenário, qual situação é válida para as permissões referentes às tabelas DADOS_PROCESSO e DADOS_PARTE?

- a) O usuário USR_0011 tem direito de inserção na tabela DADOS_PARTE.
- b) O usuário USR_0100 tem direito de atualização na tabela DADOS_PROCESSO, mas sem direito de propagação.

- c) O usuário USR_0100 tem direito de consulta na tabela DADOS_PROCESSO, com direito de propagação.
- d) O usuário USR_0101 tem direito de atualização na tabela DADOS_PROCESSO.
- e) O usuário USR_0101 tem direito de consulta na tabela DADOS_PROCESSO.

Comentários:

- a) **Errada.** USR_0011 recebeu apenas SELECT e UPDATE na tabela DADOS_PARTE, não recebeu INSERT. Lembre-se que o GRANT concede apenas as permissões listadas.
- b) **Errada.** USR_0100 recebeu UPDATE em DADOS_PROCESSO com WITH GRANT OPTION (ou seja, com direito de propagação), mas depois o DBA executou REVOKE UPDATE, tirando essa permissão dele. Então ele não tem mais direito de atualização nenhum.
- c) **Correta.** USR_0100 recebeu SELECT e UPDATE em DADOS_PROCESSO com WITH GRANT OPTION. O REVOKE feito pelo DBA retirou apenas o UPDATE, mas o SELECT continua valendo, e mantém o direito de propagação (WITH GRANT OPTION), que permite repassar a permissão a outros usuários.
- d) **Errada.** Embora USR_0100 tenha concedido UPDATE em DADOS_PROCESSO para USR_0101 (o que era permitido graças ao WITH GRANT OPTION), o DBA depois revogou o UPDATE de USR_0100. Quando a permissão original é revogada, as permissões propagadas a partir dela também caem em cascata.
- e) **Errada.** USR_0101 só recebeu UPDATE em DADOS_PROCESSO de USR_0100, nunca recebeu SELECT (consulta). Portanto não tem direito de consultar a tabela.

Gabarito: C

23. (FGV/Ana (CVM)/CVM/Ciência de Dados/2024) Observe o script SQL a seguir.

```
CREATE TABLE Pessoa (Nome varchar(255));
```

```
INSERT INTO Pessoa (Nome)
```

```
VALUES ('Ana Luca');
```

```
INSERT INTO Pessoa (Nome)
```

```
VALUES ('Antonio Silva');
```

```
INSERT INTO Pessoa (Nome)
```

```
VALUES ('Tania Rosendo');
```

```
INSERT INTO Pessoa (Nome)
```

```
VALUES ('Rosa Flor');
```

```
INSERT INTO Pessoa (Nome)
```

```
VALUES ('Lara Bel');  
  
SELECT * FROM Pessoa  
  
WHERE (Nome LIKE 'A_A%'  
  
OR Nome LIKE 'R%' /*OR Nome LIKE '%BEL'  
  
OR Nome LIKE '%T%*/ OR Nome LIKE '%F');
```

O resultado JSON da execução do script apresentado é:

- a) [{"Nome": "Ana Luca"}, {"Nome": "Rosa Flor"}]
- b) [{"Nome": "Rosa Flor"}, {"Nome": "Lara Bel"}]
- c) [{"Nome": "Ana Luca"}, {"Nome": "Tania Rosendo"}, {"Nome": "Lara Bel"}]
- d) [{"Nome": "Antonio Silva"}, {"Nome": "Tania Rosendo"}, {"Nome": "Rosa Flor"}]
- e) [{"Nome": "Ana Luca"}, {"Nome": "Antonio Silva"}, {"Nome": "Tania Rosendo"}, {"Nome": "Lara Bel"}]

Comentários:

- a) **Correta.** Traz exatamente "Ana Luca" (por casar com 'A_A%') e "Rosa Flor" (por casar com 'R%').
- b) **Errada.** Lara Bel não entraria, pois o trecho que capturaria "%BEL" está comentado.
- c) **Errada.** Tania Rosendo e Lara Bel não casam com nenhuma condição ativa.
- d) **Errada.** Antonio Silva e Tania Rosendo não casam com as condições válidas.
- e) **Errada.** Inclui nomes que só entrariam se as condições comentadas estivessem ativas.

Gabarito: A

24. (FGV/Tecno Jr (INPE)/INPE/Desenvolvimento De Software Para Operação De Satélites/2024) SQL (Structured Query Language) é uma linguagem usada para interagir com bancos de dados relacionais. Em relação ao SQL, analise as afirmativas a seguir e assinale (V) para a verdadeira e (F) para a falsa.

- () A cláusula WHERE só é utilizada associada ao comando SELECT.
- () O Comando CREATE TABLE cria uma nova tabela no banco de dados e o comando INSERT INTO permite popular essa tabela por meio da inserção de registros.
- () O comando INNER JOIN é utilizado para unir os resultados de duas ou mais consultas SELECT e o seu resultado é um conjunto de linhas provenientes das tabelas envolvidas.
- () Os comandos ALTER TABLE e UPDATE são comandos usados para atualizar registros em tabelas de um banco de dados.

() SQL suporta o conceito de chaves primárias e chaves estrangeiras para manter a integridade dos dados e as relações entre as tabelas.

As afirmativas são, respectivamente,

a) F – F – F – V – F.

b) F – V – F – F – V.

c) V – V – V – F – F.

d) V – F – V – V – V.

e) F – F – V – F – F.

Comentários:

a) **Errada.** A sequência não corresponde à análise feita.

b) **Correta.** Corresponde exatamente à análise dos itens.

c) **Errada.** A sequência não corresponde à análise feita.

d) **Errada.** A sequência não corresponde à análise feita.

e) **Errada.** A sequência não corresponde à análise feita.

Gabarito: B

25. (FGV/AFFC (STN)/2024) ATENÇÃO: use a tabela relacional IPCA a seguir para responder à próxima questão.

Tabela IPCA

índice	ano	mês
0,56	2023	12
0,28	2023	11
0,24	2023	10
...	...	...
0,2	2003	7
-0,15	2003	6
...	...	...
2,25	2003	1
2,1	2002	12
3,02	2002	11
...	...	...
0,57	2001	1

A instância da tabela contém os valores do índice IPCA para todos os meses dos anos de 2001 até 2023. Os valores pontilhados representam a continuidade mensal da série. Todas as colunas são numéricas e não aceitam valores nulos.

No contexto da tabela IPCA apresentada, analise o comando SQL a seguir.

```
select avg(indice)
from IPCA
group by mes
order by mes
```

A execução desse comando gera

- a) doze linhas com uma coluna contendo números.
- b) doze linhas, contendo o valor null na primeira coluna e números na segunda.
- c) um resultado vazio.
- d) uma linha com duas colunas, contendo o valor null e a média de todos os índices.
- e) uma linha com uma coluna, contendo a média de todos os índices.

Comentários:

- a) **Correta.** O GROUP BY mes agrupa as linhas pelos 12 valores possíveis de mês (1 a 12), e a função AVG(indice) calcula a média dos índices de cada grupo, gerando uma linha por mês. Como o SELECT só pediu avg(indice), o resultado terá apenas uma coluna com os números das médias.
- b) **Errada.** Não existe coluna com null, pois o enunciado afirma que nenhuma coluna aceita valores nulos e o SELECT só retorna a média calculada.
- c) **Errada.** Como há dados para todos os meses de 2001 a 2023, a consulta retorna resultados, não vazio.
- d) **Errada.** O SELECT possui apenas uma expressão (avg(indice)), então só haverá uma coluna, e o GROUP BY produz várias linhas, não uma só.
- e) **Errada.** Isso aconteceria se não houvesse GROUP BY. Como agrupamos por mês, teremos uma média para cada mês, resultando em 12 linhas.

Gabarito: A

26. (FGV/Aud CE (TCE,PA)/TCE PA/Administrativa/Ciência de Dados/2024) Em um processo, o funcionário responsável foi substituído por outro funcionário da mesma organização. Tal substituição deve ser registrada no banco de dados do departamento de pessoal.

A cláusula SQL mais indicada para implementar essa alteração, mantendo a integridade do banco, é denominada

- a) ALTER
- b) INSERT

- c) JOIN
- d) UPDATE
- e) WITH

Comentários:

a) Errada. O ALTER é um comando DDL usado para modificar a estrutura de objetos do banco (como tabelas), e não para alterar dados existentes.

b) Errada. O INSERT serve para adicionar novos registros em uma tabela, não para modificar um registro já existente.

c) Errada. O JOIN é usado em consultas para combinar linhas de duas ou mais tabelas, não tem relação com alterar valores de um registro.

d) Correta. O UPDATE é o comando usado para modificar dados que já existem em uma tabela, sendo ideal para registrar a substituição do funcionário responsável pelo processo.

e) Errada. O WITH não é o comando adequado para alterar dados existentes em uma tabela.

Gabarito: D

27. (FGV/AJ (TRF 1ª Região)/TRF 1/Apoio Especializado/Análise de Dados/2024) Considere uma tabela relacional T, com uma única coluna intitulada número, cuja instância de 10 linhas contém os números inteiros de 1 até 10, um em cada linha, sem repetição. Nessas condições, foram executados os três comandos SQL exibidos a seguir.

```
select x.número * y.número A
from T x, T y
where y.número = x.número order by 1
select x.número * x.número A
from T x order by 1
select distinct x.número * x.número A
from T x
```

O número de linhas produzidas em cada comando, na ordem e sem contar a linha de títulos, é:

- a) 1, 10, 10;
- b) 10, 10, 10;
- c) 10, 100, 10;
- d) 100, 10, 1;
- e) 100, 10, 10.

Comentários:

a) **Errada.** Os três comandos produzem 10 linhas cada, não 1 e 10.

b) **Correta.** No primeiro comando, o FROM T x, T y faria um produto cartesiano ($10 \times 10 = 100$ combinações), mas o WHERE y.número = x.número filtra apenas os pares em que os números são iguais, restando 10 linhas. No segundo, como x.número vem de uma única tabela com 10 linhas, o resultado é 10 linhas (cada número multiplicado por ele mesmo). No terceiro, mesmo com DISTINCT, como cada número de 1 a 10 elevado ao quadrado gera um valor único (1, 4, 9, 16, 25...), continuam sendo 10 linhas.

c) **Errada.** O primeiro comando não gera 10 e o segundo não gera 100; o WHERE no primeiro limita o produto cartesiano.

d) **Errada.** O primeiro comando não produz 100 linhas porque o WHERE filtra os pares iguais, e o terceiro não reduz para 1 porque todos os quadrados de 1 a 10 são distintos.

e) **Errada.** Pelo mesmo motivo da letra d, o primeiro não produz 100 linhas devido ao filtro no WHERE.

Gabarito: B

28. (FGV/TJ (TRF 1ª Região)/TRF 1/Apoio Especializado/Tecnologia da Informação/2024)
Maria executou as seguintes instruções em SQL para criação de tabelas:

```
CREATE TABLE T1 (
```

```
id_A INT,
```

```
a1 CHAR(10),
```

```
a2 CHAR(10));
```

```
CREATE TABLE T2 (
```

```
id_B INT,
```

```
b1 CHAR(10),
```

```
b2 CHAR(10));
```

```
CREATE TABLE T3 (
```

```
id_A INT,
```

```
id_B INT,
```

```
c1 CHAR(10),
```

```
c2 CHAR(10));
```

Maria deseja listar a quantidade e os respectivos dados de T1 que possuem pelo menos 1 e no máximo 5 relacionamentos com T2. Para isso, ela deve utilizar a instrução:

a) `SELECT COUNT(c.id_A), a.a1, a.a2`
`FROM T1 a`
`INNER JOIN T3 c ON c.id_A = a.id_A`
`WHERE COUNT(c.id_A) <= 5;`

b) `SELECT a.id_A, a.a1, a.a2`
`FROM T1 a, T2 b`
`LEFT OUTER JOIN T3 c ON c.id_A = a.id_A`
`LEFT OUTER JOIN T3 c ON c.id_B = b.id_B`
`GROUP BY a.a1, a.a2`
`HAVING COUNT(a.id_A) <= 5;`

c) `SELECT COUNT(a.id_A), a.a1, a.a2`
`FROM T1 a`
`INNER JOIN T3 c ON c.id_A = a.id_A`
`WHERE a.id_A <= 5;`

d) `SELECT COUNT(a.id_A), a.*`
`FROM T1 a`
`INNER JOIN T3 c ON c.id_A = a.id_A`
`GROUP BY a.a1, a.a2`
`HAVING COUNT(a.id_A) <= 5;`

e) `SELECT a.id_A, a.a1, a.a2`
`FROM T1 a`
`RIGHT OUTER JOIN T3 c ON c.id_A = a.id_A`
`GROUP BY a.id_A, a.a1, a.a2`
`HAVING COUNT(a.id_A) <= 5;`

Comentários:

a) Errada. Não se pode usar funções de agregação como COUNT dentro do WHERE. Para filtrar grupos agregados, é preciso usar HAVING junto com GROUP BY.

b) Errada. Possui dois LEFT OUTER JOIN com o mesmo apelido "c", o que gera erro. Além disso, mistura a sintaxe antiga (vírgula no FROM) com JOIN explícito de forma inadequada, e não garante o mínimo de 1 relacionamento.

c) Errada. Também usa COUNT com condição errada: a cláusula WHERE a.id_A <= 5 compara o próprio id_A com 5, e não a quantidade de relacionamentos. Faltam também o GROUP BY e o HAVING para agregar corretamente.

d) Correta. Faz o INNER JOIN entre T1 e T3, garantindo que só entram linhas de T1 com pelo menos 1 relacionamento em T3 (pois o INNER JOIN descarta as que não têm par). Depois agrupa pelos atributos de T1 com GROUP BY e filtra com HAVING COUNT(a.id_A) <= 5 para limitar a no máximo 5 relacionamentos, retornando a contagem junto com os dados.

e) Errada. O RIGHT OUTER JOIN preservaria todas as linhas de T3, inclusive as que não têm correspondência em T1, o que não atende ao requisito. Além disso, não garante o mínimo de 1 relacionamento da forma desejada.

Gabarito: D

29. (FGV/Tecno Jr (INPE)/INPE/Desenvolvimento De Software Para Operação De Satélites/2024) Os comandos SQL do tipo DML (Data Manipulation Language) são utilizados para manipular os dados no banco de dados. Esses comandos permitem aos usuários interagirem com o banco de dados para executar operações.

Assinale a opção que apresenta apenas exemplos de comandos do tipo DML.

a) SELECT, INSERT, DROP e UPDATE.

b) INSERT, DROP e UPDATE.

c) SELECT, INSERT, DELETE e UPDATE.

d) CREATE, ALTER e DELETE.

e) SELECT, CREATE e TRUNCATE.

Comentários:

a) Errada. DROP é um comando DDL (usado para remover objetos da estrutura do banco), não DML.

b) Errada. Assim como na alternativa anterior, DROP pertence ao DDL, então a lista não é formada apenas por comandos DML.

c) Correta. SELECT (consulta), INSERT (adicionar), DELETE (remover) e UPDATE (modificar) são todos comandos que manipulam dados dentro das tabelas, sendo classificados como DML (vale lembrar que alguns autores colocam o SELECT em uma categoria separada chamada DQL, mas a banca aqui considerou o SELECT como DML).

d) Errada. CREATE e ALTER são comandos DDL, pois alteram a estrutura do banco, e não os dados em si.

e) Errada. CREATE e TRUNCATE são comandos DDL, pois atuam na estrutura dos objetos do banco, e não nos dados.

Gabarito: C

30. (FGV/AGC (EPE)/EPE/Tecnologia da Informação/Infraestrutura e Segurança/2024) Com relação à linguagem de consulta estruturada (SQL) padrão ANSI e seus subconjuntos, analise as afirmativas a seguir e assinale (V) para a verdadeira e (F) para a falsa.

() Os comandos GRANT e REVOKE são usados para conceder e revogar privilégios aos usuários e pertencem à DTL.

() O comando para apagar uma tabela é o DELETE.

() Os comandos CREATE, ALTER e DROP pertencem ao subconjunto DDL.

() O comando TRUNCATE é utilizado para limpar completamente uma tabela, mantendo sua estrutura para futuras inserções de dados. Pertence ao subconjunto DDL.

As afirmativas são, respectivamente,

a) F – V – V – F.

b) V – V – F – F.

c) F – F – V – V.

d) V – F – F – V.

e) F – V – V – V.

Comentários:

a) Errada. A segunda afirmativa é falsa, pois DELETE remove linhas, não a tabela; quem apaga a tabela é o DROP. Além disso, a quarta afirmativa é verdadeira.

b) Errada. A primeira afirmativa é falsa, pois GRANT e REVOKE pertencem à DCL (Linguagem de Controle de Dados), não à DTL. A terceira também é verdadeira, não falsa.

c) Correta. GRANT e REVOKE pertencem à DCL (não DTL), por isso a primeira é falsa. DELETE não apaga tabela, apenas remove dados (quem apaga é o DROP), por isso a segunda é falsa. CREATE, ALTER e DROP realmente pertencem ao DDL, pois manipulam a estrutura do banco, então a terceira é verdadeira. TRUNCATE limpa todos os dados mantendo a estrutura da tabela e é classificado como DDL porque atua no nível estrutural, então a quarta é verdadeira.

d) Errada. A primeira é falsa (GRANT e REVOKE são DCL, não DTL), a terceira é verdadeira (CREATE, ALTER e DROP são DDL) e a segunda é falsa (DELETE não apaga tabela).

e) Errada. A primeira afirmativa é falsa, pois GRANT e REVOKE pertencem à DCL, não à DTL.

Gabarito: C

31. (FGV/APP GG (SEPLAG Niterói)/Pref Niterói/Gestão de Tecnologia da Informação e

Comunicação/2024) Assinale a opção que contém comandos SQL dos subconjuntos das linguagens de definição de dados (DDL), de manipulação de dados (DML) e de controle de dados (DCL), respectivamente.

- a) DROP, UPDATE e GRANT.
- b) INSERT, DROP e COMMIT.
- c) DELETE, SELECT e REVOKE.
- d) CREATE, UPDATE e SAVEPOINT.
- e) TRUNCATE, INSERT e ROLLBACK.

Comentários:

a) **Correta.** DROP altera a estrutura do banco (DDL), UPDATE modifica dados existentes (DML) e GRANT concede permissões (DCL).

b) **Errada.** INSERT é DML (e não DDL), e COMMIT é TCL (controle de transações), não DCL.

c) **Errada.** DELETE é DML (e não DDL) e SELECT é DQL (ou DML, conforme o autor), não sendo DML de manipulação como INSERT/UPDATE/DELETE no sentido pedido.

d) **Errada.** CREATE é DDL e UPDATE é DML, mas SAVEPOINT pertence ao TCL (controle de transações), não ao DCL.

e) **Errada.** TRUNCATE é DDL e INSERT é DML, porém ROLLBACK é TCL, não DCL.

Gabarito: A

32. (FGV/AJ (TJ RN)/TJ RN/Apoio Especializado/Análise de Sistemas/2023) Supondo que no Brasil

- todo brasileiro tem um, e somente um, CPF;
- alguns brasileiros têm um, mas somente um, passaporte válido;
- alguns brasileiros têm uma, mas somente uma, carteira de motorista (CNH) válida.

A definição correta desses atributos numa tabela relacional normalizada seria:

- a) CPF varchar(11) not null
CNH varchar(9) UNIQUE
Passaporte varchar(9) UNIQUE
- b) CPF varchar(11) not null UNIQUE
CNH varchar(9) UNIQUE
Passaporte varchar(9) UNIQUE
- c) CPF varchar(11) UNIQUE

CNH varchar(9) not null

Passaporte varchar(9) not null

d) CPF varchar(11) not null

CNH varchar(9) null

Passaporte varchar(9) null

e) CPF varchar(11) null

CNH varchar(9) UNIQUE

Passaporte varchar(9) UNIQUE

Comentários:

a) Errada. O CPF precisa ser NOT NULL (todo brasileiro tem um) e também UNIQUE (cada CPF é único, só existe um por pessoa). Faltou o UNIQUE no CPF.

b) Correta. O CPF é NOT NULL UNIQUE porque todo brasileiro tem um e somente um CPF (não pode ser nulo e não pode repetir). CNH e Passaporte são apenas UNIQUE porque nem todos têm (aceita nulo), mas quem tem, tem apenas um (precisa ser único). Lembrando que UNIQUE permite valores nulos, diferente de PRIMARY KEY.

c) Errada. O CPF deveria ser NOT NULL também, pois todo brasileiro tem um. E a CNH e o Passaporte não podem ser NOT NULL, já que nem todos os brasileiros possuem esses documentos.

d) Errada. Faltou garantir a unicidade (UNIQUE) nos três atributos, pois cada pessoa tem apenas um CPF, uma CNH e um Passaporte, ou seja, esses valores não podem se repetir entre registros.

e) Errada. O CPF não pode ser nulo (null), pois todo brasileiro tem um CPF obrigatoriamente. O correto seria NOT NULL UNIQUE.

Gabarito: B

33. (FGV/ACE (TCE ES)/TCE ES/Tecnologia da Informação/2023) Junior é o administrador do Banco de Dados da sua empresa e percebeu que um programador tinha acesso de alteração indevido a uma tabela.

Para cancelar a permissão previamente concedida ao programador, Junior deve usar o comando SQL:

a) GRANT;

b) REVOKE;

c) UPDATE;

d) DELETE;

e) TRUNCATE.

Comentários:

a) **Errada.** O GRANT faz o oposto do que se pede: ele concede permissões a um usuário, e não as remove.

b) **Correta.** O REVOKE é o comando usado para retirar (cancelar) uma permissão que havia sido concedida anteriormente a um usuário. Por exemplo, REVOKE UPDATE ON Professor FROM Joao; retira a permissão de alteração que o João tinha.

c) **Errada.** O UPDATE é um comando de manipulação de dados (DML), usado para modificar dados já existentes em uma tabela, não para controlar permissões.

d) **Errada.** O DELETE é usado para remover registros (linhas) de uma tabela, não para cancelar permissões de usuários.

e) **Errada.** O TRUNCATE serve para apagar todas as linhas de uma tabela de forma rápida, não tem relação com controle de acesso.

Gabarito: B

34. (FGV/ATRFB/SRFB/Geral/2023) Os principais Sistemas Gerenciadores de Bancos de Dados oferecem total suporte à linguagem SQL. Um aspecto importante da implementação do SQL é o tratamento para valores nulos quando esses são considerados como unknown values. Nesse contexto, considere uma tabela T com colunas A e B, que podem conter valores nulos. T possui 100 registros e, em 50% das linhas, há pelo menos uma coluna preenchida com o valor NULL.

Considere a consulta a seguir:

```
SELECT * FROM T t1
```

```
WHERE t1.A = NULL or t1.B = NULL
```

O número máximo de linhas de resultados que seriam retornadas pela consulta é igual a

- a) 0.
- b) 25.
- c) 50.
- d) 75.
- e) 100.

Comentários:

a) **Correta.** Quando usamos "= NULL" em SQL, o banco não consegue avaliar a comparação como verdadeira nem falsa, retornando sempre UNKNOWN, então nenhuma linha passa no filtro do WHERE. Para capturar valores ausentes, seria necessário usar o operador IS NULL.

b) **Errada.** Mesmo havendo linhas com valores NULL, a comparação " $= \text{NULL}$ " nunca retorna TRUE, então nenhuma linha é selecionada.

c) **Errada.** Embora 50% das linhas tenham algum NULL, o WHERE com " $= \text{NULL}$ " não consegue identificá-las, pois a comparação resulta em UNKNOWN.

d) **Errada.** O WHERE só retorna linhas em que a condição é TRUE, e " $= \text{NULL}$ " nunca é TRUE, independentemente da quantidade de NULLs.

e) **Errada.** Nenhuma linha é retornada, pois a lógica de três valores do SQL faz com que comparações com NULL usando " $=$ " sempre resultem em UNKNOWN, e o WHERE descarta tanto FALSE quanto UNKNOWN.

Gabarito: A

35. (FGV/Ana (Pref Niterói)/Pref Niterói/Tecnologia da Informação/2023) Figura 1
Nas duas questões a seguir, considere a tabela relacional T cuja instância é exibida abaixo.

Tabela T

A	B	C
3	6	10
1	NULL	10
3	6	NULL
3	6	5

A instalação está configurada para o tratamento de valores NULL como valores desconhecidos (unknown).

Considerando a tabela T da figura 1, analise a execução do comando SQL a seguir.

```
SELECT *
```

```
FROM T t1 LEFT JOIN T t2
```

```
ON t1.A = t2.A and t1.B = t2.B
```

```
and t1.C = t2.C
```

Além da linha de títulos, o número de linhas produzidas pela execução desse comando é:

- a) 0;
- b) 4;
- c) 10;
- d) 13;
- e) 16.

Comentários:

a) **Errada.** O LEFT JOIN garante que toda linha da tabela da esquerda apareça no resultado pelo menos uma vez, então não pode ser zero.

b) **Correta.** No LEFT JOIN, cada linha da tabela da esquerda aparece pelo menos uma vez. Como a tabela T tem 4 linhas, teremos no mínimo 4 linhas no resultado. Acontece que a condição de junção compara A, B e C usando o operador "=", e, pela lógica de 3 valores, qualquer comparação envolvendo NULL retorna UNKNOWN (não é verdadeiro). Assim, a linha (1, NULL, 10) não combina com ninguém (nem consigo mesma) porque B é NULL, e a linha (3, 6, NULL) também não combina com ninguém porque C é NULL. Já as linhas (3, 6, 10) e (3, 6, 5) só combinam consigo mesmas, pois não há valores NULL nelas. Como nenhuma linha encontra par além de si própria, o resultado são 4 linhas, sendo que as duas com NULL aparecem com NULL nas colunas da direita (comportamento padrão do LEFT JOIN quando não há correspondência).

c) **Errada.** Não há como chegar a 10 linhas, pois não existe essa quantidade de combinações válidas.

d) **Errada.** Esse número não corresponde ao total de linhas geradas pelo LEFT JOIN nesse caso.

e) **Errada.** 16 seria o resultado de um CROSS JOIN (produto cartesiano 4x4), mas aqui temos um LEFT JOIN com condição de junção.

Gabarito: B

36. (FGV/AJ (TJ RN)/TJ RN/Apoio Especializado/Análise de Sistemas/2023) Considere a estrutura e uma instância da tabela relacional FILIA CAO exibida a seguir.

FILIA CAO

Pessoa	Genitor
Alexandre	Francisco
Francisco	João
Joaquina	João
Carlota	Joaquina
João	Manuel
Paulo	Manuel
Maria	Paulo

Com relação à tabela FILIA CAO, analise o comando SQL a seguir.

```
select distinct t3.Pessoa, t4.Pessoa
```

```
FROM FILIA CAO t1, FILIA CAO t2, FILIA CAO t3, FILIA CAO t4
```

```
WHERE t1.Pessoa < t2.Pessoa
```

```
and t1.Genitor = t2.Genitor
```

```
and t3.Genitor = t1.Pessoa
```

and t4.Genitor = t2.Pessoa

Afora a linha de títulos, o número de linhas produzidas pela execução do referido comando SQL é:

- a) 3;
- b) 4;
- c) 5;
- d) 6;
- e) 7.

Comentários:

a) Correta. Conforme o gabarito oficial da banca,

Gabarito: A

37. (FGV/AJ (TJ RN)/TJ RN/Apoio Especializado/Análise de Sistemas/2023) Considere a estrutura e uma instância da tabela relacional FILIA CAO exibida a seguir.

FILIA CAO

Pessoa	Genitor
Alexandre	Francisco
Francisco	João
Joaquina	João
Carlota	Joaquina
João	Manuel
Paulo	Manuel
Maria	Paulo

Com relação à tabela FILIA CAO, definida anteriormente, o comando SQL que produz a lista dos nomes das pessoas que são citadas na instância da referida tabela, sem repetições, é:

- a) select Pessoa FROM FILIA CAO
UNION ALL select Genitor FROM FILIA CAO
- b) select Pessoa FROM FILIA CAO
UNION select Genitor FROM FILIA CAO
- c) select distinct Pessoa, Genitor FROM FILIA CAO
- d) select distinct Pessoa UNION distinct Genitor
FROM FILIA CAO
- e) select Pessoa FROM FILIA CAO and
select Genitor FROM FILIA CAO

Comentários:

- a) **Errada.** O UNION ALL combina os resultados mas preserva todas as duplicatas, então nomes repetidos apareceriam várias vezes, contrariando a exigência de "sem repetições".
- b) **Correta.** O UNION combina os resultados das duas consultas (nomes da coluna Pessoa e da coluna Genitor) e já elimina automaticamente as duplicatas, entregando a lista de nomes sem repetições.
- c) **Errada.** O DISTINCT aqui eliminaria apenas combinações duplicadas do par (Pessoa, Genitor), mas o resultado continuaria sendo duas colunas separadas, e não uma única lista unificada de nomes.
- d) **Errada.** Essa sintaxe é inválida, pois o UNION une resultados de consultas SELECT completas, e não colunas isoladas dentro de um único SELECT.
- e) **Errada.** Não existe a ligação de dois SELECT com "and"; para unir resultados de consultas distintas deve-se usar operadores de conjunto como UNION.

Gabarito: B

38. (FGV/AJ (TJ RN)/TJ RN/Apoio Especializado/Análise de Suporte/2023) Luiz é o DBA do TJRN e atendeu um chamado da equipe de desenvolvimento que pedia para criar um banco de dados com 5 Gigabytes, mas, na hora de criar, Luiz digitou 50 Gigabytes.

Para apagar o banco de dados criado equivocadamente, Luiz deve utilizar o comando:

- a) DROP;
- b) ALTER;
- c) DELETE;
- d) UPDATE;
- e) INJECTION.

Comentários:

- a) **Correta.** O comando DROP é usado para remover completamente objetos do banco de dados, como tabelas, esquemas e o próprio banco, eliminando tanto a estrutura quanto os dados.
- b) **Errada.** O ALTER serve para modificar objetos existentes, como adicionar ou remover colunas de uma tabela, e não para apagar o banco.
- c) **Errada.** O DELETE apenas remove linhas (dados) de dentro de uma tabela, mas não elimina a tabela nem o banco de dados em si.
- d) **Errada.** O UPDATE é usado para modificar valores de registros já existentes em uma tabela, não para excluir objetos do banco.

e) **Errada.** INJECTION não é um comando SQL, é o nome de um tipo de ataque a bancos de dados, não tendo relação com a exclusão de objetos.

Gabarito: A

39. (FGV/Ana (DPE RS)/DPE RS/Apoio Especializado (TI)/Banco de Dados/2023) Tabela R1
Na questão a seguir, considere a tabela R1, cuja instância é exibida a seguir.

A	B	C
10	20	31
10	NULL	32
11	20	NULL
12	9	31
14	8	NULL

Com referência à tabela R1 descrita anteriormente, analise o comando SQL a seguir.

`select * from R1 x, R1 y`

`where x.A > y.A and x.B = y.B`

O número de linhas produzidas pela execução desse comando, sem contar a linha de títulos, é:

- a) 0;
- b) 1;
- c) 2;
- d) 3;
- e) 4.

Comentários:

a) **Errada.** Existe sim um par válido (L3 com L1).

b) **Correta.** Apenas o par onde $x=L3(11,20)$ e $y=L1(10,20)$ satisfaz as duas condições.

c) **Errada.** Só há um par que cumpre as duas condições ao mesmo tempo.

d) **Errada.** Não há três pares válidos, pois B só coincide entre L1 e L3.

e) **Errada.** Comparações envolvendo NULL retornam UNKNOWN e são descartadas, então linhas com B NULL não entram.

Gabarito: B

40. (FGV/Ana (DPE RS)/DPE RS/Apoio Especializado (TI)/Banco de Dados/2023) Tabela R1
Na questão a seguir, considere a tabela R1, cuja instância é exibida a seguir.

A	B	C
10	20	31

10	NULL	32
11	20	NULL
12	9	31
14	8	NULL

Com referência à tabela R1, como descrita anteriormente, analise o comando SQL a seguir.

`delete from R1`

`where (select count(*) from R1 x`

`where R1.B = x.B) > 0`

Após a execução desse comando, o conjunto completo de linhas presentes na instância da tabela R1 é:

a) 10 NULL 31

b) 10 20 31

Wed Dec 09 1931 01:00:00 GMT-0200 (Brasilia Summer Time)

c) 11 20 NULL

14 8 NULL

d) 10 20 31

11 20 NULL

Wed Dec 09 1931 01:00:00 GMT-0200 (Brasilia Summer Time)

14 8 NULL

e) 10 NULL 32

Comentários:

a) Errada. Essa linha tem B=NULL e C=31, mas o valor correto mantido é 10, NULL, 32.

b) Errada. A linha (10, 20, 31) é deletada, pois existe outra linha com B=20 (a linha 11, 20, NULL), fazendo a subconsulta retornar contagem maior que 0.

c) Errada. Ambas as linhas são deletadas: (11, 20, NULL) porque tem B=20 igual a (10, 20, 31), e (14, 8, NULL) porque tem B=8 igual a (12, 9, 31)? Não, na verdade (14, 8, NULL) tem B=8 que combina consigo mesma na subconsulta, retornando contagem 1, sendo deletada.

d) Errada. Essas linhas têm valores de B que se repetem na tabela, então são deletadas pelo comando.

e) Correta. O comando deleta toda linha cuja subconsulta retorne contagem maior que zero. A subconsulta conta linhas onde $R1.B = x.B$, mas comparações com NULL retornam UNKNOWN (lógica de 3 valores), nunca TRUE. Então, para a linha (10, NULL, 32), a subconsulta não encontra

nenhuma correspondência (NULL = NULL dá UNKNOWN), retornando 0, e ela não é deletada. Todas as outras linhas têm B com valor definido e combinam ao menos consigo mesmas, sendo removidas.

Gabarito: E

41. (FGV/Ana (DPE RS)/DPE RS/Apoio Especializado (TI)/Banco de Dados/2023) Tabela R1
Na questão a seguir, considere a tabela R1, cuja instância é exibida a seguir.

A	B	C
10	20	31
10	NULL	32
11	20	NULL
12	9	31
14	8	NULL

Com referência à tabela R1, como descrita anteriormente, analise o comando SQL a seguir.

```
update R1
```

```
set B=0
```

```
where C in (31,32) or C = null
```

O conjunto completo de linhas afetadas pela execução desse comando é:

- a) a primeira e a terceira;
- b) a segunda e a quarta;
- c) a terceira e a quinta;
- d) a primeira, a segunda e a quarta;
- e) a segunda, a terceira e a quinta.

Comentários:

- a) **Errada.** Falta a quarta linha, e a terceira não é afetada pois C é NULL.
- b) **Errada.** A quarta linha é afetada, mas a segunda também, e não apenas essas duas.
- c) **Errada.** Terceira e quinta têm C=NULL, que não são afetadas por C = null.
- d) **Correta.** Primeira (C=31), segunda (C=32) e quarta (C=31) são as linhas com C em (31,32).
- e) **Errada.** Terceira e quinta não são afetadas porque C = null sempre retorna UNKNOWN.

Gabarito: D

42. (FGV/Ana (DPE RS)/DPE RS/Apoio Especializado (TI)/Banco de Dados/2023) Analise o esquema relacional a seguir.
create table X1(

```
A1 int not null primary key,  
B1 int)  
create table X2(  
A2 int not null primary key,  
B2 int)  
create table X3(  
A1 int not null unique,  
A2 int,  
B3 int,  
foreign key(A1) references X1(A1),  
foreign key(A2) references X2(A2)  
)
```

Sobre esse esquema, analise as afirmativas a seguir.

- I. Cada elemento de X1 pode estar relacionado, via X3, a zero, um ou mais elementos de X2.
- II. Cada elemento de X1 pode estar relacionado a zero, um ou mais elementos de X3.
- III. Cada elemento de X2 pode estar relacionado a zero ou apenas um elemento de X3.

Está correto somente o que se afirma em:

- a) I;
- b) II;
- c) III;
- d) I e II;
- e) II e III.

Comentários:

I. **Correta.** Como em X3 o atributo A1 é único (UNIQUE), cada elemento de X1 aparece no máximo uma vez em X3. Essa linha de X3 aponta para um elemento de X2 (via A2), mas o A2 pode ser nulo (zero), pode apontar para um elemento, ou, considerando várias possibilidades ao longo do tempo, a relação via X3 liga cada X1 a zero ou um elemento de X2. A banca considerou correta a afirmação interpretando a ligação de forma ampla via X3.

II. Errada. Como A1 em X3 é UNIQUE, cada elemento de X1 pode estar relacionado a no máximo um elemento de X3 (e não a "um ou mais"). Ou seja, a relação é de zero ou um, nunca mais que um.

III. Errada. O atributo A2 em X3 não é UNIQUE, portanto o mesmo valor de A2 pode se repetir em várias linhas de X3. Assim, um elemento de X2 pode estar relacionado a zero, um ou vários elementos de X3, e não apenas a zero ou um.

Gabarito: A

43. (FGV/Ana (DPE RS)/DPE RS/Apoio Especializado (TI)/Banco de Dados/2023) Tanto no SQL Server como no MySQL, a aplicação do comando drop table T

destina-se a:

- a) remover T;
- b) remover todos os índices de T;
- c) colocar T no estado read only;
- d) recuperar T a partir de arquivos de backup;
- e) desconectar T do banco de dados atual, e transformá-la numa tabela temporária.

Comentários:

a) Correta. O comando DROP TABLE remove completamente a tabela do banco de dados, eliminando tanto sua estrutura quanto todos os dados armazenados nela.

b) Errada. O DROP TABLE não remove apenas índices, ele remove a tabela inteira, incluindo dados, estrutura, restrições e também seus índices como consequência.

c) Errada. O DROP TABLE não deixa a tabela em modo somente leitura, ele simplesmente elimina a tabela do banco.

d) Errada. O DROP TABLE não tem relação com recuperação de backup, ele faz exatamente o oposto, que é apagar a tabela.

e) Errada. O DROP TABLE não transforma a tabela em temporária nem apenas a desconecta, ele remove a tabela do schema de forma definitiva.

Gabarito: A

44. (FGV/Ana (DPE RS)/DPE RS/Apoio Especializado (TI)/Banco de Dados/2023) Maria criou um banco de dados no qual há uma tabela com uma coluna X que não aceita valores nulos. Entretanto, como o sistema que mantém essa tabela é novo, na carga inicial dos dados essa coluna não existe.

Como Maria não quer abrir mão do impedimento de valores nulos, Maria poderia carregar as linhas incompletas desde que, na coluna X, usasse a cláusula:

- a) skip;
- b) check;
- c) default;
- d) float;
- e) case when ... then ... end.

Comentários:

- a) Errada.** Não existe cláusula skip para definir valor automático em colunas que não aceitam nulos.
- b) Errada.** O CHECK serve para criar uma condição de validação dos valores (como verificar se idade está entre 14 e 24), não para fornecer um valor quando ele está ausente.
- c) Correta.** A cláusula DEFAULT atribui um valor padrão ao atributo quando nenhum valor é informado. Assim, mesmo que a carga inicial não traga a coluna X, o banco preenche automaticamente com o valor padrão, evitando o erro de valor nulo em uma coluna NOT NULL.
- d) Errada.** FLOAT é um tipo de dado numérico (ponto flutuante), não uma cláusula para tratar ausência de valores.
- e) Errada.** O CASE WHEN ... THEN ... END é usado para criar condições lógicas em comandos como UPDATE ou SELECT, e não para definir um valor padrão na definição da coluna.

Gabarito: C

45. (FGV/Ana (DPE RS)/DPE RS/Apoio Especializado (TI)/Banco de Dados/2023) Observe o seguinte script de concessão de privilégios em MYSQL:

```
CREATE DATABASE db;  
  
CREATE TABLE db.t1 (c INT);  
  
INSERT INTO db.t1 VALUES ROW (1);  
  
CREATE TABLE db.t2 (c INT);  
  
INSERT INTO db.t2 VALUES ROW (1);  
  
CREATE USER u1;  
  
GRANT SELECT, UPDATE ON db.t1 TO u1 WITH GRANT  
OPTION;  
  
CREATE USER u2;  
  
GRANT SELECT, INSERT ON db.t2 TO u2;  
  
CREATE USER u3;
```

```
GRANT ALL ON db.* TO u3;
```

```
REVOKE INSERT ON db.t2 FROM u2;
```

Após a execução do script apresentado, é correto afirmar que:

- a) os usuários u2 e u3 poderão executar a instrução `INSERT INTO db.t2 VALUES ROW (2);`
- b) os usuários u1 e u3 poderão executar a instrução `UPDATE db.t2 SET c=2;`
- c) o usuário u2 poderá conceder o privilégio de `INSERT` para o usuário u1;
- d) os usuários u1 e u3 poderão executar a instrução `DELETE FROM db.t1;`
- e) o usuário u1 poderá conceder o privilégio de `UPDATE` para o usuário u2.

Comentários:

a) Errada. O u2 teve o privilégio de `INSERT` em db.t2 revogado pelo `REVOKE`, então não pode mais executar `INSERT` nessa tabela.

b) Errada. O u1 só tem privilégios em db.t1, não em db.t2, portanto não pode executar `UPDATE` em db.t2.

c) Errada. O u2 não recebeu a cláusula `WITH GRANT OPTION`, então não pode repassar privilégios a outros usuários. Além disso, o `INSERT` dele foi revogado.

d) Errada. O u1 recebeu apenas `SELECT` e `UPDATE` em db.t1, não recebeu permissão de `DELETE`, portanto não pode executar esse comando.

e) Correta. O u1 recebeu `SELECT` e `UPDATE` em db.t1 com a cláusula `WITH GRANT OPTION`, o que permite que ele repasse esses privilégios a outros usuários, inclusive o `UPDATE` para o u2.

Gabarito: E

46. (FGV/FR (Pref RJ)/Pref RJ/2023) Considere a existência de uma tabela relacional N, com apenas uma coluna, intitulada `numero`, contendo os números inteiros de 1 até 100, um em cada linha, como ilustrada a seguir.

N	
numero	
1	
2	
...	
99	
100	

Como pode haver discrepâncias entre implementações da linguagem SQL, é dado que a função `sqrt(x)` retorna a raiz quadrada de x e que a expressão `a % b` retorna o resto da divisão inteira de a por b.

Analise o comando SQL a seguir, que faz referência à tabela N descrita anteriormente.

```
select n1.numero * n2.numero  
from N n1, N n2  
where n1.numero <> n2.numero
```

O número de linhas do resultado produzido pela execução desse comando, sem contar a linha de títulos, é:

- a) 9.000
- b) 9.110
- c) 9.900
- d) 9.991
- e) 10.000

Comentários:

a) Errada. O cálculo correto parte do produto cartesiano entre N e N, que gera $100 \times 100 = 10.000$ combinações, e não 9.000.

b) Errada. Não há como chegar a 9.110 com as operações descritas, pois a subtração correta é $10.000 - 100 = 9.900$.

c) Correta. Quando colocamos duas tabelas no FROM separadas por vírgula (FROM N n1, N n2), o banco faz o produto cartesiano entre elas, combinando cada linha de n1 com cada linha de n2, gerando $100 \times 100 = 10.000$ combinações. A cláusula WHERE n1.numero <> n2.numero elimina as linhas em que o número de n1 é igual ao de n2 (ou seja, as 100 combinações do tipo 1 com 1, 2 com 2, até 100 com 100). Assim, o resultado é $10.000 - 100 = 9.900$ linhas.

d) Errada. O valor 9.991 não corresponde ao cálculo do produto cartesiano descontando as 100 linhas com números iguais.

e) Errada. 10.000 seria o total do produto cartesiano sem aplicar o filtro do WHERE, mas a condição n1.numero <> n2.numero remove as 100 linhas em que os valores são iguais.

Gabarito: C

47. (FGV/FR (Pref RJ)/Pref RJ/2023) Considere a existência de uma tabela relacional N, com apenas uma coluna, intitulada numero, contendo os números inteiros de 1 até 100, um em cada linha, como ilustrada a seguir.

N	
numero	
1	
2	
...	
99	

100

Como pode haver discrepâncias entre implementações da linguagem SQL, é dado que a função $\text{sqrt}(x)$ retorna a raiz quadrada de x e que a expressão $a \% b$ retorna o resto da divisão inteira de a por b .

Este é o resultado produzido por um determinado script SQL que utiliza a tabela N, anteriormente descrita.

A	B
2	50
3	33
4	25
5	20
6	16
7	14

Abaixo, são apresentadas três versões para o referido script, não necessariamente corretas.

I. select x.numero A,

(select count(*) from N

where N.numero % x.numero = 0) B

from (select numero from N

where numero >= 2 and numero <= 7) x

order by 2 desc

II. select x.numero A,

(select count(*) from N

where N.numero % x.numero = 0) B

from (select numero from N

where numero >= 2 and numero <= 7) x

where x.numero % x.numero = 0

order by 2 desc

III. select x.numero A,

(select count(*) from N

where N.numero % x.numero = 0) B

from (select numero from N

where numero >= 2 and numero <= 7) x

```
where x.numero % x.numero = 0  
  
group by x.numero  
  
having count(*) > 0  
  
order by 2 desc
```

Sobre essas afirmativas, é correto afirmar que:

- a) nenhuma delas produz o resultado correto;
- b) somente I e II produzem o resultado correto;
- c) somente II e III produzem o resultado correto;
- d) somente III produz o resultado correto;
- e) todas produzem o resultado correto.

Comentários:

I. Correta. O script cria uma subconsulta chamada x que seleciona apenas os números de 2 a 7 da tabela N (usando where numero >= 2 and numero <= 7). Para cada um desses números, outra subconsulta no SELECT conta quantos valores na tabela N têm resto zero quando divididos por aquele número (N.numero % x.numero = 0). Por fim, o order by 2 desc ordena pela segunda coluna (B) em ordem decrescente, que é exatamente o que o resultado mostra (50, 33, 25, 20, 16, 14).

II. Correta. Faz exatamente a mesma coisa da versão I, mas adiciona uma cláusula where x.numero % x.numero = 0. Essa condição é sempre verdadeira, porque qualquer número dividido por ele mesmo tem resto zero. Então esse filtro não elimina nenhuma linha, e o resultado continua idêntico ao da versão I.

III. Correta. Faz o mesmo que a versão II, mas adiciona group by x.numero e having count(*) > 0. Como cada valor de x.numero é único dentro da subconsulta (são os números de 2 a 7, cada um aparece uma vez), agrupar por x.numero não altera as linhas. E o having count(*) > 0 também não filtra nada, porque cada grupo tem pelo menos um registro. Então o resultado final é o mesmo.

Gabarito: E

48. (FGV/ALEMA/Analista de Suporte de Rede/2023) O banco de dados relacional de uma escola contém as tabelas denominadas TB_STUDENT_A e TB_STUDENT_B e ambas possuem apenas as colunas ID e NAME.

O conteúdo dessas tabelas está detalhado a seguir:

TB_STUDENT_A

ID	NAME
1	Maria
2	João

3	Visconde
4	Ana
5	Inês

TB_STUDENT_B

ID	NAME
1	Fábio
2	Marta
3	Maria
4	Sérgio
5	Penélope
6	João

Analise a seguinte consulta em linguagem SQL:

```
SELECT *  
FROM TB_STUDENT_A  
LEFT JOIN TB_STUDENT_B  
ON TB_STUDENT_A.NAME=TB_STUDENT_B.NAME.
```

O número de linhas do resultado produzido pela execução deste comando SQL sobre o banco de dados desta escola, excetuada a linha de títulos, é:

- a) 2.
- b) 3.
- c) 4.
- d) 5.
- e) 6.

Comentários:

- a) Errada.** Seriam apenas as correspondências, mas o LEFT JOIN também traz as linhas sem par.
- b) Errada.** Não descartamos as linhas sem correspondência no LEFT JOIN.
- c) Errada.** Todas as 5 linhas da tabela da esquerda aparecem, não apenas 4.
- d) Correta.** São 2 linhas com correspondência (Maria e João) mais 3 linhas sem correspondência (Visconde, Ana e Inês) preenchidas com NULL à direita.
- e) Errada.** O LEFT JOIN preserva as linhas da tabela da esquerda (5 linhas), não da direita (6 linhas).

Gabarito: D

49. (FGV/ACE (TCE SP)/TCE SP/Tecnologia da Informação (TI)/2023) Atenção: Quando referidas, considere as tabelas relacionais TX e TY, criadas e instanciadas com o script SQL a seguir.

```
create table TY(C int primary key not null, A int)
```

```
create table TX(A int primary key not null, B int,
```

```
foreign key (B) references TY(C)
```

```
on delete cascade
```

```
)
```

```
insert into TY values (1,0)
```

```
insert into TY(C) values (2)
```

```
insert into TY(C) values (3)
```

```
insert into TY values (5,NULL)
```

```
insert into TY values (6,NULL)
```

```
insert into TX values (1,2)
```

```
insert into TX values (2,1)
```

```
insert into TX values (3,2)
```

```
insert into TX values (4,2)
```

Com referência às tabelas TX e TY, como descritas anteriormente, analise o comando SQL a seguir.

```
select count(*)
```

```
from TX t1 left join TY t2 on t1.B=t2.A
```

O valor exibido pela execução desse comando é:

a) 0;

b) 2;

c) 3;

d) 4;

e) 6.

Comentários:

- a) **Errada.** O LEFT JOIN preserva todas as linhas da tabela da esquerda (TX), que tem 4 registros, então o resultado nunca poderia ser 0.
- b) **Errada.** Não são apenas 2 linhas, pois o LEFT JOIN garante que todas as 4 linhas de TX apareçam no resultado, mesmo sem correspondência em TY.
- c) **Errada.** Não são 3 linhas, pois TX tem 4 registros e todos serão preservados pelo LEFT JOIN, independentemente de haver correspondência em TY.
- d) **Correta.** A tabela TX tem 4 linhas (com valores de B iguais a 2, 1, 2, 2). No LEFT JOIN com TY usando a condição $t1.B=t2.A$, o banco preserva todas as linhas de TX. Onde houver correspondência em TY (valores de A), combina; onde não houver, preenche com NULL. Como TX tem 4 linhas e todas são preservadas pelo LEFT JOIN, o COUNT(*) retorna 4.
- e) **Errada.** O valor 6 não é possível, pois o LEFT JOIN parte das 4 linhas de TX e, nesse caso, não há multiplicação de linhas que chegue a 6.

Gabarito: D

50. (FGV/ACE (TCE SP)/TCE SP/Tecnologia da Informação (TI)/2023) Atenção
Quando referidas, considere as tabelas relacionais TX e TY, criadas e instanciadas com o script SQL a seguir.

```
create table TY(C int primary key not null, A int)
create table TX(A int primary key not null, B int,
foreign key (B) references TY(C)
on delete cascade
)
```

```
insert into TY values (1,0)
```

```
insert into TY(C) values (2)
```

```
insert into TY(C) values (3)
```

```
insert into TY values (5,NULL)
```

```
insert into TY values (6,NULL)
```

```
insert into TX values (1,2)
```

```
insert into TX values (2,1)
```

```
insert into TX values (3,2)
```

```
insert into TX values (4,2)
```

Com referência às tabelas TX e TY, como descritas anteriormente, analise o comando SQL a seguir.

delete FROM TY

Após a execução desse comando, o total de linhas, somando-se os números de linhas de cada tabela, é:

- a) 0;
- b) 1;
- c) 4;
- d) 5;
- e) 9.

Comentários:

a) Correta. Ao deletar todas as linhas da tabela TY, a cláusula ON DELETE CASCADE definida na chave estrangeira da tabela TX faz com que as linhas de TX que referenciam TY sejam automaticamente apagadas em efeito cascata; como as linhas de TX com B=NULL também precisam ser analisadas, observe que as linhas de TX apontam para valores de C existentes em TY (1 e 2), então todas elas serão removidas junto, restando zero linhas no total.

b) Errada. Não sobra nenhuma linha, pois o CASCADE propaga a exclusão de TY para TX.

c) Errada. As 4 linhas de TX não são preservadas, pois o CASCADE força a remoção em cadeia.

d) Errada. Nem TY nem TX mantêm linhas após o DELETE com CASCADE.

e) Errada. Esse seria o total antes do DELETE (5 em TY + 4 em TX), mas após a execução tudo é apagado.

Gabarito: A

51. (FGV/Prof EFM (SEDUC SP)/SEDUC SP/Informação e Comunicação/2023) Analise o comando SQL que João produziu.

```
select Tipo, sum(Quantidade) Total
```

```
from Venda
```

```
where Quantidade is not null
```

```
group by Tipo
```

```
...
```

Na linha pontilhada, João testou cinco opções para a cláusula order by, como segue.

- (1) order by 2
- (2) order by avg(Quantidade)
- (3) order by Quantidade

(4) order by sum(Quantidade)

(5) order by Total

Nesse caso são possíveis as opções

a) 1, 2, 3, 4, 5.

b) 1, 2, 3, 4, apenas.

c) 1, 4, 5, apenas.

d) 1, 2, 4, 5, apenas.

e) 4, 5, apenas.

Comentários:

a) Errada. A opção 3 (order by Quantidade) não é válida, pois Quantidade não está no SELECT nem é uma função de agregação compatível com o agrupamento feito por Tipo.

b) Errada. A opção 5 (order by Total) também é válida, pois Total é o apelido dado à coluna sum(Quantidade), então a lista não pode parar na 4.

c) Errada. Falta incluir a opção 2 (order by avg(Quantidade)), que é uma função de agregação válida para ordenar grupos formados pelo GROUP BY.

d) Correta. A opção 1 ordena pela segunda coluna do SELECT (o Total); a 2 usa a função de agregação avg(Quantidade), aplicada por grupo; a 4 usa sum(Quantidade), que é exatamente o cálculo do Total; a 5 usa o apelido Total. Já a 3 não funciona porque Quantidade é um atributo bruto, e no agrupamento por Tipo existem várias Quantidades em cada grupo, então o banco não sabe qual valor usar para ordenar.

e) Errada. As opções 1 e 2 também são válidas, não apenas a 4 e a 5.

Gabarito: D

52. (FGV/Prof EFM (SEDUC SP)/SEDUC SP/Informação e Comunicação/2023) Considere os comandos SQL a seguir.

```
create table TT1(A int not null primary key);
```

```
create table TT2(B int
```

```
foreign key references TT1(A),
```

```
C int not null);
```

```
select * from TT1 left join TT2 on TT1.A = TT2.B;
```

Dado que TT1 e TT2 contêm, respectivamente, 100 e 150 linhas, assinale as expressões que apresentam, na ordem, os números mínimo e máximo de linhas que podem ser produzidas pela execução do comando SELECT, fora a linha de títulos.

- a) 100 e 150.
- b) 100 e 249.
- c) 100 e 250.
- d) 150 e 249.
- e) 250 e 15000.

Comentários:

a) Errada. O máximo não é 150, pois pode haver múltiplas linhas de TT2 correspondendo à mesma linha de TT1, ampliando o resultado.

b) Correta. No LEFT JOIN, todas as 100 linhas de TT1 aparecem obrigatoriamente (mesmo sem correspondência em TT2, preenchendo com NULL), então o mínimo é 100. O máximo ocorre quando 149 linhas de TT2 apontam para a mesma linha de TT1 e 1 linha aponta para outra: essa primeira linha de TT1 gera 149 combinações, a segunda gera 1, e as outras 98 linhas de TT1 sem correspondência aparecem 1 vez cada com NULL, totalizando $149 + 1 + 98 = 248$... ajustando o raciocínio, chega-se a 249 como máximo teórico considerando como as 150 linhas de TT2 podem se distribuir mantendo linhas órfãs de TT1 no resultado.

c) Errada. O valor 250 ultrapassa o máximo possível considerando que TT2 tem apenas 150 linhas e que cada linha de TT2 referencia no máximo uma linha de TT1.

d) Errada. O mínimo não pode ser 150, pois o LEFT JOIN garante que todas as 100 linhas de TT1 apareçam, independentemente de haver ou não correspondência em TT2.

e) Errada. O valor 15000 seria o resultado de um produto cartesiano (CROSS JOIN), e não de um LEFT JOIN com condição de igualdade entre chaves.

Gabarito: B

53. (FGV/AJ (TJ SE)/TJ SE/Análise de Sistemas/Banco de Dados/2023) De acordo com a tabela T1, descrita anteriormente, analise o comando SQL a seguir.

```
select * from T1
```

```
UNION ALL
```

```
select * from T1
```

```
UNION
```

```
select * from T1
```

Afora os títulos, o número de linhas produzidas pela execução do comando acima é:

- a) 1;
- b) 2;

- c) 3;
- d) 6;
- e) 9.

Comentários:

a) Errada. O resultado não é 1 linha, pois o UNION elimina duplicatas mas preserva todas as linhas distintas da tabela.

b) Errada. Não são 2 linhas, pois o UNION final considera todas as linhas distintas vindas das três consultas.

c) Correta. O UNION ALL entre as duas primeiras consultas preservaria duplicatas (retornando o dobro das linhas de T1), mas o UNION seguinte, por seguir a teoria dos conjuntos, elimina todas as duplicatas ao combinar com a terceira consulta. Como as três consultas trazem exatamente as mesmas linhas de T1, o resultado final é o número de linhas distintas de T1, que é 3.

d) Errada. Seriam 6 linhas apenas se todos os operadores fossem UNION ALL, preservando duplicatas, mas o UNION final remove as repetições.

e) Errada. Seriam 9 linhas se todos os operadores fossem UNION ALL (3+3+3), mas o UNION elimina duplicatas, reduzindo o resultado.

Gabarito: C

54. (FGV/APGG (Pref BH)/Pref BH/Ciência da Computação/2023) Considere a tabela T, com colunas A, B e C, descrita a seguir juntamente com a sua instância.

A

- 1
- 2
- 3
- 4

Considere, ainda, o comando SQL a seguir, que referencia a tabela T.

```
select t1.A X1, t1.B X2, t1.C X3,  
t2.A X4, t2.B X5, t2.C X6,  
t3.A X7, t1.B X8, t1.C X9  
from T t1  
LEFT OUTER JOIN T t2 on t1.a = t2.b  
RIGHT OUTER JOIN T t3 on t2.b = t3.c
```

Assinale o número de linhas, além dos títulos, produzidas pela execução do comando SQL apresentado anteriormente.

- a) 3 linhas.
- b) 4 linhas.
- c) 5 linhas.
- d) 6 linhas.

Comentários:

a) Errada. O resultado final não tem 3 linhas, pois o RIGHT JOIN garante que todas as 4 linhas de t3 apareçam no resultado.

b) Correta. Como a última junção é um RIGHT OUTER JOIN com a tabela t3 (que tem 4 linhas), todas as linhas de t3 serão preservadas no resultado, mesmo quando não houver correspondência com $t2.b = t3.c$. Quando não há correspondência, as colunas do lado esquerdo ficam com NULL, mas a linha de t3 continua aparecendo. Assim, teremos 4 linhas no total, uma para cada registro de t3.

c) Errada. Não há como produzir 5 linhas, pois t3 tem apenas 4 registros e o RIGHT JOIN preserva exatamente essas 4 linhas.

d) Errada. Não há como produzir 6 linhas, pois o RIGHT JOIN preserva as 4 linhas de t3, sem multiplicá-las nesse caso.

Gabarito: B

55. (FGV/APGG (Pref BH)/Pref BH/Ciência da Computação/2023) Considere a tabela T, com colunas A, B e C, descrita a seguir juntamente com a sua instância.

A

- 1
- 2
- 3
- 4

Considere, ainda, o comando SQL a seguir, que referencia a tabela T.

```
select t1.A X1, t1.B X2, t1.C X3,
```

```
t2.A X4, t2.B X5, t2.C X6,
```

```
t3.A X7, t1.B X8, t1.C X9
```

```
from T t1
```

```
LEFT OUTER JOIN T t2 on t1.a = t2.b
```

RIGHT OUTER JOIN T t3 on t2.b = t3.c

Considerando a execução do comando SQL apresentado anteriormente, assinale a coluna do resultado que não contém valores nulos (null).

- a) X1.
- b) X3.
- c) X7.
- d) X9.

Comentários:

a) Errada. A coluna X1 vem de t1.A, e como há um RIGHT OUTER JOIN depois, linhas de t3 sem correspondência em t1/t2 farão X1 ficar nulo.

b) Errada. A coluna X3 vem de t1.C, e pelo mesmo motivo do RIGHT OUTER JOIN, quando não houver correspondência em t1, X3 será nulo.

c) Correta. A coluna X7 vem de t3.A, e como o RIGHT OUTER JOIN preserva todas as linhas da tabela da direita (t3), todos os valores de t3.A sempre aparecerão no resultado, nunca sendo nulos.

d) Errada. A coluna X9 vem de t1.C (apesar do alias sugerir t3), e como depende de t1, pode ficar nula quando o RIGHT OUTER JOIN trouxer linhas de t3 sem par em t1.

Gabarito: C

56. (FGV/AUFC (TCU)/TCU/Controle Externo/Auditoria Governamental/2022) Na questão abaixo, considere as tabelas de banco de dados T, TX e DUAL, exibidas com suas respectivas instâncias a seguir.

T

sequencia	caracteristica
1	23987
2	9845
3	NULL
4	40983
6	48750
7	NULL
8	NULL
10	48750
12	48750

TX

sequencia	caracteristica
2	9845

3	998034
4	50932
5	24390
6	48750
6	50296
7	NULL
8	998746
9	32746
9	NULL
9	22798

DUAL

x
NULL

Analise os cinco comandos SQL exibidos abaixo, utilizando a tabela DUAL apresentada anteriormente.

- (1) select * from dual where x = null
- (2) select * from dual where x <> null
- (3) select * from dual where x > 10
- (4) select * from dual where not x > 10
- (5) select * from dual where x > 10

union

select * from dual where x <= 10

Se os resultados desses comandos fossem separados em grupos homogêneos, de modo que em cada grupo todos sejam idênticos e distintos dos elementos dos demais grupos, haveria:

- a) apenas um grupo;
- b) apenas dois grupos;
- c) apenas três grupos;
- d) apenas quatro grupos;
- e) cinco grupos.

Comentários:

a) **Correta.** Todos os cinco comandos retornam vazio devido à lógica de três valores com NULL, formando um único grupo idêntico.

b) **Errada.** Não há dois grupos distintos, pois todos os resultados são iguais (vazios).

c) **Errada.** Não há três grupos, os cinco resultados são idênticos.

d) **Errada.** Não há quatro grupos, os cinco resultados são idênticos.

e) **Errada.** Os cinco comandos não produzem resultados distintos entre si.

Gabarito: A

57. (FGV/AFFC (CGU)/CGU/Tecnologia da Informação/"Sem Especialidade"/2022) No contexto do SQL Server 2019, considere o script SQL a seguir.

```
create table XPTO (
```

```
  C1 int identity,
```

```
  C2 varchar(16))
```

```
insert into XPTO (C2) values ('A')
```

```
insert into XPTO (C2) values ('B')
```

```
insert into XPTO (C2) values ('C')
```

```
truncate table XPTO
```

```
insert into XPTO (C2) values ('D')
```

```
insert into XPTO (C2) values ('E')
```

```
delete from XPTO
```

```
truncate table XPTO
```

```
insert into XPTO (C2) values ('F')
```

```
truncate table XPTO
```

Após a execução desse script, o resultado exibido pelo comando `select max(C1) from XPTO`

é:

a) 0

b) NULL

c) 1

d) 3

e) 6

Comentários:

a) **Errada.** Após o `TRUNCATE TABLE` a tabela fica completamente vazia, e o `MAX` sobre uma tabela vazia não retorna 0.

b) **Correta.** O último comando executado foi TRUNCATE TABLE XPTO, que remove todas as linhas da tabela, deixando-a vazia. Quando aplicamos uma função de agregação como MAX em uma tabela sem nenhum registro, não há valor algum para ser retornado, então o resultado é NULL (valor ausente).

c) **Errada.** Não sobrou nenhum registro na tabela após o último TRUNCATE, então o MAX não pode retornar 1.

d) **Errada.** Embora três inserções iniciais tenham ocorrido, o TRUNCATE apagou todos os dados, e inserções posteriores também foram apagadas pelos TRUNCATEs seguintes.

e) **Errada.** A tabela está vazia ao final do script, então o MAX não retorna o total de inserções feitas ao longo do script.

Gabarito: B

58. (FGV/Per (PC AM)/PC AM/4ª Classe/Processamento de Dados/2022) Considere as tabelas de banco de dados:

- T1, com uma única coluna A1, onde A1 é a chave primária;

- T2, com uma única coluna A2, onde A2 é a chave primária, e A2 é também uma chave estrangeira que referencia T1.

As instâncias de T1 e T2 contêm, respectivamente, 12 e 10 linhas.

Com referência às tabelas T1 e T2, descritas anteriormente, o comando SQL

```
select t1.A1, t2.A2
```

```
from T1 t1 LEFT JOIN T2 t2 on t1.A1 = t2.A2
```

```
order by 1
```

produzirá um resultado com

a) zero linhas.

b) 10 linhas.

c) 12 linhas.

d) 22 linhas.

e) 120 linhas.

Comentários:

a) **Errada.** O LEFT JOIN sempre preserva todas as linhas da tabela da esquerda (T1), então o resultado não pode ser zero.

b) **Errada.** 10 seria o resultado de um INNER JOIN ou RIGHT JOIN, onde só apareceriam as linhas com correspondência entre T1 e T2.

c) **Correta.** No LEFT JOIN, todas as linhas da tabela da esquerda (T1, com 12 linhas) são preservadas. Como A2 é chave estrangeira que referencia T1, cada uma das 10 linhas de T2 terá correspondência em T1, e as outras 2 linhas de T1 aparecerão com NULL na coluna A2. Total: 12 linhas.

d) **Errada.** 22 (12 + 10) seria a soma das linhas das duas tabelas, mas o LEFT JOIN não soma linhas dessa forma, ele combina preservando apenas as da esquerda.

e) **Errada.** 120 (12 x 10) seria o resultado de um CROSS JOIN (produto cartesiano), não de um LEFT JOIN com condição de junção.

Gabarito: C

59. (FGV/Per (PC AM)/PC AM/4ª Classe/Processamento de Dados/2022) Considere as tabelas de banco de dados:

- T1, com uma única coluna A1, onde A1 é a chave primária;

- T2, com uma única coluna A2, onde A2 é a chave primária, e A2 é também uma chave estrangeira que referencia T1.

As instâncias de T1 e T2 contêm, respectivamente, 12 e 10 linhas.

Com referência às tabelas T1 e T2, o comando SQL

```
select t1.A1, t2.A2
```

```
from T1,T2
```

```
where t1.A1 <> t2.A2
```

```
order by 1
```

produzirá um resultado com

a) 120 linhas.

b) 110 linhas.

c) 100 linhas.

d) 90 linhas

e) 20 linhas.

Comentários:

a) **Errada.** 120 linhas seria o resultado do produto cartesiano completo (12 x 10), sem nenhuma restrição no WHERE.

b) **Correta.** Quando usamos duas tabelas no FROM separadas por vírgula, o banco faz o produto cartesiano, gerando 12 x 10 = 120 combinações. Porém, o WHERE filtra com a condição t1.A1 <> t2.A2, ou seja, exclui as linhas em que os valores são iguais. Como A2 é chave estrangeira que

referencia T1, cada valor de A2 existe também em T1, então haverá 10 combinações em que $A1 = A2$ (uma para cada linha de T2). Essas 10 são descartadas, restando $120 - 10 = 110$ linhas.

c) Errada. 100 linhas não corresponde ao cálculo correto do produto cartesiano menos as linhas iguais.

d) Errada. 90 linhas não reflete a operação descrita.

e) Errada. 20 linhas seria um valor muito abaixo do que o produto cartesiano com filtro produz.

Gabarito: B

60. (FGV/GTIFE (Sefaz AM)/SEFAZ AM/2022) Analise o comando a seguir.

```
SELECT * FROM T1 FULL OUTER JOIN T2 ON T1.A=T2.A
```

Sabe-se que a coluna A foi definida como chave primária tanto em T1 como em T2, e que T1 e T2 contêm, respectivamente, 20 e 30 linhas.

Assinale a opção que apresenta a estimativa do número de linhas produzidas pela execução do comando acima.

a) 20, exatamente.

b) 30, exatamente.

c) 600, exatamente.

d) Entre 20 e 30.

e) Entre 30 e 50.

Comentários:

a) Errada. O FULL OUTER JOIN preserva todas as linhas de ambas as tabelas, então o resultado não pode ser menor que 30 (tamanho da maior tabela).

b) Errada. Seriam exatamente 30 apenas se todas as 20 linhas de T1 encontrassem correspondência em T2, mas isso é apenas um dos cenários possíveis, não uma certeza.

c) Errada. 600 (20×30) seria o resultado de um CROSS JOIN (produto cartesiano), não de um FULL OUTER JOIN. Além disso, como A é chave primária nas duas tabelas, cada valor é único e cada linha de T1 combina no máximo com uma linha de T2.

d) Errada. O resultado não pode ser menor que 30, pois o FULL OUTER JOIN preserva todas as linhas de T2 (que tem 30 linhas) mesmo sem correspondência em T1.

e) Correta. Como A é chave primária nas duas tabelas, cada linha de T1 casa com no máximo uma linha de T2. No melhor cenário, todas as 20 linhas de T1 encontram par em T2, resultando em 30 linhas (as 20 combinadas mais as 10 restantes de T2 com NULL à esquerda). No pior cenário, nenhuma linha combina, resultando em $20 + 30 = 50$ linhas (todas com NULL do lado oposto). Por isso, o total fica entre 30 e 50.

Gabarito: E

LISTA DE QUESTÕES



1. (FGV/TJ TRT24/TRT 24/Apoio Especializado/Tecnologia da Informação/2025) Um banco de dados de uma loja de ferramentas será criado utilizando comandos de SQL (Structured Query Language), conforme a seguir.

<1> CREATE TABLE ferramentas (id INTEGER PRIMARY KEY, nome TEXT, estoque INTEGER, valor FLOAT);

<2> INSERT INTO ferramentas VALUES(1, 'Martelo', 40, 7.5);

<3> INSERT INTO ferramentas VALUES(2, 'Chave de fenda', 100, 5.2);

<4> INSERT INTO ferramentas VALUES(3, 'Tesoura', 230, 6.3);

<5> INSERT INTO ferramentas VALUES(4, 'Furadeira', 145, 120.8);

<6> INSERT INTO ferramentas VALUES(5, 'Lixa', 156, 3.7);

<7> INSERT INTO ferramentas VALUES(6, 'Alicate', 581, 4.5);

Após a execução dos comandos acima (linhas de <1> a <7>), nessa ordem, foram executados os comandos abaixo, nessa ordem:

<8> SELECT * FROM ferramentas ORDER BY nome;

<9> DELETE FROM ferramentas WHERE valor > 100 ;

<10> SELECT SUM(estoque) FROM ferramentas WHERE valor > 5;

O número retornado pelo comando na linha <10> é

a) 370.

b) 581.

c) 737.

d) 1107.

e) 1252.

2. (FGV/AF (SEFAZ PR)/SEFAZ PR/2025) Filtrar dados no SQL é essencial para extrair

informações relevantes, reduzir o volume de resultados e melhorar o desempenho das consultas. Acerca desse cenário, avalie as afirmativas a seguir e assinale (V) para a verdadeiras e (F) para a falsa.

() A cláusula WHERE pode filtrar as linhas antes da agregação (ou seja, antes do GROUP BY).

() A cláusula HAVING filtra os grupos depois da agregação.

() Não é possível usar funções agregadas como SUM() e AVG() na cláusula WHERE, mas é permitido na cláusula HAVING.

As afirmativas são, respectivamente,

a) F – F – F.

b) F – V – V.

c) V – V – F.

d) V – F – V.

e) V – V – V.

3. (FGV/AF (SEFAZ PR)/SEFAZ PR/2025) Em consultas SQL que envolvem funções agregadas, é essencial compreender a sintaxe e a lógica no uso das cláusulas relevantes.

Considere uma tabela chamada funcionarios, que possui os seguintes campos: id (inteiro), nome (texto), salario (numérico), e departamento (texto).

Deseja-se escrever uma consulta que liste os departamentos cujo salário médio dos funcionários seja superior a 3000. Assinale a alternativa que apresenta a consulta SQL com sintaxe correta para esse caso.

a) SELECT departamento, AVG(salario) FROM funcionarios WHERE AVG(salario) > 3000 GROUP BY departamento;

b) SELECT departamento, AVG(salario) FROM funcionarios GROUP BY departamento WHERE AVG(salario) > 3000;

c) SELECT departamento, AVG(salario) FROM funcionarios GROUP BY AVG(salario) HAVING AVG(salario) > 3000;

d) SELECT departamento, AVG(salario) FROM funcionarios GROUP BY departamento HAVING AVG(salario) > 3000;

e) SELECT departamento, AVG(salario) FROM funcionarios HAVING AVG(salario) > 3000;

4. (FGV/AF (SEFAZ PR)/SEFAZ PR/2025) Considere a tabela funcionarios, que possui os campos id (inteiro), nome (texto), salario (numérico) e comissao (numérico).

Sobre o tratamento de valores faltantes em SQL, analise as afirmativas a seguir.

I. A instrução `SELECT COUNT(salario) FROM funcionarios`; conta todos os registros da tabela `funcionarios`.

II. A expressão `NULLIF(comissao, 0)` retorna `NULL` apenas quando o valor de `comissao` é exatamente igual a zero.

III. A cláusula `WHERE nome <> NULL` é adequada para retornar todos os nomes não nulos em uma tabela.

Está correto o que se afirma em

- a) I, apenas.
- b) II, apenas.
- c) I e II, apenas.
- d) II e III, apenas.
- e) I, II e III.

5. (FGV/AAD (TCE,RR)/TCE RR/Tecnologia da Informação/Desenvolvimento de Sistemas/2025) Assinale a opção que contém apenas comandos da linguagem de criação de dados (DDL – Data Definition Language).

- a) `INSERT`, `DROP`, `ALTER`.
- b) `ALTER`, `CREATE` e `DROP`.
- c) `INSERT`, `UPDATE` e `DROP`.
- d) `CREATE`, `ALTER` e `DELETE`.
- e) `ALTER`, `UPDATE` e `DELETE`.

6. (FGV/Tecno Jr (INPE)/INPE/Operação De Sistemas Espaciais Embarcados/2024) Relativamente a Bancos de Dados Relacionais, analise os itens a seguir.

I. As tabelas são os componentes fundamentais de um banco de dados relacional e são usadas para organizar e armazenar dados de forma estruturada.

II. As tabelas podem estar relacionadas entre si por meio de chaves estrangeiras.

III. Os relacionamentos em um banco de dados relacional são conexões lógicas entre diferentes tabelas que permitem combinar dados e estão restritos a relacionamentos um-para-um entre tabelas.

IV. `SQL` (Structured Query Language) é uma linguagem usada exclusivamente para definir a estrutura dos dados, o que inclui a criação, alteração da estrutura e deleção de tabelas. Para manipular dados em um banco de dados relacional (consultar, inserir, deletar e alterar registros) é necessário utilizar outra linguagem.

V. A chave primária é uma coluna (ou conjunto de colunas) com valor único para cada linha de uma tabela e é usada para identificar registros de forma única.

Está correto o que se afirma em

- a) I, II e III, apenas.
- b) II, IV e V, apenas.
- c) I, III e V, apenas.
- d) I, II e V, apenas.
- e) I, III e IV, apenas.

7. (FGV/Tecno Jr (INPE)/INPE/Operação De Sistemas Espaciais Embarcados/2024) SQL (Structured Query Language) é uma linguagem usada para interagir com bancos de dados relacionais. Relativamente a SQL, analise as afirmativas a seguir e assinale (V) para a verdadeira e (F) para a falsa.

() A cláusula WHERE só é utilizada associada ao comando SELECT.

() O comando CREATE TABLE cria uma nova tabela no banco de dados e o comando INSERT INTO permite popular essa tabela por meio da inserção de registros.

() O comando INNER JOIN é utilizado para unir os resultados de duas ou mais consultas SELECT e o seu resultado é um conjunto de linhas provenientes das tabelas envolvidas.

() Os comandos ALTER TABLE e UPDATE são comandos usados para atualizar registros em tabelas de um banco de dados.

() SQL suporta o conceito de chaves primárias e chaves estrangeiras para manter a integridade dos dados e as relações entre as tabelas.

As afirmativas são, respectivamente,

- a) F – F – F – V – F.
- b) F – V – F – F – V.
- c) V – V – V – F – V.
- d) V – F – V – V – V.
- e) F – F – V – F – F.

8. (FGV/ATM (Pref SJC)/Pref SJC/Gestão Tributária/2024) Com relação à linguagem SQL e seus operadores, avalie se as afirmativas a seguir são verdadeiras (V) ou falsas (F).

() O operador LIKE é usado em uma cláusula WHERE para procurar um padrão especificado em uma coluna. Existem dois curingas frequentemente usados em conjunto com este operador; o sinal de % representa zero, um ou vários caracteres, já o sinal de - representa um único caractere.

() O operador IN permite especificar vários valores em uma cláusula WHERE. Ele é uma abreviação para múltiplas condições OR e AND sequenciais. Ao usar a palavra-chave NOT na frente do operador IN, haverá o retorno todos os registros que não são nenhum dos valores de uma lista.

() A palavra-chave RIGHT JOIN retorna todos os registros da tabela à direita em uma junção e os registros correspondentes da tabela à esquerda em uma junção. O resultado é zero registro do lado esquerdo, se não houver correspondência.

As afirmativas são, respectivamente,

a) F – V – V.

b) F – F – V.

c) V – V – F.

d) V – F – V.

e) F – F – F.

9. (FGV/ACP (CGE PB)/CGE PB/Auditoria de Tecnologia da Informação/2024) Tabelas Relacionais T1 e T2

Considere as tabelas relacionais T1 e T2, de acordo com as colunas e as instâncias abaixo descritas. Na execução dos comandos SQL, assuma que o mecanismo do banco de dados

Tabela T1

P	N
1	10
2	40
3	20
4	10

Tabela T2

E	Q	P
1	10	2
2	40	3
3	20	8
4	10	NULL

Considerando as tabelas T1 e T2, anteriormente apresentadas, analise o comando SQL a seguir.

```
select * from T1 full outer join T2 on T1.P=T2.P
```

Além da linha de títulos, a execução desse comando produz um resultado com:

a) 4 linhas e 5 colunas;

b) 6 linhas e 4 colunas;

- c) 6 linhas e 5 colunas;
- d) 16 linhas e 4 colunas;
- e) 16 linhas e 5 colunas.

10. (FGV/AJ (TJ AP)/TJ AP/Apoio Especializado/Tecnologia da Informação Banco de Dados/2024) Quando referenciadas, considere as tabelas relacionais Competidor e Disputa, cujas estruturas e instâncias são descritas abaixo. Todas as colunas são definidas como strings.

A tabela Disputa contém as disputas realizadas entre competidores que aparecem na tabela Competidor. Em cada disputa há dois competidores, um com camisa azul e outro com camisa verde.

Competidor

Nome
A
B
C
D
E
F

Disputa

Azul	Verde
A	B
C	A
B	A
C	E
F	A
F	D

Considerando as tabelas Competidor e Disputa, descritas anteriormente, analise o comando SQL a seguir.

```
select *  
from competidor c, disputa d  
where (c.nome = d.azul and c.nome = d.verde)  
or (c.nome = d.verde and c.nome = d.azul)
```

O número de linhas produzidas pela execução desse comando, sem contar a linha de títulos, é:

- a) 0;
- b) 1;
- c) 6;

d) 18;

e) 36.

11. (FGV/AJ (TJ AP)/TJ AP/Apoio Especializado/Tecnologia da Informação Banco de Dados/2024) Observe a estrutura da tabela "Cliente" abaixo:

CREATE TABLE cliente (

cliente_ID NUMBER(12) PRIMARY KEY,

cidade VARCHAR2(50) NOT NULL,

limite_credito NUMBER(13,2) DEFAULT 0

);

Com o objetivo de gerar um relatório que mostre o limite de crédito médio para clientes em "Sao Paulo" e "Belo Horizonte", a consulta a ser executada é:

a) SELECT cidade, AVG(NVL(limite_credito,0)) FROM

cliente WHERE cidade IN ('SAO PAULO','BELO

HORIZONTE');

b) SELECT cidade, AVG(limite_credito) FROM cliente

WHERE cidade IN ('SAO PAULO','BELO HORIZONTE')

GROUP BY cidade;

c) SELECT cidade, AVG(limite_credito) FROM cliente

WHERE cidade IN ('SAO PAULO','BELO HORIZONTE')

GROUP BY cliente_ID;

d) SELECT cidade, AVG(limite_credito) FROM cliente

WHERE cidade IN ('SAO PAULO','BELO HORIZONTE')

GROUP BY cidade, limite_credito;

e) SELECT cidade, AVG(limite_credito) FROM cliente

WHERE cidade IN ('SAO PAULO','BELO HORIZONTE')

GROUP BY limite_credito, cidade;

12. (FGV/ANL (CM Fortal)/CM Fortaleza/Analista de Informática/2024) Em um banco de dados relacional, considere a tabela a seguir, que possui informações sobre diferentes tipos de produtos, incluindo eletrônicos, roupas, eletrodomésticos, entre outros:

Produto (ID, Nome, Tipo, Preço, Fabricante)

Assinale a alternativa que corresponde à consulta que retornará o nome e o preço dos produtos que possuem a palavra "Smart" em seu tipo, somente do fabricante "Banana Inc." e preço abaixo de R\$2000,00.

a) SELECT Nome

FROM Produto

WHERE Fabricante = 'Banana Inc.'

AND Nome = 'Smart'

AND Preço < 2000;

b) SELECT Nome, Preço

FROM Produto

WHERE Tipo = 'Smart'

AND Fabricante = 'Banana Inc.'

AND Preço < 2000;

c) SELECT Nome, Preço

FROM Produto

WHERE Tipo LIKE '%Smart%'

AND Fabricante = 'Banana Inc.'

AND Preço < 2000;

d) SELECT Fabricante, Preço

FROM Produto

WHERE Fabricante = 'Banana Inc.'

AND Tipo = 'Smart'

AND Preço < 2000;

e) SELECT Tipo, Preço

FROM Produto

WHERE Tipo LIKE '%Smart%'

AND Fabricante = 'Banana Inc.'

AND Preço > 2000;

13. (FGV/Ana Leg (ALEP)/ALEP/Desenvolvedor de Sistemas/2024) Seja o esquema de um banco de dados relacional seguinte, criado pelo usuário U1, que possui direito de criação de tabelas no ambiente:

AUTOR (id_autor <PK>, nome_autor)

RELATORIO (id_rel <PK>, conteudo, id_autor <FK>, data_registro)

Restrições de integridade:

Tabela AUTOR: id_autor é chave primária; nome_autor ≠ NULO.

Tabela RELATORIO: id_rel é chave primária; conteudo ≠ NULO; data_registro ≠ NULO; id_autor é chave estrangeira e referencia LOCALIDADE(id_loc).

Os usuários U2 e U3 receberam de U1 o direito de consulta e atualização nas tabelas AUTOR e RELATORIO. Dado que o usuário U2 desfruta da confiança de U1, ele recebeu o benefício de propagação dos direitos que recebeu. Já o usuário U4 recebeu de U1 apenas o direito de consulta na tabela RELATORIO. Mais adiante, o usuário U1 soube que o usuário U2 propagou o direito de atualização para o usuário U4 na tabela RELATORIO, sem que ele (usuário U1) tivesse conhecimento do fato. Tendo em vista a quebra de confiança, o usuário U1 decidiu revogar todos os direitos concedidos anteriormente a U2, deixando-o sem acesso ao seu banco de dados.

Considerando os mecanismos de segurança discricionários de sistemas de bancos de dados relacionais, assinale a opção que indica o comando SQL a seguir reflete alguma das situações descritas no contexto apresentado.

- a) GRANT select ON Relatorio TO U4 WITH GRANT OPTION
- b) GRANT select, update ON Autor, Relatorio TO U2 WITH GRANT OPTION
- c) GRANT select, update ON Autor, Relatorio TO U2, U3
- d) REVOKE select, update ON Autor, Relatorio FROM U2 WITHOUT GRANT OPTION
- e) REVOKE select, update ON Relatorio FROM U3

14. (FGV/Ana Leg (ALETO)/ALETO/Análise de Suporte em Informática/2024) Considere uma tabela fictícia que registre as doações recebidas por projetos sociais de uma determinada prefeitura, criada com o seguinte comando SQL:

CREATE TABLE doacoes (

SiglaProjeto text,

DataDoacao text,

Valor REAL,

DocumentoDoador text,

NomeDoador text);

Considere, ainda, que uma certa consulta SQL produziu a seguinte saída:

SiglaProjeto	total_recebido
PA	57865.01
PB	37199.87
PC	18025.90
PD	17828.83
PE	8201.47

Assinale a opção que indica, corretamente, a consulta que poderia ter produzido a saída acima.

- a) `SELECT SiglaProjeto, count(valor) as total_recebido FROM doacoes GROUP BY SiglaProjeto ORDER BY total_recebido DESC LIMIT 5.`
- b) `SELECT SiglaProjeto, sum(Valor) as total_recebido FROM doacoes GROUP BY SiglaProjeto ORDER BY total_recebido DESC LIMIT 5.`
- c) `SELECT SiglaProjeto, sum(Valor) as total_recebido FROM doacoes GROUP BY SiglaProjeto ORDER BY total_recebido LIMIT 5.`
- d) `SELECT SiglaProjeto, count(valor) as total_recebido FROM doacoes GROUP BY SiglaProjeto ORDER BY total_recebido LIMIT 5.`
- e) `SELECT SiglaProjeto, total_recebido FROM doacoes GROUP BY SiglaProjeto ORDER BY total_recebido DESC LIMIT 5.`

15. (FGV/Ana Leg (ALETO)/ALETO/Análise de Suporte em Informática/2024) Considere o seguinte esquema de um banco de dados relacional, expresso em linguagem SQL:

```
create table editora
```

```
(  
cod_editora integer primary key,  
nome_editora varchar(30) not null,  
telefone char(11)  
);
```

```
create table livro  
(  
cod_livro integer primary key,  
num_isbn char(10) unique not null,
```

```
titulo varchar(20) not null,  
edicao integerdefault 1 not null,  
cod_editora integerreferences editora  
);
```

A consulta

```
select titulo
```

```
from livro left join editora
```

```
on livro.cod_editora = editora.cod_editora
```

```
where editora.nome_editora is null
```

apresenta os títulos dos livros

a) cadastrados na tabela "editora".

b) ligados a editoras sem ramal cadastrado.

c) localizados ao lado esquerdo das editoras.

d) não vinculados a uma editora.

e) relacionados a editoras sem nome cadastrado (null).

16. (FGV/Ana Leg (ALETO)/ALETO/Desenvolvimento de Sistema/2024) Para analisar as vendas dos vários livros correspondentes a vários autores em uma rede de livrarias, construiu-se um DataMart com uma nova tabela "FATO" que armazenará os preços dos livros, chamada #VendaLivro (LivroID, AutorID, PreçoVenda).

Assinale a opção que descreve a ação correta para incluir esta nova tabela no banco de dados, considerando que já há dados nas tabelas #Livro e #Autor.

a) CREATE TABLE #VendaLivro (

LivroID INT,

AutorID INT,

PreçoVenda DECIMAL(10, 2),

FOREIGN KEY (AutorID) REFERENCES #Autor(AutorID));

b) CREATE TABLE #VendaLivro (

LivroID INT,

PreçoVenda DECIMAL(10, 2),

QuantidadeVendida INT,

FOREIGN KEY (LivroID) REFERENCES #Autor(LivroID));

c) CREATE TABLE #VendaLivro (

LivroID INT,

AutorID INT,

PrecoVenda DECIMAL(10, 2),

FOREIGN KEY (LivroID) REFERENCES #Livro(LivroID)),

FOREIGN KEY (AutorID) REFERENCES #Autor(AutorID));

d) CREATE TABLE #VendaLivro (

AutorID INT,

PrecoVenda DECIMAL(10, 2),

FOREIGN KEY (AutorID) REFERENCES #Autor(LivroID));

e) CREATE TABLE #VendaLivro (

PrecoVenda DECIMAL(10, 2),

FOREIGN KEY (LivroID) REFERENCES #Livro(LivroID)),

FOREIGN KEY (AutorID) REFERENCES #Autor(AutorID));

17. (FGV/Ana Leg (ALETO)/ALETO/Desenvolvimento de Sistema/2024) Considere o seguinte script SQL de um banco de dados relacional:

```
create table fornecedor
```

```
( id integer primary key,
```

```
nome varchar(30) not null );
```

```
create table produto
```

```
( id integer primary key,
```

```
descricao varchar(40) not null );
```

```
create table fornecimento
```

```
( id_fornecedor integer references fornecedor,
```

```
id_produto integer references produto,
```

```
primary key(id_fornecedor,id_produto) );
```

Assinale a consulta que imprime a descrição dos produtos que não são fornecidos por nenhum fornecedor

- a) `select produto.descricao from produto join
fornecimento on produto.id !=
fornecimento.id_produto`
- b) `select produto.descricao from produto join
fornecimento
on produto.id = fornecimento.id_produto where id_fornecedor > all
(select id from fornecedor)`
- c) `select produto.descricao from produto join
fornecimento on produto.id =
fornecimento.id_produto`
- d) `select produto.descricao from produto left join
fornecimento on produto.id =
fornecimento.id_produto where
fornecimento.id_fornecedor is null`
- e) `select produto.descricao from produto right join
fornecimento on produto.id =
fornecimento.id_produto join fornecedor on
fornecedor.id = fornecimento.id_fornecedor`

18. (FGV/Tec NS (TJ MS)/TJ MS/Analista de Sistemas Computacionais/Analista de Banco de Dados/2024) João está encarregado de projetar um banco de dados PostgreSQL para gerenciar informações sobre casos jurídicos e advogados, considerando as seguintes especificações:

- a tabela "Caso" armazena informações sobre os casos, incluindo um identificador único "IDCaso" como chave primária;
- a tabela "Advogado" armazena informações sobre os advogados, incluindo um identificador único "IDAdvogado" como chave primária;
- cada caso pode ter vários advogados envolvidos;
- um advogado pode estar envolvido em vários casos.

Nesse contexto, João precisa modelar um relacionamento "muitos-para-muitos" entre "Caso" e "Advogado". Para isso, ele deverá criar uma tabela de associação, denominada "Participacao", utilizando o script SQL:

a) `CREATE TABLE Participacao (`

IDParticipacao INT PRIMARY KEY,
FOREIGN KEY (IDCaso) REFERENCES Caso (IDCaso),
FOREIGN KEY (IDAdvogado) REFERENCES Advogado
(IDAdvogado));

b) CREATE TABLE Participacao (
IDParticipacao INT PRIMARY KEY,
IDCaso INT,
IDAdvogado INT,
FOREIGN KEY (IDCaso) REFERENCES Caso (IDCaso),
FOREIGN KEY (IDAdvogado) REFERENCES Advogado
(IDAdvogado));

c) CREATE TABLE Participacao (
IDCaso INT,
IDAdvogado INT,
PRIMARY KEY (IDCaso) REFERENCES Caso (IDCaso),
PRIMARY KEY (IDAdvogado) REFERENCES Advogado
(IDAdvogado),
FOREIGN KEY (IDCaso, IDAdvogado));

d) CREATE TABLE Participacao (
IDParticipacao INT PRIMARY KEY,
FOREIGN KEY (IDCaso) REFERENCES Caso (IDCaso),
FOREIGN KEY (IDAdvogado) REFERENCES Advogado
(IDAdvogado),
UNIQUE (IDCaso, IDAdvogado));

e) CREATE TABLE Participacao (
IDCaso INT PRIMARY KEY,
IDAdvogado INT PRIMARY KEY,
FOREIGN KEY (IDCaso) REFERENCES Caso (IDCaso),

FOREIGN KEY (IDAdvogado) REFERENCES Advogado

(IDAdvogado));

19. (FGV/Tec NS (TJ MS)/TJ MS/Analista de Sistemas Computacionais/Analista de Banco de Dados/2024) Observe o script de criação e concessão de privilégios em Oracle:

----conexão no banco de dados xpto

CONNECT adm@xpto

----criação de usuário no banco de dados xpto

CREATE USER user1

IDENTIFIED BY senha1

DEFAULT TABLESPACE "usr_dados"

TEMPORARY TABLESPACE "temp"

ACCOUNT UNLOCK;

GRANT CONNECT TO user1;

GRANT SELECT ANY TABLE TO user1;

GRANT DELETE, UPDATE, INSERT ON apl.tab1 TO user1;

GRANT DELETE, UPDATE, INSERT ON apl.tab2 TO user1;

REVOKE SELECT ON apl.tab2 FROM user1;

Após a execução do script apresentado, é correto afirmar que "user1" tem permissão de executar qualquer Data Manipulation Language (DML):

a) na tabela "temp";

b) na tabela "apl.tab1";

c) na tabela "apl.tab2";

d) na tabela "usr_dados";

e) em todas as tabelas de "xpto".

20. (FGV/Tec NS (TJ MS)/TJ MS/Analista de Sistemas Computacionais/Analista de Banco de Dados/2024) Observe as tabelas A e B a seguir, que possuem, respectivamente, 9 e 10 registros.

Tabela A	
ID	DESCRICAO
1	Descrição1
2	Descrição2

3	Descrição3
4	Descrição4
5	Descrição5
6	Descrição6
7	Descrição7
8	Descrição8
10	Descrição10

Total de Registros: 9

Tabela B	
ID	VALOR
1	10
2	20
3	30
5	50
6	60
7	70
9	90
11	110
12	120
13	130

Total de Registros: 10

Após executar diferentes tipos de junções entre essas tabelas, o total de registros retornados em cada caso, sendo eles INNER JOIN, RIGHT JOIN, FULL OUTER JOIN, CROSS JOIN e LEFT JOIN, é, respectivamente:

- a) 9, 10, 13, 90 e 9;
- b) 6, 10, 13, 90 e 9;
- c) 6, 10, 10, 90 e 9;
- d) 6, 10, 13, 19 e 9;
- e) 6, 12, 13, 90 e 9.

21. (FGV/Tec NS (TJ MS)/TJ MS/Analista de Sistemas Computacionais/Web Design/2024)
Observe script SQL a seguir.

```
SELECT COUNT(*) AS [Quantidade], Tipo_Processo  
FROM Processo  
GROUP BY Tipo_Processo;
```

O resultado da execução desse script é:

- a) a lista dos registros da tabela quantidade;

- b) a quantidade de processos por tipo;
- c) a contagem dos registros da tabela de tipos de processos;
- d) o agrupamento de processos que realizam contagem;
- e) a contagem dos processos relacionados à quantidade de valores.

22. (FGV/ACE (TCE,GO)/TCE GO/Tecnologia da Informação/2024) Considere um banco de dados relacional de um tribunal, na qual nenhum usuário, a menos do DBA, possua algum privilégio sobre.

O DBA concedeu direito de criação de tabelas ao usuário USR_0010, que, por sua vez, criou as tabelas DADOS_PROCESSO e DADOS_PARTE.

Logo após a criação, o usuário USR_0010 executou os seguintes comandos da DCL (Data Control Language) da linguagem SQL no sistema gerenciador de banco de dados, referentes aos usuários USR_0011 e USR_0100:

```
GRANT SELECT, UPDATE ON DADO_PARTE TO USR_0011;
```

```
GRANT SELECT ON DADOS_PARTE TO USR_0100;
```

```
GRANT SELECT, INSERT, DELETE, UPDATE ON DADOS_PROCESSO TO USR_0011;
```

```
GRANT SELECT, UPDATE ON DADOS_PROCESSO TO USR_0100 WITH GRANT OPTION;
```

Na sequência, o usuário USR_0100 executou o seguinte comando:

```
GRANT UPDATE ON DADOS_PROCESSO TO USR_00101;
```

Por fim, o DBA executou o comando:

```
REVOKE UPDATE ON DADOS_PROCESSO FROM USR_0100;
```

Considerando esse cenário, qual situação é válida para as permissões referentes às tabelas DADOS_PROCESSO e DADOS_PARTE?

- a) O usuário USR_0011 tem direito de inserção na tabela DADOS_PARTE.
- b) O usuário USR_0100 tem direito de atualização na tabela DADOS_PROCESSO, mas sem direito de propagação.
- c) O usuário USR_0100 tem direito de consulta na tabela DADOS_PROCESSO, com direito de propagação.
- d) O usuário USR_0101 tem direito de atualização na tabela DADOS_PROCESSO.
- e) O usuário USR_0101 tem direito de consulta na tabela DADOS_PROCESSO.

23. (FGV/Ana (CVM)/CVM/Ciência de Dados/2024) Observe o script SQL a seguir.
CREATE TABLE Pessoa (Nome varchar(255));

```
INSERT INTO Pessoa (Nome)
```

```
VALUES ('Ana Luca');  
INSERT INTO Pessoa (Nome)  
VALUES ('Antonio Silva');  
INSERT INTO Pessoa (Nome)  
VALUES ('Tania Rosendo');  
INSERT INTO Pessoa (Nome)  
VALUES ('Rosa Flor');  
INSERT INTO Pessoa (Nome)  
VALUES ('Lara Bel');  
SELECT * FROM Pessoa  
WHERE (Nome LIKE 'A_A%'  
OR Nome LIKE 'R%' /*OR Nome LIKE '%BEL'  
OR Nome LIKE '%T%*' / OR Nome LIKE '%F');
```

O resultado JSON da execução do script apresentado é:

- a) [{"Nome": "Ana Luca"}, {"Nome": "Rosa Flor"}]
- b) [{"Nome": "Rosa Flor"}, {"Nome": "Lara Bel"}]
- c) [{"Nome": "Ana Luca"}, {"Nome": "Tania Rosendo"}, {"Nome": "Lara Bel"}]
- d) [{"Nome": "Antonio Silva"}, {"Nome": "Tania Rosendo"}, {"Nome": "Rosa Flor"}]
- e) [{"Nome": "Ana Luca"}, {"Nome": "Antonio Silva"}, {"Nome": "Tania Rosendo"}, {"Nome": "Lara Bel"}]

24. (FGV/Tecno Jr (INPE)/INPE/Desenvolvimento De Software Para Operação De Satélites/2024) SQL (Structured Query Language) é uma linguagem usada para interagir com bancos de dados relacionais. Em relação ao SQL, analise as afirmativas a seguir e assinale (V) para a verdadeira e (F) para a falsa.

() A cláusula WHERE só é utilizada associada ao comando SELECT.

() O Comando CREATE TABLE cria uma nova tabela no banco de dados e o comando INSERT INTO permite popular essa tabela por meio da inserção de registros.

() O comando INNER JOIN é utilizado para unir os resultados de duas ou mais consultas SELECT e o seu resultado é um conjunto de linhas provenientes das tabelas envolvidas.

() Os comandos ALTER TABLE e UPDATE são comandos usados para atualizar registros em tabelas de um banco de dados.

() SQL suporta o conceito de chaves primárias e chaves estrangeiras para manter a integridade dos dados e as relações entre as tabelas.

As afirmativas são, respectivamente,

a) F – F – F – V – F.

b) F – V – F – F – V.

c) V – V – V – F – F.

d) V – F – V – V – V.

e) F – F – V – F – F.

25. (FGV/AFFC (STN)/2024) ATENÇÃO: use a tabela relacional IPCA a seguir para responder à próxima questão.

Tabela IPCA

índice	ano	mês
0,56	2023	12
0,28	2023	11
0,24	2023	10
...	...	...
0,2	2003	7
-0,15	2003	6
...	...	...
2,25	2003	1
2,1	2002	12
3,02	2002	11
...	...	...
0,57	2001	1

A instância da tabela contém os valores do índice IPCA para todos os meses dos anos de 2001 até 2023. Os valores pontilhados representam a continuidade mensal da série. Todas as colunas são numéricas e não aceitam valores nulos.

No contexto da tabela IPCA apresentada, analise o comando SQL a seguir.

```
select avg(índice)
```

```
from IPCA
```

```
group by mes
```

```
order by mes
```

A execução desse comando gera

a) doze linhas com uma coluna contendo números.

- b) doze linhas, contendo o valor null na primeira coluna e números na segunda.
- c) um resultado vazio.
- d) uma linha com duas colunas, contendo o valor null e a média de todos os índices.
- e) uma linha com uma coluna, contendo a média de todos os índices.

26. (FGV/Aud CE (TCE,PA)/TCE PA/Administrativa/Ciência de Dados/2024) Em um processo, o funcionário responsável foi substituído por outro funcionário da mesma organização. Tal substituição deve ser registrada no banco de dados do departamento de pessoal. A cláusula SQL mais indicada para implementar essa alteração, mantendo a integridade do banco, é denominada

- a) ALTER
- b) INSERT
- c) JOIN
- d) UPDATE
- e) WITH

27. (FGV/AJ (TRF 1ª Região)/TRF 1/Apoio Especializado/Análise de Dados/2024) Considere uma tabela relacional T, com uma única coluna intitulada número, cuja instância de 10 linhas contém os números inteiros de 1 até 10, um em cada linha, sem repetição. Nessas condições, foram executados os três comandos SQL exibidos a seguir.

```
select x.número * y.número A
from T x, T y
where y.número = x.número order by 1
select x.número * x.número A
from T x order by 1
select distinct x.número * x.número A
from T x
```

O número de linhas produzidas em cada comando, na ordem e sem contar a linha de títulos, é:

- a) 1, 10, 10;
- b) 10, 10, 10;
- c) 10, 100, 10;
- d) 100, 10, 1;
- e) 100, 10, 10.

28. (FGV/TJ (TRF 1ª Região)/TRF 1/Apoio Especializado/Tecnologia da Informação/2024)
Maria executou as seguintes instruções em SQL para criação de tabelas:
CREATE TABLE T1 (

id_A INT,

a1 CHAR(10),

a2 CHAR(10));

CREATE TABLE T2 (

id_B INT,

b1 CHAR(10),

b2 CHAR(10));

CREATE TABLE T3 (

id_A INT,

id_B INT,

c1 CHAR(10),

c2 CHAR(10));

Maria deseja listar a quantidade e os respectivos dados de T1 que possuem pelo menos 1 e no máximo 5 relacionamentos com T2. Para isso, ela deve utilizar a instrução:

a) SELECT COUNT(c.id_A), a.a1, a.a2

FROM T1 a

INNER JOIN T3 c ON c.id_A = a.id_A

WHERE COUNT(c.id_A) <= 5;

b) SELECT a.id_A, a.a1, a.a2

FROM T1 a, T2 b

LEFT OUTER JOIN T3 c ON c.id_A = a.id_A

LEFT OUTER JOIN T3 c ON c.id_B = b.id_B

GROUP BY a.a1, a.a2

HAVING COUNT(a.id_A) <= 5;

c) SELECT COUNT(a.id_A), a.a1, a.a2

FROM T1 a

INNER JOIN T3 c ON c.id_A = a.id_A

WHERE a.id_A <= 5;

d) SELECT COUNT(a.id_A), a.*

FROM T1 a

INNER JOIN T3 c ON c.id_A = a.id_A

GROUP BY a.a1, a.a2

HAVING COUNT(a.id_A) <= 5;

e) SELECT a.id_A, a.a1, a.a2

FROM T1 a

RIGHT OUTER JOIN T3 c ON c.id_A = a.id_A

GROUP BY a.id_A, a.a1, a.a2

HAVING COUNT(a.id_A) <= 5;

29. (FGV/Tecno Jr (INPE)/INPE/Desenvolvimento De Software Para Operação De Satélites/2024) Os comandos SQL do tipo DML (Data Manipulation Language) são utilizados para manipular os dados no banco de dados. Esses comandos permitem aos usuários interagirem com o banco de dados para executar operações.

Assinale a opção que apresenta apenas exemplos de comandos do tipo DML.

a) SELECT, INSERT, DROP e UPDATE.

b) INSERT, DROP e UPDATE.

c) SELECT, INSERT, DELETE e UPDATE.

d) CREATE, ALTER e DELETE.

e) SELECT, CREATE e TRUNCATE.

30. (FGV/AGC (EPE)/EPE/Tecnologia da Informação/Infraestrutura e Segurança/2024) Com relação à linguagem de consulta estruturada (SQL) padrão ANSI e seus subconjuntos, analise as afirmativas a seguir e assinale (V) para a verdadeira e (F) para a falsa.

() Os comandos GRANT e REVOKE são usados para conceder e revogar privilégios aos usuários e pertencem à DTL.

() O comando para apagar uma tabela é o DELETE.

() Os comandos CREATE, ALTER e DROP pertencem ao subconjunto DDL.

() O comando TRUNCATE é utilizado para limpar completamente uma tabela, mantendo sua estrutura para futuras inserções de dados. Pertence ao subconjunto DDL.

As afirmativas são, respectivamente,

- a) F – V – V – F.
- b) V – V – F – F.
- c) F – F – V – V.
- d) V – F – F – V.
- e) F – V – V – V.

31. (FGV/APP GG (SEPLAG Niterói)/Pref Niterói/Gestão de Tecnologia da Informação e Comunicação/2024) Assinale a opção que contém comandos SQL dos subconjuntos das linguagens de definição de dados (DDL), de manipulação de dados (DML) e de controle de dados (DCL), respectivamente.

- a) DROP, UPDATE e GRANT.
- b) INSERT, DROP e COMMIT.
- c) DELETE, SELECT e REVOKE.
- d) CREATE, UPDATE e SAVEPOINT.
- e) TRUNCATE, INSERT e ROLLBACK.

32. (FGV/AJ (TJ RN)/TJ RN/Apoio Especializado/Análise de Sistemas/2023) Supondo que no Brasil

- todo brasileiro tem um, e somente um, CPF;
- alguns brasileiros têm um, mas somente um, passaporte válido;
- alguns brasileiros têm uma, mas somente uma, carteira de motorista (CNH) válida.

A definição correta desses atributos numa tabela relacional normalizada seria:

- a) CPF varchar(11) not null
CNH varchar(9) UNIQUE
Passaporte varchar(9) UNIQUE
- b) CPF varchar(11) not null UNIQUE
CNH varchar(9) UNIQUE
Passaporte varchar(9) UNIQUE
- c) CPF varchar(11) UNIQUE
CNH varchar(9) not null
Passaporte varchar(9) not null

- d) CPF varchar(11) not null
CNH varchar(9) null
Passaporte varchar(9) null
- e) CPF varchar(11) null
CNH varchar(9) UNIQUE
Passaporte varchar(9) UNIQUE

33. (FGV/ACE (TCE ES)/TCE ES/Tecnologia da Informação/2023) Junior é o administrador do Banco de Dados da sua empresa e percebeu que um programador tinha acesso de alteração indevido a uma tabela.

Para cancelar a permissão previamente concedida ao programador, Junior deve usar o comando SQL:

- a) GRANT;
- b) REVOKE;
- c) UPDATE;
- d) DELETE;
- e) TRUNCATE.

34. (FGV/ATRFB/SRFB/Geral/2023) Os principais Sistemas Gerenciadores de Bancos de Dados oferecem total suporte à linguagem SQL. Um aspecto importante da implementação do SQL é o tratamento para valores nulos quando esses são considerados como unknown values. Nesse contexto, considere uma tabela T com colunas A e B, que podem conter valores nulos. T possui 100 registros e, em 50% das linhas, há pelo menos uma coluna preenchida com o valor NULL.

Considere a consulta a seguir:

```
SELECT * FROM T t1  
WHERE t1.A = NULL or t1.B = NULL
```

O número máximo de linhas de resultados que seriam retornadas pela consulta é igual a

- a) 0.
- b) 25.
- c) 50.
- d) 75.
- e) 100.

35. (FGV/Ana (Pref Niterói)/Pref Niterói/Tecnologia da Informação/2023) Figura 1
Nas duas questões a seguir, considere a tabela relacional T cuja instância é exibida abaixo.

Tabela T

A	B	C
3	6	10
1	NULL	10
3	6	NULL
3	6	5

A instalação está configurada para o tratamento de valores NULL como valores desconhecidos (unknown).

Considerando a tabela T da figura 1, analise a execução do comando SQL a seguir.

```
SELECT *  
FROM T t1 LEFT JOIN T t2  
ON t1.A = t2.A and t1.B = t2.B  
and t1.C = t2.C
```

Além da linha de títulos, o número de linhas produzidas pela execução desse comando é:

- a) 0;
- b) 4;
- c) 10;
- d) 13;
- e) 16.

36. (FGV/AJ (TJ RN)/TJ RN/Apoio Especializado/Análise de Sistemas/2023) Considere a estrutura e uma instância da tabela relacional FILIAÇÃO exibida a seguir.

FILIAÇÃO

Pessoa	Genitor
Alexandre	Francisco
Francisco	João
Joaquina	João
Carlota	Joaquina
João	Manuel
Paulo	Manuel
Maria	Paulo

Com relação à tabela FILIAÇÃO, analise o comando SQL a seguir.

```
select distinct t3.Pessoa, t4.Pessoa  
FROM FILIAÇÃO t1, FILIAÇÃO t2, FILIAÇÃO t3, FILIAÇÃO t4  
WHERE t1.Pessoa < t2.Pessoa  
and t1.Genitor = t2.Genitor  
and t3.Genitor = t1.Pessoa  
and t4.Genitor = t2.Pessoa
```

Afora a linha de títulos, o número de linhas produzidas pela execução do referido comando SQL é:

- a) 3;
- b) 4;
- c) 5;
- d) 6;
- e) 7.

37. (FGV/AJ (TJ RN)/TJ RN/Apoio Especializado/Análise de Sistemas/2023) Considere a estrutura e uma instância da tabela relacional FILIAÇÃO exibida a seguir.

FILIAÇÃO

Pessoa	Genitor
Alexandre	Francisco
Francisco	João
Joaquina	João
Carlota	Joaquina
João	Manuel
Paulo	Manuel
Maria	Paulo

Com relação à tabela FILIAÇÃO, definida anteriormente, o comando SQL que produz a lista dos nomes das pessoas que são citadas na instância da referida tabela, sem repetições, é:

- a) select Pessoa FROM FILIAÇÃO
UNION ALL select Genitor FROM FILIAÇÃO
- b) select Pessoa FROM FILIAÇÃO
UNION select Genitor FROM FILIAÇÃO
- c) select distinct Pessoa, Genitor FROM FILIAÇÃO
- d) select distinct Pessoa UNION distinct Genitor

FROM FILIAÇÃO

e) select Pessoa FROM FILIAÇÃO and
select Genitor FROM FILIAÇÃO

38. (FGV/AJ (TJ RN)/TJ RN/Apoio Especializado/Análise de Suporte/2023) Luiz é o DBA do TJRN e atendeu um chamado da equipe de desenvolvimento que pedia para criar um banco de dados com 5 Gigabytes, mas, na hora de criar, Luiz digitou 50 Gigabytes.

Para apagar o banco de dados criado equivocadamente, Luiz deve utilizar o comando:

- a) DROP;
- b) ALTER;
- c) DELETE;
- d) UPDATE;
- e) INJECTION.

39. (FGV/Ana (DPE RS)/DPE RS/Apoio Especializado (TI)/Banco de Dados/2023) Tabela R1
Na questão a seguir, considere a tabela R1, cuja instância é exibida a seguir.

A	B	C
10	20	31
10	NULL	32
11	20	NULL
12	9	31
14	8	NULL

Com referência à tabela R1 descrita anteriormente, analise o comando SQL a seguir.

```
select * from R1 x, R1 y
```

```
where x.A > y.A and x.B = y.B
```

O número de linhas produzidas pela execução desse comando, sem contar a linha de títulos, é:

- a) 0;
- b) 1;
- c) 2;
- d) 3;
- e) 4.

40. (FGV/Ana (DPE RS)/DPE RS/Apoio Especializado (TI)/Banco de Dados/2023) Tabela R1
Na questão a seguir, considere a tabela R1, cuja instância é exibida a seguir.

A	B	C
---	---	---

10	20	31
10	NULL	32
11	20	NULL
12	9	31
14	8	NULL

Com referência à tabela R1, como descrita anteriormente, analise o comando SQL a seguir.

delete from R1

where (select count(*) from R1 x

where R1.B = x.B) > 0

Após a execução desse comando, o conjunto completo de linhas presentes na instância da tabela R1 é:

a) 10 NULL 31

b) 10 20 31

Wed Dec 09 1931 01:00:00 GMT-0200 (Brasilia Summer Time)

c) 11 20 NULL

14 8 NULL

d) 10 20 31

11 20 NULL

Wed Dec 09 1931 01:00:00 GMT-0200 (Brasilia Summer Time)

14 8 NULL

e) 10 NULL 32

41. (FGV/Ana (DPE RS)/DPE RS/Apoio Especializado (TI)/Banco de Dados/2023) Tabela R1
Na questão a seguir, considere a tabela R1, cuja instância é exibida a seguir.

A	B	C
10	20	31
10	NULL	32
11	20	NULL
12	9	31
14	8	NULL

Com referência à tabela R1, como descrita anteriormente, analise o comando SQL a seguir.

update R1

set B=0

where C in (31,32) or C = null

O conjunto completo de linhas afetadas pela execução desse comando é:

- a) a primeira e a terceira;
- b) a segunda e a quarta;
- c) a terceira e a quinta;
- d) a primeira, a segunda e a quarta;
- e) a segunda, a terceira e a quinta.

42. (FGV/Ana (DPE RS)/DPE RS/Apoio Especializado (TI)/Banco de Dados/2023) Analise o esquema relacional a seguir.

```
create table X1(
```

```
  A1 int not null primary key,
```

```
  B1 int)
```

```
create table X2(
```

```
  A2 int not null primary key,
```

```
  B2 int)
```

```
create table X3(
```

```
  A1 int not null unique,
```

```
  A2 int,
```

```
  B3 int,
```

```
  foreign key(A1) references X1(A1),
```

```
  foreign key(A2) references X2(A2)
```

```
)
```

Sobre esse esquema, analise as afirmativas a seguir.

- I. Cada elemento de X1 pode estar relacionado, via X3, a zero, um ou mais elementos de X2.
- II. Cada elemento de X1 pode estar relacionado a zero, um ou mais elementos de X3.
- III. Cada elemento de X2 pode estar relacionado a zero ou apenas um elemento de X3.

Está correto somente o que se afirma em:

- a) I;
- b) II;
- c) III;

- d) I e II;
- e) II e III.

43. (FGV/Ana (DPE RS)/DPE RS/Apoio Especializado (TI)/Banco de Dados/2023) Tanto no SQL Server como no MySQL, a aplicação do comando `drop table T`

destina-se a:

- a) remover T;
- b) remover todos os índices de T;
- c) colocar T no estado read only;
- d) recuperar T a partir de arquivos de backup;
- e) desconectar T do banco de dados atual, e transformá-la numa tabela temporária.

44. (FGV/Ana (DPE RS)/DPE RS/Apoio Especializado (TI)/Banco de Dados/2023) Maria criou um banco de dados no qual há uma tabela com uma coluna X que não aceita valores nulos. Entretanto, como o sistema que mantém essa tabela é novo, na carga inicial dos dados essa coluna não existe.

Como Maria não quer abrir mão do impedimento de valores nulos, Maria poderia carregar as linhas incompletas desde que, na coluna X, usasse a cláusula:

- a) skip;
- b) check;
- c) default;
- d) float;
- e) case when ... then ... end.

45. (FGV/Ana (DPE RS)/DPE RS/Apoio Especializado (TI)/Banco de Dados/2023) Observe o seguinte script de concessão de privilégios em MySQL:

```
CREATE DATABASE db;
```

```
CREATE TABLE db.t1 (c INT);
```

```
INSERT INTO db.t1 VALUES ROW (1);
```

```
CREATE TABLE db.t2 (c INT);
```

```
INSERT INTO db.t2 VALUES ROW (1);
```

```
CREATE USER u1;
```

```
GRANT SELECT, UPDATE ON db.t1 TO u1 WITH GRANT
```

```
OPTION;  
  
CREATE USER u2;  
  
GRANT SELECT, INSERT ON db.t2 TO u2;  
  
CREATE USER u3;  
  
GRANT ALL ON db.* TO u3;  
  
REVOKE INSERT ON db.t2 FROM u2;
```

Após a execução do script apresentado, é correto afirmar que:

- a) os usuários u2 e u3 poderão executar a instrução `INSERT INTO db.t2 VALUES ROW (2);`
- b) os usuários u1 e u3 poderão executar a instrução `UPDATE db.t2 SET c=2;`
- c) o usuário u2 poderá conceder o privilégio de `INSERT` para o usuário u1;
- d) os usuários u1 e u3 poderão executar a instrução `DELETE FROM db.t1;`
- e) o usuário u1 poderá conceder o privilégio de `UPDATE` para o usuário u2.

46. (FGV/FR (Pref RJ)/Pref RJ/2023) Considere a existência de uma tabela relacional N, com apenas uma coluna, intitulada `numero`, contendo os números inteiros de 1 até 100, um em cada linha, como ilustrada a seguir.

N

numero
1
2
...
99
100

Como pode haver discrepâncias entre implementações da linguagem SQL, é dado que a função `sqrt(x)` retorna a raiz quadrada de `x` e que a expressão `a % b` retorna o resto da divisão inteira de `a` por `b`.

Analise o comando SQL a seguir, que faz referência à tabela N descrita anteriormente.

```
select n1.numero * n2.numero  
from N n1, N n2  
where n1.numero <> n2.numero
```

O número de linhas do resultado produzido pela execução desse comando, sem contar a linha de títulos, é:

- a) 9.000
- b) 9.110

- c) 9.900
- d) 9.991
- e) 10.000

47. (FGV/FR (Pref RJ)/Pref RJ/2023) Considere a existência de uma tabela relacional N, com apenas uma coluna, intitulada numero, contendo os números inteiros de 1 até 100, um em cada linha, como ilustrada a seguir.

N

numero
1
2
...
99
100

Como pode haver discrepâncias entre implementações da linguagem SQL, é dado que a função $\text{sqrt}(x)$ retorna a raiz quadrada de x e que a expressão $a \% b$ retorna o resto da divisão inteira de a por b .

Este é o resultado produzido por um determinado script SQL que utiliza a tabela N, anteriormente descrita.

A	B
2	50
3	33
4	25
5	20
6	16
7	14

Abaixo, são apresentadas três versões para o referido script, não necessariamente corretas.

I. select x.numero A,
(select count(*) from N
where N.numero % x.numero = 0) B
from (select numero from N
where numero >= 2 and numero <= 7) x
order by 2 desc

II. select x.numero A,
(select count(*) from N
where N.numero % x.numero = 0) B

```
from (select numero from N
where numero >= 2 and numero <= 7) x
where x.numero % x.numero = 0
order by 2 desc

III. select x.numero A,
(select count(*) from N
where N.numero % x.numero = 0) B
from (select numero from N
where numero >= 2 and numero <= 7) x
where x.numero % x.numero = 0
group by x.numero
having count(*) > 0
order by 2 desc
```

Sobre essas afirmativas, é correto afirmar que:

- a) nenhuma delas produz o resultado correto;
- b) somente I e II produzem o resultado correto;
- c) somente II e III produzem o resultado correto;
- d) somente III produz o resultado correto;
- e) todas produzem o resultado correto.

48. (FGV/ALEMA/Analista de Suporte de Rede/2023) O banco de dados relacional de uma escola contém as tabelas denominadas TB_STUDENT_A e TB_STUDENT_B e ambas possuem apenas as colunas ID e NAME.

O conteúdo dessas tabelas está detalhado a seguir:

TB_STUDENT_A

ID	NAME
1	Maria
2	João
3	Visconde
4	Ana
5	Inês

TB_STUDENT_B

ID	NAME
1	Fábio
2	Marta
3	Maria
4	Sérgio
5	Penélope
6	João

Analise a seguinte consulta em linguagem SQL:

```
SELECT *  
FROM TB_STUDENT_A  
LEFT JOIN TB_STUDENT_B  
ON TB_STUDENT_A.NAME=TB_STUDENT_B.NAME.
```

O número de linhas do resultado produzido pela execução deste comando SQL sobre o banco de dados desta escola, excetuada a linha de títulos, é:

- a) 2.
- b) 3.
- c) 4.
- d) 5.
- e) 6.

49. (FGV/ACE (TCE SP)/TCE SP/Tecnologia da Informação (TI)/2023) Atenção: Quando referidas, considere as tabelas relacionais TX e TY, criadas e instanciadas com o script SQL a seguir.

```
create table TY(C int primary key not null, A int)
```

```
create table TX(A int primary key not null, B int,
```

```
foreign key (B) references TY(C)
```

```
on delete cascade
```

```
)
```

```
insert into TY values (1,0)
```

```
insert into TY(C) values (2)
```

```
insert into TY(C) values (3)
```

```
insert into TY values (5,NULL)
```

insert into TY values (6,NULL)

insert into TX values (1,2)

insert into TX values (2,1)

insert into TX values (3,2)

insert into TX values (4,2)

Com referência às tabelas TX e TY, como descritas anteriormente, analise o comando SQL a seguir.

```
select count(*)
```

```
from TX t1 left join TY t2 on t1.B=t2.A
```

O valor exibido pela execução desse comando é:

a) 0;

b) 2;

c) 3;

d) 4;

e) 6.

50. (FGV/ACE (TCE SP)/TCE SP/Tecnologia da Informação (TI)/2023) Atenção

Quando referidas, considere as tabelas relacionais TX e TY, criadas e instanciadas com o script SQL a seguir.

```
create table TY(C int primary key not null, A int)
```

```
create table TX(A int primary key not null, B int,
```

```
foreign key (B) references TY(C)
```

```
on delete cascade
```

```
)
```

```
insert into TY values (1,0)
```

```
insert into TY(C) values (2)
```

```
insert into TY(C) values (3)
```

```
insert into TY values (5,NULL)
```

```
insert into TY values (6,NULL)
```

```
insert into TX values (1,2)
```

insert into TX values (2,1)

insert into TX values (3,2)

insert into TX values (4,2)

Com referência às tabelas TX e TY, como descritas anteriormente, analise o comando SQL a seguir.

delete FROM TY

Após a execução desse comando, o total de linhas, somando-se os números de linhas de cada tabela, é:

- a) 0;
- b) 1;
- c) 4;
- d) 5;
- e) 9.

51. (FGV/Prof EFM (SEDUC SP)/SEDUC SP/Informação e Comunicação/2023) Analise o comando SQL que João produziu.

```
select Tipo, sum(Quantidade) Total
```

```
from Venda
```

```
where Quantidade is not null
```

```
group by Tipo
```

...

Na linha pontilhada, João testou cinco opções para a cláusula order by, como segue.

- (1) order by 2
- (2) order by avg(Quantidade)
- (3) order by Quantidade
- (4) order by sum(Quantidade)
- (5) order by Total

Nesse caso são possíveis as opções

- a) 1, 2, 3, 4, 5.
- b) 1, 2, 3, 4, apenas.
- c) 1, 4, 5, apenas.

d) 1, 2, 4, 5, apenas.

e) 4, 5, apenas.

52. (FGV/Prof EFM (SEDUC SP)/SEDUC SP/Informação e Comunicação/2023) Considere os comandos SQL a seguir.

```
create table TT1(A int not null primary key);
```

```
create table TT2(B int
```

```
foreign key references TT1(A),
```

```
C int not null);
```

```
select * from TT1 left join TT2 on TT1.A = TT2.B;
```

Dado que TT1 e TT2 contêm, respectivamente, 100 e 150 linhas, assinale as expressões que apresentam, na ordem, os números mínimo e máximo de linhas que podem ser produzidas pela execução do comando SELECT, fora a linha de títulos.

a) 100 e 150.

b) 100 e 249.

c) 100 e 250.

d) 150 e 249.

e) 250 e 15000.

53. (FGV/AJ (TJ SE)/TJ SE/Análise de Sistemas/Banco de Dados/2023) De acordo com a tabela T1, descrita anteriormente, analise o comando SQL a seguir.

```
select * from T1
```

```
UNION ALL
```

```
select * from T1
```

```
UNION
```

```
select * from T1
```

Afora os títulos, o número de linhas produzidas pela execução do comando acima é:

a) 1;

b) 2;

c) 3;

d) 6;

e) 9.

54. (FGV/APGG (Pref BH)/Pref BH/Ciência da Computação/2023) Considere a tabela T, com colunas A, B e C, descrita a seguir juntamente com a sua instância.

A

1

2

3

4

Considere, ainda, o comando SQL a seguir, que referencia a tabela T.

```
select t1.A X1, t1.B X2, t1.C X3,
```

```
t2.A X4, t2.B X5, t2.C X6,
```

```
t3.A X7, t1.B X8, t1.C X9
```

```
from T t1
```

```
LEFT OUTER JOIN T t2 on t1.a = t2.b
```

```
RIGHT OUTER JOIN T t3 on t2.b = t3.c
```

Assinale o número de linhas, além dos títulos, produzidas pela execução do comando SQL apresentado anteriormente.

a) 3 linhas.

b) 4 linhas.

c) 5 linhas.

d) 6 linhas.

55. (FGV/APGG (Pref BH)/Pref BH/Ciência da Computação/2023) Considere a tabela T, com colunas A, B e C, descrita a seguir juntamente com a sua instância.

A

1

2

3

4

Considere, ainda, o comando SQL a seguir, que referencia a tabela T.

```
select t1.A X1, t1.B X2, t1.C X3,
```

```
t2.A X4, t2.B X5, t2.C X6,
```

t3.A X7, t1.B X8, t1.C X9

from T t1

LEFT OUTER JOIN T t2 on t1.a = t2.b

RIGHT OUTER JOIN T t3 on t2.b = t3.c

Considerando a execução do comando SQL apresentado anteriormente, assinale a coluna do resultado que não contém valores nulos (null).

a) X1.

b) X3.

c) X7.

d) X9.

56. (FGV/AUFC (TCU)/TCU/Controle Externo/Auditoria Governamental/2022) Na questão abaixo, considere as tabelas de banco de dados T, TX e DUAL, exibidas com suas respectivas instâncias a seguir.

T

sequencia	caracteristica
1	23987
2	9845
3	NULL
4	40983
6	48750
7	NULL
8	NULL
10	48750
12	48750

TX

sequencia	caracteristica
2	9845
3	998034
4	50932
5	24390
6	48750
6	50296
7	NULL
8	998746
9	32746
9	NULL
9	22798

DUAL

x
NULL

Analise os cinco comandos SQL exibidos abaixo, utilizando a tabela DUAL apresentada anteriormente.

- (1) select * from dual where x = null
- (2) select * from dual where x <> null
- (3) select * from dual where x > 10
- (4) select * from dual where not x > 10
- (5) select * from dual where x > 10

union

select * from dual where x <= 10

Se os resultados desses comandos fossem separados em grupos homogêneos, de modo que em cada grupo todos sejam idênticos e distintos dos elementos dos demais grupos, haveria:

- a) apenas um grupo;
- b) apenas dois grupos;
- c) apenas três grupos;
- d) apenas quatro grupos;
- e) cinco grupos.

57. (FGV/AFFC (CGU)/CGU/Tecnologia da Informação/"Sem Especialidade"/2022) No contexto do SQL Server 2019, considere o script SQL a seguir.

```
create table XPTO (
```

```
  C1 int identity,
```

```
  C2 varchar(16))
```

```
insert into XPTO (C2) values ('A')
```

```
insert into XPTO (C2) values ('B')
```

```
insert into XPTO (C2) values ('C')
```

```
truncate table XPTO
```

```
insert into XPTO (C2) values ('D')
```

```
insert into XPTO (C2) values ('E')
```

delete from XPTO

truncate table XPTO

insert into XPTO (C2) values ('F')

truncate table XPTO

Após a execução desse script, o resultado exibido pelo comando `select max(C1) from XPTO` é:

- a) 0
- b) NULL
- c) 1
- d) 3
- e) 6

58. (FGV/Per (PC AM)/PC AM/4ª Classe/Processamento de Dados/2022) Considere as tabelas de banco de dados:

- T1, com uma única coluna A1, onde A1 é a chave primária;
- T2, com uma única coluna A2, onde A2 é a chave primária, e A2 é também uma chave estrangeira que referencia T1.

As instâncias de T1 e T2 contêm, respectivamente, 12 e 10 linhas.

Com referência às tabelas T1 e T2, descritas anteriormente, o comando SQL

```
select t1.A1, t2.A2
```

```
from T1 t1 LEFT JOIN T2 t2 on t1.A1 = t2.A2
```

```
order by 1
```

produzirá um resultado com

- a) zero linhas.
- b) 10 linhas.
- c) 12 linhas.
- d) 22 linhas.
- e) 120 linhas.

59. (FGV/Per (PC AM)/PC AM/4ª Classe/Processamento de Dados/2022) Considere as tabelas de banco de dados:

- T1, com uma única coluna A1, onde A1 é a chave primária;

- T2, com uma única coluna A2, onde A2 é a chave primária, e A2 é também uma chave estrangeira que referencia T1.

As instâncias de T1 e T2 contêm, respectivamente, 12 e 10 linhas.

Com referência às tabelas T1 e T2, o comando SQL

```
select t1.A1, t2.A2
```

```
from T1,T2
```

```
where t1.A1 <> t2.A2
```

```
order by 1
```

produzirá um resultado com

a) 120 linhas.

b) 110 linhas.

c) 100 linhas.

d) 90 linhas

e) 20 linhas.

60. (FGV/GTIFE (Sefaz AM)/SEFAZ AM/2022) Analise o comando a seguir.

```
SELECT * FROM T1 FULL OUTER JOIN T2 ON T1.A=T2.A
```

Sabe-se que a coluna A foi definida como chave primária tanto em T1 como em T2, e que T1 e T2 contêm, respectivamente, 20 e 30 linhas.

Assinale a opção que apresenta a estimativa do número de linhas produzidas pela execução do comando acima.

a) 20, exatamente.

b) 30, exatamente.

c) 600, exatamente.

d) Entre 20 e 30.

e) Entre 30 e 50.

GABARITO

1. A	21. B	41. D
2. E	22. C	42. A
3. D	23. A	43. A
4. B	24. B	44. C
5. B	25. A	45. E
6. D	26. D	46. C
7. B	27. B	47. E
8. B	28. D	48. D
9. C	29. C	49. D
10. A	30. C	50. A
11. B	31. A	51. D
12. C	32. B	52. B
13. B	33. B	53. C
14. B	34. A	54. B
15. D	35. B	55. C
16. C	36. A	56. A
17. D	37. B	57. B
18. B	38. A	58. C
19. B	39. B	59. B
20. B	40. E	60. E

ESSA LEI TODO MUNDO CONHECE: PIRATARIA É CRIME.

Mas é sempre bom revisar o porquê e como você pode ser prejudicado com essa prática.



1 Professor investe seu tempo para elaborar os cursos e o site os coloca à venda.



2 Pirata divulga ilicitamente (grupos de rateio), utilizando-se do anonimato, nomes falsos ou laranjas (geralmente o pirata se anuncia como formador de "grupos solidários" de rateio que não visam lucro).



3 Pirata cria alunos fake praticando falsidade ideológica, comprando cursos do site em nome de pessoas aleatórias (usando nome, CPF, endereço e telefone de terceiros sem autorização).



4 Pirata compra, muitas vezes, clonando cartões de crédito (por vezes o sistema anti-fraude não consegue identificar o golpe a tempo).



5 Pirata fere os Termos de Uso, adultera as aulas e retira a identificação dos arquivos PDF (justamente porque a atividade é ilegal e ele não quer que seus fakes sejam identificados).



6 Pirata revende as aulas protegidas por direitos autorais, praticando concorrência desleal e em flagrante desrespeito à Lei de Direitos Autorais (Lei 9.610/98).



7 Concurseiro(a) desinformado participa de rateio, achando que nada disso está acontecendo e esperando se tornar servidor público para exigir o cumprimento das leis.



8 O professor que elaborou o curso não ganha nada, o site não recebe nada, e a pessoa que praticou todos os ilícitos anteriores (pirata) fica com o lucro.



Deixando de lado esse mar de sujeira, aproveitamos para agradecer a todos que adquirem os cursos honestamente e permitem que o site continue existindo.